
DATA SCIENCE

Série 1 — Volume III

Cahier d'exercices et d'applications

Exercices corrigés · Applications guidées · Projet
Compagnon du volume III : Apprentissage automatique

À utiliser avec le livre : chaque chapitre renvoie ici
pour s'entraîner.

Table des matières

Mode d'emploi	1
Comment utiliser ce cahier	1
Numérotation et niveaux	1
Données et environnement	2
Dictionnaire des données de <code>clients_ml.csv</code>	3
Lancer le code	3
Vérifier son installation	4
Plan du cahier	5
Chapitre 1 : La démarche d'apprentissage automatique — exercices et applications	6
Préparation	6
Applications	7
Application 1.1 — Détecter la fuite d'information (section 1.1.6)	7
Application 1.2 — La loterie du découpage unique (section 1.2.1)	8
Application 1.3 — Combien de plis ? (section 1.2.2)	9
Application 1.4 — Valider une série temporelle sans tricher (section 1.2.3)	9
Application 1.5 — Biais et variance par simulation (section 1.3.4)	10
Application 1.6 — Courbes d'apprentissage et diagnostic (sections 1.3.5 et 1.3.6)	11
Application 1.7 — Comparer deux modèles avec rigueur (section 1.4)	11
Application 1.8 — Grille, recherche aléatoire et Optuna (section 1.5)	12
Application 1.9 — L'optimisme du réglage et la validation imbriquée (section 1.5.5)	13
Exercices	14
Exercice 1.1 ★ — Risque empirique à la main (section 1.1.1)	14
Exercice 1.2 ★ — Le piège de l'exactitude (section 1.1.3)	14
Exercice 1.3 ★★ — Audit du calendrier (section 1.1.6)	14
Exercice 1.4 ★ — Un seul laissé de côté, à la main (section 1.2.2)	14
Exercice 1.5 ★★ — Construire des plis (section 1.2.3)	14
Exercice 1.6 ★★ — Découpage temporel (section 1.2.3)	14
Exercice 1.7 ★★ — Le meilleur estimateur biaisé (section 1.3.2)	14
Exercice 1.8 ★★★ — Lire des courbes d'apprentissage (section 1.3.6)	15
Exercice 1.9 ★★ — Le test t corrigé (section 1.4.2)	15
Exercice 1.10 ★★ — McNemar (section 1.4.2)	15
Exercice 1.11 ★★ — Combien d'essais aléatoires ? (section 1.5.2)	15
Exercice 1.12 ★★★ — L'optimisme du maximum (section 1.5.5)	15
Corrigés	15
Corrigé 1.1	15
Corrigé 1.2	16
Corrigé 1.3	16
Corrigé 1.4	16

Corrigé 1.5	16
Corrigé 1.6	17
Corrigé 1.7	17
Corrigé 1.8	17
Corrigé 1.9	17
Corrigé 1.10	18
Corrigé 1.11	18
Corrigé 1.12	18
Chapitre 2 : Apprentissage supervisé — exercices et applications	19
Préparation	19
Applications	20
Application 2.1 — La descente de gradient écrite à la main	20
Application 2.2 — Chemins de régularisation	21
Application 2.3 — Construire un arbre à la main, puis avec <code>scikit-learn</code>	22
Application 2.4 — Le bagging écrit à la main	23
Application 2.5 — Importances et variables de bruit	24
Application 2.6 — Le gradient boosting écrit à la main	24
Application 2.7 — XGBoost et LightGBM : réglages, valeurs manquantes, catégories	25
Application 2.8 — Le match des modèles, avec comparaison appariée	26
Application 2.9 — Les modèles classiques face au fléau de la dimension	27
Application 2.10 — Encodage ordonné, CatBoost et empilement sans fuite	28
Exercices	29
Exercice 2.1 ★ — Quelle perte pour quelle marge ? (section 2.1.1)	29
Exercice 2.2 ★★ — Un pas de gradient (section 2.1.3)	29
Exercice 2.3 ★★ — Seuillage contre rétrécissement (section 2.1.5)	29
Exercice 2.4 ★ — Quelle coupe choisir ? (section 2.2.3)	29
Exercice 2.5 ★★ — Un arbre de régression (section 2.2.4)	29
Exercice 2.6 ★★★ — Élaguer par coût-complexité (section 2.2.5)	29
Exercice 2.7 ★★ — Combien d'arbres ? (section 2.3.2)	30
Exercice 2.8 ★★ — Les clients hors sac (section 2.3.4)	30
Exercice 2.9 ★★ — Deux tours de boosting (section 2.4.1)	30
Exercice 2.10 ★★★ — Valeur de feuille et gain XGBoost (section 2.4.3)	30
Exercice 2.11 ★★ — Bayes naïf avec lissage (section 2.5.3)	30
Exercice 2.12 ★★★ — Pourquoi moyenner des modèles corrélés rapporte peu (section 2.6.2)	30
Corrigés	30
Corrigé 2.1	31
Corrigé 2.2	31
Corrigé 2.3	31
Corrigé 2.4	31
Corrigé 2.5	32
Corrigé 2.6	32
Corrigé 2.7	32
Corrigé 2.8	32
Corrigé 2.9	33
Corrigé 2.10	33
Corrigé 2.11	33
Corrigé 2.12	34
Chapitre 3 : Apprentissage non supervisé et réduction de dimension — exercices et applications	35

Applications	35
Préparation commune	35
Application 3.1 — Segmenter la clientèle de bout en bout (sections 3.1.1 à 3.1.7) . .	36
Application 3.2 — Valider contre la vérité cachée : initialisation, échelle, transfor- mation (sections 3.1.5 et 3.1.6)	37
Application 3.3 — La statistique de l'écart, écrite à la main (section 3.1.3)	38
Application 3.4 — Réduire les chiffres manuscrits : ACP, noyau, NMF, projection aléatoire (sections 3.2.2 à 3.2.6)	38
Application 3.5 — Quand les groupes ne sont pas des boules : DBSCAN et liens (section 3.3.1 et 3.3.2)	39
Application 3.6 — L'algorithme EM, de la main à la bibliothèque (section 3.3.3) . .	40
Application 3.7 — t-SNE et UMAP : perplexité, voisins, fiabilité (sections 3.4.1 à 3.4.4)	41
Application 3.8 — Réduire avant de classer : un pipeline sur les clients (section 3.2.8)	42
Exercices	42
Exercice 3.1 ★ — Silhouette à la main (section 3.1.2)	42
Exercice 3.2 ★★ — Calinski–Harabasz et Davies–Bouldin à la main (section 3.1.2) .	43
Exercice 3.3 ★★ — L'indice de Rand ajusté à partir d'un tableau croisé (section 3.1.5)	43
Exercice 3.4 ★★ — L'inertie ne peut que décroître (section 3.1.1)	43
Exercice 3.5 ★★ — Appliquer la règle de Tibshirani (section 3.1.3)	43
Exercice 3.6 ★ — Part de variance et erreur de reconstruction (section 3.2.2)	43
Exercice 3.7 ★★ — Le lemme de Johnson–Lindenstrauss en chiffres (section 3.2.6)	43
Exercice 3.8 ★★ — La concentration des distances, démontrée (section 3.2.1)	43
Exercice 3.9 ★★ — DBSCAN à la main en dimension 2 (section 3.3.1)	44
Exercice 3.10 ★★ — Un tour d'EM (section 3.3.3)	44
Exercice 3.11 ★★ — Compter les paramètres, calculer un BIC (section 3.3.3)	44
Exercice 3.12 ★★ — La perplexité de t-SNE (section 3.4.1)	44
Corrigés	44
Corrigé 3.1	44
Corrigé 3.2	45
Corrigé 3.3	45
Corrigé 3.4	45
Corrigé 3.5	45
Corrigé 3.6	46
Corrigé 3.7	46
Corrigé 3.8	46
Corrigé 3.9	46
Corrigé 3.10	47
Corrigé 3.11	47
Corrigé 3.12	47
Chapitre 4 : Ingénierie des variables et données déséquilibrées — exercices et applications	48
Préparation	48
Applications	49
Application 4.1 — Un pipeline complet, pas à pas (sections 4.1.6)	49
Application 4.2 — L'encodage par la cible, hors pli, écrit à la main (section 4.1.2) .	50
Application 4.3 — Valeurs manquantes : quand l'indicateur d'absence compte (sec- tion 4.1.5)	51
Application 4.4 — Créer des variables RFM et mesurer leur apport (section 4.2) . .	52
Application 4.5 — Toutes les stratégies pour classes rares, sur la fraude (sections 4.3.1 à 4.3.3)	54
Application 4.6 — Choisir un seuil par les coûts (section 4.3.5)	55

Application 4.7 — Sélection de variables : trois familles et un piège (section 4.4) . .	55
Application 4.8 — SMOTE et variantes dans un pipeline (section 4.5)	56
Exercices	58
Exercice 4.1 ★ — L'encodage par la cible, à la main (section 4.1.2)	58
Exercice 4.2 ★ — Mettre à l'échelle (section 4.1.3)	58
Exercice 4.3 ★★ — Box-Cox et Yeo-Johnson (section 4.1.4)	58
Exercice 4.4 ★★ — Mécanismes d'absence (section 4.1.5)	58
Exercice 4.5 ★★★ — Trouver les fuites (section 4.1.6)	59
Exercice 4.6 ★ — Créer des variables (section 4.2.1)	59
Exercice 4.7 ★★ — Coder une heure sur un cercle (section 4.2.4)	59
Exercice 4.8 ★★ — Variable disponible ou fuite ? (section 4.2.6)	59
Exercice 4.9 ★ — Lire une table de confusion (section 4.3.1)	59
Exercice 4.10 ★★ — Ce que font les poids (section 4.3.2)	59
Exercice 4.11 ★★★ — Un seuil par les coûts (section 4.3.5)	60
Exercice 4.12 ★★ — Mesurer le biais de sélection (section 4.4)	60
Exercice 4.13 ★★ — SMOTE à la main (section 4.5.1)	60
Exercice 4.14 ★★★ — La fuite par rééchantillonnage, sur des données réelles (section 4.5.4)	60
Corrigés	60
Corrigé 4.1	60
Corrigé 4.2	61
Corrigé 4.3	61
Corrigé 4.4	62
Corrigé 4.5	62
Corrigé 4.6	63
Corrigé 4.7	63
Corrigé 4.8	63
Corrigé 4.9	64
Corrigé 4.10	64
Corrigé 4.11	64
Corrigé 4.12	65
Corrigé 4.13	65
Corrigé 4.14	65
Chapitre 5 : Évaluation, calibration et interprétabilité — exercices et applications	67
Préparation	67
Applications	68
Application 5.1 — Du score à la décision : seuils, coûts et gain	68
Application 5.2 — La courbe ROC, l'AUC et la précision moyenne reconstruites à la main	69
Application 5.3 — Lift, gain et incertitude sur la comparaison de deux modèles . .	71
Application 5.4 — Régression : quelle perte pour quelle décision ?	72
Application 5.5 — Diagnostiquer et réparer la calibration	73
Application 5.6 — Trois importances, trois questions	74
Application 5.7 — Expliquer un client : SHAP, LIME et stabilité	76
Application 5.8 — Audit d'équité sur le jeu réel de crédit	77
Application 5.9 — Des seuils par groupe, et ce qu'ils coûtent	79
Application 5.10 — Prédiction conforme : classification et régression	80
Exercices	81
Exercice 5.1 ★ — Matrice de confusion à la main (section 5.1.2)	81
Exercice 5.2 ★ — Mesures F_β (section 5.1.2)	82
Exercice 5.3 ★★ — L'AUC par les paires (section 5.1.4)	82

Exercice 5.4 ★★ — ROC contre précision-rappel (section 5.1.5)	82
Exercice 5.5 ★★ — Le seuil par les coûts et la calibration (section 5.1.7)	82
Exercice 5.6 ★ — Log-loss et score de Brier (section 5.1.6)	82
Exercice 5.7 ★★ — L'ECE à la main (section 5.2.1)	82
Exercice 5.8 ★★★ — Poids de classes et décalage de l'ordonnée (section 5.2.6)	82
Exercice 5.9 ★★ — Valeurs de Shapley à la main (section 5.3.5)	83
Exercice 5.10 ★★★ — SHAP d'un modèle linéaire (section 5.3.6)	83
Exercice 5.11 ★★ — Les critères d'équité (section 5.4.2)	83
Exercice 5.12 ★★★ — L'impossibilité (section 5.4.4)	83
Exercice 5.13 ★★ — Le quantile conforme (section 5.5.2)	83
Exercice 5.14 ★★★ — La loi de la couverture (section 5.5.3)	83
Corrigés	83
Corrigé 5.1	84
Corrigé 5.2	84
Corrigé 5.3	84
Corrigé 5.4	85
Corrigé 5.5	85
Corrigé 5.6	85
Corrigé 5.7	86
Corrigé 5.8	86
Corrigé 5.9	86
Corrigé 5.10	87
Corrigé 5.11	87
Corrigé 5.12	88
Corrigé 5.13	88
Corrigé 5.14	88
Chapitre 6 : Ⓢ Détection d'anomalies et de fraude — exercices et applications	90
Préparation	90
Applications	91
Application 6.1 — Mesurer un détecteur sous déséquilibre extrême (section 6.1)	91
Application 6.2 — z-score, Mahalanobis et masquage (section 6.2.1 et 6.2.2)	93
Application 6.3 — Plus proches voisins et LOF (section 6.2.3 et 6.2.4)	94
Application 6.4 — Une forêt d'isolement écrite à la main (section 6.3)	97
Application 6.5 — Autoencodeur et ACP (section 6.4)	98
Application 6.6 — Trier les alertes : combiner les détecteurs (section 6.4.5)	100
Exercices	103
Exercice 6.1 ★ — Deux détecteurs, une même exactitude (section 6.1.3)	103
Exercice 6.2 ★ — Précision moyenne à la main (section 6.1.5)	103
Exercice 6.3 ★★ — Taux de base et formule de Bayes (section 6.1.3)	103
Exercice 6.4 ★★ — Combien d'alertes traiter ? (section 6.1.6)	103
Exercice 6.5 ★ — z-score classique contre z-score robuste (section 6.2.1)	103
Exercice 6.6 ★★ — Mahalanobis à la main (section 6.2.2)	103
Exercice 6.7 ★★ — LOF sur cinq points (section 6.2.4)	104
Exercice 6.8 ★ — La constante $c(n)$ et le score d'isolement (section 6.3.2)	104
Exercice 6.9 ★★ — Contamination et seuil (section 6.3.4)	104
Exercice 6.10 ★★★ — Un autoencodeur linéaire est une ACP (section 6.4.2)	104
Exercice 6.11 ★★ — Pourquoi limiter le goulot ? (section 6.4.3)	104
Exercice 6.12 ★★★ — Un budget, deux détecteurs (section 6.4.5)	104
Corrigés	105
Corrigé 6.1	105
Corrigé 6.2	105

Corrigé 6.3	105
Corrigé 6.4	105
Corrigé 6.5	106
Corrigé 6.6	106
Corrigé 6.7	106
Corrigé 6.8	107
Corrigé 6.9	107
Corrigé 6.10	107
Corrigé 6.11	108
Corrigé 6.12	109
Chapitre 7 : ☉ Systèmes de recommandation — exercices et applications	110
Préparation	110
Applications	111
Application 7.1 — La matrice d'interactions, la popularité et le contenu à la main	111
Application 7.2 — Les voisins de produits et de clients écrits à la main	112
Application 7.3 — L'ALS implicite écrit à la main	114
Application 7.4 — La factorisation sur les notes explicites, par SGD	115
Application 7.5 — Les métriques et leurs intervalles de confiance, écrits à la main	116
Application 7.6 — Le démarrage à froid	118
Application 7.7 — Découpage aléatoire contre découpage temporel	120
Application 7.8 — La boucle de rétroaction	120
Application 7.9 — Mélanger les méthodes, et diversifier	122
Exercices	123
Exercice 7.1 ★ — Cosinus et Jaccard à la main (section 7.1.4)	123
Exercice 7.2 ★ — Popularité et contenu pour un petit client (sections 7.1.5 et 7.1.6)	123
Exercice 7.3 ★★ — Pourquoi stocker en format creux (section 7.1.3)	123
Exercice 7.4 ★ — Voisins de produits pour U4 (section 7.2.3)	123
Exercice 7.5 ★★ — Le rétrécissement renverse un classement (section 7.2.4)	123
Exercice 7.6 ★★ — Prédire une note avec les voisins (section 7.2.2)	123
Exercice 7.7 ★★ — Une étape d'ALS à la main (section 7.3.4)	124
Exercice 7.8 ★★ — L'astuce de calcul de l'ALS implicite (section 7.3.5)	124
Exercice 7.9 ★★ — Un pas de gradient stochastique (section 7.3.3)	124
Exercice 7.10 ★ — Les cinq métriques pour une liste de dix (section 7.4.1)	124
Exercice 7.11 ★★ — Quand les métriques ne s'accordent pas (section 7.4.1)	124
Exercice 7.12 ★★ — Sans dérive, le découpage aléatoire est-il innocent ? (sections 7.4.3 et 7.4.6)	124
Corrigés	124
Corrigé 7.1	125
Corrigé 7.2	125
Corrigé 7.3	125
Corrigé 7.4	126
Corrigé 7.5	126
Corrigé 7.6	126
Corrigé 7.7	127
Corrigé 7.8	127
Corrigé 7.9	127
Corrigé 7.10	127
Corrigé 7.11	128
Corrigé 7.12	128

Chapitre 8 : ☉ Apprentissage semi-supervisé et actif — exercices et applications	129
---	------------

Préparation	129
Applications	130
Application 8.1 — La courbe d'apprentissage selon le budget d'étiquettes	130
Application 8.2 — Auto-apprentissage : écrire la boucle, regarder les pseudo-étiquettes	131
Application 8.3 — La propagation d'étiquettes, du graphe à la solution fermée	132
Application 8.4 — Co-apprentissage : un essai instructif	133
Application 8.5 — Quand les graphes échouent : et si l'on choisissait mieux les variables ?	135
Application 8.6 — La boucle d'apprentissage actif, écrite à la main	136
Application 8.7 — Comité et densité	137
Application 8.8 — Biais d'échantillonnage, démarrage à froid et coût	138
Exercices	140
Exercice 8.1 ★ — Une étape d'EM dur à la main (section 8.1.2)	140
Exercice 8.2 ★★ — Combien de tirages ? (section 8.1.5)	140
Exercice 8.3 ★★ — Quelle hypothèse est en jeu ? (section 8.1.3)	140
Exercice 8.4 ★ — Pseudo-étiquettes et erreurs attendues (section 8.2.1)	140
Exercice 8.5 ★★ — Propagation sur quatre nœuds (section 8.2.4)	140
Exercice 8.6 ★★★ — Pourquoi la propagation converge (section 8.2.4)	140
Exercice 8.7 ★ — Trois critères, un classement (section 8.3.2)	141
Exercice 8.8 ★★ — Équivalence à deux classes (section 8.3.2)	141
Exercice 8.9 ★★ — Entropie du vote (section 8.3.3)	141
Exercice 8.10 ★★ — Un test des signes (section 8.3.5)	141
Exercice 8.11 ★★★ — Corriger le biais d'échantillonnage par les proportions ? (section 8.3.6)	141
Exercice 8.12 ★★ — Quand l'apprentissage actif est-il rentable ? (section 8.3.8)	141
Corrigés	141
Corrigé 8.1	141
Corrigé 8.2	142
Corrigé 8.3	142
Corrigé 8.4	142
Corrigé 8.5	143
Corrigé 8.6	143
Corrigé 8.7	144
Corrigé 8.8	144
Corrigé 8.9	144
Corrigé 8.10	145
Corrigé 8.11	145
Corrigé 8.12	146
Chapitre 9 : ☉ Bases de l'apprentissage par renforcement — exercices et applications	147
Applications	147
Préparation (à exécuter d'abord)	147
Application 9.1 — Résoudre le problème de stock et mesurer l'effet des frais fixes (section 9.1.9)	148
Application 9.2 — Les quatre bannières : comparer les stratégies de bandit (section 9.2)	149
Application 9.3 — Un bandit contextuel selon le canal d'arrivée (section 9.2.8)	151
Application 9.4 — Le Q-learning sur le problème de stock (section 9.3)	151
Application 9.5 — SARSA contre Q-learning sur le couloir d'entrepôt (section 9.3.4)	153
Application 9.6 — Une récompense mal posée (section 9.4.6)	154
Exercices	155

Exercice 9.1 ★ — Retour actualisé (section 9.1.2)	155
Exercice 9.2 ★★ — Évaluer deux politiques (section 9.1.5)	155
Exercice 9.3 ★★ — Itération de la valeur à la main (section 9.1.7)	155
Exercice 9.4 ★★★ — Combien d'itérations ? (section 9.1.8)	155
Exercice 9.5 ★ — Calculer un regret (section 9.2.2)	155
Exercice 9.6 ★★ — L'indice UCB (section 9.2.5)	155
Exercice 9.7 ★★ — Thompson à la main (section 9.2.6)	155
Exercice 9.8 ★★★ — Test A/B contre bandit (sections 9.2.3 et 9.2.9)	155
Exercice 9.9 ★ — L'erreur de différence temporelle (section 9.3.2)	156
Exercice 9.10 ★★ — Q-learning ou SARSA ? (sections 9.3.3 et 9.3.4)	156
Exercice 9.11 ★★ — Quels pas d'apprentissage garantissent la convergence ? (section 9.3.6)	156
Exercice 9.12 ★★★ — La malédiction de la dimension (sections 9.3.8 et 9.4.1)	156
Corrigés	156
Corrigé 9.1	156
Corrigé 9.2	156
Corrigé 9.3	157
Corrigé 9.4	157
Corrigé 9.5	157
Corrigé 9.6	157
Corrigé 9.7	158
Corrigé 9.8	158
Corrigé 9.9	158
Corrigé 9.10	159
Corrigé 9.11	159
Corrigé 9.12	159
Projet du volume et auto-évaluation — exercices et applications	160
Projet du volume	160
P.1 Le cahier des charges	160
P.2 Étape 1 : cadrer	161
P.3 Étape 2 : auditer les données	161
P.4 Étape 3 : séparer et fixer des références	162
P.5 Étape 4 : des variables qui parlent au métier	162
P.6 Étape 5 : comparer des modèles, avec leur incertitude	163
P.7 Étape 6 : régler le boosting avec un budget limité	164
P.8 Étape 7 : choisir un seuil, en fonction des coûts	165
P.9 Étape 8 : les probabilités sont-elles fiables ?	165
P.10 Étape 9 : évaluer une seule fois sur le test	166
P.11 Étape 10 : comprendre, puis vérifier l'équité	166
P.12 Étape 11 : le rapport	168
P.13 Les limites de l'étude	169
Auto-évaluation	169
La démarche (chapitre 1)	169
Les modèles supervisés (chapitre 2)	170
Le non supervisé (chapitre 3)	170
Les variables et le déséquilibre (chapitre 4)	170
Évaluation, calibration, interprétabilité (chapitre 5)	170
Les chapitres facultatifs (6 à 9)	171
Corrigés des questions	171
Votre grille d'auto-évaluation	174

Mode d'emploi

« On comprend en lisant, on retient en pratiquant. »

Ce cahier est le **compagnon du livre** du volume III (*Apprentissage automatique*). Le livre explique les idées ; le cahier les fait travailler. Il contient, chapitre par chapitre, des **applications guidées** sur données, des **exercices** et leurs **corrigés**, puis le **projet du volume** (un pipeline complet sur un jeu réel) et l'auto-évaluation.

Comment utiliser ce cahier

1. **Lisez d'abord la section du livre.** Chaque section qui a un prolongement ici se termine par une ligne de ce genre : > ■ **Pour s'entraîner.** Cahier, chapitre 2 : application 2.1, exercices 2.1 à 2.4.
2. **Essayez avant de regarder le corrigé.** Les énoncés sont regroupés dans la partie *Exercices*, les corrigés dans la partie *Corrigés* du même chapitre : on peut ainsi chercher sans voir la réponse.
3. **Tapez le code vous-même** plutôt que de le copier : modifiez-le, cassez-le, lisez les messages d'erreur. En apprentissage automatique, on apprend surtout en changeant un réglage et en regardant ce que devient le score.
4. **Faites les applications dans l'ordre** : chacune raconte une petite étude, avec une question, des étapes, du code et une lecture des résultats.
5. **Gardez le jeu de test pour la fin.** C'est la règle de tout le volume : dans les applications, quand on vous demande de régler quelque chose, on le fait avec la validation croisée sur le jeu d'entraînement, jamais en regardant le jeu de test.

Numérotation et niveaux

Chaque chapitre du cahier correspond au chapitre du livre de même numéro.

Élément	Numérotation	Exemple
Application	Application N.k	Application 2.1 : première application du chapitre 2
Exercice	Exercice N.k	Exercice 2.3 : troisième exercice du chapitre 2
Corrigé	Corrigé N.k	Corrigé 2.3 : correction de l'exercice 2.3

La difficulté des exercices est indiquée par des étoiles :

Niveau	Signification
★	application directe d'une formule ou d'une idée vue dans la section
★★	demande de combiner deux idées, ou un petit raisonnement
★★★	demande de la réflexion, une démonstration ou une petite expérience

Un exercice cite la section du livre qu'il exerce, par exemple « (section 1.2) ».

Données et environnement

Les données sont celles du livre, dans le dossier `donnees/`. Toutes celles de la boutique sont **simulées** avec des graines fixes ; vos résultats seront identiques à ceux du livre.

Fichier	Contenu
<code>clients_ml.csv</code>	12 000 clients : 19 variables d'entrée et 3 cibles (<code>churn_90j</code> , <code>depense_6m</code> , <code>segment_vrai</code>)
<code>transactions.csv</code> <code>interactions.csv</code> , <code>produits_ml.csv</code>	60 000 commandes en ligne, dont 0,81 % de fraudes achats « client × produit » et description des 150 produits
<code>credit_default.csv</code>	jeu réel : 30 000 clients d'une banque (UCI, licence CC0) ; cible <code>default</code>

Certains chapitres ajoutent leurs propres jeux (`ch0N-*.csv`) : ils sont fournis, et le script qui les produit se trouve dans `build/`. Chaque chapitre du cahier est **autonome** : il commence par recharger ses données et refaire ses imports.

Colonne	Statut	Remarque
---------	--------	----------

Dictionnaire des données de clients_ml.csv

Le statut d'une colonne décide de ce qu'on a le droit d'en faire. Le voici pour les 24 colonnes du fichier :

Colonne	Statut	Remarque
id_client	identifiant	à ne jamais utiliser comme variable d'entrée
age, ville, canal_acquisition, appareil, anciennete_mois, nb_commandes_12m, panier_moyen, montant_12m, recence_jours, nb_retours_12m, satisfaction_moy, nb_tickets_support_12m, programme_fidelite, nb_promos_recues_12m, part_achats_promo, taux_ouverture_email, delai_livraison_moy, categorie_preferee, revenu_zone	entrée (19 variables)	observées au 31/12/2025 ; certaines ont des valeurs manquantes
churn_90j	cible (classification)	1 si le client ne commande plus dans les 90 jours suivants
depense_6m	cible (régression)	dépense, en €, sur les 6 mois suivants
segment_vrai	vérité de simulation	classe latente ayant servi à générer les données ; sert uniquement à valider les méthodes non supervisées (chapitre 3) ; n'existe pas dans la vie réelle
commandes_apres_cible	⚠ à ne pas utiliser en entrée	nombre de commandes dans les 3 mois suivant la date d'observation : une information qu'on n'a pas encore au moment de prédire. Elle sert à étudier la fuite d'information (chapitre 1)

Le dictionnaire de transactions.csv, de interactions.csv et de credit_default.csv figure dans la section « Les données et l'environnement du volume » du livre ; celui de credit_default.csv est repris dans le projet.

Lancer le code

Placez-vous dans le dossier du volume (celui qui contient donnees/), car les chemins sont relatifs (donnees/clients_ml.csv). Vous pouvez travailler dans un notebook Jupyter, dans un éditeur avec la commande python, ou dans un terminal interactif.

Les chemins. Si Python répond `FileNotFoundError`, c'est presque toujours que vous n'êtes pas dans le bon dossier : vérifiez avec `import os; print(os.getcwd())`.

Le temps de calcul. Les modèles de ce volume sont plus lents que ceux du volume II (forêts, boosting, validations croisées). Les applications sont dimensionnées pour tourner en quelques dizaines de secondes sur un ordinateur ordinaire ; si l'une d'elles est trop lente chez vous, réduisez `n_estimators` ou travaillez sur un sous-échantillon des clients.

Vérifier son installation

Avant de commencer, ce court bloc vérifie que les données sont trouvées, que les bibliothèques sont installées et qu'un modèle à graine fixe donne le résultat attendu.

```
from importlib import import_module
from importlib.metadata import version
paquets = [("numpy", "numpy"), ("pandas", "pandas"), ("scipy", "scipy"), ("scikit-learn", "sklearn"),
↪ ("xgboost", "xgboost"),
↪ ("lightgbm", "lightgbm"), ("catboost", "catboost"), ("shap", "shap"), ("lime", "lime"),
↪ ("optuna", "optuna"),
↪ ("imbalanced-learn", "imblearn"), ("umap-learn", "umap")]
for paquet, module in paquets:
    import_module(module)                                # échoue si la bibliothèque n'est pas installée
    print(f"{paquet:17s}", version(paquet))
import pandas as pd
print(pd.read_csv("donnees/clients_ml.csv").shape)
```

```
numpy          2.5.3
pandas         3.0.6
scipy          1.18.1
scikit-learn   1.9.1
xgboost        3.4.1
lightgbm       4.7.0
catboost       1.2.10
shap           0.52.0
lime           0.2.0.1
optuna         5.0.0
imbalanced-learn 0.14.2
umap-learn     0.5.12
(12000, 24)
```

Vous devez lire une version pour chaque bibliothèque (le livre utilise celles de la section « L'environnement de travail »), puis (12000, 24). Si une bibliothèque manque, installez-la avec `pip` (voir le livre).

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
X, y = load_breast_cancer(return_X_y=True)
print(round(LogisticRegression(max_iter=5000).fit(X, y).score(X, y), 4))
```

```
0.9578
```

Cette seconde vérification doit afficher une exactitude de 0,9578 (à la dernière décimale près selon la version de la bibliothèque) : c'est le signe que le calcul numérique se comporte comme prévu.

Plan du cahier

Chapitre	Contenu
1. La démarche	séparation des données, validation croisée, biais-variance, modèles de référence, fuite d'information ; ⊕ réglage des hyperparamètres
2. Apprentissage supervisé	modèles linéaires et logistiques, arbres, forêts, gradient boosting ; ⊕ SVM, k plus proches voisins, Bayes naïf, stacking
3. Sans étiquettes	validation d'un regroupement, réduction de dimension ; ⊕ DBSCAN, mélanges gaussiens, t-SNE et UMAP
4. Variables et déséquilibre	encodage, transformations, création de variables, classes rares ; ⊕ sélection de variables, SMOTE
5. Évaluation et interprétabilité	métriques, calibration, SHAP et LIME ; ⊕ équité, prédiction conforme
6 à 9 (facultatifs)	anomalies et fraude, recommandation, semi-supervisé et actif, renforcement
Projet et auto-évaluation	un pipeline complet sur le jeu réel de défaut de crédit, puis des questions pour vérifier ses acquis

Chapitre 1 : La démarche d'apprentissage automatique — exercices et applications

Ce chapitre du cahier accompagne le chapitre 1 du livre (formulation, validation croisée, biais-variance, rigueur expérimentale, réglage). Les **applications** reprennent, avec tout leur code, les expériences dont le livre ne montre que les résultats ; les **exercices** font manipuler les idées à la main. Données : `donnees/clients_ml.csv` (12 000 clients simulés, voir le livre). Prérequis : Python, pandas, scikit-learn.

Préparation

Toutes les applications partent des mêmes lignes : charger les clients, exclure les colonnes qui ne sont pas des variables d'entrée, et réserver 25 % des clients pour un test final que l'on n'ouvrira qu'à l'application 1.7.

```
import numpy as np, pandas as pd, warnings
warnings.filterwarnings("ignore")
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.metrics import roc_auc_score
from sklearn.pipeline import make_pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import HistGradientBoostingClassifier

df = pd.read_csv("donnees/clients_ml.csv")
CATEG = ["ville", "canal_acquisition", "appareil", "categorie_preferee"]
EXCLUS = ["id_client", "churn_90j", "depense_6m", "segment_vrai", "commandes_apres_cible"]
NUM = [c for c in df.columns if c not in CATEG + EXCLUS]
X = df[NUM + CATEG].copy()
X[CATEG] = X[CATEG].astype("category")
y = df["churn_90j"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
print(X_tr.shape, X_te.shape, round(y_tr.mean(), 4))
```

```
(9000, 19) (3000, 19) 0.1404
```

Puis les deux modèles du chapitre : une régression logistique précédée d'un prétraitement (imputation par la médiane avec indicateur de valeur manquante, centrage-réduction, encodage des

catégories) et un gradient boosting qui gère seul les valeurs manquantes et les catégories.

```
def modele_logit(C=1.0):
    prep = ColumnTransformer([
        ("num", make_pipeline(SimpleImputer(strategy="median", add_indicator=True),
        ↪ StandardScaler()), NUM),
        ("cat", OneHotEncoder(handle_unknown="ignore"), CATEG)]
    return make_pipeline(pre, LogisticRegression(C=C, max_iter=2000))

def modele_hgb(**kw):
    return HistGradientBoostingClassifier(categorical_features="from_dtype", random_state=0, **kw)

cv5 = StratifiedKFold(5, shuffle=True, random_state=0)
for nom, m in [("régression logistique", modele_logit()), ("gradient boosting", modele_hgb())]:
    print(nom, cross_val_score(m, X_tr, y_tr, cv=cv5, scoring="roc_auc").mean().round(4))
```

```
régression logistique 0.8603
gradient boosting 0.8881
```

Applications

Application 1.1 — Détecter la fuite d'information (section 1.1.6)

Contexte. Le tableau contient une colonne `commandes_apres_cible` (commandes des trois mois suivants). Un stagiaire l'a ajoutée aux variables d'entrée et annonce un gain spectaculaire. Vous devez le quantifier, comprendre pourquoi il est faux, puis étudier un second type de fuite : le prétraitement fait avant la validation.

Étape 1 : mesurer le « gain ».

```
Xl = X_tr.copy()
Xl["commandes_apres_cible"] = df.loc[X_tr.index, "commandes_apres_cible"]
sans = cross_val_score(modele_hgb(), X_tr, y_tr, cv=5, scoring="roc_auc").mean()
avec = cross_val_score(modele_hgb(), Xl, y_tr, cv=5, scoring="roc_auc").mean()
print("sans :", round(sans, 4), "| avec :", round(avec, 4), "| écart :", round(avec - sans, 4))
```

```
sans : 0.8895 | avec : 0.9221 | écart : 0.0325
```

Étape 2 : regarder l'importance de la colonne (perte d'AUC quand on mélange ses valeurs, sur un jeu de validation).

```
from sklearn.inspection import permutation_importance
a, b, ya, yb = train_test_split(Xl, y_tr, test_size=0.3, stratify=y_tr, random_state=1)
m = modele_hgb().fit(a, ya)
pi = permutation_importance(m, b, yb, scoring="roc_auc", n_repeats=3, random_state=0)
print(pd.Series(pi.importances_mean, index=b.columns).sort_values(ascending=False).head(4).round(3))
```

```
commandes_apres_cible    0.156
age                      0.047
recence_jours            0.034
satisfaction_moy         0.024
dtype: float64
```

Étape 3 : l'audit du calendrier. On construit le tableau « variable → connue au 31 décembre ? » : c'est lui, et non les chiffres, qui tranche.

```
audit = pd.DataFrame({"variable": ["recence_jours", "nb_tickets_support_12m", "satisfaction_moy",
    ↪ "commandes_apres_cible"],
    "calculée à partir de": ["historique jusqu'au 31/12", "tickets avant le 31/12",
    ↪ "enquêtes avant le 31/12", "commandes de janvier à mars"],
```

```
"connue à t0 ?": ["oui", "oui", "oui", "NON"]})
print(audit.to_string(index=False))
```

variable	calculée à partir de	connue à t0 ?
recence_jours	historique jusqu'au 31/12	oui
nb_tickets_support_12m	tickets avant le 31/12	oui
satisfaction_moy	enquêtes avant le 31/12	oui
commandes_apres_cible	commandes de janvier à mars	NON

Étape 4 : la contamination par la sélection de variables. Sur 100 clients fictifs et 2 000 variables de bruit, comparez une sélection faite *avant* la validation croisée et une sélection faite *dans* le pipeline, pour 5 graines.

```
from sklearn.feature_selection import SelectKBest, f_classif
avant, dans = [], []
for graine in range(5):
    rng = np.random.default_rng(graine)
    Xn, yn = rng.normal(size=(100, 2000)), rng.integers(0, 2, 100)
    cv = StratifiedKFold(5, shuffle=True, random_state=graine)
    Xs = SelectKBest(f_classif, k=10).fit_transform(Xn, yn)
    avant.append(cross_val_score(LogisticRegression(max_iter=1000), Xs, yn, cv=cv,
    ↳ scoring="roc_auc").mean())
    pipe = make_pipeline(SelectKBest(f_classif, k=10), LogisticRegression(max_iter=1000))
    dans.append(cross_val_score(pipe, Xn, yn, cv=cv, scoring="roc_auc").mean())
print("avant :", np.round(avant, 3), "| dans le pipeline :", np.round(dans, 3))
```

```
avant : [0.879 0.857 0.895 0.892 0.877] | dans le pipeline : [0.602 0.435 0.599 0.543 0.617]
```

À vous. (a) Pourquoi l'AUC « avec » n'atteint-elle pas 1,0, alors que la colonne est tirée de l'avenir ? (b) Quel est le rôle d'un Pipeline dans la seconde expérience ? (c) Donnez un exemple de fuite temporelle dans un projet de prévision de ventes.

Application 1.2 — La loterie du découpage unique (section 1.2.1)

Objectif. Mesurer la variabilité d'une estimation par découpage unique, et voir comment elle diminue avec la taille du jeu de validation.

```
def auc_decoupage(n, graine):
    sub = X_tr.sample(n, random_state=0)
    ys = y_tr.loc[sub.index]
    a, b, ya, yb = train_test_split(sub, ys, test_size=0.2, stratify=ys, random_state=graine)
    return roc_auc_score(yb, modele_logit().fit(a, ya).predict_proba(b)[: , 1])

for n in (500, 2000, 6000):
    aucs = np.array([auc_decoupage(n, s) for s in range(100)])
    print(f"n = {n:5d} : AUC moyenne {aucs.mean():.3f} | écart-type {aucs.std():.3f} | min
    ↳ {aucs.min():.3f} | max {aucs.max():.3f}")
```

```
n = 500 : AUC moyenne 0.872 | écart-type 0.046 | min 0.748 | max 0.948
n = 2000 : AUC moyenne 0.862 | écart-type 0.025 | min 0.790 | max 0.914
n = 6000 : AUC moyenne 0.864 | écart-type 0.013 | min 0.830 | max 0.890
```

À vous. Le nombre de clients partis dans le jeu de validation vaut environ $0,14 \times 0,2 n$. Vérifiez que l'écart-type de l'AUC décroît à peu près comme l'inverse de la racine de ce nombre. Que conclure sur la taille minimale d'un jeu de validation ?

Application 1.3 — Combien de plis ? (section 1.2.2)

Objectif. Comparer plusieurs k en comparant chaque estimation par validation croisée à la vraie performance du modèle (mesurée sur des clients laissés de côté).

```
resultats = {k: [] for k in (2, 5, 10, 20)}
vrai = []
for r in range(10):
    sb = X_tr.sample(1000, random_state=100 + r)
    ys = y_tr.loc[sb.index]
    hors = X_tr.drop(sb.index)
    vrai.append(roc_auc_score(y_tr.loc[hors.index], modele_logit().fit(sb, ys).predict_proba(hors)[: ,
↪ 1]))
    for k in resultats:
        cv = StratifiedKFold(k, shuffle=True, random_state=r)
        resultats[k].append(cross_val_score(modele_logit(), sb, ys, cv=cv, scoring="roc_auc").mean())
print("vraie AUC :", round(np.mean(vrai), 4))
for k, v in resultats.items():
    print(f"k = {k:2d} : estimation {np.mean(v):.4f} | biais {np.mean(v) - np.mean(vrai):+.4f} |
↪ écart-type {np.std(v):.4f}")
```

```
vraie AUC : 0.8441
k = 2 : estimation 0.8335 | biais -0.0106 | écart-type 0.0212
k = 5 : estimation 0.8454 | biais +0.0013 | écart-type 0.0139
k = 10 : estimation 0.8455 | biais +0.0015 | écart-type 0.0145
k = 20 : estimation 0.8471 | biais +0.0030 | écart-type 0.0133
```

À vous. Ajoutez la validation « un seul laissé de côté » sur 100 clients (LeaveOneOut) : pourquoi la mesure d'AUC par pli y est-elle impossible, et comment la contourner (indice : `cross_val_predict`) ?

Application 1.4 — Valider une série temporelle sans tricher (section 1.2.3)

Objectif. Montrer qu'une validation croisée aléatoire est trompeuse sur une série temporelle. On simule une série mensuelle (tendance, saisonnalité, bruit corrélé), on construit des variables retardées, et on compare trois estimations de l'erreur.

```
from sklearn.model_selection import KFold, TimeSeriesSplit
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(7)
T = 144
t = np.arange(T)
bruit = np.zeros(T)
for i in range(1, T):
    bruit[i] = 0.8 * bruit[i - 1] + rng.normal(0, 2)
d = pd.DataFrame({"y": 100 + 0.5 * t + 8 * np.sin(2 * np.pi * t / 12) + bruit})
for lag in (1, 2, 3, 12):
    d[f"l{lag}"] = d.y.shift(lag)
d["mois"] = t % 12
d = d.dropna().reset_index(drop=True)
Xs, ys = d.drop(columns="y"), d.y

def modele():
    return RandomForestRegressor(100, min_samples_leaf=2, random_state=0, n_jobs=1)

def rmse(u, v):
    return np.sqrt(mean_squared_error(u, v))

alea = -cross_val_score(modele(), Xs, ys, cv=KFold(5, shuffle=True, random_state=0),
↪ scoring="neg_root_mean_squared_error").mean()
```

```

temps = -cross_val_score(modele(), Xs, ys, cv=TimeSeriesSplit(5),
    ↪ scoring="neg_root_mean_squared_error").mean()
n = len(d)
futur = rmse(ys[n - 24:], modele().fit(Xs[:n - 24], ys[:n - 24]).predict(Xs[n - 24:]))
print("RMSE : CV aléatoire", round(alea, 2), "| CV temporelle", round(temps, 2), "| 24 derniers mois",
    ↪ round(futur, 2))

```

RMSE : CV aléatoire 3.51 | CV temporelle 7.31 | 24 derniers mois 6.75

À vous. (a) Ajoutez `gap=12` à `TimeSeriesSplit` : que change cet espace entre entraînement et validation, et dans quelle situation est-il utile ? (b) Refaites l'expérience sans la variable `l1` : l'écart entre validation aléatoire et temporelle diminue-t-il ? Pourquoi ?

Application 1.5 — Biais et variance par simulation (section 1.3.4)

Objectif. Estimer, pour des polynômes de degré croissant, le biais carré et la variance en simulant de nombreux jeux d'entraînement, et vérifier la décomposition erreur = $\sigma^2 + \text{biais}^2 + \text{variance}$.

```

from numpy.polynomial import Polynomial

rng = np.random.default_rng(1)
f = lambda x: np.sin(1.5 * np.pi * x)
sigma, N, R = 0.3, 30, 500
grille = np.linspace(0, 1, 200)
lignes = []
for degre in range(1, 8):
    P = np.empty((R, len(grille)))
    for r in range(R):
        x = rng.uniform(0, 1, N)
        P[r] = Polynomial.fit(x, f(x) + rng.normal(0, sigma, N), degre)(grille)
    lignes.append((degre, ((P.mean(0) - f(grille)) ** 2).mean(), P.var(0).mean()))
tab = pd.DataFrame(lignes, columns=["degré", "biais²", "variance"])
tab["erreur attendue"] = tab["biais²"] + tab["variance"] + sigma ** 2
print(tab.round(4).to_string(index=False))

```

degré	biais²	variance	erreur attendue
1	0.1835	0.0241	0.2976
2	0.0330	0.0166	0.1396
3	0.0033	0.0175	0.1108
4	0.0003	0.0252	0.1155
5	0.0001	0.0758	0.1659
6	0.0003	0.1434	0.2337
7	0.0012	0.4150	0.5062

Vérification directe de la décomposition. On mesure l'erreur quadratique sur de **nouvelles** observations (avec leur bruit) pour le degré 3, et on la compare à la somme des trois termes.

```

errs = []
for r in range(R):
    x = rng.uniform(0, 1, N)
    p = Polynomial.fit(x, f(x) + rng.normal(0, sigma, N), 3)
    xn = rng.uniform(0, 1, 200)
    errs.append(((f(xn) + rng.normal(0, sigma, 200) - p(xn)) ** 2).mean())
print("erreur mesurée (degré 3) :", round(np.mean(errs), 4), "| somme biais² + variance + bruit :",
    ↪ tab.loc[2, "erreur attendue"].round(4))

```

erreur mesurée (degré 3) : 0.1086 | somme biais² + variance + bruit : 0.1108

À vous. Doublez la taille de l'échantillon ($N = 60$) : que deviennent le biais et la variance, et le degré optimal ? Redémontrez la décomposition sur une feuille, sans regarder le livre.

Application 1.6 — Courbes d'apprentissage et diagnostic (sections 1.3.5 et 1.3.6)

Objectif. Dessiner les courbes d'apprentissage de trois modèles et poser un diagnostic pour chacun.

```
from sklearn.model_selection import learning_curve
from sklearn.tree import DecisionTreeClassifier

tailles = [200, 500, 1000, 2000, 4000, 6000]
cv3 = StratifiedKFold(3, shuffle=True, random_state=0)
arbre = make_pipeline(SimpleImputer(strategy="median"), DecisionTreeClassifier(random_state=0))
for nom, mod, Xm in [("régression logistique", modele_logit(), X_tr), ("gradient boosting",
↪ modele_hgb(), X_tr), ("arbre profond", arbre, X_tr[NUM])]:
    _, train, val = learning_curve(mod, Xm, y_tr, train_sizes=tailles, cv=cv3, scoring="roc_auc")[:3]
    print(f"{nom:22s} entraînement {train.mean(1).round(3)} | validation {val.mean(1).round(3)}")
```

```
régression logistique entraînement [0.974 0.91 0.892 0.883 0.872 0.868] | validation [0.826 0.833
↪ 0.845 0.851 0.857 0.86 ]
gradient boosting entraînement [1. 1. 1. 1. 1. 0.999] | validation [0.827 0.847
↪ 0.863 0.869 0.877 0.886]
arbre profond entraînement [1. 1. 1. 1. 1. 1.] | validation [0.657 0.651 0.675 0.686 0.677
↪ 0.682]
```

À vous. Pour chaque modèle, répondez : biais ou variance ? Collecter deux fois plus de clients aiderait-il ? Quelle action concrète proposez-vous ? Puis tracez la courbe de validation de l'arbre selon `max_depth` (de 1 à 20) avec `validation_curve`, et retrouvez la profondeur optimale.

Application 1.7 — Comparer deux modèles avec rigueur (section 1.4)

Objectif. Comparer la régression logistique et le gradient boosting avec un test t corrigé, un test de McNemar et un bootstrap, puis ouvrir le jeu de test **une seule fois**.

```
from scipy import stats
from sklearn.model_selection import ShuffleSplit

J = 15
al, ah = [], []
for a, b in ShuffleSplit(J, test_size=0.25, random_state=0).split(X_tr, y_tr):
    al.append(roc_auc_score(y_tr.iloc[b], modele_logit().fit(X_tr.iloc[a],
↪ y_tr.iloc[a]).predict_proba(X_tr.iloc[b])[:, 1]))
    ah.append(roc_auc_score(y_tr.iloc[b], modele_hgb().fit(X_tr.iloc[a],
↪ y_tr.iloc[a]).predict_proba(X_tr.iloc[b])[:, 1]))
d = np.array(ah) - np.array(al)
n1, n2 = len(a), len(b)
t_naif = d.mean() / np.sqrt(d.var(ddof=1) / J)
t_corr = d.mean() / np.sqrt((1 / J + n2 / n1) * d.var(ddof=1))
print("écart moyen", d.mean().round(4), "| t naïf", t_naif.round(2), "| t corrigé", t_corr.round(2),
↪ "| p corrigée", round(2 * stats.t.sf(abs(t_corr), J - 1), 6))
```

```
écart moyen 0.0296 | t naïf 17.71 | t corrigé 7.23 | p corrigée 4e-06
```

```
a, b, ya, yb = train_test_split(X_tr, y_tr, test_size=0.3, stratify=y_tr, random_state=2)
pl = modele_logit().fit(a, ya).predict_proba(b)[:, 1]
ph = modele_hgb().fit(a, ya).predict_proba(b)[:, 1]
juste_l, juste_h = (pl > 0.25) == yb.values, (ph > 0.25) == yb.values
bb, cc = int((juste_l & ~juste_h).sum()), int((~juste_l & juste_h).sum())
chi2 = (abs(bb - cc) - 1) ** 2 / (bb + cc)
print("McNemar : b =", bb, "c =", cc, "chi2 =", round(chi2, 2), "p =", stats.chi2.sf(chi2, 1))
rng = np.random.default_rng(0)
ecarts = []
for _ in range(1000):
    i = rng.choice(len(yb), len(yb))
```

```

ecarts.append(roc_auc_score(yb.values[i], ph[i]) - roc_auc_score(yb.values[i], pl[i]))
print("bootstrap : écart d'AUC", round(roc_auc_score(yb, ph) - roc_auc_score(yb, pl), 4), "| IC95",
      ↪ np.percentile(ecarts, [2.5, 97.5]).round(4))

```

```

McNemar : b = 120 c = 211 chi2 = 24.47 p = 7.54250626803909e-07
bootstrap : écart d'AUC 0.0235 | IC95 [0.0112 0.0364]

```

Le jeu de test, une seule fois.

```

ml, mh = modele_logit().fit(X_tr, y_tr), modele_hgb().fit(X_tr, y_tr)
p_l, p_h = ml.predict_proba(X_te)[: , 1], mh.predict_proba(X_te)[: , 1]
rng = np.random.default_rng(1)
bl, bh = [], []
for _ in range(1000):
    i = rng.choice(len(y_te), len(y_te))
    bl.append(roc_auc_score(y_te.values[i], p_l[i]))
    bh.append(roc_auc_score(y_te.values[i], p_h[i]))
for nom, v, bs in [("régression logistique", roc_auc_score(y_te, p_l), bl), ("gradient boosting",
    ↪ roc_auc_score(y_te, p_h), bh)]:
    print(f"{nom} : AUC test {v:.4f}, IC95 {np.percentile(bs, [2.5, 97.5]).round(4)}")

```

```

régression logistique : AUC test 0.8656, IC95 [0.8476 0.8829]
gradient boosting : AUC test 0.8996, IC95 [0.8829 0.9137]

```

À vous. Quelle question chacun des trois outils (test t corrigé, McNemar, bootstrap) pose-t-il réellement ? Pourquoi le test t naïf conclut-il avec une statistique 2,5 fois plus grande ? Si le résultat du test vous déplaisait, que feriez-vous *sans* le consommer ?

Application 1.8 — Grille, recherche aléatoire et Optuna (section 1.5)

Objectif. Comparer trois stratégies de réglage d'un gradient boosting, avec un budget de 20 essais, sur un sous-échantillon de 3 000 clients.

```

import optuna
from scipy.stats import loguniform, randint
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
optuna.logging.set_verbosity(optuna.logging.WARNING)

X3 = X_tr.sample(3000, random_state=0)
y3 = y_tr.loc[X3.index]
cv = StratifiedKFold(3, shuffle=True, random_state=0)
grille = {"learning_rate": [0.03, 0.1, 0.3], "max_leaf_nodes": [8, 31, 63]}
gs = GridSearchCV(modele_hgb(), grille, cv=cv, scoring="roc_auc").fit(X3, y3)
print("grille (9 essais) :", gs.best_params_, round(gs.best_score_, 4))

```

```

grille (9 essais) : {'learning_rate': 0.03, 'max_leaf_nodes': 8} 0.894

```

```

dist = {"learning_rate": loguniform(0.01, 0.5), "max_leaf_nodes": randint(4, 128),
        "l2_regularization": loguniform(1e-3, 10), "min_samples_leaf": randint(5, 100)}
rs = RandomizedSearchCV(modele_hgb(), dist, n_iter=20, cv=cv, scoring="roc_auc",
    ↪ random_state=0).fit(X3, y3)
print("aléatoire (20 essais) :", round(rs.best_score_, 4))

```

```

def objectif(essai):
    p = {"learning_rate": essai.suggest_float("learning_rate", 0.01, 0.5, log=True),
         "max_leaf_nodes": essai.suggest_int("max_leaf_nodes", 4, 128),
         "l2_regularization": essai.suggest_float("l2_regularization", 1e-3, 10, log=True),
         "min_samples_leaf": essai.suggest_int("min_samples_leaf", 5, 100)}
    return cross_val_score(modele_hgb(**p), X3, y3, cv=cv, scoring="roc_auc").mean()

```

```

etude = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=0))

```

```
etude.optimize(objectif, n_trials=20)
print("TPE (20 essais) :", round(etude.best_value, 4))
```

```
aléatoire (20 essais) : 0.8932
TPE (20 essais) : 0.8928
```

L'arrêt précoce.

```
m = modele_hgb(early_stopping=True, validation_fraction=0.15, n_iter_no_change=10,
↳ max_iter=500).fit(X_tr, y_tr)
print("arrêt après", m.n_iter_, "itérations ; AUC en validation croisée des paramètres retenus :",
      round(cross_val_score(modele_hgb(max_iter=m.n_iter_), X_tr, y_tr, cv=cv5,
↳ scoring="roc_auc").mean(), 4))
```

```
arrêt après 55 itérations ; AUC en validation croisée des paramètres retenus : 0.8892
```

À vous. Répétez la recherche aléatoire avec 10 graines : la dispersion des meilleurs scores est-elle plus grande que l'écart entre les trois méthodes ? Que conclure sur « la meilleure méthode » ?

Application 1.9 — L'optimisme du réglage et la validation imbriquée (section 1.5.5)

Objectif. Mesurer comment l'optimisme d'un réglage croît avec le nombre d'essais, sur des étiquettes aléatoires (aucun signal), et vérifier que la validation imbriquée le corrige.

```
from sklearn.tree import DecisionTreeClassifier

rng = np.random.default_rng(0)
Xb = X_tr[NUM].fillna(X_tr[NUM].median()).sample(1150, random_state=1).values
yb_ = rng.integers(0, 2, 1150)
Xa, Xh, ya_, yh = Xb[:150], Xb[150:], yb_[:150], yb_[150:]
dist_arbre = {"max_depth": randint(1, 12), "min_samples_leaf": randint(1, 20), "max_features":
↳ randint(1, len(NUM))}
for n_essais in (1, 10, 100, 300):
    r = RandomizedSearchCV(DecisionTreeClassifier(random_state=0), dist_arbre, n_iter=n_essais, cv=5,
↳ scoring="roc_auc", random_state=0).fit(Xa, ya_)
    neuf = roc_auc_score(yh, r.predict_proba(Xh)[: , 1])
    print(f"{n_essais:4d} essais : meilleur AUC en validation croisée {r.best_score_:.3f} | AUC sur
↳ 1000 clients neufs {neuf:.3f}")
```

```
1 essais : meilleur AUC en validation croisée 0.503 | AUC sur 1000 clients neufs 0.514
10 essais : meilleur AUC en validation croisée 0.623 | AUC sur 1000 clients neufs 0.493
100 essais : meilleur AUC en validation croisée 0.651 | AUC sur 1000 clients neufs 0.533
300 essais : meilleur AUC en validation croisée 0.661 | AUC sur 1000 clients neufs 0.533
```

```
rech = RandomizedSearchCV(DecisionTreeClassifier(random_state=0), dist_arbre, n_iter=30, cv=3,
↳ scoring="roc_auc", random_state=0)
imbrique = cross_val_score(rech, Xa, ya_, cv=StratifiedKFold(5, shuffle=True, random_state=0),
↳ scoring="roc_auc")
print("validation imbriquée :", imbrique.mean().round(3))
```

```
validation imbriquée : 0.483
```

À vous. Tracez l'optimisme (meilleur score de validation moins score sur données neuves) en fonction du nombre d'essais. À quelle vitesse croît-il ? Quel lien avec l'espérance du maximum de n variables aléatoires ?

Exercices

Exercice 1.1 ★ — Risque empirique à la main (section 1.1.1)

Cinq clients ont pour réponse $y = (1, 0, 0, 1, 0)$ et pour probabilités prédites $\hat{p} = (0,8; 0,3; 0,6; 0,4; 0,1)$. Calculez l'exactitude (seuil 0,5), la perte logarithmique, l'erreur de Brier (moyenne de $(\hat{p} - y)^2$) et l'AUC.

Exercice 1.2 ★ — Le piège de l'exactitude (section 1.1.3)

Sur un jeu de transactions où 2 % sont frauduleuses, le modèle A répond toujours « non frauduleuse ». Le modèle B détecte 60 % des fraudes mais déclenche une fausse alerte sur 3 % des transactions légitimes. Calculez l'exactitude de chacun. Lequel est utile ? Quelle métrique choisiriez-vous ?

Exercice 1.3 ★★ — Audit du calendrier (section 1.1.6)

On prédit, au 31 décembre, si un client partira dans les 90 jours. Pour chacune des variables suivantes, dites si l'on peut l'utiliser, et justifiez : (a) le nombre de commandes de l'année écoulée ; (b) le montant du dernier remboursement demandé, enregistré en février ; (c) la satisfaction déclarée dans l'enquête de novembre ; (d) l'indicateur « compte clôturé » (valeur au 31 mars) ; (e) la moyenne, calculée sur tous les clients (y compris ceux du futur jeu de test), des paniers de chaque ville ; (f) le numéro du client (croissant avec la date d'inscription).

Exercice 1.4 ★ — Un seul laissé de côté, à la main (section 1.2.2)

Trois montants : $y = (1, 4, 10)$. Le « modèle » est la moyenne des observations d'entraînement. Calculez l'erreur quadratique d'entraînement, l'erreur LOO (un seul laissé de côté), et vérifiez le rapport $\left(\frac{n}{n-1}\right)^2$.

Exercice 1.5 ★★ — Construire des plis (section 1.2.3)

- (a) On a 23 observations et on fait une validation croisée à 5 plis (sans mélange) : donnez la taille de chaque pli, le nombre de modèles entraînés et la taille de l'entraînement à chaque essai. (b) Sur 50 clients dont 7 sont partis, à 5 plis **stratifiés** : combien de partis dans chaque pli ?

Exercice 1.6 ★★ — Découpage temporel (section 1.2.3)

On a 12 mois numérotés de 0 à 11 et on utilise `TimeSeriesSplit(n_splits=3)`. Écrivez à la main les indices d'entraînement et de validation de chaque essai, puis vérifiez avec le code. Pourquoi ce découpage est-il préférable à un tirage aléatoire de plis pour prévoir les ventes ?

Exercice 1.7 ★★ — Le meilleur estimateur biaisé (section 1.3.2)

On estime une moyenne $\mu = 3$ avec $n = 9$ observations de variance $\sigma^2 = 9$. Montrez que l'estimateur $c\bar{y}$ a pour erreur quadratique $(c-1)^2\mu^2 + c^2\sigma^2/n$, trouvez c^* , calculez l'erreur minimale, et comparez avec celle de $\bar{y}$.

Exercice 1.8 ★★★ — Lire des courbes d'apprentissage (section 1.3.6)

Deux modèles, A et B, ont été évalués avec des quantités croissantes de données. A : entraînement 0,99 / 0,97 / 0,94 / 0,92 ; validation 0,70 / 0,76 / 0,80 / 0,82 (pour 100, 300, 1 000 et 3 000 clients). B : entraînement 0,76 / 0,75 / 0,74 / 0,74 ; validation 0,70 / 0,72 / 0,73 / 0,73. Posez un diagnostic pour chacun (biais ou variance), dites si collecter des données aiderait, et proposez deux actions pour chaque modèle.

Exercice 1.9 ★★ — Le test t corrigé (section 1.4.2)

Sur $J = 10$ découpages (800 observations d'entraînement et 200 de validation), les écarts d'exactitude entre deux modèles valent, en points de pourcentage : 2; 4; 3; 5; 1; 3; 4; 2; 3; 3. Calculez la statistique t naïve, la statistique corrigée, et l'intervalle de confiance à 95 % corrigé (on donne $t_{0,975;9} = 2,262$).

Exercice 1.10 ★★ — McNemar (section 1.4.2)

Deux classifieurs sont comparés sur 500 clients. Ils divergent sur 35 d'entre eux : A a raison et B tort pour 25 clients, A tort et B raison pour 10. Calculez la statistique de McNemar avec correction de continuité et la probabilité critique. La différence est-elle significative à 5 % ? à 1 % ?

Exercice 1.11 ★★ — Combien d'essais aléatoires ? (section 1.5.2)

Une zone de réglages « excellents » occupe 2 % de l'espace de recherche. (a) Quelle est la probabilité d'en toucher au moins un avec 50 tirages aléatoires ? (b) Combien de tirages pour 90 % ? (c) Une grille à 5 valeurs pour 4 hyperparamètres compte combien de points ? Si seuls 2 hyperparamètres comptent, combien de valeurs distinctes chacun reçoit-il dans la grille ? et dans 50 tirages aléatoires ?

Exercice 1.12 ★★★ — L'optimisme du maximum (section 1.5.5)

On évalue 100 modèles **sans aucune compétence** (scores aléatoires) sur un jeu de validation de 200 clients dont 28 sont partis. (a) Estimez par simulation l'AUC espérée du meilleur des 100. (b) Que vaut alors l'AUC « annoncée » du modèle sélectionné, et celle qu'il aura sur de nouvelles données ? (c) Reliez ce résultat à la validation imbriquée.

Corrigés

Corrigé 1.1

Exactitude : prédictions à 0,5 : (1,0,1,0,0) contre $y = (1,0,0,1,0)$: justes pour les clients 1, 2 et 5, donc $3/5 = 0,6$. *Perte logarithmique* : les termes $-\ln(\text{probabilité attribuée au réel})$ valent 0,223 ; 0,357 ; 0,916 ; 0,916 ; 0,105, de somme 2,518, soit $2,518/5 = 0,504$. *Brier* : $(0,04 + 0,09 + 0,36 + 0,36 + 0,01)/5 = 0,172$. *AUC* : positifs (0,8 ; 0,4), négatifs (0,3 ; 0,6 ; 0,1) : 6 paires. 0,8 bat les trois négatifs (3) ; 0,4 bat 0,3 et 0,1 mais pas 0,6 (2) : $5/6 \approx 0,833$.

```
from sklearn.metrics import log_loss, brier_score_loss
yy, pp = np.array([1, 0, 0, 1, 0]), np.array([0.8, 0.3, 0.6, 0.4, 0.1])
print((pp > 0.5).astype(int).tolist(), round(((pp > 0.5) == yy).mean(), 3), round(log_loss(yy, pp),
↪ 4), round(brier_score_loss(yy, pp), 4), round(roc_auc_score(yy, pp), 4))
```

```
[1, 0, 1, 0, 0] 0.6 0.5036 0.172 0.8333
```

Corrigé 1.2

Modèle A : exactitude = 98 % (il ne se trompe que sur les fraudes), mais il ne détecte **aucune** fraude : il est inutile. Modèle B : exactitude = $0,02 \times 0,60 + 0,98 \times 0,97 = 0,012 + 0,9506 = 96,3\%$, **inférieure** à celle de A, alors qu'il détecte 60 % des fraudes. L'exactitude classe mal ces deux modèles. On choisit plutôt le **rappel** (part des fraudes détectées), la **précision** (part des alertes justifiées) et surtout le **coût** : coût d'une fraude manquée contre coût d'une fausse alerte. La précision de B vaut $\frac{0,012}{0,012+0,0294} \approx 29\%$: environ trois alertes sur dix sont de vraies fraudes.

Corrigé 1.3

- (a) **Oui** : calculée sur le passé. (b) **Non** : enregistré en février, donc après t_0 (fuite par la cible : un remboursement tardif est une conséquence du comportement futur). (c) **Oui** : l'enquête de novembre précède t_0 . (d) **Non** : c'est la cible déguisée (compte clôturé au 31 mars). (e) **À éviter telle quelle** : une moyenne calculée sur *tous* les clients, y compris ceux du futur jeu de test, fait entrer de l'information du test dans l'apprentissage (contamination) ; elle doit être calculée **sur le jeu d'entraînement seulement**, à l'intérieur du pipeline. (f) **Prudence** : un numéro croissant encode la date d'inscription (ancienneté), ce qui peut être légitime, mais il n'a aucun sens causal et se dégrade si la numérotation change ; on préfère l'ancienneté explicite.

Corrigé 1.4

Moyenne $\bar{y} = 5$; erreur d'entraînement $\frac{(1-5)^2 + (4-5)^2 + (10-5)^2}{3} = \frac{16+1+25}{3} = 14$. LOO : on retire 1, on prédit $\frac{4+10}{2} = 7$, erreur -6 ; on retire 4, on prédit $\frac{1+10}{2} = 5,5$, erreur $-1,5$; on retire 10, on prédit $\frac{1+4}{2} = 2,5$, erreur $7,5$. Erreur LOO : $\frac{36+2,25+56,25}{3} = 31,5$. Rapport : $31,5/14 = 2,25 = (3/2)^2$ ✓.

```
yv = np.array([1., 4., 10.])
loo = np.array([yv[i] - np.delete(yv, i).mean() for i in range(3)])
print(loo, (loo ** 2).mean(), ((yv - yv.mean()) ** 2).mean(), (loo ** 2).mean() / ((yv - yv.mean()) **
↳ 2).mean())
```

```
[-6. -1.5  7.5] 31.5 14.0 2.25
```

Corrigé 1.5

- (a) $23 = 5 \times 4 + 3$: trois plis de 5 observations et deux de 4 (ou l'inverse, selon l'ordre) : tailles (5,5,5,4,4). **5 modèles** sont entraînés ; à chaque essai l'entraînement compte $23 - 5 = 18$ ou $23 - 4 = 19$ observations. (b) $50/5 = 10$ clients par pli ; les 7 partis se répartissent aussi également que possible : deux plis en contiennent 2 et trois plis en contiennent 1.

```
from sklearn.model_selection import KFold
print([len(v) for _, v in KFold(5).split(np.arange(23))])
yy50 = np.array([1] * 7 + [0] * 43)
print([int(yy50[v].sum()) for _, v in StratifiedKFold(5, shuffle=True,
↳ random_state=0).split(np.zeros(50), yy50)])
```

```
[5, 5, 5, 4, 4]
[2, 2, 1, 1, 1]
```

Corrigé 1.6

La taille de validation est $12 / (3+1) = 3$. Essai 1 : entraînement $\{0, 1, 2\}$, validation $\{3, 4, 5\}$; essai 2 : entraînement $\{0, \dots, 5\}$, validation $\{6, 7, 8\}$; essai 3 : entraînement $\{0, \dots, 8\}$, validation $\{9, 10, 11\}$. Chaque validation est **postérieure** à l'entraînement, comme en production ; un tirage aléatoire de plis entraînerait sur des mois situés après ceux à prévoir (fuite temporelle, application 1.4).

```
from sklearn.model_selection import TimeSeriesSplit
for tr, va in TimeSeriesSplit(3).split(np.arange(12)):
    print(tr.tolist(), va.tolist())
```

```
[0, 1, 2] [3, 4, 5]
[0, 1, 2, 3, 4, 5] [6, 7, 8]
[0, 1, 2, 3, 4, 5, 6, 7, 8] [9, 10, 11]
```

Corrigé 1.7

$\mathbb{E}[c\bar{y}] = c\mu$, donc biais = $(c - 1)\mu$; variance $c^2\sigma^2/n$; l'erreur quadratique est la somme du biais carré et de la variance : $(c - 1)^2\mu^2 + c^2\sigma^2/n$. Sa dérivée en c vaut $2(c - 1)\mu^2 + 2c\sigma^2/n$, qui s'annule pour $c^* = \frac{\mu^2}{\mu^2 + \sigma^2/n}$. Ici $\mu^2 = 9$ et $\sigma^2/n = 1$: $c^* = 0,9$. L'erreur minimale vaut $(0,1)^2 \times 9 + 0,81 \times 1 = 0,09 + 0,81 = 0,90$, contre 1 pour $\bar{y}$: un gain de 10 %.

```
c = np.linspace(0.5, 1.2, 8); mu, s2, n = 3.0, 9.0, 9
print(np.round((c - 1) ** 2 * mu ** 2 + c ** 2 * s2 / n, 3), "| optimum", mu ** 2 / (mu ** 2 + s2 /
↪ n), "| erreur minimale", (0.9 - 1) ** 2 * 9 + 0.81)
```

```
[2.5 1.8 1.3 1. 0.9 1. 1.3 1.8] | optimum 0.9 | erreur minimale 0.9
```

Corrigé 1.8

A : l'entraînement reste très supérieur à la validation (0,92 contre 0,82 à 3 000 clients), mais l'écart se **réduit** et la validation **monte encore** (0,70 → 0,82) : c'est la **variance**. Collecter des données aiderait. Actions : régulariser ou simplifier le modèle (profondeur, pénalité), moyenner plusieurs modèles (forêts). **B** : les deux courbes **se rejoignent** à un niveau bas (0,74 et 0,73) et plafonnent : c'est le **biais**. Plus de données n'aidera presque pas. Actions : un modèle plus souple (plus de profondeur, interactions), de meilleures variables (chapitre 4).

Corrigé 1.9

Moyenne $\bar{d} = 3$; écarts à la moyenne : $-1; 1; 0; 2; -2; 0; 1; -1; 0; 0$, carrés de somme 12, donc $s_d^2 = 12/9 = 1,333$. Naïf : $t = \frac{3}{\sqrt{1,333/10}} = \frac{3}{0,365} = 8,2$. Facteur de correction $\frac{1}{10} + \frac{200}{800} = 0,35$: $t = \frac{3}{\sqrt{0,35 \times 1,333}} = \frac{3}{0,683} = 4,4$. Erreur-type corrigée 0,683 ; intervalle : $3 \pm 2,262 \times 0,683 = 3 \pm 1,55$, soit $[1,5 ; 4,5]$ points : la différence est significative, mais deux fois moins nettement que ne le laissait croire le test naïf.

```
d = np.array([2, 4, 3, 5, 1, 3, 4, 2, 3, 3], float)
tn = d.mean() / np.sqrt(d.var(ddof=1) / 10); tc = d.mean() / np.sqrt(0.35 * d.var(ddof=1)); se =
↪ np.sqrt(0.35 * d.var(ddof=1))
print(round(tn, 2), round(tc, 2), round(d.mean() - 2.262 * se, 2), round(d.mean() + 2.262 * se, 2))
```

```
8.22 4.39 1.45 4.55
```

Corrigé 1.10

$\chi^2 = \frac{(|25 - 10| - 1)^2}{35} = \frac{14^2}{35} = \frac{196}{35} = 5,6$ (1 degré de liberté), soit une probabilité critique de 0,018 : significatif à 5 %, **non significatif à 1 %**. (Le test exact, binomial de paramètre 1/2 sur 35 désaccords, donne une valeur voisine.)

```
from scipy import stats
print(round((abs(25 - 10) - 1) ** 2 / 35, 2), round(stats.chi2.sf(5.6, 1), 4),
      ↪ round(stats.binomtest(10, 35, 0.5).pvalue, 4))
```

5.6 0.018 0.0167

Corrigé 1.11

(a) $1 - 0,98^{50} = 1 - 0,364 = 0,636$. (b) $n \geq \frac{\ln 0,1}{\ln 0,98} \approx 114$ tirages. (c) $5^4 = 625$ points de grille. Si seuls 2 hyperparamètres comptent, chacun ne reçoit que **5 valeurs distinctes** dans la grille (les 625 points ne font que 25 combinaisons utiles, répétées 25 fois), alors que 50 tirages aléatoires lui en donnent **50 valeurs distinctes** : c'est l'argument de Bergstra et Bengio.

```
print(round(1 - 0.98 ** 50, 3), int(np.ceil(np.log(0.1) / np.log(0.98))), 5 ** 4)
```

0.636 114 625

Corrigé 1.12

On simule 100 scorers aléatoires sur 200 clients (28 positifs) et on retient le meilleur ; on répète 300 fois.

```
rng = np.random.default_rng(0)
yv = np.array([1] * 28 + [0] * 172)
maxima = [max(roc_auc_score(yv, rng.random(200)) for _ in range(100)) for _ in range(300)]
print("AUC du meilleur des 100 : moyenne", np.mean(maxima).round(3), "| 5e-95e centiles",
      ↪ np.percentile(maxima, [5, 95]).round(3))
```

AUC du meilleur des 100 : moyenne 0.647 | 5e-95e centiles [0.614 0.688]

(a) Le meilleur de 100 modèles sans compétence obtient en moyenne une AUC voisine de **0,65** (de 0,61 à 0,69 pour 90 % des répétitions), alors que chacun vaut 0,5 en espérance : l'écart-type d'une AUC mesurée sur 28 positifs et 172 négatifs est $\sqrt{201/(12 \times 28 \times 172)} \approx 0,06$, et le maximum de 100 variables presque normales dépasse leur moyenne d'environ 2,5 écarts-types, soit $0,5 + 2,5 \times 0,06 \approx 0,65$. (b) L'AUC **annoncée** par la validation est donc environ 0,65 ; sur de nouvelles données, le modèle sélectionné retombe à **0,5**. (c) La validation imbriquée évalue la *procédure* « essayer 100 modèles, retenir le meilleur » sur des clients que la sélection n'a pas vus ; elle donne 0,5, ce qui est la vérité.

Chapitre 2 : Apprentissage supervisé

— exercices et applications

Ce chapitre du cahier accompagne le chapitre 2 du livre. Les **applications** reprennent, pas à pas et avec du code, les calculs que le livre n'a fait que résumer (descente de gradient, arbre, bagging et boosting écrits à la main, comparaisons de modèles) ; les **exercices** sont corrigés à la fin. Il se lit **de haut en bas** : les objets créés dans la section « Préparation » servent partout, et quelques applications réutilisent les précédentes. Les données sont celles du chapitre : `clients_ml.csv`, des clients d'une boutique simulée, avec la résiliation à 90 jours `churn_90j` comme cible.

Préparation

Une seule cellule charge les bibliothèques, les données et le découpage entraînement/test (identique à celui du livre). Les colonnes `depense_6m`, `segment_vrai` et `commandes_apres_cible` ne sont **jamais** des variables d'entrée (cette dernière contient l'avenir : c'est une fuite d'information).

```
import warnings
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.model_selection import StratifiedKFold, cross_val_score, train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

warnings.filterwarnings("ignore")
clients = pd.read_csv("donnees/clients_ml.csv")
y = clients["churn_90j"]
X = clients.drop(columns=["id_client", "churn_90j", "depense_6m", "segment_vrai",
↪ "commandes_apres_cible"])
cat = ["ville", "canal_acquisition", "appareil", "categorie_preferee"]
num = [c for c in X.columns if c not in cat]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
Xtr_o = pd.get_dummies(Xtr, columns=cat, dtype=float) # catégories en
↪ indicatrices (arbres, forêts)
Xte_o = pd.get_dummies(Xte, columns=cat, dtype=float).reindex(columns=Xtr_o.columns, fill_value=0.0)
Xtr_c, Xte_c = Xtr.copy(), Xte.copy() # catégories
↪ pandas (boosting)
```

```

for c in cat:
    Xtr_c[c] = pd.Categorical(Xtr[c], categories=sorted(Xtr[c].dropna().unique()))
    Xte_c[c] = pd.Categorical(Xte[c], categories=Xtr_c[c].cat.categories)
pre_lin = ColumnTransformer([("num", make_pipeline(SimpleImputer(strategy="median",
↪ add_indicator=True), StandardScaler()), num),
                             ("cat", OneHotEncoder(handle_unknown="ignore"), cat)])
cv5 = StratifiedKFold(5, shuffle=True, random_state=0)
print(len(Xtr), "clients d'entraînement,", len(Xte), "de test ; taux de départ :", round(y.mean(), 3))

```

```
9000 clients d'entraînement, 3000 de test ; taux de départ : 0.14
```

Applications

Application 2.1 — La descente de gradient écrite à la main

Sections du livre : 2.1.2 à 2.1.4. **Objectif :** programmer la régression logistique « de zéro », vérifier qu'on retrouve scikit-learn, et voir l'effet du pas et de la mise à l'échelle.

Étape 1 — un pas sur les quatre clients du livre. On reprend $x = (-2, -1, 1, 3)$, $y = (0, 0, 1, 1)$, $w = b = 0$ et un pas de 0,5. Le gradient est $\frac{1}{n} \sum (p_i - y_i)x_i$.

```

x4 = np.array([-2.0, -1.0, 1.0, 3.0]); y4 = np.array([0, 0, 1, 1])
w, b, pas = 0.0, 0.0, 0.5
p = 1 / (1 + np.exp(-(w * x4 + b)))
grad_w, grad_b = np.mean((p - y4) * x4), np.mean(p - y4)
print("gradient :", grad_w, grad_b, "-> nouveau w :", w + pas * grad_w)

```

```
gradient : -0.875 0.0 -> nouveau w : 0.4375
```

On retrouve $-0,875$ et $w = 0,4375$: le calcul à la main du livre.

Étape 2 — la descente complète sur trois variables. On standardise la récence, le nombre de commandes et la satisfaction (valeurs manquantes remplacées par la médiane) et l'on itère 500 fois.

```

cols = ["recence_jours", "nb_commandes_12m", "satisfaction_moy"]
A = Xtr[cols].fillna(Xtr[cols].median()); mu, sd = A.mean(), A.std()
Z = ((A - mu) / sd).to_numpy(); Z1 = np.column_stack([np.ones(len(Z)), Z]); yy = ytr.to_numpy(float)

def descente(pas, n_iter=500, taille_lot=None, graine=0):
    rng = np.random.default_rng(graine); th = np.zeros(Z1.shape[1]); hist = []
    for _ in range(n_iter):
        idx = np.arange(len(Z1)) if taille_lot is None else rng.choice(len(Z1), taille_lot,
↪ replace=False)
        p = 1 / (1 + np.exp(-Z1[idx] @ th))
        th = th - pas * Z1[idx].T @ (p - yy[idx]) / len(idx)
        pc = 1 / (1 + np.exp(-Z1 @ th)); hist.append(-np.mean(yy * np.log(pc + 1e-12) + (1 - yy) *
↪ np.log(1 - pc + 1e-12)))
    return th, np.array(hist)

th, hist = descente(0.5)
sk = LogisticRegression(C=1e6, max_iter=5000).fit(Z, yy)
print("à la main :", th.round(3))
print("scikit-learn:", np.r_[sk.intercept_, sk.coef_[0]].round(3))

```

```
à la main : [-2.343  0.554 -0.711 -0.538]
scikit-learn: [-2.342  0.553 -0.713 -0.537]
```

Les deux jeux de coefficients coïncident à 0,002 près : la régression logistique de scikit-learn

n'est pas autre chose qu'une descente de gradient (plus sophistiquée) sur cette perte. La perte atteinte est 0,328.

Étape 3 — le pas d'apprentissage. Comparons trois pas, puis la descente **stochastique** (mini-lots de 64 clients). (Avec des variables standardisées, la courbure de la perte est modérée : un pas de 0,5 est confortable, un pas de 60 est bien trop grand.)

```
res = {}
for nom, kw in {"pas 0,02": dict(pas=0.02), "pas 0,5": dict(pas=0.5), "pas 60 (trop grand)":
    dict(pas=60.0), "SGD, lots de 64, pas 0,2": dict(pas=0.2, taille_lot=64)}.items():
    res[nom] = descente(**kw)[1]
print(pd.DataFrame({"perte à 50 itérations": {k: v[49] for k, v in res.items()}, "perte à 500": {k:
    v[-1] for k, v in res.items()}).round(4))
```

	perte à 50 itérations	perte à 500
pas 0,02	0.5711	0.3544
pas 0,5	0.3321	0.3285
pas 60 (trop grand)	3.4504	2.1627
SGD, lots de 64, pas 0,2	0.3536	0.3285

Un pas trop petit (0,02) n'a pas fini d'avancer après 500 itérations ; un pas bien choisi (0,5) atteint le minimum en quelques dizaines ; un pas **trop grand** (60) fait osciller la descente, qui ne converge pas (perte à 500 itérations : 2,16). La SGD, bruitée, arrive près du minimum en ne regardant que 64 clients par itération.

Pour aller plus loin. Refaites l'étape 2 **sans standardiser** (utilisez `A.to_numpy()` à la place de `Z`) avec le plus grand pas qui ne diverge pas : mesurez la perte après 500 itérations.

Application 2.2 — Chemins de régularisation

Sections du livre : 2.1.5. **Objectif :** voir comment la pénalité ℓ_1 sélectionne des variables, et si cette sélection est **stable**.

Étape 1 — le chemin. Pour une grille de valeurs de C (petit = pénalité forte), on compte les coefficients non nuls et l'on mesure l'AUC en validation croisée.

```
grille = np.logspace(-2.5, 0.5, 7)
lignes = []
for C in grille:
    mod = make_pipeline(pre_lin, LogisticRegression(C=C, penalty="l1", solver="liblinear",
        max_iter=2000, random_state=0))
    auc = cross_val_score(mod, Xtr, ytr, cv=cv5, scoring="roc_auc").mean()
    lignes.append({"C": round(C, 4), "coefficients non nuls": int((mod.fit(Xtr, ytr)[-1].coef_ !=
        0).sum()), "AUC CV": round(auc, 4)})
print(pd.DataFrame(lignes).to_string(index=False))
```

C	coefficients non nuls	AUC CV
0.0032	5	0.8244
0.0100	9	0.8487
0.0316	12	0.8561
0.1000	25	0.8587
0.3162	40	0.8604
1.0000	42	0.8604
3.1623	43	0.8602

Étape 2 — la stabilité. On refait la sélection sur 12 échantillons bootstrap avec $C=0.05$ et l'on compte, pour chaque variable, la fréquence où elle est retenue.

```

rng = np.random.default_rng(1)
noms = pre_lin.fit(Xtr, ytr).get_feature_names_out()
freq = np.zeros(len(noms))
for b in range(12):
    idx = rng.integers(0, len(Xtr), len(Xtr))
    m = make_pipeline(pre_lin, LogisticRegression(C=0.05, penalty="l1", solver="liblinear",
    ↪ max_iter=2000, random_state=0)).fit(Xtr.iloc[idx], ytr.iloc[idx])
    freq += (m[-1].coef_[0] != 0)
st = pd.Series(freq / 12, index=noms).sort_values(ascending=False)
print(st.head(8).round(2).to_string()); print("\nvariables retenues à chaque fois :", int((st ==
    ↪ 1).sum()), "; retenues parfois :", int(((st > 0) & (st < 1)).sum()))

```

```

num_age                1.0
num_anciennete_mois    1.0
num_nb_commandes_12m  1.0
num_panier_moyen       1.0
num_recence_jours      1.0
num_satisfaction_moy    1.0
num_missingindicator_satisfaction_moy  1.0
num_part_achats_promo  1.0
...
variables retenues à chaque fois : 10 ; retenues parfois : 24

```

Sur les 12 tirages, 10 variables sont retenues **à chaque fois** et 24 seulement **parfois** : le Lasso est fiable pour les signaux **forts** et **hésitant** pour les variables corrélées entre elles (laquelle choisir ?). Ne présentez jamais « les variables sélectionnées par le Lasso » comme *les* variables importantes sans cette vérification.

Application 2.3 — Construire un arbre à la main, puis avec scikit-learn

Sections du livre : 2.2.1 à 2.2.5. **Objectif** : écrire la recherche de la meilleure coupe, puis la récursion.

Étape 1 — le meilleur seuil d'une variable.

```

ex = pd.DataFrame({"recence": [15, 30, 45, 70, 95, 120, 160, 200, 260, 320], "satisfaction": [4.5,
    ↪ 3.0, 4.0, 4.1, 3.1, 4.4, 3.0, 4.3, 2.9, 3.8],
    "parti": [0, 0, 0, 0, 0, 0, 1, 0, 1, 1]})
gini = lambda v: 1 - np.mean(v) ** 2 - (1 - np.mean(v)) ** 2 if len(v) else 0.0

def meilleure_coupe(df, variables, cible="parti"):
    meilleur = (0.0, None, None); g0 = gini(df[cible])
    for var in variables:
        v = np.sort(df[var].unique())
        for s in (v[:-1] + v[1:]) / 2:
            gche, drt = df[df[var] <= s], df[df[var] > s]
            gain = g0 - (len(gche) * gini(gche[cible]) + len(drt) * gini(drt[cible])) / len(df)
            if gain > meilleur[0] + 1e-12: meilleur = (gain, var, s)
    return meilleur
print(meilleure_coupe(ex, ["recence", "satisfaction"]))

```

```
(np.float64(0.270000000000000013), 'recence', np.float64(140.0))
```

Étape 2 — la récursion. On construit l'arbre jusqu'à des groupes purs ou une profondeur maximale.

```

def construire(df, variables, prof=0, prof_max=3):
    if df["parti"].nunique() == 1 or prof == prof_max: return {"classe": round(df["parti"].mean(), 2),
    ↪ "n": len(df)}
    gain, s = meilleure_coupe(df, variables)
    if var is None: return {"classe": round(df["parti"].mean(), 2), "n": len(df)}
    return {"si": f"{var} <= {s:g}", "oui": construire(df[df[var] <= s], variables, prof + 1,
    ↪ prof_max), "non": construire(df[df[var] > s], variables, prof + 1, prof_max)}

```

```
def afficher(arbre, retrait=0):
    pad = "    " * retrait
    if "classe" in arbre: print(f"{pad}-> probabilité de départ {arbre['classe']} ({arbre['n']}
    ↪ clients)")
    else:
        print(f"{pad}si {arbre['si']} :"); afficher(arbre["oui"], retrait + 1); print(f"{pad}sinon
        ↪ :"); afficher(arbre["non"], retrait + 1)
    afficher(construire(ex, ["recence", "satisfaction"]))
```

```
si recence <= 140 :
    -> probabilité de départ 0.0 (6 clients)
sinon :
    si satisfaction <= 4.05 :
        -> probabilité de départ 1.0 (3 clients)
    sinon :
        -> probabilité de départ 0.0 (1 clients)
```

On retrouve les deux questions du livre ($\text{recence} \leq 140$, puis $\text{satisfaction} \leq 4,05$).

Étape 3 — sur les vraies données : profondeur et élagage.

```
from sklearn.tree import DecisionTreeClassifier
profs = [2, 3, 4, 5, 6, 8, 12]
for d in profs:
    m = DecisionTreeClassifier(max_depth=d, min_samples_leaf=5, random_state=0)
    print(d, round(cross_val_score(m, Xtr_o, ytr, cv=cv5, scoring="roc_auc").mean(), 4))
chemin = DecisionTreeClassifier(min_samples_leaf=5,
    ↪ random_state=0).cost_complexity_pruning_path(Xtr_o, ytr)
print("nombre de valeurs de alpha possibles :", len(chemin.ccp_alphas))
```

```
2 0.7926
3 0.8385
4 0.8613
5 0.863
6 0.8579
8 0.813
12 0.7631
nombre de valeurs de alpha possibles : 273
```

La validation croisée retient la profondeur 5 (AUC 0,863) : au-delà, l'arbre mémorise.

Application 2.4 — Le bagging écrit à la main

Sections du livre : 2.3.1 à 2.3.4. Objectif : construire une forêt de bagging avec une boucle, calculer soi-même l'erreur hors sac, et vérifier la baisse de variance.

```
rng = np.random.default_rng(0)
B, n = 60, len(Xtr_o)
Xn, yn = Xtr_o.to_numpy(), ytr.to_numpy()
arbres, masque_in = [], np.zeros((B, n), dtype=bool)
for b in range(B):
    idx = rng.integers(0, n, n); masque_in[b, np.unique(idx)] = True
    arbres.append(DecisionTreeClassifier(max_depth=8, min_samples_leaf=5,
    ↪ random_state=b).fit(Xn[idx], yn[idx]))
P_train = np.array([t.predict_proba(Xn)[:, 1] for t in arbres]) # B x n
oob = np.where(~masque_in, P_train, np.nan); p_oob = np.nanmean(oob, axis=0)
print("part de clients hors sac par arbre :", round(float((~masque_in).mean()), 3))
print("AUC hors sac (fait main) :", round(roc_auc_score(yn[~np.isnan(p_oob)],
    ↪ p_oob[~np.isnan(p_oob)]), 4))
P_test = np.array([t.predict_proba(Xte_o.to_numpy())[ :, 1] for t in arbres])
print("AUC test, 1 arbre :", round(roc_auc_score(yte, P_test[0]), 4), "; 60 arbres :",
    ↪ round(roc_auc_score(yte, P_test.mean(0)), 4))
```

```
part de clients hors sac par arbre : 0.369
AUC hors sac (fait main) : 0.8853
AUC test, 1 arbre : 0.8174 ; 60 arbres : 0.8962
```

Chaque arbre laisse de côté 37 % des clients (théorie : 36,8 %). L'AUC hors sac calculée à la main (0,885) estime la performance sans toucher au jeu de test, et la moyenne de 60 arbres (0,896) bat nettement un arbre isolé (0,817).

Pour aller plus loin. Remplacez `DecisionTreeClassifier(...)` par `DecisionTreeClassifier(..., max_features="sqrt")` : vous obtenez une vraie forêt aléatoire. Comparez l'AUC de test de 60 arbres.

Application 2.5 — Importances et variables de bruit

Sections du livre : 2.3.6. **Objectif :** constater les biais de l'importance par impureté et la dilution entre variables corrélées.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
rb = np.random.default_rng(5)
Xb, Xbt = Xtr_o.copy(), Xte_o.copy()
Xb["bruit_continu"], Xbt["bruit_continu"] = rb.normal(size=len(Xb)), rb.normal(size=len(Xbt))
Xb["bruit_id"], Xbt["bruit_id"] = rb.integers(0, 2000, len(Xb)).astype(float), rb.integers(0, 2000,
↳ len(Xbt)).astype(float)
Xb["recence_bis"], Xbt["recence_bis"] = Xb["recence_jours"] + rb.normal(0, 3, len(Xb)),
↳ Xbt["recence_jours"] + rb.normal(0, 3, len(Xbt)) # quasi-doublon
f = RandomForestClassifier(150, min_samples_leaf=5, max_features="sqrt", n_jobs=1,
↳ random_state=0).fit(Xb, ytr)
mdi = pd.Series(f.feature_importances_, index=Xb.columns)
perm = pd.Series(permutation_importance(f, Xbt, yte, scoring="roc_auc", n_repeats=3, random_state=0,
↳ n_jobs=1).importances_mean, index=Xb.columns)
cible = ["recence_jours", "recence_bis", "bruit_continu", "bruit_id", "satisfaction_moy"]
print(pd.DataFrame({"impureté": mdi[cible], "permutation": perm[cible]}).round(4))
```

	impureté	permutation
recence_jours	0.0946	0.0118
recence_bis	0.0975	0.0073
bruit_continu	0.0366	-0.0005
bruit_id	0.0323	-0.0004
satisfaction_moy	0.1221	0.0276

Lisez le tableau : (1) les deux variables de **bruit** reçoivent une importance par impureté non négligeable et une importance par permutation proche de zéro ; (2) **recence_jours** et son quasi-doublon **recence_bis** **se partagent** l'importance : chacune paraît moins utile qu'elle ne l'est. Pour mesurer l'importance d'un groupe de variables corrélées, on permute le groupe entier, ou l'on retire le groupe et l'on remesure la performance (importance par suppression).

Application 2.6 — Le gradient boosting écrit à la main

Sections du livre : 2.4.1 et 2.4.2. **Objectif :** programmer le boosting pour la perte quadratique (prédire la dépense `depense_6m`), puis pour la perte logistique (la résiliation), et le comparer à `scikit-learn`.

Étape 1 — perte quadratique : les résidus.

```
from sklearn.tree import DecisionTreeRegressor
dep = clients.loc[Xtr.index, "depense_6m"].to_numpy(); dep_te = clients.loc[Xte.index,
↳ "depense_6m"].to_numpy()
```

```

Xn_te = Xte_o.to_numpy()
F, F_te, nu, hist = np.full(len(Xn), dep.mean()), np.full(len(Xn_te), dep.mean()), 0.1, []
for m in range(150):
    t = DecisionTreeRegressor(max_depth=3, min_samples_leaf=20, random_state=m).fit(Xn, dep - F)
    ↪ # on ajuste les RÉSIDUS
    F += nu * t.predict(Xn); F_te += nu * t.predict(Xn_te)
    hist.append((m + 1, np.sqrt(np.mean((dep - F) ** 2)), np.sqrt(np.mean((dep_te - F_te) ** 2))))
h = pd.DataFrame(hist, columns=["arbres", "RMSE entraînement", "RMSE test"]).set_index("arbres")
print(h.loc[[1, 10, 50, 100, 150]].round(2)); print("RMSE de la prédiction constante :",
    ↪ round(float(np.sqrt(np.mean((dep_te - dep.mean()) ** 2))), 2))

```

	RMSE entraînement	RMSE test
arbres		
1	118.41	120.71
10	97.83	100.45
50	88.40	94.09
100	85.26	94.00
150	82.51	94.25

RMSE de la prédiction constante : 125.62

Le RMSE de test passe de 125,6 (prédiction constante) à 94,0 après 100 arbres ; ensuite il ne s'améliore plus et remonte légèrement (94,25 à 150 arbres) alors que celui de l'entraînement continue de baisser (82,5) : le modèle commence à s'ajuster au bruit, d'où l'intérêt de l'arrêt précoce.

Étape 2 — perte logistique : $y - p$. On travaille sur la log-cote F , avec des arbres de profondeur 3.

```

F, F_te, hist = np.full(len(Xn), np.log(yn.mean() / (1 - yn.mean()))), np.full(len(Xn_te),
    ↪ np.log(yn.mean() / (1 - yn.mean()))), []
for m in range(200):
    p = 1 / (1 + np.exp(-F))
    t = DecisionTreeRegressor(max_depth=3, min_samples_leaf=20, random_state=m).fit(Xn, yn - p)
    ↪ # pseudo-résidu : y - p
    F += 0.1 * t.predict(Xn); F_te += 0.1 * t.predict(Xn_te)
    if (m + 1) in (1, 25, 50, 100, 200): hist.append((m + 1, roc_auc_score(yte, F_te), log_loss(yte, 1
    ↪ / (1 + np.exp(-F_te)))))
print(pd.DataFrame(hist, columns=["arbres", "AUC test", "perte logistique
    ↪ test"]).round(4).to_string(index=False))

```

arbres	AUC test	perte logistique test
1	0.8412	0.4018
25	0.8779	0.3374
50	0.8821	0.3072
100	0.8912	0.2830
200	0.8969	0.2650

Notre boosting artisanal atteint 0,897 après 200 arbres ; HistGradientBoostingClassifier, avec des réglages comparables, 0,901. Les deux ne diffèrent que par les détails (histogrammes, valeurs de feuilles de Newton, traitement des valeurs manquantes).

Application 2.7 — XGBoost et LightGBM : réglages, valeurs manquantes, catégories

Sections du livre : 2.4.3 à 2.4.6. Objectif : explorer les hyperparamètres principaux et les traitements natifs.

```

import lightgbm as lgb
Xa, Xv, ya, yv = train_test_split(Xtr_c, ytr, test_size=0.2, stratify=ytr, random_state=1)
lignes = []
for lr in (0.1, 0.03):
    for nl in (7, 15, 31):
        m = lgb.LGBMClassifier(n_estimators=800, learning_rate=lr, num_leaves=nl, subsample=0.8,
    ↪ subsample_freq=1, colsample_bytree=0.8, n_jobs=1, random_state=0, verbose=-1)

```

```

m.fit(Xa, ya, eval_set=[(Xv, yv)], callbacks=[lgb.early_stopping(30, verbose=False)])
lignes.append({"pas": lr, "feuilles": nl, "arbres retenus": m.best_iteration_, "AUC test":
↳ round(roc_auc_score(yte, m.predict_proba(Xte_c)[: , 1]), 4)})
print(pd.DataFrame(lignes).to_string(index=False))

```

pas	feuilles	arbres retenus	AUC test
0.10	7	115	0.9013
0.10	15	57	0.9013
0.10	31	58	0.8991
0.03	7	305	0.9037
0.03	15	246	0.9019
0.03	31	229	0.8987

Les six réglages donnent des AUC de test entre 0,899 et 0,904 : le boosting est **peu sensible** aux réglages fins une fois l'arrêt précoce utilisé.

Valeurs manquantes et catégories. Comparons le traitement natif à une imputation par la médiane et à un encodage des villes en codes entiers arbitraires.

```

def auc_lgb(A, B):
    m = lgb.LGBMClassifier(n_estimators=800, learning_rate=0.05, num_leaves=15, subsample=0.8,
↳ subsample_freq=1, colsample_bytree=0.8, n_jobs=1, random_state=0, verbose=-1)
    Aa, Av, ya_, yv_ = train_test_split(A, ytr, test_size=0.2, stratify=ytr, random_state=1)
    m.fit(Aa, ya_, eval_set=[(Av, yv_)], callbacks=[lgb.early_stopping(30, verbose=False)])
    return round(roc_auc_score(yte, m.predict_proba(B)[: , 1]), 4)
Ai, Bi = Xtr_c.copy(), Xte_c.copy(); Ai[num] = Ai[num].fillna(Xtr[num].median()); Bi[num] =
↳ Bi[num].fillna(Xtr[num].median())
Ak, Bk = Xtr_c.copy(), Xte_c.copy()
for c in cat: Ak[c], Bk[c] = Xtr_c[c].cat.codes, Xte_c[c].cat.codes
print({"natif": auc_lgb(Xtr_c, Xte_c), "valeurs manquantes imputées": auc_lgb(Ai, Bi), "catégories en
↳ codes entiers": auc_lgb(Ak, Bk)})

```

```
{'natif': 0.9026, 'valeurs manquantes imputées': 0.9028, 'catégories en codes entiers': 0.9027}
```

Les écarts sont faibles sur ces données, mais le traitement natif ne coûte rien. Essayez de **supprimer volontairement** 40 % des valeurs de `recence_jours` au hasard puis au hasard **pour les clients partis seulement** (une absence informative) : l'imputation perd l'information « manquant », le traitement natif la garde.

Application 2.8 — Le match des modèles, avec comparaison appariée

Sections du livre : 2.4.7. **Objectif :** comparer trois modèles proprement : validation croisée **répétée**, différences **appariées**, test corrigé et bootstrap.

Étape 1 — validation croisée répétée à 5 plis × 3 répétitions (les mêmes plis pour tous les modèles).

```

from sklearn.model_selection import RepeatedStratifiedKFold
rcv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=0)
mods = {"logistique": (make_pipeline(pre_lin, LogisticRegression(C=0.3, max_iter=1000)), Xtr),
        "forêt": (RandomForestClassifier(100, min_samples_leaf=5, max_features="sqrt", n_jobs=1,
↳ random_state=0), Xtr_o),
        "boosting": (HistGradientBoostingClassifier(learning_rate=0.05, max_iter=200,
↳ early_stopping=False, categorical_features="from_dtype", random_state=0), Xtr_c)}
S = {k: cross_val_score(m, A, ytr, cv=rcv, scoring="roc_auc") for k, (m, A) in mods.items()}
print(pd.DataFrame({k: [v.mean(), v.std()] for k, v in S.items()}, index=["moyenne", "écart-type (15
↳ plis)"]).round(4))

```

	logistique	forêt	boosting
moyenne	0.8603	0.8856	0.8898
écart-type (15 plis)	0.0117	0.0131	0.0096

Étape 2 — les différences appariées. Pour chaque pli, on calcule la différence d'AUC entre deux modèles ; un test de Student **corrigé** (Nadeau et Bengio) tient compte de ce que les plis se recouvrent : la variance de la moyenne des différences est multipliée par $\frac{1}{k} + \frac{n_{\text{test}}}{n_{\text{train}}}$ au lieu de $\frac{1}{k}$ ($k = 15$ plis ici).

```
from scipy import stats
def compare(a, b):
    d = S[a] - S[b]; k = len(d); n_test, n_train = len(ytr) // 5, len(ytr) - len(ytr) // 5
    var_corr = d.var(ddof=1) * (1 / k + n_test / n_train)
    t = d.mean() / np.sqrt(var_corr); p = 2 * stats.t.sf(abs(t), k - 1)
    return {"écart moyen d'AUC": round(d.mean(), 4), "t corrigé": round(t, 2), "p": round(p, 4)}
print("boosting - logistique :", compare("boosting", "logistique")); print("boosting - forêt      :",
↪ compare("boosting", "forêt"))
```

```
boosting - logistique : {"écart moyen d'AUC": np.float64(0.0295), 't corrigé': np.float64(9.35), 'p':
↪ np.float64(0.0)}
boosting - forêt      : {"écart moyen d'AUC": np.float64(0.0042), 't corrigé': np.float64(0.99), 'p':
↪ np.float64(0.3392)}
```

Le boosting l'emporte clairement sur la régression logistique (écart 0,029, $p < 0,001$). Face à la forêt, l'écart est de 0,004 avec $p = 0,339$: à ce niveau de précision, la différence n'est pas établie.

Le test de Student ordinaire sur ces 15 différences ignorerait le recouvrement des plis et serait **trop optimiste** (il rejeterait trop souvent l'égalité). La correction ci-dessus est l'usage recommandé.

Application 2.9 — Les modèles classiques face au fléau de la dimension

Sections du livre : 2.5. **Objectif :** mesurer comment k-NN, SVM et boosting réagissent à l'ajout de variables **inutiles**.

```
from sklearn.neighbors import KNeighborsClassifier
rng = np.random.default_rng(2)
lignes = []
for k_bruit in (0, 20, 100, 300):
    bruit_tr, bruit_te = rng.normal(size=(len(Xtr), k_bruit)), rng.normal(size=(len(Xte), k_bruit))
    pre = pre_lin.fit(Xtr, ytr); A, B = pre.transform(Xtr), pre.transform(Xte)
    A = np.hstack([A.toarray() if hasattr(A, "toarray") else A, bruit_tr]); B = np.hstack([B.toarray()
↪ if hasattr(B, "toarray") else B, bruit_te])
    knn = KNeighborsClassifier(75).fit(A, ytr)
    hgb = HistGradientBoostingClassifier(learning_rate=0.1, max_iter=100, early_stopping=False,
↪ random_state=0).fit(np.hstack([Xtr_o.fillna(Xtr_o.median()).to_numpy(), bruit_tr]), ytr)
    lignes.append({"variables de bruit ajoutées": k_bruit, "AUC k-NN": round(roc_auc_score(yte,
↪ knn.predict_proba(B[:, 1]), 4),
                    "AUC boosting": round(roc_auc_score(yte,
↪ hgb.predict_proba(np.hstack([Xte_o.fillna(Xte_o.median()).to_numpy(),
↪ bruit_te])), 4)})
print(pd.DataFrame(lignes).to_string(index=False))
```

variables de bruit ajoutées	AUC k-NN	AUC boosting
0	0.8562	0.9011
20	0.8442	0.9015
100	0.8166	0.8981
300	0.7975	0.9024

Le k-NN passe de 0,856 à 0,797 quand on ajoute 300 variables de pur bruit : les distances sont noyées. Le boosting, qui **choisit** les variables à chaque coupe, passe de 0,901 à 0,902. C'est l'illustration, sur données réelles, de la différence entre un modèle qui **mesure des distances** et

un modèle qui **sélectionne**.

Application 2.10 — Encodage ordonné, CatBoost et empilement sans fuite

Sections du livre : 2.6. Objectif : implémenter l'encodage ordonné d'une catégorie, le comparer à l'encodage naïf, puis empiler deux modèles en respectant la validation hors pli.

Étape 1 — une variable à nombreuses modalités, encodée de deux façons. On croise la ville et la tranche d'âge de dix ans : environ 140 « quartiers » de quelques dizaines de clients. On les encode par la moyenne de la cible, **naïvement** (en incluant le client lui-même) puis de façon **ordonnée** (seulement les clients qui le précèdent dans un ordre aléatoire), et l'on mesure l'AUC d'un modèle logistique sur cette seule colonne, sur l'entraînement **et** sur le test.

```
rng = np.random.default_rng(0)
q_tr = (Xtr["ville"] + "_" + (Xtr["age"] // 10).astype(int).astype(str)).to_numpy()
q_te = (Xte["ville"] + "_" + (Xte["age"] // 10).astype(int).astype(str)).to_numpy()
prior, a = ytr.mean(), 20
stat = pd.DataFrame({"q": q_tr, "y": ytr.to_numpy()}).groupby("q")["y"].agg(["sum", "count"])
enc_naif = pd.Series(q_tr).map(stat["sum"] / stat["count"]).to_numpy() # moyenne de la
↳ cible, client compris
enc_te = pd.Series(q_te).map((stat["sum"] + a * prior) / (stat["count"] + a)).fillna(prior).to_numpy()
somme, nb, enc_ord = {}, {}, np.zeros(len(q_tr))
for i in rng.permutation(len(q_tr)): # encodage ORDONNÉ
↳ : seulement les clients précédents
    z = q_tr[i]; enc_ord[i] = (somme.get(z, 0.0) + a * prior) / (nb.get(z, 0) + a)
    somme[z] = somme.get(z, 0.0) + ytr.iloc[i]; nb[z] = nb.get(z, 0) + 1
def auc_tr_te(e_tr):
    m = LogisticRegression().fit(e_tr.reshape(-1, 1), ytr)
    return round(roc_auc_score(ytr, m.predict_proba(e_tr.reshape(-1, 1))[:, 1]), 4),
↳ round(roc_auc_score(yte, m.predict_proba(enc_te.reshape(-1, 1))[:, 1]), 4)
print(len(stat), "modalités ; naïf (AUC entraînement, test) :", auc_tr_te(enc_naif), "; ordonné :",
↳ auc_tr_te(enc_ord))
```

```
134 modalités ; naïf (AUC entraînement, test) : (0.7545, 0.7135) ; ordonné : (0.7019, 0.7135)
```

Avec 134 modalités (environ 67 clients chacune), l'encodage naïf donne une AUC de 0,754 sur l'entraînement mais de 0,714 seulement sur le test : l'écart est la **fuite** (le client a contribué à la valeur qui le décrit). L'encodage ordonné donne 0,702 sur l'entraînement et 0,714 sur le test : **l'entraînement et le test racontent la même histoire**, ce qui est le signe d'une variable honnête. Plus les modalités sont rares, plus l'écart grandit (voir l'identifiant de zone du livre, 2.6.1).

Étape 2 — l'empilement hors pli de deux modèles.

```
from sklearn.model_selection import cross_val_predict
from scipy.special import logit
bases = [(make_pipeline(pre_lin, LogisticRegression(C=0.3, max_iter=1000)), Xtr, Xte),
          (HistGradientBoostingClassifier(learning_rate=0.05, max_iter=200, early_stopping=False,
↳ categorical_features="from_dtype", random_state=0), Xtr_c, Xte_c)]
oof = np.column_stack([cross_val_predict(m, A, ytr, cv=cv5, method="predict_proba")[:, 1] for m, A, _
↳ in bases])
pte = np.column_stack([m.fit(A, ytr).predict_proba(B)[:, 1] for m, A, B in bases])
L = lambda P: logit(np.clip(P, 1e-4, 1 - 1e-4))
meta = LogisticRegression().fit(L(oof), ytr)
print("coefficients du méta-modèle (logistique, boosting) :", meta.coef_[0].round(2))
print("AUC test : logistique", round(roc_auc_score(yte, pte[:, 0]), 4), "| boosting",
↳ round(roc_auc_score(yte, pte[:, 1]), 4), "| empilement", round(roc_auc_score(yte,
↳ meta.predict_proba(L(pte))[:, 1]), 4))
```

```
coefficients du méta-modèle (logistique, boosting) : [0.2 0.68]
AUC test : logistique 0.8659 | boosting 0.9032 | empilement 0.9028
```

L'empilement apprend à donner l'essentiel du poids au boosting (0,68) et un poids moindre au modèle linéaire (0,20). Mais son AUC de test (0,903) n'est pas supérieure à celle du boosting seul (0,903) : le modèle linéaire n'apporte presque rien de neuf (rappel des corrélations élevées entre modèles, section 2.6.2). Vérifiez que, si vous entraînez le méta-modèle sur les prédictions **faites sur l'entraînement** (`m.fit(A, ytr).predict_proba(A)`), le poids du modèle le plus « sur-ajusté » gonfle.

Exercices

Exercice 2.1 ★ — Quelle perte pour quelle marge ? (section 2.1.1)

Pour des marges $m = -1, 0, 5$ et 3 , calculez les pertes charnière $\max(0, 1 - m)$, logistique $\ln(1 + e^{-m})$ et quadratique $(1 - m)^2$. Laquelle punit un client **très bien classé** ? Laquelle ignore complètement les exemples de marge supérieure à 1 ?

Exercice 2.2 ★★ — Un pas de gradient (section 2.1.3)

Trois clients : $x = (0, 1, 2)$, $y = (0, 1, 1)$. Partant de $w = 0$, $b = 0$, calculez le gradient de la perte logistique moyenne en w et en b , puis les nouveaux paramètres après un pas de $\eta = 0,4$. Calculez la perte avant et après.

Exercice 2.3 ★★ — Seuillage contre rétrécissement (section 2.1.5)

Dans le cas orthonormé, les estimations non pénalisées sont $z = (2,4; -0,7; 0,2)$. Donnez les coefficients du Lasso ($\text{signe}(z)(|z| - \lambda)_+$) et de Ridge ($z/(1 + \lambda)$) pour $\lambda = 0,5$. Combien de variables le Lasso conserve-t-il ? Pour quelle valeur de λ élimine-t-il la deuxième variable ?

Exercice 2.4 ★ — Quelle coupe choisir ? (section 2.2.3)

Un nœud contient 100 clients dont 30 sont partis. La coupe A sépare 40 clients (dont 25 partis) à gauche et 60 (dont 5 partis) à droite. La coupe B sépare 70 clients (dont 20 partis) à gauche et 30 (dont 10 partis) à droite. Calculez l'impureté de Gini du nœud et le gain de chaque coupe. Laquelle l'arbre choisit-il ?

Exercice 2.5 ★★ — Un arbre de régression (section 2.2.4)

Huit clients : $x = (1, 2, 3, 4, 5, 6, 7, 8)$, dépense $y = (5, 7, 6, 8, 30, 28, 33, 31)$. Trouvez la meilleure coupe de la forme « $x \leq s$ » au sens de la somme des carrés des écarts, puis la prédiction de chaque feuille et la somme des carrés résiduelle.

Exercice 2.6 ★★★ — Élaguer par coût-complexité (section 2.2.5)

Une suite d'arbres emboîtés pour nombres de feuilles et erreurs d'entraînement : T_0 (7 feuilles, erreur 10), T_1 (5, 12), T_2 (3, 18), T_3 (1, 30). Pour chaque arbre, calculez $R_\alpha(T) = R(T) + \alpha|T|$ avec $\alpha = 0,5, 2$ et 5 . Quel arbre est optimal pour chaque α ? À partir de quelle valeur de α passe-t-on de T_0 à T_1 , de T_1 à T_2 , de T_2 à T_3 ?

Exercice 2.7 ★★ — Combien d'arbres ? (section 2.3.2)

Des arbres ont une variance $\sigma^2 = 4$ et une corrélation $\rho = 0,2$. (a) Quelle est la variance de la moyenne de $B = 25$ arbres ? (b) Quelle est la limite quand $B \rightarrow \infty$? (c) Combien d'arbres faut-il pour que la part « évitable » $\frac{1-\rho}{B}\sigma^2$ soit inférieure à 10 % de la part incompressible ? (d) Que devient la limite si une forêt aléatoire ramène ρ à 0,1 ?

Exercice 2.8 ★★ — Les clients hors sac (section 2.3.4)

Une forêt compte $B = 300$ arbres sur $n = 9\,000$ clients. (a) Combien de clients distincts contient en moyenne un échantillon bootstrap ? (b) Pour un client donné, combien d'arbres ne l'ont jamais vu, en moyenne ? (c) Avec seulement $B = 10$ arbres, quelle est la probabilité qu'un client donné ne soit **hors sac pour aucun** arbre ?

Exercice 2.9 ★★ — Deux tours de boosting (section 2.4.1)

Cinq clients, $x = (1, 2, 3, 4, 5)$ et $y = (2, 4, 3, 11, 15)$. Avec des souches et un pas $\nu = 0,5$: calculez F_0 , les résidus, la première souche (meilleure coupe par somme des carrés), F_1 , puis les nouveaux résidus.

Exercice 2.10 ★★★ — Valeur de feuille et gain XGBoost (section 2.4.3)

Un nœud contient six clients avec les gradients $g = (-0,5; -0,5; -0,4; 0,5; 0,4; -0,6)$ et les hessiens $h = (0,25; 0,25; 0,24; 0,25; 0,24; 0,24)$, $\lambda = 2$, $\gamma = 0,1$. (a) Valeur de feuille si l'on ne coupe pas. (b) Gain de la coupe C1 qui sépare les clients $\{1, 2, 3\}$ des clients $\{4, 5, 6\}$, et de la coupe C2 qui sépare $\{1, 2, 6\}$ de $\{3, 4, 5\}$. (c) Laquelle choisir, et est-elle effectuée ?

Exercice 2.11 ★★ — Bayes naïf avec lissage (section 2.5.3)

Parmi 20 clients (8 partis, 12 restés), on observe : $P(\text{pas de ticket} \mid \text{parti}) = 2/8$, $P(\text{pas de ticket} \mid \text{resté}) = 9/12$, $P(\text{courriel jamais ouvert} \mid \text{parti}) = 0/8$, $P(\text{courriel jamais ouvert} \mid \text{resté}) = 3/12$. (a) Probabilité de départ d'un client sans ticket et au courriel jamais ouvert, sans lissage. (b) Quel problème pose le 0 ? (c) Refaites le calcul avec le lissage de Laplace (ajouter 1 au numérateur et 2 au dénominateur de chaque probabilité conditionnelle).

Exercice 2.12 ★★★ — Pourquoi moyenner des modèles corrélés rapporte peu (section 2.6.2)

Deux modèles ont la même variance d'erreur $\sigma^2 = 1$ et une corrélation d'erreur ρ_e . (a) Quelle est la variance d'erreur de leur moyenne ? Calculez-la pour $\rho_e = 0, 0,5$ et $0,9$. (b) Si le second modèle a une variance d'erreur $\sigma_2^2 = 4$ (il est nettement moins bon) et $\rho_e = 0$, la moyenne **égale** est-elle meilleure que le premier modèle seul ? Quelle pondération optimale ($w, 1 - w$) minimise la variance ? (c) Simulez (b) avec 100 000 tirages.

Corrigé 2.1

```
m = np.array([-1.0, 0.5, 3.0])
print(pd.DataFrame({"marge": m, "charnière": np.maximum(0, 1 - m), "logistique":
↳ np.log1p(np.exp(-m)), "quadratique": (1 - m) ** 2}).round(3).to_string(index=False))
```

marge	charnière	logistique	quadratique
-1.0	2.0	1.313	4.00
0.5	0.5	0.474	0.25
3.0	0.0	0.049	4.00

La perte **quadratique** punit le client très bien classé ($m = 3 : (1 - 3)^2 = 4$) : c'est absurde pour la classification. La perte **charnière** vaut exactement 0 dès que $m \geq 1$: les exemples bien classés avec marge suffisante sont **ignorés** (les autres sont les vecteurs de support). La logistique, elle, reste faiblement positive (0,049 pour $m = 3$) et continue à pousser vers des marges plus grandes.

Corrigé 2.2

Avec $w = b = 0$, $p_i = 0,5$; erreurs $p_i - y_i = (0,5; -0,5; -0,5)$. Donc $\partial L / \partial w = \frac{1}{3}[0,5 \times 0 + (-0,5) \times 1 + (-0,5) \times 2] = -0,5$ et $\partial L / \partial b = \frac{1}{3}(0,5 - 0,5 - 0,5) = -\frac{1}{6} \approx -0,1667$. Après le pas $\eta = 0,4$: $w = 0,2$, $b = 0,0667$.

```
x3, y3 = np.array([0.0, 1.0, 2.0]), np.array([0, 1, 1])
def perte3(w, b):
    p = 1 / (1 + np.exp(-(w * x3 + b))) ; return -np.mean(y3 * np.log(p) + (1 - y3) * np.log(1 - p))
print("gradient :", np.mean((0.5 - y3) * x3), np.mean(0.5 - y3), "| paramètres :", 0.4 * 0.5, 0.4 / 6,
↳ "| pertes :", round(perte3(0, 0), 4), "->", round(perte3(0.2, 0.4 / 6), 4))
```

```
gradient : -0.5 -0.16666666666666666 | paramètres : 0.2 0.06666666666666667 | pertes : 0.6931 ->
↳ 0.5942
```

Corrigé 2.3

Lasso : $(2,4 - 0,5; -(0,7 - 0,5); 0) = (1,9; -0,2; 0)$: **deux** variables conservées (la troisième, $|0,2| < 0,5$, est annulée). Ridge : $z/1,5 = (1,6; -0,467; 0,133)$: aucune n'est annulée. Le Lasso élimine la deuxième variable dès que $\lambda \geq 0,7$.

Corrigé 2.4

$G(\text{nœud}) = 1 - 0,3^2 - 0,7^2 = 0,42$. Coupe A : gauche $p = \frac{25}{40} = 0,625$, $G = 1 - 0,625^2 - 0,375^2 = 0,469$; droite $p = \frac{5}{60}$, $G = 0,153$; impureté $= 0,4 \times 0,469 + 0,6 \times 0,153 = 0,279$; **gain** 0,141. Coupe B : gauche $p = \frac{20}{70}$, $G = 0,408$; droite $p = \frac{1}{3}$, $G = 0,444$; impureté $= 0,7 \times 0,408 + 0,3 \times 0,444 = 0,419$; gain 0,001. L'arbre choisit **A** : B ne sépare presque rien (les deux groupes restent aussi mélangés que le nœud).

```
def gini2(p): return 1 - p ** 2 - (1 - p) ** 2
g0 = gini2(0.3)
print({"A": round(g0 - 0.4 * gini2(25 / 40) - 0.6 * gini2(5 / 60), 4), "B": round(g0 - 0.7 * gini2(20
↳ / 70) - 0.3 * gini2(10 / 30), 4)})
```

```
{'A': 0.1408, 'B': 0.001}
```

Corrigé 2.5

Les y passent de 5-8 (clients 1 à 4) à 28-33 (clients 5 à 8) : la coupe naturelle est $x \leq 4,5$.

```

xr, yr = np.arange(1, 9), np.array([5, 7, 6, 8, 30, 28, 33, 31.0])
sse = lambda v: ((v - v.mean()) ** 2).sum()
res = {s + 0.5: sse(yr[xr <= s]) + sse(yr[xr > s]) for s in range(1, 8)}
meilleur = min(res, key=res.get)
print({k: round(v, 1) for k, v in res.items()}); print("meilleure coupe : x <=", meilleur, "| feuilles
↳ :", yr[xr <= meilleur].mean(), yr[xr > meilleur].mean(), "| SSE :", round(res[meilleur], 1), "|
↳ SSE avant :", sse(yr))

```

```

{1.5: np.float64(961.7), 2.5: np.float64(753.3), 3.5: np.float64(420.0), 4.5: np.float64(18.0), 5.5:
↳ np.float64(459.5), 6.5: np.float64(684.0), 7.5: np.float64(991.4)}
meilleure coupe : x <= 4.5 | feuilles : 6.5 30.5 | SSE : 18.0 | SSE avant : 1170.0

```

À gauche la moyenne est $\frac{5+7+6+8}{4} = 6,5$ (somme des carrés $2,25 + 0,25 + 0,25 + 2,25 = 5$), à droite $\frac{30+28+33+31}{4} = 30,5$ (somme $0,25 + 6,25 + 6,25 + 0,25 = 13$) : somme résiduelle 18, contre 1170 avant la coupe (moyenne générale 18,5).

Corrigé 2.6

```

arbres = {"T0": (7, 10), "T1": (5, 12), "T2": (3, 18), "T3": (1, 30)}
tab = pd.DataFrame({"alpha = {a}": {k: R + a * L for k, (L, R) in arbres.items()} for a in (0.5, 2,
↳ 5)})
print(tab.round(1)); print(tab.idxmin().to_dict())

```

	alpha = 0.5	alpha = 2	alpha = 5
T0	13.5	24	45
T1	14.5	22	37
T2	19.5	24	33
T3	30.5	32	35

```

{'alpha = 0.5': 'T0', 'alpha = 2': 'T1', 'alpha = 5': 'T2'}

```

$\alpha = 0,5$: T_0 ($10 + 3,5 = 13,5$) ; $\alpha = 2$: T_1 ($12 + 10 = 22$ contre 24 pour T_0 et 24 pour T_2) ; $\alpha = 5$: T_2 ($18 + 15 = 33$ contre 37 pour T_1 et 35 pour T_3). Les seuils de passage valent $\frac{R(T_{k+1}) - R(T_k)}{|T_k| - |T_{k+1}|}$: de T_0 à T_1 , $\frac{12-10}{7-5} = 1$; de T_1 à T_2 , $\frac{18-12}{5-3} = 3$; de T_2 à T_3 , $\frac{30-18}{3-1} = 6$.

Corrigé 2.7

(a) $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 = 0,2 \times 4 + \frac{0,8 \times 4}{25} = 0,8 + 0,128 = 0,928$. (b) $\rho\sigma^2 = 0,8$. (c) On veut $\frac{(1-\rho)\sigma^2}{B} \leq 0,1\rho\sigma^2$, soit $B \geq \frac{1-\rho}{0,1\rho} = \frac{0,8}{0,02} = 40$ arbres. (d) La limite tombe à $0,1 \times 4 = 0,4$: la décorrélation divise le plancher par deux.

```

f = lambda B, rho, s2: rho * s2 + (1 - rho) * s2 / B
print(f(25, 0.2), f(10**9, 0.2), f(40, 0.2) - 0.8, 0.1 * 0.8, f(10**9, 0.1))

```

```

0.928 0.80000000032000001 0.07999999999999996 0.08000000000000002 0.400000000360000004

```

Corrigé 2.8

(a) $n(1 - (1 - \frac{1}{n})^n) \approx 0,632 \times 9\,000 \approx 5\,689$ clients distincts. (b) Un arbre ne voit pas un client donné avec probabilité 0,368 : $300 \times 0,368 \approx 110$ arbres en moyenne. (c) Il est hors sac pour un arbre avec probabilité 0,368, donc pour **aucun** des 10 avec probabilité $(1 - 0,368)^{10} = 0,632^{10} \approx 0,0102$: environ **1 client sur 100** n'aurait aucune prédiction hors sac.

```
n = 9000; p_out = (1 - 1 / n) ** n
print(round(n * (1 - p_out)), round(300 * p_out, 1), round((1 - p_out) ** 10, 4))
```

```
5689 110.4 0.0102
```

Corrigé 2.9

$F_0 = \frac{2+4+3+11+15}{5} = 7$; résidus $r = (-5; -3; -4; 4; 8)$. Meilleure coupe : $x \leq 3$ (moyennes des résidus -4 à gauche et $+6$ à droite ; le détail de toutes les coupes est donné par le code). Souche : -4 à gauche, $+6$ à droite ; avec $v = 0,5$: $F_1 = 7 - 2 = 5$ pour $x \leq 3$ et $7 + 3 = 10$ pour $x \geq 4$. Nouveaux résidus : $y - F_1 = (-3; -1; -2; 1; 5)$.

```
xs5, ys5 = np.arange(1, 6), np.array([2, 4, 3, 11, 15.0])
r = ys5 - ys5.mean(); sse5 = lambda v: ((v - v.mean()) ** 2).sum()
print("F0 =", ys5.mean(), "| résidus :", r, "| somme des carrés selon la coupe :", {s + 0.5:
↳ round(sse5(r[:s]) + sse5(r[s:]), 2) for s in range(1, 5)})
F1 = np.where(xs5 <= 3, ys5.mean() + 0.5 * r[:3].mean(), ys5.mean() + 0.5 * r[3:].mean()); print("F1
↳ =", F1, "| résidus 2 :", ys5 - F1)
```

```
F0 = 7.0 | résidus : [-5. -3. -4.  4.  8.] | somme des carrés selon la coupe : {1.5:
↳ np.float64(98.75), 2.5: np.float64(76.67), 3.5: np.float64(10.0), 4.5: np.float64(50.0)}
F1 = [ 5.  5.  5. 10. 10.] | résidus 2 : [-3. -1. -2.  1.  5.]
```

La coupe $x \leq 3$ est la meilleure : la somme des carrés des résidus tombe à 10 (gauche : $(-5 + 4)^2 + (-3 + 4)^2 + 0 = 2$; droite : $(4 - 6)^2 + (8 - 6)^2 = 8$), contre 98,8 pour $x \leq 1$, 76,7 pour $x \leq 2$ et 50 pour $x \leq 4$. Au tour 2, le client 5, dont le résidu vaut encore 5, serait la cible de la souche suivante.

Corrigé 2.10

(a) $G = \sum g = -0,5 - 0,5 - 0,4 + 0,5 + 0,4 - 0,6 = -1,1$; $H = 1,47$; $w^* = -\frac{G}{H+\lambda} = \frac{1,1}{3,47} \approx 0,317$.

```
g = np.array([-0.5, -0.5, -0.4, 0.5, 0.4, -0.6]); h = np.array([0.25, 0.25, 0.24, 0.25, 0.24, 0.24]);
↳ lam, gam = 2.0, 0.1
def gain(gauche):
    gl, hl = g[gauche].sum(), h[gauche].sum(); gr, hr = g.sum() - gl, h.sum() - hl
    return 0.5 * (gl ** 2 / (hl + lam) + gr ** 2 / (hr + lam) - g.sum() ** 2 / (h.sum() + lam)) - gam
print("w* =", round(-g.sum() / (h.sum() + lam), 4), "| gain C1 =", round(gain([0, 1, 2]), 4), "| gain
↳ C2 =", round(gain([0, 1, 5]), 4))
```

```
w* = 0.317 | gain C1 = 0.0998 | gain C2 = 0.2386
```

(b) La coupe C1 sépare $\{1, 2, 3\}$ ($G = -1,4$, $H = 0,74$) de $\{4, 5, 6\}$ ($G = 0,3$, $H = 0,73$) : gain 0,100. La coupe C2 sépare $\{1, 2, 6\}$ ($G = -1,6$) de $\{3, 4, 5\}$ ($G = +0,5$) : gain 0,239. (c) On choisit la coupe de **gain maximal**, ici C2 : elle regroupe presque tous les gradients négatifs d'un côté et isole les gradients positifs de l'autre. Elle est **effectuée**, parce que son gain est positif après soustraction de γ .

Corrigé 2.11

(a) Score « parti » : $\frac{8}{20} \times \frac{2}{8} \times \frac{0}{8} = 0$; score « resté » : $\frac{12}{20} \times \frac{9}{12} \times \frac{3}{12} = 0,1125$; probabilité de départ = 0. (b) Un **zéro** dans une probabilité conditionnelle annule tout le produit : le modèle affirme qu'aucun client parti n'a jamais « courriel jamais ouvert », donc exclut le départ **définitivement**, sur la seule base de 8 observations. (c) Avec le lissage : $P(\text{pas de ticket} | \text{parti}) = \frac{2+1}{8+2} = 0,3$, $P(\text{courriel} | \text{parti}) = \frac{0+1}{8+2} = 0,1$, $P(\text{pas de ticket} | \text{resté}) = \frac{10}{14} = 0,714$,

$P(\text{courriel} \mid \text{resté}) = \frac{4}{14} = 0,286$. Scores : parti $0,4 \times 0,3 \times 0,1 = 0,012$; resté $0,6 \times 0,714 \times 0,286 = 0,1224$; probabilité de départ $\frac{0,012}{0,1344} \approx 0,089$.

```
sp = 8 / 20 * 3 / 10 * 1 / 10; sr = 12 / 20 * 10 / 14 * 4 / 14
print(round(sp, 4), round(sr, 4), round(sp / (sp + sr), 3))
```

```
0.012 0.1224 0.089
```

Corrigé 2.12

- (a) La variance de $\frac{e_1 + e_2}{2}$ est $\frac{\sigma^2 + \sigma^2 + 2\rho_e \sigma^2}{4} = \frac{(1 + \rho_e)\sigma^2}{2}$: 0,5 pour $\rho_e = 0$, 0,75 pour $\rho_e = 0,5$, 0,95 pour $\rho_e = 0,9$: plus les erreurs sont corrélées, moins on gagne (au mieux, on divise par 2).
- (b) La moyenne égale a pour variance $\frac{1+4}{4} = 1,25$, **plus** que le premier modèle seul (1) : mélanger avec un modèle nettement plus mauvais fait **perdre**. La pondération ($w, 1 - w$) minimise $w^2 \cdot 1 + (1 - w)^2 \cdot 4$ pour $w = \frac{4}{5} = 0,8$ (poids inversement proportionnel aux variances), avec une variance de 0,8.

```
rng = np.random.default_rng(0); e1, e2 = rng.normal(0, 1, 100000), rng.normal(0, 2, 100000)
print("seul :", round(e1.var(), 3), "| moyenne égale :", round(((e1 + e2) / 2).var(), 3), "| pondérée ↵
↵ 0,8/0,2 :", round((0.8 * e1 + 0.2 * e2).var(), 3))
```

```
seul : 1.0 | moyenne égale : 1.256 | pondérée 0,8/0,2 : 0.802
```

Chapitre 3 : Apprentissage non supervisé et réduction de dimension — exercices et applications

Ce chapitre du cahier accompagne le chapitre 3 du livre. Les **applications** reprennent, pas à pas et avec le code, les études du livre (segmenter, valider, réduire, dessiner) ; les **exercices** se travaillent d'abord à la main, puis se vérifient par le calcul. Données : les 12 000 clients simulés de la boutique (`donnees/clients_ml.csv`) et les chiffres manuscrits de `scikit-learn` (jeu **réel** embarqué). Prérequis : le chapitre 3 du livre ; Python avec NumPy, pandas, `scikit-learn`.

Applications

Préparation commune

À exécuter une fois : elle charge les clients, prépare les sept variables de comportement (le montant en logarithme, puis centrage-réduction) et importe les outils. Les applications 3.1 à 3.3 et 3.5 à 3.8 la réutilisent.

```
import warnings
import numpy as np
import pandas as pd
warnings.filterwarnings("ignore")
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import (silhouette_score, calinski_harabasz_score, davies_bouldin_score,
                             adjusted_rand_score, normalized_mutual_info_score)

c = pd.read_csv("donnees/clients_ml.csv")
cols = ["age", "nb_commandes_12m", "montant_12m", "recence_jours",
        "part_achats_promo", "taux_ouverture_email", "nb_promos_recues_12m"]
X = c[cols].copy()
X["montant_12m"] = np.log1p(X["montant_12m"])      # montant très asymétrique : logarithme
Z = StandardScaler().fit_transform(X)
print(Z.shape, "| moyennes ~ 0 :", Z.mean(0).round(2).max(), "| écarts-types = 1 :",
      ↪ Z.std(0).round(2).min())
```

```
(12000, 7) | moyennes ~ 0 : -0.0 | écarts-types = 1 : 1.0
```

Application 3.1 — Segmenter la clientèle de bout en bout (sections 3.1.1 à 3.1.7)

Contexte. La gérante veut des groupes de clients pour adapter ses envois. **Objectif.** Passer de la table brute à une segmentation justifiée : choix de k par plusieurs critères, stabilité, profils, utilité.

Étape 1 — Balayer k avec trois critères internes. Pour chaque k de 2 à 8, on ajuste k-means et on calcule silhouette (sur 4 000 clients), Calinski–Harabasz et Davies–Bouldin.

```
lignes, modeles = [], {}
for k in range(2, 9):
    km = KMeans(k, n_init=10, random_state=0).fit(Z)
    modeles[k] = km
    lignes.append({"k": k, "silhouette": round(silhouette_score(Z, km.labels_, sample_size=4000,
    ↪ random_state=0), 3),
                  "CH": round(calinski_harabasz_score(Z, km.labels_)), "DB":
    ↪ round(davies_bouldin_score(Z, km.labels_), 3)})
critere = pd.DataFrame(lignes)
print(critere.to_string(index=False))
```

k	silhouette	CH	DB
2	0.304	4041	1.193
3	0.331	5040	1.118
4	0.284	5135	1.271
5	0.261	4678	1.311
6	0.244	4241	1.321
7	0.231	3825	1.428
8	0.226	3520	1.334

Lecture. La silhouette et Davies–Bouldin préfèrent $k = 3$ (0,331 et 1,118) ; Calinski–Harabasz culmine à $k = 4$ (5 135). Aucun critère ne tranche seul : reprenez une plage (3 à 5) et départagez avec la stabilité et l'utilité.

Étape 2 — Stabilité. On tire 15 sous-échantillons de 80 %, on réajuste, et on compare au modèle de référence avec l'ARI.

```
def stabilite(A, k, B=15):
    r = np.random.default_rng(0)
    ref = KMeans(k, n_init=10, random_state=0).fit(A)
    sc = []
    for b in range(B):
        idx = r.choice(len(A), int(0.8 * len(A)), replace=False)
        km = KMeans(k, n_init=5, random_state=b).fit(A[idx])
        sc.append(adjusted_rand_score(ref.predict(A[idx]), km.labels_))
    return round(float(np.mean(sc)), 3), round(float(np.std(sc)), 3)
print({k: stabilite(Z, k) for k in (3, 4, 5, 6)})
```

```
{3: (0.996, 0.004), 4: (0.986, 0.007), 5: (0.981, 0.016), 6: (0.964, 0.029)}
```

Étape 3 — Profils et utilité. On retient $k = 4$ et on regarde, pour chaque groupe, sa taille, deux variables de comportement et le départ à 90 jours (que la segmentation n'a pas vu).

```
c["groupe"] = modeles[4].labels_
profil = c.groupby("groupe").agg(part=("age", lambda s: len(s) / len(c)),
    ↪ commandes=("nb_commandes_12m", "mean"),
    ↪ "mean"),
    ↪ recence=("recence_jours", "mean"), part_promo=("part_achats_promo",
    ↪ "mean"),
    ↪ depart=("churn_90j", "mean"), depense_6m=("depense_6m",
    ↪ "mean")).round(3)
print(profil.to_string())
```

	part	commandes	recence	part_promo	depart	depense_6m
groupe						
0	0.186	4.283	62.151	0.746	0.198	52.372

1	0.296	7.305	39.021	0.191	0.010	168.239
2	0.145	0.108	363.855	0.187	0.413	22.161
3	0.374	2.276	91.661	0.158	0.110	57.801

Lecture. Quatre profils nets. Le groupe 1 (30 % des clients) réunit les fidèles : 7,3 commandes, récurrence de 39 jours, départ à 1,0 %, dépense à 6 mois de 168 €. Le groupe 2 (14,5 %) réunit les dormants : 0,11 commande, récurrence de 364 jours, départ à 41,3 %. Le groupe 0 est celui des chasseurs de promotions (74,6 % des achats en promotion) et le groupe 3, le plus nombreux (37,4 %), celui des occasionnels réguliers. Stabilité (étape 2) : l'ARI moyen vaut 0,996, 0,986, 0,981 et 0,964 pour $k = 3, \dots, 6$: tous ces découpages sont reproductibles, donc la stabilité ne dépend pas de 3, 4, 5 et 6.

Pour aller plus loin. Reprenez l'étape 1 avec `random_state=1` : les critères et le k retenu changent-ils ? Recommencez avec seulement `age`, `recence_jours` et `nb_commandes_12m` : que deviennent les profils ?

Application 3.2 — Valider contre la vérité cachée : initialisation, échelle, transformation (sections 3.1.5 et 3.1.6)

Objectif. Mesurer, avec `segment_vrai`, ce qui change l'ARI : le hasard de l'initialisation, l'échelle, la transformation du montant.

Étape 1 — Le hasard de l'initialisation. Vingt graines, avec une seule initialisation par ajustement (`n_init=1`), puis avec dix.

```
def ari_graines(A, n_init, graines=range(20)):
    out = [adjusted_rand_score(c["segment_vrai"], KMeans(4, n_init=n_init,
        random_state=s).fit_predict(A)) for s in graines]
    return round(float(np.mean(out)), 3), round(float(np.min(out)), 3), round(float(np.max(out)), 3)
print("n_init=1 (moyenne, min, max) :", ari_graines(Z, 1))
print("n_init=10 (moyenne, min, max) :", ari_graines(Z, 10, range(5)))
```

```
n_init=1 (moyenne, min, max) : (0.486, 0.482, 0.489)
n_init=10 (moyenne, min, max) : (0.487, 0.486, 0.489)
```

Lecture. Surprise : sur ces données bien structurées, l'initialisation compte **très peu** : avec une seule initialisation, l'ARI moyen est 0,486 (de 0,482 à 0,489 sur 20 graines) ; avec dix, 0,487. Ne généralisez pas : sur des données moins nettes, `n_init=1` donne des résultats bien plus dispersés. Gardez `n_init=10` (c'est le coût d'une assurance bon marché).

Étape 2 — Échelle et transformation. On compare quatre préparations des mêmes variables.

```
from sklearn.preprocessing import RobustScaler, MinMaxScaler
brut = c[cols].to_numpy(dtype=float)
prepas = {"standardisation (log du montant)": Z,
          "min-max (log du montant)": MinMaxScaler().fit_transform(X),
          "robuste (log du montant)": RobustScaler().fit_transform(X),
          "standardisation, montant brut": StandardScaler().fit_transform(brut)}
for nom, A in prepas.items():
    lab = KMeans(4, n_init=10, random_state=0).fit_predict(A)
    print(f"{nom:36s} ARI = {adjusted_rand_score(c['segment_vrai'], lab):.3f} NMI = {normalized_mutual_info_score(c['segment_vrai'], lab):.3f}")
```

```
standardisation (log du montant)  ARI = 0.487  NMI = 0.535
min-max (log du montant)         ARI = 0.502  NMI = 0.545
robuste (log du montant)         ARI = 0.397  NMI = 0.498
standardisation, montant brut    ARI = 0.354  NMI = 0.490
```

Lecture. L'échelle pèse beaucoup plus que l'initialisation : l'ARI va de 0,354 (montant brut) à

0,502 (min-max). La standardisation (0,487) n'est pas la meilleure ici : le min-max fait légèrement mieux (0,502) et l'échelle **robuste** nettement moins bien (0,397), parce qu'elle ne réduit pas les valeurs extrêmes de `nb_promos_recues_12m` et de `recence_jours`. Il n'existe pas d'échelle universelle : **testez plusieurs préparations** et jugez-les avec les épreuves de 3.1.

Application 3.3 — La statistique de l'écart, écrite à la main (section 3.1.3)

Objectif. Programmer la statistique de l'écart et comparer deux références. Pour limiter le temps de calcul, on travaille sur 3 000 clients tirés au hasard.

Étape 1 — La fonction. Pour chaque k de 1 à 8, on calcule $\log W_k$ sur les données, puis sur B jeux de référence ; l'écart est la différence des moyennes, et la règle de Tibshirani choisit le plus petit k tel que $\text{Gap}(k) \geq \text{Gap}(k+1) - s_{k+1}$.

```
def log_w(A, k):
    return np.log(KMeans(k, n_init=3, random_state=0).fit(A).inertia_)

def ecart(A, reference, ks=range(1, 9), B=10, graine=0):
    r = np.random.default_rng(graine)
    lw = np.array([log_w(A, k) for k in ks])
    ref = np.array([log_w(reference(A, r), k) for k in ks for _ in range(B)])
    gap, s = ref.mean(0) - lw, ref.std(0) * np.sqrt(1 + 1 / B)
    ks = list(ks)
    choix = next((k for i, k in enumerate(ks[:-1]) if gap[i] >= gap[i + 1] - s[i + 1]), ks[-1])
    return gap.round(3), choix
```

Étape 2 — Deux références. La permutation des colonnes garde les distributions marginales ; la boîte uniforme ne les garde pas.

```
sous = Z[np.random.default_rng(0).choice(len(Z), 3000, replace=False)]
perm = lambda A, r: np.column_stack([r.permutation(A[:, j]) for j in range(A.shape[1])])
unif = lambda A, r: r.uniform(A.min(0), A.max(0), A.shape)
for nom, ref in ("permutation", perm), ("boîte uniforme", unif)):
    gap, choix = ecart(sous, ref)
    print(f"{nom:15s} gap = {gap} -> k retenu : {choix}")
```

```
permutation    gap = [0.    0.17  0.374 0.486 0.503 0.484 0.447 0.44 ] -> k retenu : 5
boîte uniforme gap = [0.838 0.806 1.011 1.122 1.177 1.207 1.2   1.217] -> k retenu : 1
```

Lecture. Avec la permutation, l'écart augmente puis s'aplatit : il culmine à $k = 5$ (0,503) et la règle retient 5. Avec la boîte uniforme, l'écart est déjà de 0,838 pour **un seul groupe** et ne présente pas de maximum net ; ici la règle s'arrête à $k = 1$ (dans le livre, avec d'autres tirages de référence, elle s'arrête à $k = 6$: le choix **n'est pas stable**). C'est la preuve que cette référence est inadaptée à des variables asymétriques et corrélées.

Pour aller plus loin. Écrivez une troisième référence : tirer des points uniformes dans la boîte englobante des données **après rotation par ACP** (variante de Tibshirani), puis tourner en sens inverse. Change-t-elle le k retenu ?

Application 3.4 — Réduire les chiffres manuscrits : ACP, noyau, NMF, projection aléatoire (sections 3.2.2 à 3.2.6)

Objectif. Comparer cinq réductions à nombre de dimensions égal, par la précision d'un classifieur (5 plus proches voisins) en validation croisée. **Précaution** : la réduction fait partie du modèle ; on l'insère donc **dans** un Pipeline, pour qu'elle soit réajustée sur chaque pli d'entraînement.

```
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA, KernelPCA, NMF, TruncatedSVD
```

```

from sklearn.random_projection import GaussianRandomProjection
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score
d = load_digits(); Xd, yd = d.data / 16.0, d.target
reductions = {"ACP": lambda m: PCA(m), "SVD tronquée": lambda m: TruncatedSVD(m, random_state=0),
              "ACP à noyau (rbf, gamma=0,02)": lambda m: KernelPCA(m, kernel="rbf", gamma=0.02),
              "NMF": lambda m: NMF(m, init="nndsvda", random_state=0, max_iter=400),
              "projection aléatoire": lambda m: GaussianRandomProjection(m, random_state=0)}

lignes = []
for nom, f in reductions.items():
    ligne = {"méthode": nom}
    for m in (5, 10, 20, 40):
        ligne[f"m={m}"] = round(cross_val_score(make_pipeline(f(m), KNeighborsClassifier(5)), Xd, yd,
        ↪ cv=5).mean(), 3)
    lignes.append(ligne)
print(pd.DataFrame(lignes).to_string(index=False))

```

```

           méthode  m=5  m=10  m=20  m=40
           ACP  0.884  0.940  0.958  0.962
SVD tronquée  0.844  0.935  0.959  0.962
ACP à noyau (rbf, gamma=0,02)  0.883  0.943  0.957  0.962
           NMF  0.795  0.824  0.843  0.851
projection aléatoire  0.586  0.757  0.905  0.934

```

Lecture. À $m = 40$, l'ACP, la SVD tronquée et l'ACP à noyau atteignent la même précision (0,962) ; pour $m = 5$, l'ACP (0,884) et l'ACP à noyau (0,883) devancent la SVD tronquée (0,844), qui gaspille une direction sur la moyenne. La **NMF** plafonne à 0,851 : sa contrainte de positivité rend les composantes lisibles, au prix d'une information moins bien conservée. La **projection aléatoire** est la plus faible en petite dimension (0,586 à $m = 5$) mais rattrape son retard avec m (0,934 à 40) : conforme à Johnson–Lindenstrauss, qui demande beaucoup de dimensions.

Pour aller plus loin. Faites varier gamma de l'ACP à noyau (0,002, 0,02, 0,2) : à partir de quelle valeur la méthode se dégrade-t-elle, et pourquoi ?

Application 3.5 — Quand les groupes ne sont pas des boules : DBSCAN et liens (sections 3.3.1 et 3.3.2)

Objectif. Explorer la grille (ϵ, m) de DBSCAN sur deux formes classiques, et comparer avec les liens hiérarchiques.

```

from sklearn.datasets import make_moons, make_circles
from sklearn.cluster import DBSCAN, AgglomerativeClustering
jeux = {"lunes": make_moons(600, noise=0.07, random_state=0), "cercles": make_circles(600,
↪ factor=0.4, noise=0.05, random_state=0)}
for nom, (A, y) in jeux.items():
    print(nom, "| k-means :", round(adjusted_rand_score(y, KMeans(2, n_init=10,
    ↪ random_state=0).fit_predict(A)), 3))
    for eps in (0.08, 0.12, 0.18, 0.25):
        ligne = [round(adjusted_rand_score(y, DBSCAN(eps=eps, min_samples=m).fit_predict(A)), 2) for m
        ↪ in (3, 5, 10, 20)]
        print(f"   epsilon = {eps:4.2f}   ARI pour min_samples = 3, 5, 10, 20 :", ligne)

```

```

lunes | k-means : 0.252
  epsilon = 0.08 ARI pour min_samples = 3, 5, 10, 20 : [0.73, 0.54, 0.07, 0.0]
  epsilon = 0.12 ARI pour min_samples = 3, 5, 10, 20 : [1.0, 1.0, 0.97, 0.08]
  epsilon = 0.18 ARI pour min_samples = 3, 5, 10, 20 : [1.0, 1.0, 1.0, 0.99]
  epsilon = 0.25 ARI pour min_samples = 3, 5, 10, 20 : [1.0, 1.0, 1.0, 1.0]
cercles | k-means : -0.001
  epsilon = 0.08 ARI pour min_samples = 3, 5, 10, 20 : [0.55, 0.52, 0.97, 0.01]
  epsilon = 0.12 ARI pour min_samples = 3, 5, 10, 20 : [0.99, 0.99, 0.54, 1.0]

```

```
epsilon = 0.18 ARI pour min_samples = 3, 5, 10, 20 : [1.0, 1.0, 1.0, 0.88]
epsilon = 0.25 ARI pour min_samples = 3, 5, 10, 20 : [1.0, 1.0, 1.0, 1.0]
```

Lecture. Sur les lunes comme sur les cercles, k-means échoue (0,252 et -0,001). DBSCAN réussit ($ARI \approx 1$) dès que ε est assez grand, mais la bonne valeur dépend de `min_samples` : sur les lunes, $\varepsilon = 0,12$ convient pour $m \leq 10$ mais s'effondre à $m = 20$ (0,08) ; sur les cercles, $\varepsilon = 0,08$ avec $m = 10$ donne 0,97 alors que $m = 20$ donne 0,01. Un voisinage trop exigeant fait disparaître les points de bordure : plus m est grand, plus il faut agrandir ε .

```
for nom, (A, y) in jeux.items():
    res = {l: round(adjusted_rand_score(y, AgglomerativeClustering(2, linkage=l).fit_predict(A)), 3)
    for l in ("single", "complete", "average", "ward")}
    print(nom, res)
```

```
lunes {'single': 1.0, 'complete': 0.426, 'average': 0.557, 'ward': 0.557}
cercles {'single': 1.0, 'complete': 0.236, 'average': 0.115, 'ward': 0.001}
```

Lecture. Le lien **simple** retrouve parfaitement les deux formes (1,0 dans les deux cas), les autres échouent, surtout sur les cercles où le lien de Ward donne 0,001. Le lien simple suit des chaînes de points proches ; il serait détruit par un peu de bruit reliant les deux formes (testez en ajoutant 20 points au hasard).

Application 3.6 — L'algorithme EM, de la main à la bibliothèque (section 3.3.3)

Objectif. Écrire EM pour un mélange de deux lois normales en dimension 1, vérifier que la vraisemblance ne baisse jamais, puis comparer à `GaussianMixture`.

Étape 1 — Données et algorithme. Cent valeurs tirées de $\mathcal{N}(0;1)$ (40 %) et $\mathcal{N}(4;1,5^2)$ (60 %).

```
from scipy.stats import norm
r = np.random.default_rng(1)
x = np.concatenate([r.normal(0, 1, 40), r.normal(4, 1.5, 60)])
def em(x, mu, sd, pi, tours=40):
    ll = []
    for _ in range(tours):
        dens = pi * norm.pdf(x[:, None], mu, sd) # étape E
        ll.append(np.log(dens.sum(1)).sum()); g = dens / dens.sum(1, keepdims=True)
        N = g.sum(0); mu = (g * x[:, None]).sum(0) / N # étape M
        sd = np.sqrt((g * (x[:, None] - mu) ** 2).sum(0) / N); pi = N / len(x)
    return mu, sd, pi, np.array(ll)
mu, sd, pi, ll = em(x, np.array([1.0, 3.0]), np.array([1.0, 1.0]), np.array([0.5, 0.5]))
print("mu =", mu.round(3), "sd =", sd.round(3), "pi =", pi.round(3), "| la vraisemblance ne baisse
↳ jamais :", bool(np.all(np.diff(ll) >= -1e-9)))
```

```
mu = [0.358 4.135] sd = [1.166 0.933] pi = [0.489 0.511] | la vraisemblance ne baisse jamais : True
```

Étape 2 — Comparaison avec la bibliothèque.

```
from sklearn.mixture import GaussianMixture
g = GaussianMixture(2, n_init=5, random_state=0).fit(x.reshape(-1, 1))
o = np.argsort(g.means_.ravel())
print("sklearn (tol=1e-3 par défaut, %d tours) : mu =" % g.n_iter_, g.means_.ravel()[o].round(3), "sd
↳ =", np.sqrt(g.covariances_.ravel()[o].round(3), "pi =", g.weights_[o].round(3))
g = GaussianMixture(2, n_init=5, random_state=0, tol=1e-9, max_iter=2000).fit(x.reshape(-1, 1)); o =
↳ np.argsort(g.means_.ravel())
print("sklearn (tol=1e-9, %d tours) : mu =" % g.n_iter_, g.means_.ravel()[o].round(3), "sd
↳ =", np.sqrt(g.covariances_.ravel()[o].round(3), "pi =", g.weights_[o].round(3))
print("EM écrit à la main, 2 000 tours : mu =", em(x, np.array([1.0, 3.0]), np.array([1.0,
↳ 1.0]), np.array([0.5, 0.5]), 2000)[0].round(3))
```

```
sklearn (tol=1e-3 par défaut, 3 tours) : mu = [0.331 4.115] sd = [1.147 0.947] pi = [0.483 0.517]
sklearn (tol=1e-9, 132 tours)           : mu = [0.386 4.157] sd = [1.186 0.917] pi = [0.495 0.505]
EM écrit à la main, 2 000 tours         : mu = [0.387 4.157]
```

Lecture. À 40 tours, le EM écrit à la main donne $\mu = (0,358 ; 4,135)$ et la vraisemblance ne baisse jamais, comme la théorie le garantit. La bibliothèque donne $(0,331 ; 4,115)$ avec sa tolérance par défaut (elle s'arrête après trois tours, dès que l'amélioration passe sous 10^{-3}) ; avec $\text{tol}=1e-9$, elle converge vers $\mu = (0,386 ; 4,157)$, **exactement ce que donne notre EM après 2 000 tours** $(0,387 ; 4,157)$. Les vraies valeurs étaient 0 et 4 : l'estimation est correcte, avec l'erreur d'un échantillon de 100 points. Pour le BIC sur les clients (étape 3), la chute est forte de $k = 3$ à $k = 4$ ($31\,244 \rightarrow 28\,484$), puis lente ; le minimum est atteint à $k = 7$ ($27\,577$) sur ce sous-échantillon de 3 000 clients : le BIC ne désigne pas le même nombre que les critères de 3.1, ce qui illustre à nouveau qu'un critère n'est pas une vérité.

Étape 3 — BIC sur les clients. On ajuste des mélanges à covariance complète, de 1 à 8 composantes, sur les 3 000 clients de l'application 3.3.

```
for k in range(1, 9):
    gm = GaussianMixture(k, n_init=3, random_state=0).fit(sous)
    print(k, round(gm.bic(sous)))
```

```
1 52124
2 45583
3 31244
4 28484
5 28113
6 27944
7 27577
8 27620
```

Application 3.7 — t-SNE et UMAP : perplexité, voisins, fiabilité (sections 3.4.1 à 3.4.4)

Objectif. Mesurer l'effet des hyperparamètres sur la fiabilité et sur la précision d'un classifieur entraîné sur les coordonnées du dessin. On travaille sur 600 chiffres pour que le calcul reste court.

```
from sklearn.manifold import TSNE, trustworthiness
import umap
idx = np.random.default_rng(0).choice(len(Xd), 600, replace=False)
Xs, ys = Xd[idx], yd[idx]
def bilan(E):
    return round(trustworthiness(Xs, E, n_neighbors=10), 3),
    ↪ round(cross_val_score(KNeighborsClassifier(5), E, ys, cv=5).mean(), 3)
lignes = []
for perp in (5, 30, 60):
    lignes.append({"méthode": f"t-SNE, perplexité {perp}", "fiabilité, précision": bilan(TSNE(2,
    ↪ perplexity=perp, random_state=0, init="pca").fit_transform(Xs))})
for nv in (5, 15, 40):
    lignes.append({"méthode": f"UMAP, {nv} voisins", "fiabilité, précision":
    ↪ bilan(umap.UMAP(n_neighbors=nv, random_state=0).fit_transform(Xs))})
print(pd.DataFrame(lignes).to_string(index=False))
```

méthode	fiabilité, précision
t-SNE, perplexité 5	(0.984, 0.985)
t-SNE, perplexité 30	(0.985, 0.97)
t-SNE, perplexité 60	(0.984, 0.975)
UMAP, 5 voisins	(0.982, 0.982)
UMAP, 15 voisins	(0.983, 0.968)
UMAP, 40 voisins	(0.983, 0.962)

Lecture. La fiabilité est quasi identique partout (0,982 à 0,985) : tous les réglages conservent bien

les voisinages. La précision, elle, est la plus haute pour les plus petits voisinages (0,985 pour t-SNE à perplexité 5 contre 0,970 à 30 et 0,975 à 60 ; 0,982 pour UMAP à 5 voisins, puis 0,968 et 0,962 à 15 et 40) : un petit voisinage sépare mieux les classes **sur le dessin** même si la structure globale est moins bien rendue. Rappel : ces précisions sont optimistes (plongement transductif).

Mise en garde. La précision d'un classifieur sur les coordonnées d'un dessin est **optimiste** : le plongement a vu **toutes** les données, y compris celles du pli de test (c'est un plongement *transductif*). Pour une mesure honnête, ajustez UMAP sur le pli d'entraînement seulement et projetez le pli de test avec `transform`.

Application 3.8 — Réduire avant de classer : un pipeline sur les clients (section 3.2.8)

Objectif. Mesurer l'effet du nombre de composantes gardées sur la segmentation, avec trois épreuves : la silhouette dans l'espace réduit, l'ARI avec la vérité cachée et la **séparation du départ** entre groupes (utilité).

```
from sklearn.pipeline import make_pipeline
lignes = []
for m in (2, 3, 4, 5, 7):
    pipe = make_pipeline(PCA(m), KMeans(4, n_init=10, random_state=0)).fit(Z)
    lab = pipe.predict(Z); R = pipe[0].transform(Z)
    depart = c.groupby(lab)["churn_90j"].mean()
    lignes.append({"composantes": m, "variance gardée":
        round(pipe[0].explained_variance_ratio_.sum(), 3),
        "silhouette": round(silhouette_score(R, lab, sample_size=4000, random_state=0), 3),
        "ARI": round(adjusted_rand_score(c["segment_vrai"], lab), 3),
        "départ min - max": f"{depart.min():.3f} - {depart.max():.3f}"})
print(pd.DataFrame(lignes).to_string(index=False))
```

composantes	variance gardée	silhouette	ARI	départ min - max
2	0.641	0.467	0.480	0.009 - 0.418
3	0.762	0.381	0.489	0.013 - 0.409
4	0.857	0.342	0.502	0.012 - 0.414
5	0.932	0.312	0.479	0.010 - 0.412
7	1.000	0.284	0.487	0.010 - 0.413

Lecture. Garder 4 composantes (85,7 % de la variance) donne le meilleur ARI (0,502) ; 2 composantes suffisent presque (0,480). La séparation du départ entre groupes reste **stable** quel que soit m (de 1 % environ à 41 %) : pour la décision, la réduction ne coûte rien. Attention à la colonne silhouette : elle **décroît** avec m (0,467 $\rightarrow$ 0,284), non parce que les groupes se dégradent, mais parce que les distances se resserrent en dimension plus grande (3.2.1) : on ne compare pas des silhouettes calculées dans des espaces de dimensions différentes.

Exercices

Exercice 3.1 ★ — Silhouette à la main (section 3.1.2)

Cinq points sur une droite : 1, 3, 7, 9, 11, partagés en $A = \{1, 3\}$ et $B = \{7, 9, 11\}$. Calculez la silhouette de chaque point et la silhouette moyenne.

Exercice 3.2 ★★ — Calinski–Harabasz et Davies–Bouldin à la main (section 3.1.2)

Pour la même partition que l'exercice 3.1, calculez la dispersion intra W , la dispersion inter B , l'indice de Calinski–Harabasz et l'indice de Davies–Bouldin (avec la distance moyenne au centre comme s_j).

Exercice 3.3 ★★ — L'indice de Rand ajusté à partir d'un tableau croisé (section 3.1.5)

Un algorithme classe 70 clients en trois groupes, alors que la vérité compte trois classes. Le tableau croisé (lignes : groupes trouvés ; colonnes : vraies classes) est

$$\begin{pmatrix} 20 & 5 & 0 \\ 3 & 15 & 2 \\ 0 & 4 & 21 \end{pmatrix}.$$

Calculez l'ARI et la pureté.

Exercice 3.4 ★★ — L'inertie ne peut que décroître (section 3.1.1)

Montrez que l'inertie minimale W_k^* de k-means vérifie $W_{k+1}^* \leq W_k^*$, puis expliquez pourquoi cette monotonie interdit de choisir k en minimisant l'inertie. Vérifiez numériquement sur les clients.

Exercice 3.5 ★★ — Appliquer la règle de Tibshirani (section 3.1.3)

Une statistique de l'écart a donné, pour $k = 1, \dots, 6$: $\text{Gap} = (0; 0,30; 0,62; 0,75; 0,77; 0,74)$ et $s = (0,01; 0,01; 0,01; 0,02; 0,04; 0,02)$. Quel k retient la règle « plus petit k tel que $\text{Gap}(k) \geq \text{Gap}(k+1) - s_{k+1}$ » ? Quel est l'argmax de l'écart ? Pourquoi les deux diffèrent-ils ?

Exercice 3.6 ★ — Part de variance et erreur de reconstruction (section 3.2.2)

Les valeurs propres de la matrice de corrélation de six variables sont 4,2 ; 2,1 ; 0,9 ; 0,5 ; 0,2 ; 0,1. Quelle part de variance gardent 1, 2, 3 composantes ? Combien de composantes pour garder au moins 90 % ? Quelle est l'erreur relative de reconstruction avec 2 composantes ?

Exercice 3.7 ★★ — Le lemme de Johnson–Lindenstrauss en chiffres (section 3.2.6)

On a $n = 10\,000$ observations décrites par $p = 50\,000$ variables. Quelle dimension k garantit, d'après la borne du livre, une conservation des distances à $\pm\varepsilon$ pour $\varepsilon = 0,25$ puis $\varepsilon = 0,1$? Commentez le résultat par rapport à p et à n .

Exercice 3.8 ★★★ — La concentration des distances, démontrée (section 3.2.1)

Soient $\mathbf{X}, \mathbf{Y}$ deux points indépendants, uniformes dans $[0, 1]^d$. (a) Calculez l'espérance et la variance de $D^2 = \|\mathbf{X} - \mathbf{Y}\|^2$. (b) Montrez que $\sqrt{\text{Var}(D^2)}/\mathbb{E}[D^2] = \sqrt{1,4/d}$. (c) En déduire, par la méthode delta, la dispersion relative de D , et la comparer à celle du livre en dimension 50 et 1 000.

Exercice 3.9 ★★ — DBSCAN à la main en dimension 2 (section 3.3.1)

Neuf points : $P_1(0;0), P_2(1;0), P_3(0;1), P_4(1;1), P_5(2;1,5), P_6(5;5), P_7(5,5;5), P_8(5;5,5), P_9(10;0)$. Avec $\varepsilon = 1,2$ et $m = 3$ (le point compte dans son voisinage), classez chaque point (cœur, bordure, bruit) et donnez les groupes.

Exercice 3.10 ★★ — Un tour d'EM (section 3.3.3)

Quatre valeurs 0 ; 2 ; 3 ; 6, un mélange de deux lois normales de même écart-type 1 et de poids égaux, de moyennes initiales $\mu_1 = 0$ et $\mu_2 = 6$. Calculez les responsabilités, puis les nouvelles moyennes et poids après un tour.

Exercice 3.11 ★★ — Compter les paramètres, calculer un BIC (section 3.3.3)

- (a) Combien de paramètres libres a un mélange gaussien à covariances complètes de K composantes en dimension p ? Évaluez pour $p = 7$, $K = 4$ puis $K = 5$. (b) Avec $n = 12\,000$, $\ln \hat{L}_4 = -52\,500$ et $\ln \hat{L}_5 = -51\,300$, quel K minimise le BIC ? (c) De combien devrait au moins augmenter la log-vraisemblance quand on passe de $K = 4$ à $K = 5$ pour que le BIC préfère $K = 5$?

Exercice 3.12 ★★ — La perplexité de t-SNE (section 3.4.1)

La probabilité de choisir comme voisin chacun des quatre voisins d'un point est (0,5 ; 0,25 ; 0,125 ; 0,125). (a) Calculez son entropie en bits puis sa perplexité. (b) Quelle est la perplexité d'une loi uniforme sur m voisins ? (c) Interprétez : que signifie « choisir la perplexité 30 » ?

Corrigés**Corrigé 3.1**

- Point 1 : $a = |1 - 3| = 2$; $b = (6 + 8 + 10)/3 = 8$; $s = 1 - 2/8 = 0,75$.
- Point 3 : $a = 2$; $b = (4 + 6 + 8)/3 = 6$; $s = 1 - 2/6 \approx 0,667$.
- Point 7 : $a = (|7 - 9| + |7 - 11|)/2 = (2 + 4)/2 = 3$; $b = (|7 - 1| + |7 - 3|)/2 = (6 + 4)/2 = 5$; $s = 1 - 3/5 = 0,4$.
- Point 9 : $a = (2 + 2)/2 = 2$; $b = (8 + 6)/2 = 7$; $s = 1 - 2/7 \approx 0,714$.
- Point 11 : $a = (4 + 2)/2 = 3$; $b = (10 + 8)/2 = 9$; $s = 1 - 3/9 \approx 0,667$.

Silhouette moyenne : $(0,75 + 0,667 + 0,4 + 0,714 + 0,667)/5 \approx 0,640$. Vérification :

```
from sklearn.metrics import silhouette_samples
pts = np.array([[1.], [3.], [7.], [9.], [11.]]) ; lab = np.array([0, 0, 1, 1, 1])
print(silhouette_samples(pts, lab).round(3), silhouette_samples(pts, lab).mean().round(3))
```

```
[0.75  0.667 0.4   0.714 0.667] 0.64
```

Corrigé 3.2

Les moyennes sont 2 (groupe A) et 9 (groupe B) ; la moyenne générale est $31/5 = 6,2$. $W = (1 + 1) + (4 + 0 + 4) = 10$; $B = 2(2 - 6,2)^2 + 3(9 - 6,2)^2 = 35,28 + 23,52 = 58,8$ (on vérifie $W + B = 68,8 = \sum (x_i - 6,2)^2$). $CH = \frac{58,8/(2-1)}{10/(5-2)} = 17,64$. Distances moyennes au centre : $s_A = 1$, $s_B = (2 + 0 + 2)/3 \approx 1,333$; distance entre centres : 7 ; $DB = (1 + 1,333)/7 \approx 0,333$.

```
print(round(calinski_harabasz_score(pts, lab), 2), round(davies_bouldin_score(pts, lab), 3))
```

```
17.64 0.333
```

Corrigé 3.3

$\sum_{ij} \binom{n_{ij}}{2} = \binom{20}{2} + \binom{5}{2} + \binom{3}{2} + \binom{15}{2} + \binom{2}{2} + \binom{4}{2} + \binom{21}{2} = 190 + 10 + 3 + 105 + 1 + 6 + 210 = 525$.
Totaux des lignes 25, 20, 25 : $\sum_i \binom{a_i}{2} = 300 + 190 + 300 = 790$. Totaux des colonnes 23, 24, 23 :
 $\sum_j \binom{b_j}{2} = 253 + 276 + 253 = 782$. $\binom{70}{2} = 2415$. Espérance au hasard : $790 \times 782 / 2415 \approx 255,8$;
maximum : $(790 + 782)/2 = 786$. $ARI = \frac{525 - 255,8}{786 - 255,8} \approx 0,508$. Pureté : $(20 + 15 + 21)/70 = 0,8$:
la pureté (80 %) paraît bien meilleure que l'ARI, qui retire la part due au hasard.

```
M = np.array([[20, 5, 0], [3, 15, 2], [0, 4, 21]])
lig, col = np.indices((3, 3))
trouve, vrai = np.repeat(lig.ravel(), M.ravel()), np.repeat(col.ravel(), M.ravel()) # une ligne par
↪ client
print(round(adjusted_rand_score(vrai, trouve), 3), M.max(axis=1).sum() / M.sum())
```

```
0.508 0.8
```

(Attention : la pureté prend le maximum de chaque **ligne** ; ici 20, 15, 21.)

Corrigé 3.4

Preuve. Soit $(C_1, \dots, C_k; \mu_1, \dots, \mu_k)$ une solution optimale à k groupes, d'inertie W_k^* . Si un groupe contient au moins deux points distincts, on en retire un point x du groupe j pour en faire un groupe à lui seul, de centre x : l'inertie de ce point devient 0, et celle du groupe j , recalculée avec sa nouvelle moyenne, ne peut pas augmenter (la moyenne minimise la somme des carrés). On obtient une partition à $k + 1$ groupes d'inertie $\leq W_k^*$, donc $W_{k+1}^* \leq W_k^*$. Au maximum ($k = n$), l'inertie est nulle. *Conséquence* : minimiser l'inertie sur k conduirait toujours à prendre k le plus grand possible ; elle ne peut servir qu'à repérer un **coude**, ou doit être remplacée par un critère qui pénalise k .

```
print([round(KMeans(k, n_init=5, random_state=0).fit(Z).inertia_) for k in range(1, 9)])
```

```
[84000, 62970, 45645, 36775, 32812, 30349, 28889, 27542]
```

Corrigé 3.5

Règle : $k = 1 : 0 \geq 0,30 - 0,01$? non. $k = 2 : 0,30 \geq 0,62 - 0,01$? non. $k = 3 : 0,62 \geq 0,75 - 0,02 = 0,73$? non. $k = 4 : 0,75 \geq 0,77 - 0,04 = 0,73$? **oui** : la règle retient $k = 4$. L'argmax est $k = 5$ (0,77). Les deux diffèrent parce que la règle accepte un k plus petit dès que le gain suivant (0,02) est inférieur à l'incertitude de l'estimation ($s_5 = 0,04$) : elle préfère le modèle le plus simple à gain statistiquement indiscernable.

Corrigé 3.6

Somme des valeurs propres : 8 (six variables standardisées). Part de variance : 1 composante : $4,2/8 = 52,5\%$; 2 : $6,3/8 = 78,75\%$; 3 : $7,2/8 = 90\%$. Il faut donc 3 composantes pour au moins 90 %. Erreur relative de reconstruction avec 2 composantes : $1 - 0,7875 = 0,2125$, soit 21,25 % (théorème d'Eckart–Young).

```
vp = np.array([4.2, 2.1, 0.9, 0.5, 0.2, 0.1]); print(np.cumsum(vp) / vp.sum())
```

```
[0.525  0.7875 0.9      0.9625 0.9875 1.      ]
```

Corrigé 3.7

$k \geq 4 \ln n / (\varepsilon^2/2 - \varepsilon^3/3)$ avec $\ln 10\,000 \approx 9,21$. Pour $\varepsilon = 0,25$: dénominateur $0,03125 - 0,00521 = 0,02604$, donc $k \geq 1\,415$ (la bibliothèque donne 1 414, par arrondi). Pour $\varepsilon = 0,1$: $0,005 - 0,00033 = 0,00467$, donc $k \geq 7\,894$. On passe de 50 000 variables à environ 1 400 (réduction par 35) pour $\pm 25\%$ sur les distances, mais à 7 900 (réduction par 6) pour $\pm 10\%$. La borne ne dépend pas de p , et seulement logarithmiquement de n : elle est précieuse quand p est énorme, mais elle devient **moins que triviale** quand ε est petit.

```
from sklearn.random_projection import johnson_lindenstrauss_min_dim
print(johnson_lindenstrauss_min_dim(10_000, eps=0.25), johnson_lindenstrauss_min_dim(10_000,
↪ eps=0.1))
```

```
1414 7894
```

Corrigé 3.8

- (a) Pour une coordonnée, $U = X_j - Y_j$ est triangulaire sur $[-1, 1]$: $\mathbb{E}[U^2] = \mathbb{E}[X^2] - 2\mathbb{E}[X]\mathbb{E}[Y] + \mathbb{E}[Y^2] = \frac{1}{3} - \frac{1}{2} + \frac{1}{3} = \frac{1}{6}$ et $\mathbb{E}[U^4] = \frac{1}{15}$ (intégrale de la densité triangulaire). Donc $\text{Var}(U^2) = \frac{1}{15} - \frac{1}{36} = \frac{7}{180}$. Par indépendance des coordonnées, $\mathbb{E}[D^2] = d/6$ et $\text{Var}(D^2) = 7d/180$.
- (b) $\frac{\sqrt{7d/180}}{d/6} = \sqrt{\frac{7 \times 36}{180d}} = \sqrt{\frac{1,4}{d}}$.
- (c) Par la méthode delta, $D = \sqrt{D^2}$ a une dispersion relative moitié moindre : $\approx 0,5\sqrt{1,4/d} \approx 0,5916/\sqrt{d}$. Pour $d = 50$: 0,0837 ; pour $d = 1\,000$: 0,0187. Le livre mesure 0,086 et 0,018 (tableau de 3.2.1) : l'accord est bon.

```
for d in (50, 1000): print(d, round(0.5 * np.sqrt(1.4 / d), 4))
```

```
50 0.0837
1000 0.0187
```

Corrigé 3.9

Distances utiles : $P_1P_2 = P_1P_3 = P_2P_4 = P_3P_4 = 1$; $P_2P_3 = P_1P_4 = \sqrt{2} \approx 1,414 > 1,2$; $P_4P_5 = \sqrt{1+0,25} \approx 1,118 \leq 1,2$; $P_6P_7 = P_6P_8 = 0,5$; $P_7P_8 \approx 0,707$. Voisinages (avec le point lui-même) : $P_1 : \{P_1, P_2, P_3\}$ (3, cœur) ; $P_2 : \{P_2, P_1, P_4\}$ (3, cœur) ; $P_3 : \{P_3, P_1, P_4\}$ (3, cœur) ; $P_4 : \{P_4, P_2, P_3, P_5\}$ (4, cœur) ; $P_5 : \{P_5, P_4\}$ (2, pas un cœur, mais voisin du cœur P_4 : **bordure**) ; P_6, P_7, P_8 : chacun a 3 points : **cœurs** ; P_9 : seul, **bruit**. Groupes : $\{P_1, \dots, P_5\}$ et $\{P_6, P_7, P_8\}$; bruit : P_9 .

```
P = np.array([[0, 0], [1, 0], [0, 1], [1, 1], [2, 1.5], [5, 5], [5.5, 5], [5, 5.5], [10, 0]])
db = DBSCAN(eps=1.2, min_samples=3).fit(P); print(db.labels_, [int(i) + 1 for i in
↪ sorted(db.core_sample_indices_)])
```

```
[ 0  0  0  0  0  1  1  1 -1] [1, 2, 3, 4, 6, 7, 8]
```

Corrigé 3.10

Responsabilité du groupe 1 : $\gamma_1(x) = 1/(1 + e^{-(x-6)^2-x^2/2})$. $x = 0 : (36 - 0)/2 = 18, \gamma_1 \approx 1$; $x = 2 : (16 - 4)/2 = 6, \gamma_1 = 1/(1 + e^{-6}) \approx 0,9975$; $x = 3 : (9 - 9)/2 = 0, \gamma_1 = 0,5$; $x = 6 : (0 - 36)/2 = -18, \gamma_1 \approx 0$. Nouvelles moyennes : $\mu_1 = \frac{0 \times 1 + 2 \times 0,9975 + 3 \times 0,5}{1 + 0,9975 + 0,5} \approx 1,399$; avec $\gamma_2 = 1 - \gamma_1 : \mu_2 = \frac{2 \times 0,0025 + 3 \times 0,5 + 6 \times 1}{0 + 0,0025 + 0,5 + 1} \approx 4,995$. Poids : $\pi_1 = 2,4975/4 \approx 0,624$ et $\pi_2 \approx 0,376$.

```
xx = np.array([0.0, 2.0, 3.0, 6.0]); g = 1 / (1 + np.exp(-((xx - 6) ** 2 - xx ** 2) / 2)); G =
↪ np.column_stack([g, 1 - g])
print(g.round(4), (G * xx[:, None]).sum(0) / G.sum(0), G.sum(0) / 4)
```

```
[1.      0.9975 0.5      0.      ] [1.39940602 4.99506283] [0.62438184 0.37561816]
```

Corrigé 3.11

(a) $q = Kp + K \frac{p(p+1)}{2} + (K-1) : p = 7$ donne $K \times (7 + 28) + (K-1) = 35K + K - 1 = 36K - 1$: $q = 143$ pour $K = 4, q = 179$ pour $K = 5$. (b) $\ln n = \ln 12\,000 \approx 9,393$. $\text{BIC}_4 = 105\,000 + 143 \times 9,393 \approx 106\,343$; $\text{BIC}_5 = 102\,600 + 179 \times 9,393 \approx 104\,281$: le BIC retient $K = 5$. (c) Il faut $-2\Delta \ln \hat{L} > (179 - 143) \ln n$, soit $\Delta \ln \hat{L} > 36 \times 9,393/2 \approx 169$: le passage à $K = 5$ doit gagner **au moins 169 unités** de log-vraisemblance. Ici, le gain est de 1 200 : très au-delà du seuil.

```
n = 12000
print(round(105000 + 143 * np.log(n)), round(102600 + 179 * np.log(n)), round(36 * np.log(n) / 2, 1))
```

```
106343 104281 169.1
```

Corrigé 3.12

(a) $H = -\sum p \log_2 p = 0,5 \times 1 + 0,25 \times 2 + 2 \times 0,125 \times 3 = 0,5 + 0,5 + 0,75 = 1,75$ bits ; perplexité $2^{1,75} \approx 3,36$. (b) Pour la loi uniforme sur m voisins, $H = \log_2 m$, donc la perplexité vaut exactement m . (c) Choisir la perplexité 30 revient à régler, pour **chaque** point, la largeur σ_i de sorte que sa distribution de voisinage ait une entropie équivalente à celle d'une loi uniforme sur environ 30 voisins : c'est un nombre **effectif** de voisins, qui s'adapte à la densité locale.

```
p = np.array([0.5, 0.25, 0.125, 0.125]); print(2 ** (-(p * np.log2(p)).sum()))
```

```
3.363585661014858
```

Chapitre 4 : Ingénierie des variables et données déséquilibrées — exercices et applications

Ce chapitre du cahier accompagne le chapitre 4 du livre. Les **applications** reprennent, pas à pas, ce que le livre n'a fait que résumer : un pipeline complet, l'encodage par la cible écrit à la main, la comparaison de stratégies pour les valeurs manquantes, la création de variables, les stratégies pour classes rares, la sélection de variables et SMOTE. Les **exercices** (à la main d'abord, puis avec du code) sont corrigés à la fin. Données : `clients_ml.csv` (le départ des clients, 14 %) et `transactions.csv` (la fraude, 0,8 %). Prérequis : les sections 4.1 à 4.5 du livre, selon l'exercice.

Préparation

Une seule cellule charge les données et refait le découpage du livre (75 % / 25 %, stratifié, graine 0) ; les applications s'en servent.

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score, cross_validate,
↳ StratifiedKFold, cross_val_predict
from sklearn.metrics import roc_auc_score, average_precision_score, precision_score, recall_score,
↳ f1_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline

clients = pd.read_csv("donnees/clients_ml.csv")
y = clients["churn_90j"]
tr, te = train_test_split(clients.index, test_size=0.25, random_state=0, stratify=y)
ytr, yte = y[tr], y[te]
NUM = ["age", "anciennete_mois", "nb_commandes_12m", "montant_12m", "recence_jours", "nb_retours_12m",
        "nb_tickets_support_12m", "programme_fidelite", "nb_promos_recues_12m", "part_achats_promo",
        "taux_ouverture_email", "satisfaction_moy", "delai_livraison_moy", "panier_moyen"]
QUAL = ["ville", "canal_acquisition", "appareil", "categorie_preferee"]
```

```
trans = pd.read_csv("donnees/transactions.csv")
Xt = pd.get_dummies(trans.drop(columns=["id_commande", "fraude", "type_fraude"]), dtype=float)
yt = trans["fraude"]
Xa, Xb, ya, yb = train_test_split(Xt, yt, test_size=0.3, random_state=0, stratify=yt)
print(len(clients), "clients, part de départs", round(y.mean(), 3), "|", len(trans), "commandes, part  
↪ de fraudes", round(yt.mean(), 4))
```

12000 clients, part de départs 0.14 | 60000 commandes, part de fraudes 0.0081

Applications

Application 4.1 — Un pipeline complet, pas à pas (sections 4.1.6)

Objectif. Construire un modèle de départ de clients qui prend la table **brute** en entrée et refait tout son prétraitement lui-même, puis l'utiliser sur un nouveau client.

Étape 1 : quelles colonnes de quel type ? Quatorze variables numériques (dont quatre avec des valeurs manquantes) et quatre qualitatives.

```
print("valeurs manquantes par variable numérique :")
print(clients[NUM].isna().sum()[lambda s: s > 0].to_dict())
print("modalités par variable qualitative :", clients[QUAL].nunique().to_dict())
```

```
valeurs manquantes par variable numérique :
{'satisfaction_moy': 1533, 'delai_livraison_moy': 1434, 'panier_moyen': 1630}
modalités par variable qualitative : {'ville': 20, 'canal_acquisition': 3, 'appareil': 3,  
↪ 'categorie_preferee': 4}
```

Étape 2 : le prétraitement. Pour les numériques : imputation par la médiane avec indicateurs d'absence, puis standardisation. Pour les qualitatives : imputation par la modalité la plus fréquente, puis *one-hot*.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

numeriques = make_pipeline(SimpleImputer(strategy="median", add_indicator=True), StandardScaler())
categories = make_pipeline(SimpleImputer(strategy="most_frequent"),
↪ OneHotEncoder(handle_unknown="ignore"))
prep = ColumnTransformer([("num", numeriques, NUM), ("cat", categories, QUAL)])
modele = make_pipeline(pre, LogisticRegression(max_iter=3000))
cv = cross_val_score(modele, clients.loc[tr], ytr, cv=5, scoring="roc_auc")
print(f"validation croisée : AUC {cv.mean():.3f} (± {cv.std():.3f})")
```

validation croisée : AUC 0.860 (± 0.013)

Étape 3 : qu'a produit le prétraitement ? On ajuste sur tout l'entraînement et on regarde les colonnes créées.

```
modele.fit(clients.loc[tr], ytr)
noms = modele[:-1].get_feature_names_out()
print(len(noms), "colonnes en entrée du modèle :", sum(n.startswith("num__") for n in noms),
↪ "numériques (dont indicateurs)",
sum(n.startswith("cat__") for n in noms), "issues de l'encodage disjonctif")
print("exemples :", list(noms[:3]), "...", list(noms[-3:]))
print(f"AUC test : {roc_auc_score(yte, modele.predict_proba(clients.loc[te])[:, 1]):.3f}")
```

47 colonnes en entrée du modèle : 17 numériques (dont indicateurs), 30 issues de l'encodage disjonctif
 exemples : ['num_age', 'num_anciennete_mois', 'num_nb_commandes_12m'] ...
 ↪ ['cat_categorie_preferee_B', 'cat_categorie_preferee_C', 'cat_categorie_preferee_D']
 AUC test : 0.866

Étape 4 : le même problème avec un boosting. Un boosting ne demande ni mise à l'échelle ni imputation, et sait traiter des modalités directement si on lui déclare les colonnes qualitatives (type category).

```
brut = clients[NUM + QUAL].copy()
for col in QUAL:
    brut[col] = brut[col].astype("category")
gb = HistGradientBoostingClassifier(random_state=0, max_iter=100,
    ↪ categorical_features="from_dtype").fit(brut.loc[tr], ytr)
print(f"boosting sans aucun prétraitement : AUC test {roc_auc_score(yte,
    ↪ gb.predict_proba(brut.loc[te])[:, 1]):.3f}")
```

boosting sans aucun prétraitement : AUC test 0.900

Étape 5 : un seul objet pour la production. Le pipeline accepte une ligne brute, avec ses valeurs manquantes.

```
nouveau = pd.DataFrame([{"age": 33, "anciennete_mois": 20, "nb_commandes_12m": 1, "montant_12m":
    ↪ 40.0, "recence_jours": 210,
    "nb_retours_12m": 0, "nb_tickets_support_12m": 3, "programme_fidelite": 0,
    ↪ "nb_promos_recues_12m": 5,
    "part_achats_promo": 0.7, "taux_ouverture_email": 0.1, "satisfaction_moy":
    ↪ np.nan, "delai_livraison_moy": 4.0,
    "panier_moyen": 40.0, "ville": "Ville C", "canal_acquisition": "Réseaux",
    ↪ "appareil": np.nan,
    "categorie_preferee": "B"}])
print(f"probabilité de départ du nouveau client :", round(float(modele.predict_proba(nouveau)[0, 1]),
    ↪ 3))
```

probabilité de départ du nouveau client : 0.367

Pour aller plus loin. Remplacez OneHotEncoder par l'encodage de la fréquence puis comparez les deux pipelines en validation croisée : l'écart dépasse-t-il l'écart-type d'un pli à l'autre ?

Application 4.2 — L'encodage par la cible, hors pli, écrit à la main (section 4.1.2)

Objectif. Écrire les trois variantes de l'encodage par la cible et mesurer leur fuite sur une variable de pur bruit, un code postal à 400 modalités tiré au hasard.

Étape 1 : la variable de bruit.

```
rng = np.random.default_rng(4)
code_postal = pd.Series(rng.integers(0, 400, len(clients)), index=clients.index)
print(f"clients par code (entraînement), en moyenne :", round(len(tr) / 400, 1))
```

clients par code (entraînement), en moyenne : 22.5

Étape 2 : l'encodage naïf et l'encodage lissé. Le lissage $\frac{n_m \bar{y}_m + k \mu}{n_m + k}$ rapproche les petites modalités de la moyenne générale.

```
def encodage_naif(col_tr, y_tr, col_te, k=0):
    mu = y_tr.mean()
    g = y_tr.groupby(col_tr).agg(["sum", "count"])
    moy = (g["sum"] + k * mu) / (g["count"] + k)
    return col_tr.map(moy), col_te.map(moy).fillna(mu)
```

```
for k in (0, 20):
    a, b = encodage_naif(code_postal[tr], ytr, code_postal[te], k)
    print(f"lissage k = {k:2d} : AUC entraînement {roc_auc_score(ytr, a):.3f} | AUC test
        ↳ {roc_auc_score(yte, b):.3f}")
```

```
lissage k = 0 : AUC entraînement 0.678 | AUC test 0.510
lissage k = 20 : AUC entraînement 0.678 | AUC test 0.510
```

Étape 3 : le calcul hors pli. Les lignes du pli j sont encodées avec les statistiques des autres plis.

```
from sklearn.model_selection import KFold

def encodage_hors_pli(col_tr, y_tr, plis=5, graine=0):
    sortie = pd.Series(index=col_tr.index, dtype=float)
    for i, j in KFold(plis, shuffle=True, random_state=graine).split(col_tr):
        moy = y_tr.iloc[i].groupby(col_tr.iloc[i]).mean()
        sortie.iloc[j] = col_tr.iloc[j].map(moy).fillna(y_tr.iloc[i].mean()).values
    return sortie

oof = encodage_hors_pli(code_postal[tr], ytr)
print("hors pli, AUC entraînement :", round(roc_auc_score(ytr, oof), 3))
```

```
hors pli, AUC entraînement : 0.509
```

Étape 4 : le piège du « leave-one-out ». On retire seulement la ligne courante. Dans une même modalité, la valeur encodée dépend alors de **sa propre étiquette** : on le vérifie sur la vraie ville.

```
ville = clients.loc[tr, "ville"]
somme, effectif = ytr.groupby(ville).transform("sum"), ytr.groupby(ville).transform("count")
loo = (somme - ytr) / (effectif - 1)
centre = loo - loo.groupby(ville).transform("mean")           # on retire l'effet de la ville : il
↳ reste l'écart intra-ville
print("écart intra-ville moyen, clients PARTIS :", round(centre[ytr == 1].mean(), 4))
print("écart intra-ville moyen, clients RESTÉS :", round(centre[ytr == 0].mean(), 4))
print("AUC de -(encodage L00 centré) pour retrouver la cible :", round(roc_auc_score(ytr, -centre),
↳ 3))
```

```
écart intra-ville moyen, clients PARTIS : -0.0018
écart intra-ville moyen, clients RESTÉS : 0.0003
AUC de -(encodage L00 centré) pour retrouver la cible : 1.0
```

Lecture. Le calcul naïf, même lissé (les deux lignes sont identiques : le lissage atténue les valeurs extrêmes mais ne supprime pas la fuite), donne une AUC d'entraînement de 0,678 pour une variable sans aucun lien avec la cible, qui retombe à 0,510 sur le test. Le calcul hors pli ne fuit pas (0,509). Le *leave-one-out* fuit à l'envers : dans une même ville, un client parti reçoit une valeur plus basse qu'un client resté, puisqu'on lui a retiré sa propre étiquette. Une fois l'effet de la ville retiré, cet écart est **exact** et sépare parfaitement les deux groupes : un arbre peut en tirer parti pour retrouver la cible.

Application 4.3 — Valeurs manquantes : quand l'indicateur d'absence compte (section 4.1.5)

Objectif. Comparer des stratégies d'imputation en validation croisée, puis mesurer comment l'intérêt de l'indicateur d'absence évolue avec la force du lien entre absence et cible.

Étape 1 : six stratégies, avec leur incertitude.

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import KNNImputer, IterativeImputer
```

```

strategies = {"moyenne": SimpleImputer(strategy="mean"), "médiane": SimpleImputer(strategy="median"),
              "médiane + indicateurs": SimpleImputer(strategy="median", add_indicator=True),
              "kNN (k = 5)": KNNImputer(n_neighbors=5), "itérative": IterativeImputer(random_state=0,
              ↪ max_iter=5)}
cv = StratifiedKFold(5, shuffle=True, random_state=0)
for nom, imp in strategies.items():
    m = make_pipeline(imp, StandardScaler(), LogisticRegression(max_iter=3000))
    s = cross_val_score(m, clients.loc[tr, NUM], ytr, cv=cv, scoring="roc_auc")
    print(f"{nom:24s} AUC {s.mean():.3f} (± {s.std():.3f})")

```

moyenne	AUC 0.857 (± 0.009)
médiane	AUC 0.856 (± 0.009)
médiane + indicateurs	AUC 0.857 (± 0.009)
kNN (k = 5)	AUC 0.855 (± 0.010)
itérative	AUC 0.855 (± 0.009)

Étape 2 : le cas simulé, avec une force de lien variable. La variable x est liée à la cible ; elle est manquante avec une probabilité q_1 pour les départs et $q_0 = 0,10$ pour les autres.

```

def auc_selon_absence(q1, graine=11, n=6000):
    r = np.random.default_rng(graine)
    x = r.normal(size=n)
    yy = r.binomial(1, 1 / (1 + np.exp(-(-1.5 + 0.8 * x))))
    manquant = r.random(n) < np.where(yy == 1, q1, 0.10)
    X = pd.DataFrame({"x": np.where(manquant, np.nan, x)})
    a, b, ya_, yb_ = train_test_split(X, yy, test_size=0.3, random_state=0, stratify=yy)
    sortie = []
    for ind in (False, True):
        m = make_pipeline(SimpleImputer(strategy="mean", add_indicator=ind),
        ↪ LogisticRegression()).fit(a, ya_)
        sortie.append(roc_auc_score(yb_, m.predict_proba(b)[:, 1]))
    return sortie

for q1 in (0.10, 0.25, 0.40, 0.55, 0.70):
    sans, avec = auc_selon_absence(q1)
    print(f"absence chez les départs {q1:.2f} : AUC sans indicateur {sans:.3f}, avec indicateur
    ↪ {avec:.3f}")

```

absence chez les départs 0.10	: AUC sans indicateur 0.714, avec indicateur 0.715
absence chez les départs 0.25	: AUC sans indicateur 0.688, avec indicateur 0.732
absence chez les départs 0.40	: AUC sans indicateur 0.644, avec indicateur 0.778
absence chez les départs 0.55	: AUC sans indicateur 0.610, avec indicateur 0.819
absence chez les départs 0.70	: AUC sans indicateur 0.569, avec indicateur 0.864

Lecture. Dans les vraies données, toutes les stratégies sont indiscernables. Dans la simulation, l'écart entre « sans » et « avec » indicateur **croît avec le lien entre absence et cible**, et il est nul quand l'absence est aléatoire ($q_1 = 0,10$, comme chez les autres). L'indicateur ne coûte presque rien : ajoutez-le dès que vous doutez.

Application 4.4 — Créer des variables RFM et mesurer leur apport (section 4.2)

Objectif. Fabriquer des variables métier, découvrir des seuils sur l'entraînement, et vérifier par permutation que les nouvelles variables servent vraiment.

Étape 1 : variables génériques et profils RFM.

```

def fabriquer(df):
    f = df[NUM].copy()
    f["frequence_mensuelle"] = df["nb_commandes_12m"] / np.minimum(df["anciennete_mois"], 12)
    f["taux_retour"] = (df["nb_retours_12m"] / df["nb_commandes_12m"].replace(0, np.nan)).fillna(0)

```

```
f["tickets_par_commande"] = df["nb_tickets_support_12m"] / (df["nb_commandes_12m"] + 1)
f["jamais_commande"] = (df["nb_commandes_12m"] == 0).astype(int)
f["log_recence"] = np.log1p(df["recence_jours"])
return f
```

```
F = fabriquer(clients)
print(F[["frequence_mensuelle", "taux_retour", "tickets_par_commande"]].describe().loc[["mean",
↪ "50%", "max"]].round(2))
```

	frequence_mensuelle	taux_retour	tickets_par_commande
mean	0.37	0.07	0.22
50%	0.25	0.00	0.00
max	12.00	1.00	8.00

Étape 2 : des classes de récence, avec des bornes lues sur l'entraînement. On coupe la récence en quintiles de l'entraînement et on applique les mêmes bornes au test.

```
bornes = clients.loc[tr, "recence_jours"].quantile([0.2, 0.4, 0.6, 0.8]).values
F["classe_recence"] = np.digitize(clients["recence_jours"], bornes)
print("bornes (jours) :", bornes.round(0))
print(clients.loc[tr].assign(classe=F.loc[tr,
↪ "classe_recence"]).groupby("classe")["churn_90j"].mean().round(3).to_dict())
```

```
bornes (jours) : [ 15.  38.  76. 203.]
{0: 0.062, 1: 0.055, 2: 0.079, 3: 0.137, 4: 0.367}
```

Étape 3 : les règles découvertes. Les seuils (160 jours, satisfaction 3,15, promotions 0,60) viennent d'un arbre de profondeur 3 ajusté sur l'entraînement (livre, 4.2.2).

```
F["inactif_et_mecontent"] = ((clients["recence_jours"] > 160) & (clients["satisfaction_moy"] <
↪ 3.15)).astype(int)
F["promo_sans_fidelite"] = ((clients["part_achats_promo"] > 0.60) & (clients["programme_fidelite"] ==
↪ 0)).astype(int)
brutes, riches = NUM, [c for c in F.columns if c != "classe_recence"]
for nom, cols in [("14 variables brutes", brutes), ("variables enrichies", riches)]:
    m = make_pipeline(SimpleImputer(strategy="median", add_indicator=True), StandardScaler(),
    ↪ LogisticRegression(max_iter=3000))
    m.fit(F.loc[tr, cols], ytr)
    print(f"{nom:22s} ({len(cols):2d} colonnes) AUC test {roc_auc_score(yte,
    ↪ m.predict_proba(F.loc[te, cols])[0, 1]):.3f}")
```

```
14 variables brutes      (14 colonnes) AUC test 0.858
variables enrichies      (21 colonnes) AUC test 0.886
```

Étape 4 : les nouvelles variables servent-elles ? Importance par permutation, sur une partie de l'entraînement mise de côté.

```
from sklearn.inspection import permutation_importance

a_in, a_val, y_in, y_val = train_test_split(F.loc[tr, riches], ytr, test_size=0.3, random_state=0,
    ↪ stratify=ytr)
m = make_pipeline(SimpleImputer(strategy="median", add_indicator=True), StandardScaler(),
    ↪ LogisticRegression(max_iter=3000)).fit(a_in, y_in)
perm = permutation_importance(m, a_val, y_val, scoring="roc_auc", n_repeats=5, random_state=0,
    ↪ n_jobs=1)
top = pd.Series(perm.importances_mean, index=riches).sort_values(ascending=False).head(6)
print(top.round(4).to_string())
```

age	0.0596
nb_commandes_12m	0.0560
promo_sans_fidelite	0.0516
inactif_et_mecontent	0.0416

recence_jours	0.0214
panier_moyen	0.0028

Lecture. Les variables enrichies font passer l'AUC de 0,858 à 0,886 (la version complète du livre, avec une règle de plus et deux variables supplémentaires, atteint 0,900). L'importance par permutation montre que les **indicateurs de règles** servent vraiment : promo_sans_fidelite (0,052) et inactif_et_mecontent (0,042) sont dans les quatre premières, devant la récence brute (0,021). L'âge arrive en tête (0,060) : une relation en U, que la régression logistique doit se contenter d'approcher par une droite.

Application 4.5 — Toutes les stratégies pour classes rares, sur la fraude (sections 4.3.1 à 4.3.3)

Objectif. Comparer sur les mêmes plis la régression logistique et le boosting avec : rien, poids équilibrés, sous- et sur-échantillonnage, en rapportant la PR-AUC avec son écart-type. Le rééchantillonnage est **dans le pipeline**.

```
from imblearn.pipeline import Pipeline as PipeIB
from imblearn.over_sampling import RandomOverSampler
from imblearn.under_sampling import RandomUnderSampler

cv = StratifiedKFold(5, shuffle=True, random_state=0)
lg = lambda **k: LogisticRegression(max_iter=3000, **k)
gb = lambda **k: HistGradientBoostingClassifier(random_state=0, max_iter=100, **k)
modeles = {
    "logistique": PipeIB([("sc", StandardScaler()), ("m", lg())]),
    "logistique + poids": PipeIB([("sc", StandardScaler()), ("m", lg(class_weight="balanced"))]),
    "logistique + sous-éch. (1:5)": PipeIB([("sc", StandardScaler()), ("u",
        ↪ RandomUnderSampler(sampling_strategy=0.2, random_state=0)), ("m", lg())]),
    "boosting": PipeIB([("m", gb())]),
    "boosting + poids": PipeIB([("m", gb(class_weight="balanced"))]),
}
for nom, mod in modeles.items():
    r = cross_validate(mod, Xa, ya, cv=cv, scoring={"auc": "roc_auc", "ap": "average_precision"})
    print(f"{nom:<30s} AUC {r['test_auc'].mean():.3f} (± {r['test_auc'].std():.3f}) | PR-AUC
        ↪ {r['test_ap'].mean():.3f} (± {r['test_ap'].std():.3f})")
```

logistique	AUC 0.907 (± 0.019) PR-AUC 0.280 (± 0.059)
logistique + poids	AUC 0.910 (± 0.018) PR-AUC 0.219 (± 0.041)
logistique + sous-éch. (1:5)	AUC 0.910 (± 0.017) PR-AUC 0.255 (± 0.040)
boosting	AUC 0.950 (± 0.010) PR-AUC 0.575 (± 0.040)
boosting + poids	AUC 0.966 (± 0.013) PR-AUC 0.544 (± 0.072)

Lecture. Le classement des modèles est stable (le boosting domine la régression logistique de très loin : PR-AUC de 0,54 à 0,58 contre 0,22 à 0,28). Les **traitements** de déséquilibre n'améliorent pas la PR-AUC ; ils la dégradent même un peu (de 0,03 à 0,06), un écart du même ordre que l'écart-type entre plis (0,04 à 0,07). L'AUC-ROC, elle, reste stable ou monte (0,950 à 0,966 pour le boosting). Pour trancher entre deux traitements proches, il faut un **test apparié** ou un *bootstrap* (livre, 4.3.6).

Application 4.6 — Choisir un seuil par les coûts (section 4.3.5)

Objectif. Pour trois rapports de coûts, calculer le seuil théorique $s^* = \frac{c_{FP}}{c_{FP} + c_{FN}}$, le comparer au seuil optimal empirique trouvé sur des prédictions **hors pli**, puis évaluer le coût sur le test.

Étape 1 : prédictions hors pli et modèle final.

```
modele = HistGradientBoostingClassifier(random_state=0, max_iter=100)
oof = cross_val_predict(modele, Xa, ya, cv=StratifiedKFold(5, shuffle=True, random_state=0),
    ↪ method="predict_proba")[:, 1]
p_test = modele.fit(Xa, ya).predict_proba(Xb)[:, 1]
```

Étape 2 : une fonction de coût et trois scénarios.

```
def cout(p, vrai, s, c_fn, c_fp):
    bloque = p >= s
    return int((~bloque) & (vrai == 1)).sum() * c_fn + (bloque & (vrai == 0)).sum() * c_fp

grille = np.linspace(0.005, 0.95, 190)
for c_fn, c_fp in [(10, 5), (100, 5), (500, 5)]:
    s_theo = c_fp / (c_fp + c_fn)
    s_emp = grille[int(np.argmax([cout(oof, ya.values, s, c_fn, c_fp) for s in grille]))]
    print(f"c_FN = {c_fn:3d}, c_FP = {c_fp} : seuil théorique {s_theo:.3f} | seuil hors pli
    ↪ {s_emp:.3f} | coût test à 0,5 : "
        f"{cout(p_test, yb.values, 0.5, c_fn, c_fp):6d} | au seuil théorique : {cout(p_test,
    ↪ yb.values, s_theo, c_fn, c_fp):6d} | au seuil hors pli : {cout(p_test, yb.values,
    ↪ s_emp, c_fn, c_fp):6d}")
```

```
c_FN = 10, c_FP = 5 : seuil théorique 0.333 | seuil hors pli 0.515 | coût test à 0,5 : 855 | au
↪ seuil théorique : 860 | au seuil hors pli : 855
c_FN = 100, c_FP = 5 : seuil théorique 0.048 | seuil hors pli 0.025 | coût test à 0,5 : 7155 | au
↪ seuil théorique : 5575 | au seuil hors pli : 5420
c_FN = 500, c_FP = 5 : seuil théorique 0.010 | seuil hors pli 0.005 | coût test à 0,5 : 35155 | au
↪ seuil théorique : 16495 | au seuil hors pli : 14865
```

Lecture. Plus la fraude coûte cher par rapport à une fausse alerte, plus le seuil optimal descend et plus le seuil de 0,5 est coûteux : de 855 € (rapport 2) à 35 155 € (rapport 100), contre 16 495 € au seuil théorique et 14 865 € au seuil hors pli. Les deux seuils calculés donnent des **coûts voisins** (860 contre 855 ; 5 575 contre 5 420 ; 16 495 contre 14 865) même quand ils diffèrent (0,333 contre 0,515 dans le premier scénario, où le modèle est si confiant que beaucoup de seuils se valent) : tant que les probabilités du modèle sont à peu près calibrées, la formule suffit.

Application 4.7 — Sélection de variables : trois familles et un piège (section 4.4)

Objectif. Comparer filtre, enveloppe et méthode intégrée sur les clients enrichis de variables de bruit, puis mesurer comment le biais de sélection dépend du nombre de variables et de clients.

Étape 1 : les clients, plus cinq variables de bruit.

```
from sklearn.feature_selection import mutual_info_classif, RFE

rng = np.random.default_rng(5)
Xs = clients[NUM].copy()
for k in range(3):
    Xs[f"bruit_continu_{k + 1}"] = rng.normal(size=len(Xs))
for k in range(2):
    Xs[f"bruit_discret_{k + 1}"] = rng.integers(0, 12, len(Xs))
A = pd.DataFrame(SimpleImputer(strategy="median").fit_transform(Xs.loc[tr]), columns=Xs.columns,
    ↪ index=tr)
As = pd.DataFrame(StandardScaler().fit_transform(A), columns=A.columns, index=tr)
bruits = [c for c in A.columns if c.startswith("bruit")]
```

Étape 2 : trois sélections de 8 variables.

```
mi = pd.Series(mutual_info_classif(A, ytr, random_state=0),
  ↳ index=A.columns).sort_values(ascending=False)
sel_mi = list(mi.index[:8])
sel_rfe = list(As.columns[RFE(LogisticRegression(max_iter=3000), n_features_to_select=8).fit(As,
  ↳ ytr).support_])
l1 = LogisticRegression(penalty="l1", solver="liblinear", C=0.05, max_iter=5000).fit(As, ytr)
sel_l1 = list(As.columns[l1.coef_[0] != 0])
for nom, sel in [("information mutuelle", sel_mi), ("RFE", sel_rfe), ("pénalité L1", sel_l1)]:
    print(f"{nom:22s} {len(sel):2d} variables, bruit retenu : {[v for v in sel if v in bruits]}")
```

```
information mutuelle    8 variables, bruit retenu : []
RFE                    8 variables, bruit retenu : []
pénalité L1            14 variables, bruit retenu : ['bruit_continu_1', 'bruit_continu_2',
  ↳ 'bruit_discret_1']
```

Étape 3 : le biais de sélection selon p et n . Cible aléatoire ; on sélectionne les 10 meilleures variables, avant ou dans la validation croisée.

```
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.pipeline import Pipeline

def auc_biais(n, p, graines=15):
    avant, dans = [], []
    cv = StratifiedKFold(5, shuffle=True, random_state=0)
    for g in range(graines):
        r = np.random.default_rng(g)
        X, yy = r.normal(size=(n, p)), r.integers(0, 2, n)
        Xk = SelectKBest(f_classif, k=10).fit_transform(X, yy)
        avant.append(cross_val_score(LogisticRegression(max_iter=2000), Xk, yy, cv=cv,
  ↳ scoring="roc_auc").mean())
        pipe = Pipeline([("sel", SelectKBest(f_classif, k=10)), ("lr",
  ↳ LogisticRegression(max_iter=2000))])
        dans.append(cross_val_score(pipe, X, yy, cv=cv, scoring="roc_auc").mean())
    return np.mean(avant), np.mean(dans)

for n, p in [(150, 50), (150, 500), (150, 2000), (600, 500), (2000, 500)]:
    a, d = auc_biais(n, p)
    print(f"n = {n:4d}, p = {p:4d} : AUC sélection avant la validation {a:.3f} | dans la validation
  ↳ {d:.3f}")
```

```
n = 150, p = 50 : AUC sélection avant la validation 0.652 | dans la validation 0.481
n = 150, p = 500 : AUC sélection avant la validation 0.790 | dans la validation 0.474
n = 150, p = 2000 : AUC sélection avant la validation 0.836 | dans la validation 0.475
n = 600, p = 500 : AUC sélection avant la validation 0.656 | dans la validation 0.498
n = 2000, p = 500 : AUC sélection avant la validation 0.594 | dans la validation 0.503
```

Lecture. Le biais croît avec le nombre de variables candidates p et décroît avec le nombre de clients n : il est énorme quand $p \gg n$ (génomique, textes) et négligeable avec beaucoup de lignes. La sélection faite dans la validation est toujours honnête (autour de 0,5).

Application 4.8 — SMOTE et variantes dans un pipeline (section 4.5)

Objectif. Comparer SMOTE, Borderline-SMOTE, ADASYN et SMOTE-NC dans un pipeline ; mesurer l'effet de k ; refaire la fuite par rééchantillonnage.

Étape 1 : le pipeline SMOTE avec le nombre de voisins.

```
from imblearn.over_sampling import SMOTE, BorderlineSMOTE, ADASYN

cv = StratifiedKFold(5, shuffle=True, random_state=0)
```

```
for k in (2, 5, 15):
    pipe = PipeIB([("sc", StandardScaler()), ("o", SMOTE(k_neighbors=k, random_state=0)), ("m",
    ↳ LogisticRegression(max_iter=3000))])
    s = cross_val_score(pipe, Xa, ya, cv=cv, scoring="average_precision")
    print(f"SMOTE, k = {k:2d} : PR-AUC {s.mean():.3f} (± {s.std():.3f})")
```

```
SMOTE, k = 2 : PR-AUC 0.220 (± 0.043)
SMOTE, k = 5 : PR-AUC 0.214 (± 0.039)
SMOTE, k = 15 : PR-AUC 0.215 (± 0.037)
```

Étape 2 : trois variantes, avec boosting.

```
for nom, o in [("SMOTE", SMOTE(random_state=0)), ("Borderline-SMOTE",
    ↳ BorderlineSMOTE(random_state=0)), ("ADASYN", ADASYN(random_state=0))]:
    pipe = PipeIB([("o", o), ("m", HistGradientBoostingClassifier(random_state=0, max_iter=100))])
    s = cross_val_score(pipe, Xa, ya, cv=cv, scoring="average_precision")
    print(f"{nom:18s} + boosting : PR-AUC {s.mean():.3f} (± {s.std():.3f})")
```

```
SMOTE          + boosting : PR-AUC 0.500 (± 0.033)
Borderline-SMOTE + boosting : PR-AUC 0.573 (± 0.043)
ADASYN         + boosting : PR-AUC 0.495 (± 0.038)
```

Étape 3 : SMOTE-NC avec une variable qualitative. On rend les départs rares (3 %) en gardant tous les clients restés et seulement une partie des partis, et on traite ville comme variable nominale.

```
from imblearn.over_sampling import SMOTENC

rare = pd.concat([clients[clients.churn_90j == 0], clients[clients.churn_90j == 1].sample(frac=0.2,
    ↳ random_state=0)])
Xr = rare[["age", "recence_jours", "nb_commandes_12m", "montant_12m"]].copy()
Xr["ville_code"] = rare["ville"].astype("category").cat.codes
yr = rare["churn_90j"]
pipe = PipeIB([("o", SMOTENC(categorical_features=[4], random_state=0)), ("m",
    ↳ HistGradientBoostingClassifier(random_state=0, max_iter=100))])
s = cross_val_score(pipe, Xr, yr, cv=cv, scoring="average_precision")
s0 = cross_val_score(HistGradientBoostingClassifier(random_state=0, max_iter=100), Xr, yr, cv=cv,
    ↳ scoring="average_precision")
print("part de départs :", round(yr.mean(), 3))
print("SMOTE-NC + boosting : PR-AUC", round(s.mean(), 3), "(± %.3f)" % s.std())
print("sans rééchantillonnage : PR-AUC", round(s0.mean(), 3), "(± %.3f)" % s0.std())
```

```
part de départs : 0.032
SMOTE-NC + boosting : PR-AUC 0.207 (± 0.035)
sans rééchantillonnage : PR-AUC 0.187 (± 0.008)
```

Étape 4 : la fuite. Sur-échantillonner avant la validation croisée.

```
Xo, yo = RandomOverSampler(random_state=0).fit_resample(Xr, yr)
fuite = cross_val_score(HistGradientBoostingClassifier(random_state=0, max_iter=100), Xo, yo, cv=cv,
    ↳ scoring="average_precision")
honnete = cross_val_score(HistGradientBoostingClassifier(random_state=0, max_iter=100), Xr, yr,
    ↳ cv=cv, scoring="average_precision")
print(f"PR-AUC avec fuite (rééchantillonnage avant la validation) : {fuite.mean():.3f} | sans
    ↳ rééchantillonnage : {honnete.mean():.3f}")
```

```
PR-AUC avec fuite (rééchantillonnage avant la validation) : 0.972 | sans rééchantillonnage : 0.187
```

Lecture. Avec la régression logistique, le nombre de voisins k ne change rien (0,214 à 0,220). Avec le boosting, **Borderline-SMOTE (0,573) égale le boosting sans traitement (0,575)**, tandis que SMOTE (0,500) et ADASYN (0,495) le dégradent d'environ 0,08 : une explication plausible

est qu'en ne fabriquant des points qu'à la frontière, Borderline modifie moins la zone centrale des classes (c'est une hypothèse, que ces données ne démontrent pas). SMOTE-NC obtient 0,207 contre 0,187 sans rééchantillonnage : un écart apparent, mais **plus petit que son écart-type entre plis (0,035)**, donc sans valeur probante. La fuite, elle, donne 0,972 contre 0,187 : un score absurde pour un problème difficile.

Exercices

Les exercices sont rangés par difficulté (★ calcul direct, ★★ raisonnement, ★★★ synthèse ou expérience). Essayez-les **à la main d'abord** ; les corrigés suivent.

Exercice 4.1 ★ — L'encodage par la cible, à la main (section 4.1.2)

Huit clients habitent trois villes. La cible vaut 1 si le client est parti : ville P, quatre clients : 1, 1, 0, 0 ; ville Q, trois clients : 1, 0, 0 ; ville R, un client : 1.

- Calculez l'encodage par la cible naïf de chaque ville, et la moyenne générale.
- Calculez l'encodage lissé avec $k = 4$: $\frac{n_m \bar{y}_m + k \mu}{n_m + k}$.
- Quelle ville est la plus suspecte, et pourquoi ?
- Quelle valeur attribuer à un client d'une ville S absente de l'entraînement ?

Exercice 4.2 ★ — Mettre à l'échelle (section 4.1.3)

Cinq montants : 2, 4, 6, 8, 10.

- Standardisez-les (écart-type de la population).
- Appliquez le min-max, puis la transformation robuste (médiane, écart interquartile $Q_3 - Q_1$ avec $Q_1 = 4$ et $Q_3 = 8$).
- Remplacez le dernier montant par 100. Quelles transformations changent pour les quatre premières valeurs ?

Exercice 4.3 ★★ — Box-Cox et Yeo-Johnson (section 4.1.4)

- Appliquez la transformation de Box-Cox avec $\lambda = \frac{1}{2}$ puis avec $\lambda = 0$ aux valeurs 1, 4, 9, 16. Que remarquez-vous pour $\lambda = \frac{1}{2}$?
- Calculez la transformation de Yeo-Johnson de $x = -3$ puis de $x = 5$ pour $\lambda = 1$. Que fait cette transformation pour $\lambda = 1$?

Exercice 4.4 ★★ — Mécanismes d'absence (section 4.1.5)

Pour chaque situation, donnez le mécanisme (MCAR, MAR, MNAR ou structurel) et le traitement que vous conseillez.

- Un capteur de température tombe en panne de temps en temps, sans rapport avec la température.
- Dans un questionnaire, les personnes aux revenus les plus élevés refusent plus souvent de déclarer leur revenu.
- La tension artérielle n'est mesurée que pour les patients de plus de 60 ans (l'âge est toujours renseigné).
- Le « panier moyen » est vide pour les clients qui n'ont jamais commandé.

Exercice 4.5 ★★★ — Trouver les fuites (section 4.1.6)

Voici un script qui prétend évaluer un modèle de départ de clients :

```
X = SimpleImputer(strategy="median").fit_transform(df.drop(columns="y"))
X = StandardScaler().fit_transform(X)
X = SelectKBest(f_classif, k=10).fit_transform(X, df["y"])
Xtr, Xte, ytr, yte = train_test_split(X, df["y"], stratify=df["y"], random_state=0)
Xtr, ytr = SMOTE(random_state=0).fit_resample(Xtr, ytr)
modele = LogisticRegression().fit(Xtr, ytr)
print(roc_auc_score(yte, modele.predict_proba(Xte)[: , 1]))
```

- Repérez les étapes qui fuient et celles qui sont correctes.
- Réécrivez le script avec un Pipeline.
- Mesurez l'écart sur des données de pur bruit (200 lignes, 100 variables, cible aléatoire).

Exercice 4.6 ★ — Créer des variables (section 4.2.1)

Trois clients : A (4 commandes, 1 retour, 2 tickets, ancienneté 30 mois, 200 € sur 12 mois), B (aucune commande, ancienneté 8 mois) et C (10 commandes, 4 retours, 5 tickets, ancienneté 5 mois, 800 €). Calculez pour chacun : le taux de retour (0 s'il n'y a pas de commande), les tickets par commande $\frac{\text{tickets}}{\text{commandes}+1}$, la fréquence mensuelle $\frac{\text{commandes}}{\min(\text{ancienneté}, 12)}$ et le montant par commande. Pour quel client une variable est-elle indéfinie ?

Exercice 4.7 ★★ — Coder une heure sur un cercle (section 4.2.4)

- Donnez les coordonnées $(\sin \frac{2\pi h}{24}, \cos \frac{2\pi h}{24})$ pour $h = 23$, $h = 1$ et $h = 12$.
- Calculez les distances euclidiennes entre 23 h et 1 h, puis entre 23 h et 12 h, et comparez avec les distances sur l'entier h .

Exercice 4.8 ★★ — Variable disponible ou fuite ? (section 4.2.6)

On veut prédire, au 1er du mois, si un abonné **résiliera dans le mois**. Pour chaque variable, dites si elle est utilisable ou si elle constitue une fuite : (a) le nombre de tickets d'assistance des six mois précédents ; (b) la date de résiliation (vide si l'abonné reste) ; (c) le montant facturé pendant le mois ; (d) la note d'une enquête de satisfaction envoyée à la fin du mois ; (e) l'ancienneté au 1er du mois ; (f) un « segment client » calculé avec un modèle entraîné sur l'historique complet, y compris le mois à prédire.

Exercice 4.9 ★ — Lire une table de confusion (section 4.3.1)

Sur 1 000 commandes : 40 fraudes détectées, 60 fausses alertes, 10 fraudes manquées, 890 commandes normales laissées passer. Calculez l'exactitude, la précision, le rappel et le F1. Comparez l'exactitude avec celle d'un modèle qui répond toujours « pas de fraude ».

Exercice 4.10 ★★ — Ce que font les poids (section 4.3.2)

La fréquence de la classe rare est $\pi = 0,02$; on pondère la classe rare par $w = 10$.

- Quelle est la cote de la classe rare, et celle estimée par un modèle sans variable explicative après pondération ?
- Quelle probabilité cela donne-t-il ?
- Un modèle pondéré annonce $p_w = 0,5$ pour un client. Quelle est la probabilité corrigée ?
- À quelle probabilité réelle correspond le seuil de 0,5 du modèle pondéré ?

Exercice 4.11 ★★★ — Un seuil par les coûts (section 4.3.5)

Une fraude manquée coûte $c_{FN} = 50$ € et une fausse alerte $c_{FP} = 2$ €.

- Calculez le seuil théorique.
- Dix commandes, avec leur probabilité prédite (calibrée) et leur vraie classe : $p = (0,9; 0,6; 0,3; 0,2; 0,1; 0,08; 0,05; 0,04; 0,02; 0,01)$, $y = (1; 0; 1; 0; 0; 0; 1; 0; 0; 0)$. Calculez le coût total au seuil 0,5 puis au seuil théorique.
- Les probabilités viennent d'un modèle pondéré avec $w = 20$. Quel seuil faut-il appliquer à ces probabilités gonflées pour obtenir la même décision que le seuil théorique ?

Exercice 4.12 ★★ — Mesurer le biais de sélection (section 4.4)

Simulez 100 clients, 1 000 variables de bruit et une cible aléatoire. Sélectionnez les 10 meilleures variables, puis évaluez par validation croisée, (a) en sélectionnant **avant** la validation et (b) **dans** la validation (20 tirages). (c) Comment l'écart évolue-t-il si l'on passe à 1 000 clients ? Expliquez.

Exercice 4.13 ★★ — SMOTE à la main (section 4.5.1)

- Deux fraudes voisines sont $x = (2; 10)$ et $x' = (6; 18)$. Quel point synthétique SMOTE crée-t-il pour $\lambda = 0,25$, puis pour $\lambda = 0,9$?
- Pourquoi les points synthétiques sont-ils toujours sur le segment $[x, x']$?
- Pour une variable qualitative canal, les cinq voisins d'une fraude ont pour canaux : Site, Site, Réseaux, Boutique, Site. Que fait SMOTE-NC ?

Exercice 4.14 ★★★ — La fuite par rééchantillonnage, sur des données réelles (section 4.5.4)

Le jeu `load_breast_cancer` de `scikit-learn` (569 tumeurs réelles) est livré avec la bibliothèque. Gardez toutes les tumeurs bénignes et seulement 30 tumeurs malignes (la classe rare, à détecter). Mesurez la PR-AUC d'un classifieur des plus proches voisins ($k = 1$) en validation croisée : (a) avec le sur-échantillonnage **dans** un pipeline, (b) avec le sur-échantillonnage **avant** la validation croisée. Expliquez pourquoi le $k = 1$ rend la fuite si spectaculaire.

Corrigés**Corrigé 4.1**

- Ville P : $\frac{2}{4} = 0,5$; ville Q : $\frac{1}{3} \approx 0,333$; ville R : $\frac{1}{1} = 1$. Moyenne générale : $\frac{4}{8} = 0,5$.
- Avec $k = 4$: P : $\frac{4 \times 0,5 + 4 \times 0,5}{4+4} = 0,5$; Q : $\frac{3 \times \frac{1}{3} + 4 \times 0,5}{3+4} = \frac{3}{7} \approx 0,429$; R : $\frac{1 \times 1 + 4 \times 0,5}{1+4} = 0,6$.
- La ville R** : elle ne contient qu'un client, et son encodage naïf (1,0) est exactement l'étiquette de ce client. Le lissage ramène la valeur à 0,6.
- La moyenne générale, 0,5 (valeur de repli).

```
y_ex = pd.Series([1, 1, 0, 0, 1, 0, 0, 1], index=list("PPPPQQQR"))
g = y_ex.groupby(level=0).agg(["mean", "count"])
lisse = (g["count"] * g["mean"] + 4 * y_ex.mean()) / (g["count"] + 4)
print(pd.DataFrame({"naïf": g["mean"], "lissé (k = 4)": lisse}).round(3).T.to_string())
print("moyenne générale :", y_ex.mean())
```

	P	Q	R
naïf	0.5	0.333	1.0
lissé (k = 4)	0.5	0.429	0.6
moyenne générale :	0.5		

Corrigé 4.2

- (a) Moyenne 6, écart-type de la population $\sqrt{8} \approx 2,828$: $z = (-1,414; -0,707; 0; 0,707; 1,414)$.
 (b) Min-max : (0; 0,25; 0,5; 0,75; 1). Robuste : $(x - 6)/4 = (-1; -0,5; 0; 0,5; 1)$.
 (c) Avec 100, la moyenne vaut 24 et l'écart-type environ 38,05 : la standardisation donne $(-0,578; -0,526; -0,473; -0,420; 2,0)$ et le min-max (0; 0,020; 0,041; 0,061; 1) : les quatre premières valeurs sont **écrasées**. La transformation robuste laisse les quatre premières **inchangées** $(-1; -0,5; 0; 0,5)$ et envoie la dernière à 23,5.

```
for valeurs in ([2, 4, 6, 8, 10], [2, 4, 6, 8, 100]):
    v = np.array(valeurs, float)
    q1, q3 = np.percentile(v, [25, 75])
    print(valeurs, "\n standardisé", ((v - v.mean()) / v.std()).round(3), "\n min-max ", ((v -
    ↪ v.min()) / (v.max() - v.min())).round(3),
          "\n robuste ", ((v - np.median(v)) / (q3 - q1)).round(3))
```

```
[2, 4, 6, 8, 10]
standardisé [-1.414 -0.707  0.      0.707  1.414]
min-max     [0.    0.25 0.5   0.75 1.   ]
robuste     [-1.   -0.5  0.    0.5  1.   ]
[2, 4, 6, 8, 100]
standardisé [-0.578 -0.526 -0.473 -0.42  1.997]
min-max     [0.    0.02 0.041 0.061 1.   ]
robuste     [-1.   -0.5  0.    0.5 23.5]
```

Corrigé 4.3

- (a) Pour $\lambda = \frac{1}{2}$: $\frac{\sqrt{x}-1}{1/2} = 2(\sqrt{x} - 1)$, soit (0; 2; 4; 6) : l'écart entre valeurs devient **constant** (2). C'est normal : 1, 4, 9, 16 sont des carrés parfaits, et la racine les ramène à 1, 2, 3, 4 régulièrement espacés. Pour $\lambda = 0$: $\ln x = (0; 1,386; 2,197; 2,773)$: les écarts diminuent, la queue de droite est resserrée.
 (b) Pour $\lambda = 1$: si $x = 5 \geq 0$, $\frac{(5+1)^1-1}{1} = 5$; si $x = -3 < 0$, $-\frac{(1+3)^{2-1}-1}{2-1} = -3$. Pour $\lambda = 1$ la transformation de Yeo-Johnson est l'**identité** : on ne change rien, ce qui est la valeur « neutre » du paramètre.

```
x = np.array([1, 4, 9, 16.0])
print("Box-Cox lambda = 0,5 :", (np.sqrt(x) - 1) / 0.5, "| lambda = 0 :", np.log(x).round(3))
from scipy.stats import yeojohnson
print("Yeo-Johnson, lambda = 1 :", yeojohnson(np.array([-3.0, 5.0]), lmbda=1.0))
```

```
Box-Cox lambda = 0,5 : [0.  2.  4.  6.] | lambda = 0 : [0.    1.386 2.197 2.773]
Yeo-Johnson, lambda = 1 : [-3.   5.]
```

Corrigé 4.4

- (a) **MCAR** (au hasard, sans lien avec la valeur) : supprimer les lignes ou imputer par la médiane est sans biais ; l'indicateur d'absence est inutile.
- (b) **MNAR** : l'absence dépend de la valeur manquante elle-même. On ne peut pas la corriger complètement ; on ajoute un **indicateur d'absence** et on le signale comme limite de l'étude.
- (c) **MAR** : l'absence dépend d'une variable **observée** (l'âge). On peut imputer en s'appuyant sur l'âge (imputation conditionnelle) et garder l'indicateur.
- (d) **Structurel** : la valeur n'existe pas (aucune commande, donc aucun panier). Ne pas imputer par la moyenne : on crée un indicateur (« jamais commandé ») et on traite ces clients comme un groupe à part.

Corrigé 4.5

- (a) **Fuites** : l'imputation, la standardisation et surtout la **sélection de variables** sont faites avant la séparation, donc elles utilisent aussi les lignes du test (et, pour la sélection, **les étiquettes du test**). **Correct** : SMOTE n'est appliqué qu'après la séparation, sur l'entraînement.
- (b) Le pipeline suivant apprend toutes les étapes sur les plis d'entraînement seulement.
- (c) Sur du bruit pur, l'AUC devrait valoir 0,5. Le script fautif donne une valeur trompeusement élevée.

```
from sklearn.feature_selection import SelectKBest, f_classif
from imblearn.over_sampling import SMOTE

def fautif(X, yy):
    Z = StandardScaler().fit_transform(SimpleImputer(strategy="median").fit_transform(X))
    Z = SelectKBest(f_classif, k=10).fit_transform(Z, yy)
    ztr, zte, ytr_, yte_ = train_test_split(Z, yy, stratify=yy, random_state=0)
    ztr, ytr_ = SMOTE(random_state=0).fit_resample(ztr, ytr_)
    return roc_auc_score(yte_, LogisticRegression().fit(ztr, ytr_).predict_proba(zte)[: , 1])

def correct(X, yy):
    Xtr_, Xte_, ytr_, yte_ = train_test_split(X, yy, stratify=yy, random_state=0)
    pipe = Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler()), ("sel",
    SelectKBest(f_classif, k=10)),
    ("smote", SMOTE(random_state=0)), ("lr", LogisticRegression())]).fit(Xtr_, ytr_)
    return roc_auc_score(yte_, pipe.predict_proba(Xte_)[: , 1])

avant, apres = [], []
for g in range(20):
    r = np.random.default_rng(g)
    X, yy = r.normal(size=(200, 100)), r.integers(0, 2, 200)
    avant.append(fautif(X, yy)); apres.append(correct(X, yy))
print(f"AUC de test sur du bruit pur : script fautif {np.mean(avant):.3f} | pipeline
    {np.mean(apres):.3f}")
```

AUC de test sur du bruit pur : script fautif 0.714 | pipeline 0.485

La différence tient à la sélection : choisir les 10 meilleures variables sur **toutes** les lignes (étiquettes du test comprises) trouve des variables qui, par hasard, ressemblent à la cible aussi sur le test.

client	taux de retour	tickets par commande	fréquence mensuelle	montant par commande
--------	----------------	----------------------	---------------------	----------------------

Corrigé 4.6

client	taux de retour	tickets par commande	fréquence mensuelle	montant par commande
A	$\frac{1}{4} = 0,25$	$\frac{2}{4+1} = 0,4$	$\frac{4}{12} \approx 0,333$	$\frac{200}{4} = 50$
B	0 (convention)	$\frac{0}{0+1} = 0$	$\frac{0}{8} = 0$	indéfini (division par zéro)
C	$\frac{4}{10} = 0,4$	$\frac{5}{11} \approx 0,455$	$\frac{10}{5} = 2$	$\frac{800}{10} = 80$

Pour le client B, le **montant par commande** est indéfini : il faut le laisser manquant (avec un indicateur « jamais commandé »), pas le remplacer par 0, qui voudrait dire « il commande pour 0 € ».

```
t = pd.DataFrame({"cmd": [4, 0, 10], "ret": [1, 0, 4], "tick": [2, 0, 5], "anc": [30, 8, 5], "mont": [200, 0, 800]}, index=list("ABC"))
print(pd.DataFrame({"taux_retour": (t.ret / t.cmd.replace(0, np.nan)).fillna(0), "tickets_par_cmd": t.tick / (t.cmd + 1), "freq_mensuelle": t.cmd / np.minimum(t.anc, 12), "montant_par_cmd": t.mont / t.cmd.replace(0, np.nan)}).round(3))
```

	taux_retour	tickets_par_cmd	freq_mensuelle	montant_par_cmd
A	0.25	0.400	0.333	50.0
B	0.00	0.000	0.000	NaN
C	0.40	0.455	2.000	80.0

Corrigé 4.7

- (a) Angles de $15^\circ \times h$. $h = 23 : 345^\circ$, $(\sin, \cos) = (-0,2588; 0,9659)$; $h = 1 : 15^\circ$, $(0,2588; 0,9659)$; $h = 12 : 180^\circ$, $(0; -1)$.
- (b) $d(23 \text{ h}, 1 \text{ h}) = \sqrt{(0,2588 + 0,2588)^2 + 0^2} = 0,518$; $d(23 \text{ h}, 12 \text{ h}) = \sqrt{0,2588^2 + 1,9659^2} \approx 1,983$. Sur l'entier, les distances sont 22 et 11 : minuit semble **plus loin** de 23 h que midi. Sur le cercle, 23 h et 1 h sont proches (0,518) et 23 h et midi opposés (1,983), comme dans la réalité.

```
h = np.array([23, 1, 12])
pts = np.c_[np.sin(2 * np.pi * h / 24), np.cos(2 * np.pi * h / 24)]
print(pts.round(4))
print("d(23 h, 1 h) =", np.linalg.norm(pts[0] - pts[1]).round(3), "| d(23 h, 12 h) =", np.linalg.norm(pts[0] - pts[2]).round(3))
```

```
[[-0.2588  0.9659]
 [ 0.2588  0.9659]
 [ 0.      -1.     ]]
d(23 h, 1 h) = 0.518 | d(23 h, 12 h) = 1.983
```

Corrigé 4.8

- (a) **Utilisable** : connu au 1er du mois (historique). (b) **Fuite** : la date de résiliation est précisément ce qu'on veut prédire. (c) **Fuite** : le montant facturé dans le mois dépend de la résiliation (un abonné résilié ne reçoit pas de facture complète). (d) **Fuite** : une enquête envoyée à la fin du mois suit la résiliation et peut en dépendre. (e) **Utilisable** : connu au

1er du mois. (f) **Fuite possible** : le modèle a vu le mois à prédire pendant son entraînement ; la variable contient de l'information du futur. Question de contrôle pour chaque variable : à quelle date cette valeur est-elle connue, par rapport à la date de prédiction ?

Corrigé 4.9

Exactitude : $\frac{40+890}{1000} = 0,93$. Précision : $\frac{40}{40+60} = 0,4$. Rappel : $\frac{40}{40+10} = 0,8$. F1 : $\frac{2 \times 0,4 \times 0,8}{0,4+0,8} \approx 0,533$.
Le modèle paresseux a une exactitude de $\frac{950}{1000} = 0,95$, **supérieure** à celle du modèle utile (0,93), alors que son rappel est nul : l'exactitude est trompeuse.

```
vp, fp, fn, vn = 40, 60, 10, 890
prec, rap = vp / (vp + fp), vp / (vp + fn)
print("exactitude", (vp + vn) / 1000, "| précision", prec, "| rappel", rap, "| F1", round(2 * prec *
↪ rap / (prec + rap), 3), "| modèle paresseux :", (vn + fp) / 1000)
```

```
exactitude 0.93 | précision 0.4 | rappel 0.8 | F1 0.533 | modèle paresseux : 0.95
```

Corrigé 4.10

- (a) Cote de la classe rare : $\frac{0,02}{0,98} \approx 0,0204$. Après pondération, la cote estimée est multipliée par $w : 0,204$.
- (b) $p^* = \frac{0,204}{1+0,204} \approx 0,169$: une probabilité de 2 % est devenue 16,9 %.
- (c) Cote pondérée $\frac{0,5}{0,5} = 1$, divisée par $w = 10 : 0,1$, soit $p = \frac{0,1}{1,1} \approx 0,0909$.
- (d) La même : un seuil de 0,5 sur le modèle pondéré revient à un seuil de **9,1 %** sur la probabilité réelle.

```
pi, w = 0.02, 10
cote = pi / (1 - pi)
print("cote", round(cote, 4), "-> pondérée", round(w * cote, 4), "-> probabilité", round(w * cote / (1
↪ + w * cote), 4))
pw = 0.5
print("probabilité corrigée de p_w = 0,5 :", round(pw / (pw + (1 - pw) * w), 4))
```

```
cote 0.0204 -> pondérée 0.2041 -> probabilité 0.1695
probabilité corrigée de p_w = 0,5 : 0.0909
```

Corrigé 4.11

- (a) $s^* = \frac{2}{2+50} \approx 0,0385$.
- (b) Au seuil 0,5, on bloque les commandes 1 et 2 (probabilités 0,9 et 0,6). La commande 2 est une fausse alerte (2 €) ; les fraudes 3 et 7 passent (2 fraudes manquées, 100 €) : coût **102 €**. Au seuil 0,0385, on bloque les huit commandes dont $p \geq 0,04$: aucune fraude manquée, cinq fausses alertes (commandes 2, 4, 5, 6, 8) : coût **10 €**.
- (c) Le seuil sur la probabilité gonflée est $\frac{s^* w}{1-s^*+s^* w} = \frac{0,0385 \times 20}{1-0,0385+0,769} \approx 0,444$: on bloque dès que le modèle pondéré dépasse **0,444**.

```
p = np.array([0.9, 0.6, 0.3, 0.2, 0.1, 0.08, 0.05, 0.04, 0.02, 0.01]); v = np.array([1, 0, 1, 0, 0, 0,
↪ 1, 0, 0, 0])
cout_ex = lambda s: int(((p < s) & (v == 1)).sum() * 50 + ((p >= s) & (v == 0)).sum() * 2)
s_star = 2 / 52
print("seuil théorique", round(s_star, 4), "| coût à 0,5 :", cout_ex(0.5), "| coût au seuil théorique
↪ :", cout_ex(s_star))
print("seuil équivalent pour un modèle pondéré (w = 20) :", round(s_star * 20 / (1 - s_star + s_star *
↪ 20), 3))
```

seuil théorique 0.0385 | coût à 0,5 : 102 | coût au seuil théorique : 10
seuil équivalent pour un modèle pondéré ($w = 20$) : 0.444

Corrigé 4.12

```
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.pipeline import Pipeline

def biais(n, p, graines=20):
    avant, dans = [], []
    cv = StratifiedKFold(5, shuffle=True, random_state=0)
    for g in range(graines):
        r = np.random.default_rng(g)
        X, yy = r.normal(size=(n, p)), r.integers(0, 2, n)
        avant.append(cross_val_score(LogisticRegression(max_iter=2000), SelectKBest(f_classif,
↪ k=10).fit_transform(X, yy), yy, cv=cv, scoring="roc_auc").mean())
        dans.append(cross_val_score(Pipeline([("s", SelectKBest(f_classif, k=10)), ("m",
↪ LogisticRegression(max_iter=2000))]), X, yy, cv=cv, scoring="roc_auc").mean())
    return np.mean(avant), np.mean(dans)

for n in (100, 1000):
    a, d = biais(n, 1000)
    print(f"n = {n:4d}, p = 1000 : AUC sélection avant {a:.3f} | dans la validation {d:.3f}")
```

n = 100, p = 1000 : AUC sélection avant 0.866 | dans la validation 0.489
n = 1000, p = 1000 : AUC sélection avant 0.639 | dans la validation 0.492

(a)-(b) Avec 100 clients et 1 000 variables, la sélection **avant** la validation donne une AUC de 0,866 sur du bruit pur, la sélection **dans** la validation 0,489 (le hasard). (c) Avec 1 000 clients le biais diminue (0,639 au lieu de 0,866) sans disparaître : parmi 1 000 variables aléatoires, il est plus difficile qu'avec 100 clients d'en trouver 10 qui ressemblent à la cible **par hasard**, car la corrélation fortuite d'une variable avec la cible décroît en $1/\sqrt{n}$.

Corrigé 4.13

- (a) $x + \lambda(x' - x)$ avec $x' - x = (4; 8)$: pour $\lambda = 0,25$: $(2 + 1; 10 + 2) = (3; 12)$; pour $\lambda = 0,9$: $(2 + 3,6; 10 + 7,2) = (5,6; 17,2)$.
- (b) Parce que $x + \lambda(x' - x) = (1 - \lambda)x + \lambda x'$ avec $\lambda \in [0, 1]$ est une **combinaison convexe** de x et x' : elle décrit exactement le segment qui les relie.
- (c) SMOTE-NC ne peut pas interpoler une catégorie : le point synthétique reçoit la modalité **la plus fréquente parmi les voisins**, ici « Site » (3 voisins sur 5).

Corrigé 4.14

```
from sklearn.datasets import load_breast_cancer
from sklearn.neighbors import KNeighborsClassifier

d = load_breast_cancer(as_frame=True)
X_bc, y_bc = d.data, (d.target == 0).astype(int) # 1 = tumeur maligne (la classe rare)
r = np.random.default_rng(0)
idx = np.r_[np.where(y_bc == 0)[0], r.choice(np.where(y_bc == 1)[0], 30, replace=False)]
Xr, yr = X_bc.iloc[idx], y_bc.iloc[idx]
cv = StratifiedKFold(5, shuffle=True, random_state=0)
dans = cross_val_score(Pipeline([("sc", StandardScaler()), ("o", RandomOverSampler(random_state=0)),
↪ ("m", KNeighborsClassifier(1))]), Xr, yr, cv=cv, scoring="average_precision")
Xo, yo = RandomOverSampler(random_state=0).fit_resample(StandardScaler().fit_transform(Xr), yr)
avant = cross_val_score(KNeighborsClassifier(1), Xo, yo, cv=cv, scoring="average_precision")
```

```
print("part de tumeurs malignes :", round(yr.mean(), 3))
print(f"suréchantillonnage DANS le pipeline : PR-AUC {dans.mean():.3f} | AVANT la validation croisée  
↪ : {avant.mean():.3f}")
```

```
part de tumeurs malignes : 0.078
suréchantillonnage DANS le pipeline : PR-AUC 0.881 | AVANT la validation croisée : 0.994
```

Le sur-échantillonnage avant la validation recopie chaque tumeur maligne plusieurs fois : une copie tombe dans le pli d'entraînement, une autre dans le pli de validation. Avec $k = 1$, le classifieur retrouve **la copie exacte** du point à évaluer (distance 0) et reprend son étiquette : le score est artificiellement parfait. Ici, 7,8 % des tumeurs retenues sont malignes ; la PR-AUC passe de 0,881 (pipeline correct) à 0,994 (fuite). Le $k = 1$ est le cas extrême, parce que ce classifieur ne fait que retrouver le point le plus proche.

Chapitre 5 : Évaluation, calibration et interprétabilité — exercices et applications

Ce chapitre du cahier accompagne le chapitre 5 du livre. Les **applications** refont pas à pas, sur les données de la boutique et sur le jeu réel de crédit, les calculs que le livre n'a fait que résumer (métriques, seuils, calibration, importances, SHAP, équité, prédiction conforme) ; les **exercices** se travaillent d'abord à la main, et leurs **corrigés** viennent à la fin. Chaque exercice indique la section du livre qu'il met en pratique.

Préparation

Une seule cellule charge les bibliothèques et les données, répartit les clients en trois jeux (entraînement 7 200, calibration 2 400, test 2 400, stratifiés sur `churn_90j`) et ajuste les trois modèles du chapitre : régression logistique, forêt aléatoire et gradient boosting. Les applications suivantes en réutilisent les noms : `P` (probabilités de test des trois modèles), `p` (celles du boosting), `yt` (les départs observés sur le jeu de test).

```
import numpy as np, pandas as pd, warnings, math, itertools
warnings.filterwarnings("ignore")
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from sklearn import metrics as M
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.preprocessing import StandardScaler

c = pd.read_csv("donnees/clients_ml.csv")
X = pd.get_dummies(c.drop(columns=["churn_90j", "depense_6m", "segment_vrai",
    ↪ "commandes_apres_cible", "id_client"]),
    columns=["ville", "canal_acquisition", "appareil", "categorie_preferee"],
    ↪ dtype=float)
X["panier_manquant"] = c["panier_moyen"].isna().astype(float); X["satisfaction_manquante"] =
    ↪ c["satisfaction_moy"].isna().astype(float)
y = c["churn_90j"]
Xa, Xte, ya, yte = train_test_split(X, y, test_size=0.2, random_state=0, stratify=y)
Xtr, Xca, ytr, yca = train_test_split(Xa, ya, test_size=0.25, random_state=0, stratify=ya)
med = Xtr.median(); Xtr, Xca, Xte = Xtr.fillna(med), Xca.fillna(med), Xte.fillna(med) # médianes
    ↪ de l'entraînement seul
sc = StandardScaler().fit(Xtr)
lr = LogisticRegression(max_iter=3000).fit(sc.transform(Xtr), ytr)
```

```

rf = RandomForestClassifier(150, min_samples_leaf=5, random_state=0, n_jobs=1).fit(Xtr, ytr)
gb = HistGradientBoostingClassifier(random_state=0).fit(Xtr, ytr)
P = {"logistique": lr.predict_proba(sc.transform(Xte))[:, 1], "forêt": rf.predict_proba(Xte)[:, 1],
     ↪ "boosting": gb.predict_proba(Xte)[:, 1]}
yt = yte.to_numpy(); p = P["boosting"]
print(len(Xtr), len(Xca), len(Xte), round(yt.mean(), 3), {k: round(M.roc_auc_score(yt, v), 4) for k,
     ↪ v in P.items()})

```

```
7200 2400 2400 0.14 {'logistique': 0.8593, 'forêt': 0.8881, 'boosting': 0.8971}
```

Les AUC du chapitre (0,859 ; 0,888 ; 0,897) sont retrouvées : les découpages et les graines sont les mêmes que dans le livre.

Applications

Application 5.1 — Du score à la décision : seuils, coûts et gain

Sections du livre : 5.1.2 à 5.1.7. **Objectif :** transformer les probabilités du boosting en une décision de campagne, et vérifier jusqu'où la formule du seuil $t^* = c/(sV)$ est fiable. Un contact coûte $c = 5$ €, sauve un partant avec la probabilité $s = 0,4$, et un client sauvé vaut $V = 60$ €.

Étape 1 — Le gain de la campagne pour quelques seuils. La fonction `gain_net` additionne, pour les clients dont le score atteint le seuil, le gain attendu ($s * V$ pour un partant) et retranche le coût de contact.

```

def gain_net(score, t, c_contact=5.0, s=0.4, V=60.0):
    contacte = score >= t
    return float((s * V * yt[contacte]).sum() - c_contact * contacte.sum())

for t in [0.05, 0.1, 0.2, 0.3, 0.5]:
    yh = (p >= t).astype(int)
    print(f"t={t:.2f} alertes={yh.sum():4d} précision={M.precision_score(yt, yh):.3f}
    ↪ rappel={M.recall_score(yt, yh):.3f} gain={gain_net(p, t):7.1f} €")

```

t=0.05	alertes= 954	précision=0.322	rappel=0.911	gain= 2598.0 €
t=0.10	alertes= 692	précision=0.399	rappel=0.819	gain= 3164.0 €
t=0.20	alertes= 460	précision=0.517	rappel=0.706	gain= 3412.0 €
t=0.30	alertes= 350	précision=0.603	rappel=0.626	gain= 3314.0 €
t=0.50	alertes= 230	précision=0.709	rappel=0.484	gain= 2762.0 €

Parmi les cinq seuils essayés, le meilleur est 0,2 (3 412 €) ; le seuil habituel de 0,5 n'en rapporte que 2 762 €, parce qu'il ne détecte que 48 % des partants.

Étape 2 — Le seuil optimal sur une grille fine.

```

grille = np.round(np.arange(0.02, 0.8, 0.01), 2)
g = np.array([gain_net(p, t) for t in grille])
print("seuil optimal :", grille[g.argmax()], "| gain :", round(g.max(), 1), "| seuil théorique c/(sV)
    ↪ :", round(5 / (0.4 * 60), 3))

```

```
seuil optimal : 0.18 | gain : 3473.0 | seuil théorique c/(sV) : 0.208
```

Le maximum observé est de 3 473 € au seuil 0,18. Le seuil théorique $5/24 = 0,208$ en donne 3 391 € : 82 € de moins (2,4 %), car le gain d'un jeu de 2 400 clients est bruité et la courbe est plate autour de l'optimum.

Étape 3 — Sensibilité à l'économie du problème. On fait varier la probabilité de succès s et le coût de contact c , et l'on compare le seuil théorique au seuil optimal de la grille.

```
lignes = []
for s in [0.2, 0.4, 0.6]:
    for cc in [2.0, 5.0, 10.0]:
        gg = np.array([gain_net(p, t, cc, s) for t in grille])
        lignes.append((s, cc, round(cc / (s * 60), 3), grille[gg.argmax()], round(gg.max(), 0)))
print(pd.DataFrame(lignes, columns=["succès s", "coût c", "seuil théorique", "seuil optimal",
    ↳ (grille)", "gain max"]).to_string(index=False))
```

succès s	coût c	seuil théorique	seuil optimal (grille)	gain max
0.2	2.0	0.167	0.12	1988.0
0.2	5.0	0.417	0.40	832.0
0.2	10.0	0.833	0.79	58.0
0.4	2.0	0.083	0.05	5460.0
0.4	5.0	0.208	0.18	3473.0
0.4	10.0	0.417	0.40	1664.0
0.6	2.0	0.056	0.05	9144.0
0.6	5.0	0.139	0.12	6596.0
0.6	10.0	0.278	0.30	4096.0

Dans les neuf cas, le seuil optimal observé reste proche du seuil théorique (écart maximal de 0,05 avec la grille). La logique est celle attendue : plus un contact coûte cher, ou moins il réussit, plus le seuil monte et moins on contacte. Dans le cas extrême ($s = 0,2$, $c = 10$ €), le seuil théorique est de 0,83 et le gain maximal tombe à 58 € : presque aucun client ne justifie un contact.

Étape 4 — La formule suppose de bonnes probabilités, donc dépend du modèle. On applique le même seuil théorique aux trois modèles.

```
t_th = 5 / (0.4 * 60)
for k, pk in P.items():
    print(f"{k:11s} contacts {int((pk >= t_th).sum()):4d} gain au seuil théorique {gain_net(pk,
    ↳ t_th):7.1f} € | meilleur gain sur la grille {max(gain_net(pk, t) for t in grille):7.1f} €")
```

logistique	contacts	568	gain au seuil théorique	2776.0 €	meilleur gain sur la grille	2829.0 €
forêt	contacts	550	gain au seuil théorique	3226.0 €	meilleur gain sur la grille	3352.0 €
boosting	contacts	445	gain au seuil théorique	3391.0 €	meilleur gain sur la grille	3473.0 €

À seuil identique, les modèles ne contactent pas le même nombre de clients (568, 550 et 445) et ne rapportent pas autant : 2 776 € pour la logistique, 3 226 € pour la forêt, 3 391 € pour le boosting. Un écart d'AUC de 0,04 représente donc, ici, environ 600 € de gain par campagne de 2 400 clients. Remarquez aussi que, pour la logistique et la forêt, le seuil théorique est un peu moins bon que le meilleur seuil de la grille (écart de 53 € et de 126 €) : leurs probabilités sont un peu moins justes (voir l'application 5.5).

Pour aller plus loin. Ajoutez une contrainte de budget (« au plus 300 contacts ») : le seuil optimal devient celui qui sélectionne exactement 300 clients. Comparez le gain à celui du seuil théorique (445 contacts).

Application 5.2 — La courbe ROC, l'AUC et la précision moyenne reconstruites à la main

Sections du livre : 5.1.4 et 5.1.5. Objectif : ne plus voir l'AUC comme une boîte noire en la recalculant de trois façons, puis voir de ses propres yeux pourquoi elle ne réagit pas à la prévalence.

Étape 1 — La courbe ROC par tri des scores. On trie les clients par score décroissant ; en descendant la liste, le rappel (parmi les positifs) et le taux de fausses alertes (parmi les négatifs) augmentent. L'aire se calcule par trapèzes.

```
def roc_manuelle(y_vrai, score):
    ordre = np.argsort(-score, kind="stable"); ys, ss = y_vrai[ordre], score[ordre]
```

```

tp, fp = np.cumsum(ys), np.cumsum(1 - ys)
garde = np.r_[np.diff(ss) != 0, True] # un point par valeur distincte du score
return np.r_[0, fp[garde] / fp[-1]], np.r_[0, tp[garde] / tp[-1]]

fpr, tpr = roc_manuelle(yt, p)
auc_trapeze = np.sum(np.diff(fpr) * (tpr[1:] + tpr[:-1]) / 2)
print("AUC par trapèzes :", round(auc_trapeze, 4), "| scikit-learn :", round(M.roc_auc_score(yt, p),
↪ 4), "| points de la courbe :", len(fpr))

```

```
AUC par trapèzes : 0.8971 | scikit-learn : 0.8971 | points de la courbe : 2401
```

L'aire par trapèzes (0,8971) est celle de scikit-learn. La courbe a 2 401 points : un par valeur distincte de score, plus l'origine.

Étape 2 — L'AUC comme statistique de Mann-Whitney. Par les rangs : on additionne les rangs des positifs, on retranche le minimum possible, et l'on divise par le nombre de paires.

```

from scipy.stats import rankdata
rangs = rankdata(p)
n_pos, n_neg = int(yt.sum()), int((1 - yt).sum())
U = rangs[yt == 1].sum() - n_pos * (n_pos + 1) / 2 # statistique de Mann-Whitney
print("U / (n+ n-) =", round(U / (n_pos * n_neg), 4), "| nombre de paires :", n_pos * n_neg)

```

```
U / (n+ n-) = 0.8971 | nombre de paires : 695231
```

Le résultat est le même, 0,8971, calculé sur 695 231 paires (337 partants fois 2 063 fidèles) : l'AUC est la probabilité qu'un partant ait un score supérieur à celui d'un fidèle (volume I, section 3.7.2).

Étape 3 — La précision moyenne.

```

prec, rap, _ = M.precision_recall_curve(yt, p) # le rappel est décroissant dans ces
↪ tableaux
ap_manuelle = -np.sum(np.diff(rap) * prec[:-1])
print("AP à la main :", round(ap_manuelle, 4), "| scikit-learn :",
↪ round(M.average_precision_score(yt, p), 4))

```

```
AP à la main : 0.6496 | scikit-learn : 0.6496
```

Étape 4 — L'effet de la prévalence. On donne à chaque client fidèle un poids m : c'est comme si l'on avait m fois plus de fidèles, avec les mêmes scores.

```

lignes = []
for m in [1, 3, 10, 30]:
    w = np.where(yt == 0, float(m), 1.0)
    lignes.append((m, round((w * yt).sum() / w.sum(), 3), round(M.roc_auc_score(yt, p,
↪ sample_weight=w), 4), round(M.average_precision_score(yt, p, sample_weight=w), 4)))
print(pd.DataFrame(lignes, columns=["négatifs x m", "prévalence", "AUC",
↪ "AP"]).to_string(index=False))

```

négatifs x m	prévalence	AUC	AP
1	0.140	0.8971	0.6496
3	0.052	0.8971	0.4399
10	0.016	0.8971	0.2343
30	0.005	0.8971	0.1152

L'AUC reste figée à 0,8971 quel que soit m ; la précision moyenne s'effondre de 0,650 à 0,115 quand la prévalence passe de 14 % à 0,5 %. Pour un détecteur de fraude (prévalence de l'ordre du pour-cent), c'est la seconde colonne qui décrit l'expérience réelle de l'utilisateur.

Application 5.3 — Lift, gain et incertitude sur la comparaison de deux modèles

Section du livre : 5.1.8. **Objectif** : raisonner en budget (« je ne peux contacter que 20 % des clients »), puis se demander si l'avantage d'un modèle sur un autre est plus grand que le bruit.

Étape 1 — Part des partants atteints et lift, pour trois budgets.

```
def capture(score, part):
    k = int(part * len(score)); o = np.argsort(-score)[:k]
    return yt[o].sum() / yt.sum(), yt[o].mean() / yt.mean()    # part des partants atteints, lift

lignes = [(k, part, *np.round(capture(pk, part), 3)) for k, pk in P.items() for part in (0.1, 0.2,
↪ 0.3)]
print(pd.DataFrame(lignes, columns=["modèle", "part contactée", "partants atteints",
↪ "lift"]).to_string(index=False))
```

modèle	part contactée	partants atteints	lift
logistique	0.1	0.412	4.125
logistique	0.2	0.641	3.205
logistique	0.3	0.760	2.532
forêt	0.1	0.481	4.807
forêt	0.2	0.706	3.531
forêt	0.3	0.813	2.710
boosting	0.1	0.499	4.985
boosting	0.2	0.724	3.620
boosting	0.3	0.831	2.770

En contactant 10 % des clients, le boosting atteint 49,9 % des partants (lift 4,99), la forêt 48,1 % et la logistique 41,2 %. À 20 % de budget : 72,4 %, 70,6 % et 64,1 %. Les deux modèles à arbres sont proches ; la logistique est nettement en retrait.

Étape 2 — Du budget au gain en euros. Avec les mêmes économies que l'application 5.1 (24 € d'espérance par partant contacté, 5 € par contact), on compare le gain du modèle à celui d'un ciblage au hasard.

```
for part in [0.05, 0.1, 0.2, 0.3, 0.5]:
    k = int(part * len(p)); o = np.argsort(-p)[:k]
    modele = 24 * yt[o].sum() - 5 * k    # 24 = 0,4 x 60 : gain attendu d'un
    ↪ partant contacté ; 5 € par contact
    hasard = 24 * yt.mean() * k - 5 * k    # espérance avec un ciblage au hasard
    print(f"{part:4.0%} contactés ({k:4d}) : gain du modèle {modele:7.1f} € | gain d'un ciblage au
    ↪ hasard {hasard:7.1f} €")
```

5% contactés (120) :	gain du modèle	1776.0 €	gain d'un ciblage au hasard	-195.6 €
10% contactés (240) :	gain du modèle	2832.0 €	gain d'un ciblage au hasard	-391.2 €
20% contactés (480) :	gain du modèle	3456.0 €	gain d'un ciblage au hasard	-782.4 €
30% contactés (720) :	gain du modèle	3120.0 €	gain d'un ciblage au hasard	-1173.6 €
50% contactés (1200) :	gain du modèle	1608.0 €	gain d'un ciblage au hasard	-1956.0 €

Le ciblage au hasard perd de l'argent à tous les budgets (chaque contact coûte 5 € et ne rapporte en moyenne que $24 \times 0,14 = 3,4$ €). Le modèle atteint son maximum vers 20 % de budget (3 456 €) puis décroît, puisque les contacts supplémentaires visent des clients dont le risque est inférieur au seuil de rentabilité.

Étape 3 — Le bruit du jeu de test. L'AUC du boosting dépasse celle de la logistique de 0,038 : cet écart est-il plus grand que le hasard ? On rééchantillonne 300 fois les clients du jeu de test (*bootstrap*, volume I, section 3.3.5) et l'on recalcule l'écart à chaque fois, **pour les deux modèles sur les mêmes clients** (comparaison appariée).

```
rng = np.random.default_rng(0); n = len(yt); diffs = []
for _ in range(300):
    i = rng.integers(0, n, n)    # rééchantillonnage des clients du
    ↪ jeu de test
```

```

diffs.append(M.roc_auc_score(yt[i], P["boosting"][i]) - M.roc_auc_score(yt[i],
↪ P["logistique"][i]))
obs = M.roc_auc_score(yt, P["boosting"]) - M.roc_auc_score(yt, P["logistique"])
print("écart d'AUC observé :", round(obs, 4), "| intervalle à 95 % (bootstrap apparié) :",
↪ np.round(np.percentile(diffs, [2.5, 97.5]), 4))

```

```
écart d'AUC observé : 0.0378 | intervalle à 95 % (bootstrap apparié) : [0.0254 0.05 ]
```

L'intervalle à 95 % de l'écart d'AUC est de [0,025 ; 0,050] : il exclut 0, et l'avantage du boosting sur la logistique est donc réel, du moins sur ce type de données. *Pour aller plus loin* : refaites l'exercice pour la forêt contre le boosting. L'écart observé est de 0,009 : l'intervalle contient-il 0 ?

Application 5.4 — Régression : quelle perte pour quelle décision ?

Section du livre : 5.1.10. Objectif : constater que le « meilleur » modèle dépend de la métrique, donc de la décision. Un stock doit être préparé pour la dépense des six mois à venir de chaque client. Manquer 1 € de demande (rupture) coûte 3 €, en prévoir 1 € de trop (surstock) coûte 1 €.

Étape 1 — Trois modèles, trois pertes d'entraînement. Un boosting entraîné à minimiser l'erreur quadratique (il prédit la moyenne), un autre l'erreur absolue (la médiane), un troisième la perte pinball de niveau 0,75 (le troisième quartile).

```

from sklearn.ensemble import HistGradientBoostingRegressor as HGBR
yr = c["depense_6m"]; ytr_r, yte_r = yr.loc[Xtr.index].to_numpy(), yr.loc[Xte.index].to_numpy()
modeles = {"moyenne (carré)": HGBR(max_iter=150, random_state=0), "médiane (absolu)":
↪ HGBR(loss="absolute_error", max_iter=150, random_state=0),
          "quantile 0,75": HGBR(loss="quantile", quantile=0.75, max_iter=150, random_state=0)}
pred = {k: np.clip(m.fit(Xtr, ytr_r).predict(Xte), 0, None) for k, m in modeles.items()}
def pinball(yv, q, tau): return float(np.mean(np.maximum(tau * (yv - q), (tau - 1) * (yv - q))))
for k, q in pred.items():
    print(f"{k:17s} MAE {M.mean_absolute_error(yte_r, q):6.2f} RMSE {M.mean_squared_error(yte_r, q)
↪ ** 0.5:6.2f} pinball(0,75) {pinball(yte_r, q, 0.75):6.2f} part des y <= prévision
↪ {np.mean(yte_r <= q):.3f}")

```

moyenne (carré)	MAE	59.21	RMSE	98.61	pinball(0,75)	29.23	part des y <= prévision	0.612
médiane (absolu)	MAE	55.42	RMSE	97.14	pinball(0,75)	31.41	part des y <= prévision	0.558
quantile 0,75	MAE	67.10	RMSE	104.40	pinball(0,75)	25.94	part des y <= prévision	0.729

Chacun gagne sur la métrique qui est sa perte : le modèle de la médiane a la plus petite erreur absolue (55,4 €), le modèle quantile la plus petite perte pinball de niveau 0,75 (25,9). (Le modèle de la médiane a aussi un RMSE légèrement meilleur que celui de la moyenne, 97,1 contre 98,6 : l'écart est faible et la perte d'entraînement n'a pas à coïncider avec la métrique d'évaluation.) Le modèle quantile a la pire erreur absolue (67,1 €), et il n'est pas biaisé « par erreur » : il vise volontairement le troisième quartile ; 72,9 % des clients ont une dépense inférieure à sa prévision (la cible est 75 %).

Étape 2 — La perte qui compte : le coût du stock. Le coût d'une rupture est trois fois celui d'un surstock : le niveau optimal est le quantile de niveau $\tau = \frac{c_{\text{rupture}}}{c_{\text{rupture}} + c_{\text{surstock}}} = 0,75$ (c'est le problème du « vendeur de journaux »).

```

c_rupture, c_surstock = 3.0, 1.0 # manquer 1 € de demande coûte 3 € ; en prévoir 1 € de trop
↪ coûte 1 €
print("niveau optimal tau = c_rupture / (c_rupture + c_surstock) =", c_rupture / (c_rupture +
↪ c_surstock))
for k, q in pred.items():
    cout = np.mean(np.where(yte_r > q, c_rupture * (yte_r - q), c_surstock * (q - yte_r)))
    print(f"{k:17s} coût moyen par client : {cout:6.2f} €")

```

```
niveau optimal tau = c_rupture / (c_rupture + c_surstock) = 0.75
moyenne (carré)   coût moyen par client : 116.92 €
médiane (absolu) coût moyen par client : 125.64 €
quantile 0,75     coût moyen par client : 103.75 €
```

Le coût moyen par client est de **103,8 €** avec le modèle quantile, contre **116,9 €** avec le modèle de la moyenne et **125,6 €** avec celui de la médiane : 11 % d'économie, alors que ce modèle a la pire erreur absolue de la liste. Le bon modèle est celui dont **la perte est la perte de la décision**.

Application 5.5 — Diagnostiquer et réparer la calibration

Sections du livre : 5.2.1 à 5.2.6. **Objectif :** mesurer la calibration de modèles, réparer une logistique pondérée, et chiffrer en euros le coût d'une mauvaise calibration.

Étape 1 — Diagnostic. On ajoute aux trois modèles la logistique pondérée (`class_weight="balanced"`), puis l'on affiche le score de Brier, l'ECE et la probabilité moyenne prédite. On regarde enfin le diagramme de fiabilité de la forêt sous forme de tableau.

```
def fiabilite(yv, pv, n_bins=10):
    groupes = np.array_split(np.argsort(pv), n_bins)          # classes de même effectif
    return np.array([[pv[g].mean(), yv[g].mean(), len(g)] for g in groupes])
def ece(yv, pv, n_bins=10):
    t = fiabilite(yv, pv, n_bins); return float(np.sum(t[:, 2] * np.abs(t[:, 0] - t[:, 1])) / t[:,
    ↪ 2].sum())

lr_pond = LogisticRegression(max_iter=3000, class_weight="balanced").fit(sc.transform(Xtr), ytr)
P["logistique pondérée"] = lr_pond.predict_proba(sc.transform(Xte))[:, 1]
for k, pk in P.items():
    print(f"{k:21s} Brier {M.brier_score_loss(yt, pk):.4f} ECE {ece(yt, pk):.4f} probabilité moyenne
    ↪ {pk.mean():.3f}")
print(pd.DataFrame(fiabilite(yt, P["forêt"]), columns=["prob. moyenne", "taux observé",
    ↪ "effectif"]).round(3).to_string(index=False))
```

logistique	Brier 0.0872	ECE 0.0120	probabilité moyenne 0.136
forêt	Brier 0.0808	ECE 0.0314	probabilité moyenne 0.139
boosting	Brier 0.0758	ECE 0.0137	probabilité moyenne 0.128
logistique pondérée	Brier 0.1542	ECE 0.2087	probabilité moyenne 0.349
prob. moyenne	taux observé	effectif	
0.003	0.004	240.0	
0.011	0.000	240.0	
0.022	0.008	240.0	
0.040	0.033	240.0	
0.062	0.038	240.0	
0.093	0.079	240.0	
0.135	0.100	240.0	
0.193	0.150	240.0	
0.284	0.317	240.0	
0.543	0.675	240.0	

La logistique pondérée classe bien (voir son AUC dans le livre) mais annonce en moyenne 35 % de départs alors qu'ils sont 14 % : Brier de 0,154, ECE de 0,209. La forêt est sous-confiante : dans la dernière classe, elle annonce 54 % pour 68 % observés.

Étape 2 — Platt et isotonique, ajustés sur le jeu de calibration.

```
from sklearn.isotonic import IsotonicRegression
def logit(s): s = np.clip(s, 1e-4, 1 - 1e-4); return np.log(s / (1 - s)).reshape(-1, 1)

yc = yca.to_numpy(); s_cal = lr_pond.predict_proba(sc.transform(Xca))[:, 1]          # calibration sur le
    ↪ jeu de CALIBRATION
platt = LogisticRegression(C=1e6, max_iter=1000).fit(logit(s_cal), yc)
```

```
iso = IsotonicRegression(out_of_bounds="clip").fit(s_cal, yc)
s_te = P["logistique pondérée"]
versions = {"brut": s_te, "Platt": platt.predict_proba(logit(s_te))[:, 1], "isotonique":
↳ iso.predict(s_te)}
for k, pk in versions.items(): print(f"{k:11s} Brier {M.brier_score_loss(yt, pk):.4f} ECE {ece(yt,
↳ pk):.4f}")
print("Platt : pente", round(float(platt.coef_[0, 0]), 3), "| ordonnée",
↳ round(float(platt.intercept_[0]), 3), "| ln(pi/(1-pi)) =", round(float(np.log(ytr.mean() / (1 -
↳ ytr.mean()))), 3))
```

```
brut      Brier 0.1542  ECE 0.2087
Platt     Brier 0.0881  ECE 0.0145
isotonique Brier 0.0887  ECE 0.0191
Platt : pente 1.063 | ordonnée -1.789 | ln(pi/(1-pi)) = -1.812
```

Les deux méthodes ramènent le Brier de 0,154 à 0,088-0,089 (celui d'une logistique ordinaire est de 0,087) et l'ECE de 0,209 à 0,015-0,019. L'ordonnée de Platt, $-1,789$, est proche de $\ln \frac{\pi}{1-\pi} = -1,812$: la méthode a retrouvé, sans qu'on le lui dise, le décalage de prévalence créé par les poids de classes (exercice 5.8).

Étape 3 — Ce que cela change pour la décision. On applique le seuil théorique de l'application 5.1 (0,208) aux probabilités brutes, puis recalibrées.

```
t_th = 5 / (0.4 * 60) # seuil théorique de la campagne (application 5.1) :
↳ valable pour des probabilités CALIBRÉES
for k, pk in versions.items():
    print(f"{k:11s} contacts {int((pk >= t_th).sum()):4d} gain net {gain_net(pk, t_th):8.1f} €")
```

```
brut      contacts 1363 gain net 937.0 €
Platt     contacts 589 gain net 2671.0 €
isotonique contacts 571 gain net 2617.0 €
```

Avec les probabilités brutes, le seuil théorique fait contacter **1 363 clients** et ne rapporte que **937 €** ; avec les probabilités recalibrées, 589 clients et **2 671 €** (571 clients et 2 617 € pour l'isotonique). Le classement des clients est pourtant identique : la mauvaise calibration a coûté plus de 1 700 € sans que l'AUC ne bouge.

Application 5.6 — Trois importances, trois questions

Sections du livre : 5.3.2 et 5.3.6. **Objectif** : comparer l'importance par permutation, la moyenne des valeurs SHAP et l'importance « gain » de LightGBM, et comprendre pourquoi elles diffèrent.

Étape 1 — Les trois tableaux dans un seul.

```
import lightgbm as lgb, shap
from sklearn.inspection import permutation_importance
mg = lgb.LGBMClassifier(n_estimators=150, learning_rate=0.05, num_leaves=15, min_child_samples=40,
↳ subsample=0.8, subsample_freq=1,
    colsample_bytree=0.8, random_state=0, verbose=-1, n_jobs=1).fit(Xtr, ytr)
perm = permutation_importance(mg, Xte, yte, scoring="roc_auc", n_repeats=5, random_state=0,
↳ n_jobs=1).importances_mean
sv = shap.TreeExplainer(mg).shap_values(Xte.iloc[:500]); sv = sv[1] if isinstance(sv, list) else sv
gain = mg.booster_.feature_importance("gain")
tab = pd.DataFrame({"permutation": perm, "SHAP moyen": np.abs(sv).mean(axis=0), "gain (part)": gain /
↳ gain.sum()}, index=Xte.columns)
print(tab.sort_values("SHAP moyen", ascending=False).head(8).round(4).to_string())
```

```
          permutation  SHAP moyen  gain (part)
age                0.0655      0.6834      0.1962
montant_12m        0.0403      0.6476      0.1271
```

recence_jours	0.0612	0.5583	0.1909
part_achats_promo	0.0200	0.2780	0.0744
satisfaction_moy	0.0409	0.2666	0.1380
nb_commandes_12m	0.0072	0.2179	0.0402
programme_fidelite	0.0088	0.2036	0.0281
taux_ouverture_email	-0.0008	0.1394	0.0347

L'âge, la récence et le montant annuel dominant selon les trois mesures. Mais voyez `taux_ouverture_email` : sa permutation est négative ($-0,0008$, c'est-à-dire nulle au bruit près), alors que sa contribution SHAP moyenne (0,139) et sa part de gain (3,5 %) sont substantielles.

Étape 2 — Les rangs.

```
rangs = tab.rank(ascending=False)
print(rangs.corr(method="spearman").round(3))
print(rangs.loc[["taux_ouverture_email", "panier_moyen", "anciennete_mois"]].astype(int))
```

	permutation	SHAP moyen	gain (part)
permutation	1.000	0.642	0.618
SHAP moyen	0.642	1.000	0.951
gain (part)	0.618	0.951	1.000

	permutation	SHAP moyen	gain (part)
taux_ouverture_email	47	8	7
panier_moyen	45	10	10
anciennete_mois	11	11	8

Les deux mesures qui regardent l'intérieur du modèle (SHAP et gain) concordent presque parfaitement (corrélation de rangs de Spearman 0,951) ; la permutation, qui mesure la **perte de performance sur le jeu de test**, s'accorde moins avec elles (0,64 et 0,62). L'écart le plus net : `taux_ouverture_email`, au 7^e ou 8^e rang des deux premières mesures et au 47^e pour la permutation. Chaque mesure répond à sa question.

Étape 3 — Les variables corrélées se partagent l'importance. On ajoute à `recence_jours` une copie bruitée (écart-type du bruit : 0, 15 puis 50 jours) et l'on mesure l'importance par permutation de l'originale et de la copie.

```
for bruit in [0, 15, 50]:
    rng = np.random.default_rng(0) # même graine pour chaque niveau
    ↪ de bruit
    A, B = Xtr.copy(), Xte.copy()
    A["copie"] = A["recence_jours"] + rng.normal(0, bruit, len(A)); B["copie"] = B["recence_jours"] +
    ↪ rng.normal(0, bruit, len(B))
    m = lgb.LGBMClassifier(n_estimators=150, learning_rate=0.05, num_leaves=15, min_child_samples=40,
    ↪ subsample=0.8, subsample_freq=1,
                           colsample_bytree=0.8, random_state=0, verbose=-1, n_jobs=1).fit(A, ytr)
    imp = pd.Series(permutation_importance(m, B, yte, scoring="roc_auc", n_repeats=5, random_state=0,
    ↪ n_jobs=1).importances_mean, index=B.columns)
    print(f"bruit de {bruit:2d} jours : récence {imp['recence_jours']:.4f} copie {imp['copie']:.4f}
    ↪ somme {imp['recence_jours'] + imp['copie']:.4f}")
```

```
bruit de 0 jours : récence 0.0551 copie -0.0002 somme 0.0549
bruit de 15 jours : récence 0.0189 copie 0.0126 somme 0.0315
bruit de 50 jours : récence 0.0330 copie 0.0026 somme 0.0356
```

Sans bruit (copie exacte), LightGBM n'utilise en pratique qu'une des deux colonnes : la récence garde 0,0551 et la copie 0. Avec un bruit de 15 jours, l'importance se **partage** (0,0189 et 0,0126) et la somme (0,0315) est bien inférieure à l'importance de la récence seule (0,0612 dans le livre). Avec un bruit de 50 jours, la copie est trop bruitée pour servir (0,0330 contre 0,0026). Morale : la **répartition** de l'importance entre variables voisines n'est pas stable, ni interprétable seule.

Application 5.7 — Expliquer un client : SHAP, LIME et stabilité

Sections du livre : 5.3.4 à 5.3.6. **Objectif** : expliquer une prédiction individuelle de trois façons, et vérifier ce qui est exact et ce qui est approché.

Étape 1 — SHAP et l'efficacité. On choisit un client dont le risque prédit est proche de 55 %.

```
pm = mg.predict_proba(Xte)[:, 1]; idx = int(np.argsort(np.abs(pm - 0.55))[0]) # un client dont le
↳ risque prédit est proche de 55 %
phi = shap.TreeExplainer(mg).shap_values(Xte.iloc[[idx]]); phi = (phi[1] if isinstance(phi, list)
↳ else phi)[0]
haut = np.argsort(-np.abs(phi))[:5]
print("client", idx, "| probabilité prédite", round(float(pm[idx]), 3), "| départ observé",
↳ int(yte.iloc[idx]))
print([(Xte.columns[j], round(float(phi[j]), 3), float(Xte.iloc[idx, j])) for j in haut])
base = float(np.ravel(shap.TreeExplainer(mg).expected_value)[-1])
print("base + somme des SHAP =", round(base + phi.sum(), 4), "| sortie brute du modèle =",
↳ round(float(mg.predict(Xte.iloc[[idx]], raw_score=True)[0]), 4))
```

```
client 961 | probabilité prédite 0.548 | départ observé 0
[('age', 1.019, 26.0), ('recence_jours', 0.848, 365.0), ('part_achats_promo', 0.622, 0.804),
↳ ('montant_12m', 0.565, 0.0), ('satisfaction_moy', -0.485, 5.0)]
base + somme des SHAP = 0.1945 | sortie brute du modèle = 0.1945
```

Le client numéro 961 a un risque de 0,548 (il est finalement resté). Ses cinq plus fortes contributions : l'âge (26 ans, +1,02), une dernière commande vieille d'un an (+0,85), une part d'achats en promotion de 80 % (+0,62), aucun achat dans l'année (+0,57), et une satisfaction de 5 qui fait baisser le risque (-0,49). La valeur de base plus la somme des 47 contributions redonne la sortie brute du modèle, 0,1945 en log-cote : c'est l'efficacité de Shapley, exacte.

Étape 2 — L'instabilité de LIME dépend du nombre d'échantillons. Pour 300, 1 500 et 6 000 clients fictifs, on relance LIME avec cinq graines et l'on mesure le recouvrement des cinq variables les plus importantes.

```
from lime.lime_tabular import LimeTabularExplainer
def vars_lime(graine, n_ech):
    ex = LimeTabularExplainer(Xtr.to_numpy(), feature_names=list(Xtr.columns), mode="classification",
↳ random_state=graine)
    e = ex.explain_instance(Xte.iloc[idx].to_numpy(), mg.predict_proba, num_features=5,
↳ num_samples=n_ech)
    return {next(v for v in Xtr.columns if v in t) for t, _ in e.as_list()}
for n_ech in [300, 1500, 6000]:
    ens = [vars_lime(g, n_ech) for g in range(5)]
    jac = [len(ens[i] & ens[j]) / len(ens[i] | ens[j]) for i in range(5) for j in range(i + 1, 5)]
    print(f"{n_ech:5d} échantillons : recouvrement moyen {np.mean(jac):.2f}, minimum {min(jac):.2f}")
```

```
300 échantillons : recouvrement moyen 0.63, minimum 0.43
1500 échantillons : recouvrement moyen 0.80, minimum 0.67
6000 échantillons : recouvrement moyen 1.00, minimum 1.00
```

Avec 300 échantillons, deux graines partagent en moyenne 63 % de leurs variables (minimum 43 %) ; avec 1 500, 80 % (minimum 67 %) ; avec 6 000, 100 %. LIME se stabilise quand on lui donne assez d'échantillons ; le prix est le temps de calcul. Une explication LIME sans mention du nombre d'échantillons et de la graine n'est pas reproductible.

Étape 3 — Shapley exact par énumération. Pour un modèle simple (une régression logistique sur trois variables, sortie en log-cote), on calcule les valeurs de Shapley du même client en énumérant toutes les coalitions (avec un fond de 300 clients), puis on les compare à la formule $\beta_j(x_j - \bar{x}_j)$.

```
cols = ["recence_jours", "satisfaction_manquante", "nb_tickets_support_12m"]
m3 = LogisticRegression(max_iter=1000).fit(Xtr[cols], ytr)
```

```
fond = Xtr[cols].sample(300, random_state=0).to_numpy(); x0 = Xte[cols].iloc[idx].to_numpy()
def v(S):
    # valeur d'une coalition S : on fixe les variables de S au
    ↪ client, les autres suivent le fond
    Z = fond.copy(); Z[:, list(S)] = x0[list(S)]
    return float((Z @ m3.coef_[0] + m3.intercept_[0]).mean())
phi_exact = [sum(math.factorial(len(S)) * math.factorial(2 - len(S)) / 6 * (v(S + (j,)) - v(S))
    for r in range(3) for S in itertools.combinations([k for k in range(3) if k != j], r))
    ↪ for j in range(3)]
print("Shapley exact :", np.round(phi_exact, 4), "| formule beta_j (x_j - moyenne) :",
    ↪ np.round(m3.coef_[0] * (x0 - fond.mean(axis=0)), 4))
```

```
Shapley exact : [ 1.6919  0.0497 -0.1831] | formule beta_j (x_j - moyenne) : [ 1.6919  0.0497 -0.1831]
```

Les deux calculs coïncident au dernier chiffre (1,6919 ; 0,0497 ; -0,1831) : pour un modèle linéaire, la valeur de Shapley d'une variable est exactement son coefficient fois l'écart du client à la moyenne (exercice 5.10).

Application 5.8 — Audit d'équité sur le jeu réel de crédit

Sections du livre : 5.4.1 à 5.4.4. **Objectif** : auditer un modèle de défaut par groupe, mesurer l'incertitude des écarts, puis tester l'équité « par ignorance ». Les données sont **réelles** (UCI, licence CC0) et l'étude est une illustration méthodologique (voir l'avertissement de la section 5.4.1).

Étape 1 — Le modèle.

```
cr = pd.read_csv("donnees/credit_default.csv")
cr["groupe_age"] = pd.cut(cr["age"], [0, 29, 44, 200], labels=["moins de 30 ans", "30 à 44 ans", "45
    ↪ ans et plus"]).astype(str)
cr["sexe"] = cr["sex"].map({1: "hommes", 2: "femmes"})
var_avec = [k for k in cr.columns if k not in ("default", "groupe_age", "sexe")]
tr, te = train_test_split(cr, test_size=0.3, random_state=0, stratify=cr["default"])
m_avec = HistGradientBoostingClassifier(max_iter=150, random_state=0).fit(tr[var_avec],
    ↪ tr["default"])
s = m_avec.predict_proba(te[var_avec])[:, 1]; y5 = te["default"].to_numpy(); t_alerte =
    ↪ float(np.quantile(s, 0.75))
print(cr.shape, "| taux de défaut", round(cr["default"].mean(), 4), "| AUC test",
    ↪ round(M.roc_auc_score(y5, s), 4), "| seuil (25 % d'alertes)", round(t_alerte, 3))
```

```
(30000, 26) | taux de défaut 0.2212 | AUC test 0.7765 | seuil (25 % d'alertes) 0.271
```

Le jeu compte 30 000 clients et 26 colonnes (24 d'origine plus deux colonnes de groupes), pour un taux de défaut de 22,1 %. Le modèle atteint une AUC de 0,7765 ; on signale les 25 % de dossiers les plus risqués (score supérieur à 0,271).

Étape 2 — L'audit par groupe.

```
def par_groupe(score, y, g, t):
    lignes = {}
    for nom in np.unique(g):
        m = g == nom; al = score[m] >= t; yy = y[m]
        tp, fp = (al & (yy == 1)).sum(), (al & (yy == 0)).sum(); fn, tn = (~al & (yy == 1)).sum(),
        ↪ (~al & (yy == 0)).sum()
        lignes[nom] = dict(effectif=int(m.sum()), taux_defaut=yy.mean(), taux_alerte=al.mean(),
        ↪ rappel=tp / (tp + fn), fausses_alertes=fp / (fp + tn), precision=tp / (tp + fp))
    return pd.DataFrame(lignes).T
for var in ["sexe", "groupe_age"]:
    print(par_groupe(s, y5, te[var].to_numpy(), t_alerte).round(3).to_string(), "\n")
```

	effectif	taux_defaut	taux_alerte	rappel	fausses_alertes	precision
femmes	5353.0	0.206	0.234	0.571	0.147	0.502
hommes	3647.0	0.244	0.273	0.574	0.176	0.513

	effectif	taux_defaut	taux_alerte	rappel	fausses_alertes	precision
30 à 44 ans	4497.0	0.212	0.234	0.533	0.154	0.483
45 ans et plus	1618.0	0.255	0.279	0.610	0.165	0.559
moins de 30 ans	2885.0	0.216	0.258	0.608	0.162	0.509

Selon le sexe, le rappel est presque identique (0,571 et 0,574), mais le taux d'alerte (0,234 contre 0,273) et le taux de fausses alertes (0,147 contre 0,176) diffèrent. Selon l'âge, le rappel varie de 0,533 (30-44 ans) à 0,61 (les deux autres classes).

Étape 3 — Les écarts sont-ils plus grands que le bruit ? On rééchantillonne 300 fois les 9 000 dossiers du jeu de test et l'on recalcule les écarts maximaux selon le sexe.

```
def ecarts(score, y, g, t):
    tab = par_groupe(score, y, g, t); return np.array([np.ptp(tab["taux_alerte"]),
    ↪ np.ptp(tab["rappel"])]))
g_sexe = te["sexe"].to_numpy(); rng = np.random.default_rng(0); n = len(y5); b = []
for _ in range(300):
    i = rng.integers(0, n, n); b.append(ecarts(s[i], y5[i], g_sexe[i], t_alerte))
b = np.array(b); obs = ecarts(s, y5, g_sexe, t_alerte)
print("écart de taux d'alerte :", round(obs[0], 3), "IC 95 %", np.round(np.percentile(b[:, 0], [2.5,
    ↪ 97.5]), 3), "| écart de rappel :", round(obs[1], 3), "IC 95 %", np.round(np.percentile(b[:, 1],
    ↪ [2.5, 97.5]), 3))
```

```
écart de taux d'alerte : 0.039 IC 95 % [0.023 0.057] | écart de rappel : 0.003 IC 95 % [0.001 0.053]
```

L'écart de taux d'alerte (0,039) a un intervalle à 95 % de [0,023 ; 0,057] : il est solidement positif. L'écart de rappel (0,003) a un intervalle de [0,001 ; 0,053] : l'égalité des chances est « presque réalisée » sur l'échantillon, mais **les données ne permettent pas d'exclure** un écart de l'ordre de 5 points. (Un écart maximal entre groupes est toujours positif par construction : son intervalle de bootstrap ne contient jamais 0.)

Étape 4 — Retirer le sexe, et chercher des proxys.

```
var_sans = [k for k in var_avec if k != "sex"]
m_sans = HistGradientBoostingClassifier(max_iter=150, random_state=0).fit(tr[var_sans],
    ↪ tr["default"])
s2 = m_sans.predict_proba(te[var_sans])[:, 1]; t2 = float(np.quantile(s2, 0.75))
print("AUC sans le sexe :", round(M.roc_auc_score(y5, s2), 4), "| écarts (alerte, rappel) sans le sexe
    ↪ :", np.round(ecarts(s2, y5, g_sexe, t2), 4), "| avec :", np.round(obs, 4))
from sklearn.model_selection import cross_val_predict
proxy = cross_val_predict(HistGradientBoostingClassifier(max_iter=100, random_state=0), cr[var_sans],
    ↪ (cr["sex"] == 2).astype(int), cv=3, method="predict_proba")[:, 1]
print("AUC pour retrouver le sexe à partir des autres variables :", round(M.roc_auc_score((cr["sex"]
    ↪ == 2).astype(int), proxy), 4))
```

```
AUC sans le sexe : 0.7741 | écarts (alerte, rappel) sans le sexe : [0.0301 0.0009] | avec : [0.0393
    ↪ 0.0029]
AUC pour retrouver le sexe à partir des autres variables : 0.6441
```

Sans la variable sexe, l'AUC passe de 0,7765 à 0,7741 ; les écarts d'alerte et de rappel selon le sexe passent de (0,0393 ; 0,0029) à (0,0301 ; 0,0009) : ils diminuent un peu sans disparaître. Et les autres variables suffisent à retrouver le sexe avec une AUC de 0,644 : elles en sont des proxys.

Application 5.9 — Des seuils par groupe, et ce qu'ils coûtent

Section du livre : 5.4.5. **Objectif** : comparer trois politiques de décision (seuil unique, égalité des chances, parité démographique) sur le sexe et sur l'âge, avec un coût d'erreur (un défaut manqué coûte 5, une fausse alerte 1).

Étape 1 — Les trois politiques.

```
def politique(score, y, g, mode, t0):
    cible = ((score >= t0) & (y == 1)).sum() / (y == 1).sum(); al = np.zeros(len(y), dtype=bool)
    for nom in np.unique(g):
        m = g == nom
        if mode == "unique": t = t0
        elif mode == "chances": t = np.quantile(score[m & (y == 1)], 1 - cible)  # même rappel
        else: t = np.quantile(score[m], 0.75)  # même taux
        al[m] = score[m] >= t
    return al
c_fn, c_fp = 5.0, 1.0
for var in ["sexe", "groupe_age"]:
    g = te[var].to_numpy()
    for mode in ["unique", "chances", "parite"]:
        al = politique(s, y5, g, mode, t_alerte); tab = par_groupe(al.astype(float), y5, g, 0.5)
        cout = (c_fn * (~al & (y5 == 1)).sum() + c_fp * (al & (y5 == 0)).sum()) / len(y5) * 1000
        print(f"{var:10s} {mode:8s} coût {cout:6.1f} | écarts : alerte
              ↳ {np.ptp(tab['taux_alerte']):.3f}  rappel {np.ptp(tab['rappel']):.3f}  précision
              ↳ {np.ptp(tab['precision']):.3f}  fausses alertes {np.ptp(tab['fausses_alertes']):.3f}")
```

sexe	unique	coût	596.1	écarts : alerte 0.039	rappel 0.003	précision 0.011	fausses
↳ alertes							
sexe	chances	coût	597.0	écarts : alerte 0.036	rappel 0.001	précision 0.016	fausses
↳ alertes							
sexe	parite	coût	594.9	écarts : alerte 0.000	rappel 0.040	précision 0.052	fausses
↳ alertes							
groupe_age	unique	coût	596.1	écarts : alerte 0.044	rappel 0.077	précision 0.076	fausses
↳ alertes							
groupe_age	chances	coût	601.1	écarts : alerte 0.036	rappel 0.002	précision 0.143	fausses
↳ alertes							
groupe_age	parite	coût	598.3	écarts : alerte 0.000	rappel 0.051	précision 0.121	fausses
↳ alertes							

Les résultats sont ceux du tableau de 5.4.5. Pour l'âge, par exemple, l'égalité des chances ramène l'écart de rappel de 0,077 à 0,002, mais fait passer l'écart de précision de 0,076 à 0,143 et l'écart de fausses alertes de 0,011 à 0,054. Le coût moyen (595 à 601 pour 1 000 dossiers) varie peu.

Étape 2 — D'autres groupes, et le piège des petits effectifs.

```
te2 = te.copy()
te2["etat_civil"] = te2["marriage"].map({1: "marié(e)", 2: "célibataire"}).fillna("autre")
te2["etudes"] = te2["education"].map({1: "supérieures", 2: "université", 3: "lycée"}).fillna("autre")
for var in ["etat_civil", "etudes"]:
    print(par_groupe(s, y5, te2[var].to_numpy(), t_alerte).round(3).to_string(), "\n")
```

	effectif	taux_defaut	taux_alerte	rappel	fausses_alertes	precision
autre	122.0	0.197	0.221	0.500	0.153	0.444
célibataire	4780.0	0.205	0.235	0.569	0.149	0.495
marié(e)	4098.0	0.241	0.268	0.578	0.169	0.520

	effectif	taux_defaut	taux_alerte	rappel	fausses_alertes	precision
autre	139.0	0.108	0.043	0.133	0.032	0.333
lycée	1450.0	0.268	0.300	0.624	0.182	0.556
supérieures	3170.0	0.190	0.216	0.564	0.134	0.496
université	4241.0	0.232	0.265	0.564	0.174	0.495

Selon la situation matrimoniale, les taux d'alerte (0,235 pour les célibataires, 0,268 pour les mariés) suivent les taux de défaut (0,205 et 0,241). Selon le niveau d'études, les clients de niveau « lycée » ont le taux de défaut le plus élevé (0,268). Le groupe « autre » (139 clients, 10,8 % de défaut, rappel 0,133) est trop petit pour que ses taux soient interprétables : **un écart mesuré sur quelques dizaines de cas positifs est essentiellement du bruit**, et un audit sérieux le dit.

Application 5.10 — Prédiction conforme : classification et régression

Sections du livre : 5.5.1 à 5.5.6. **Objectif** : construire des ensembles de prédiction avec garantie, constater que la garantie est marginale, la vérifier par simulation pour deux modèles et plusieurs tailles de calibration, puis passer à la régression.

Étape 1 — Ensembles LAC et correction de Mondrian.

```
alpha = 0.10
def q_conf(sc_, alpha):
    sc_ = np.sort(sc_); k = int(np.ceil((len(sc_) + 1) * (1 - alpha))) # k-ième plus petit
    ↪ score, k = ceil((n+1)(1-alpha))
    return float(sc_[k - 1]) if k <= len(sc_) else float("inf")
pcal, ptest = gb.predict_proba(Xca), gb.predict_proba(Xte); ycal = yca.to_numpy()
s_cal = 1 - pcal[np.arange(len(yca)), ycal]; q = q_conf(s_cal, alpha)
ens = (1 - ptest) <= q; couv = ens[np.arange(len(yt)), yt]
print("q =", round(q, 4), "| couverture", round(couv.mean(), 4), "| partants", round(couv[yt ==
    ↪ 1].mean(), 4), "| fidèles", round(couv[yt == 0].mean(), 4), "| taille moyenne",
    ↪ round(ens.sum(1).mean(), 3))
qk = [q_conf(s_cal[ycal == k], alpha) for k in (0, 1)] # Mondrian : un
    ↪ quantile par classe
ens_m = np.column_stack([(1 - ptest[:, 0]) <= qk[0], (1 - ptest[:, 1]) <= qk[1]]); cm =
    ↪ ens_m[np.arange(len(yt)), yt]
print("Mondrian : partants", round(cm[yt == 1].mean(), 4), "| fidèles", round(cm[yt == 0].mean(), 4),
    ↪ "| taille moyenne", round(ens_m.sum(1).mean(), 3))
```

```
q = 0.5221 | couverture 0.9025 | partants 0.4955 | fidèles 0.969 | taille moyenne 1.006
Mondrian : partants 0.9021 | fidèles 0.9157 | taille moyenne 1.213
```

La couverture globale est de 90,25 %, mais elle n'est que de 49,6 % pour les partants. Avec un quantile par classe (Mondrian), elle devient valable dans chaque classe (90,2 % et 91,6 %) au prix d'ensembles un peu plus gros (1,21 classe en moyenne contre 1,01).

Étape 2 — La garantie ne dépend pas du modèle. On répète 400 fois le tirage d'un jeu de calibration de n clients dans le pool (calibration + test) et l'on mesure la couverture sur les autres, pour le boosting puis pour la régression logistique, à comparer à la loi Bêta théorique.

```
from scipy.stats import beta as beta_
def simule(scores, n_cal, reps=400, seed=1):
    rng = np.random.default_rng(seed); cov = []
    for _ in range(reps):
        perm = rng.permutation(len(scores)); q = q_conf(scores[perm[:n_cal]], alpha);
        ↪ cov.append((scores[perm[n_cal:]] <= q).mean())
    return np.mean(cov), np.std(cov)
yy = np.concatenate([ycal, yt]); Z = pd.concat([Xca, Xte])
pools = {"boosting": gb.predict_proba(Z), "logistique": lr.predict_proba(sc.transform(Z))}
for nom, pp in pools.items():
    sp = 1 - pp[np.arange(len(yy)), yy]
    for n_cal in [50, 100, 300, 1000]:
        m_, sd_ = simule(sp, n_cal); k = int(np.ceil((n_cal + 1) * (1 - alpha))); th = beta_(k, n_cal
        ↪ + 1 - k)
        print(f"{nom:10s} n = {n_cal:4d} : couverture {m_: .4f} (écart-type {sd_: .4f}) | théorie
            ↪ {th.mean(): .4f} ({th.std(): .4f})")
```

```

boosting n = 50 : couverture 0.9009 (écart-type 0.0416) | théorie 0.9020 (0.0412)
boosting n = 100 : couverture 0.8994 (écart-type 0.0303) | théorie 0.9010 (0.0296)
boosting n = 300 : couverture 0.9001 (écart-type 0.0171) | théorie 0.9003 (0.0172)
boosting n = 1000 : couverture 0.8998 (écart-type 0.0104) | théorie 0.9001 (0.0095)
logistique n = 50 : couverture 0.9030 (écart-type 0.0400) | théorie 0.9020 (0.0412)
logistique n = 100 : couverture 0.9000 (écart-type 0.0294) | théorie 0.9010 (0.0296)
logistique n = 300 : couverture 0.9004 (écart-type 0.0175) | théorie 0.9003 (0.0172)
logistique n = 1000 : couverture 0.8998 (écart-type 0.0105) | théorie 0.9001 (0.0095)

```

Pour les deux modèles, la couverture moyenne est de 90 % à 0,3 point près, **quelle que soit la taille n** de la calibration, et l'écart-type colle à la théorie (0,0416 contre 0,0412 pour $n = 50$; 0,0171 contre 0,0172 pour $n = 300$). Le modèle compte pour la **taille des ensembles**, pas pour la validité. (À $n = 1000$, l'écart-type observé, 0,0104, est légèrement supérieur à celui de la théorie, 0,0095 : le jeu de test restant n'a plus que 3 800 clients, et ses fluctuations propres s'ajoutent.)

Étape 3 — Régression : largeur constante contre CQR.

```

yr = c["depense_6m"]; ytr_r, ycal_r, yte_r = (yr.loc[X_.index].to_numpy() for X_ in (Xtr, Xca, Xte))
reg = HGBR(max_iter=150, random_state=0).fit(Xtr, ytr_r)
q_r = q_conf(np.abs(ycal_r - reg.predict(Xca)), alpha); mu = reg.predict(Xte); lo, hi = np.maximum(mu
↳ - q_r, 0), mu + q_r
gl, gh = (HGBR(loss="quantile", quantile=a, max_iter=150, random_state=0).fit(Xtr, ytr_r) for a in
↳ (alpha / 2, 1 - alpha / 2))
q_c = q_conf(np.maximum(gl.predict(Xca) - ycal_r, ycal_r - gh.predict(Xca)), alpha)
lo2, hi2 = np.maximum(gl.predict(Xte) - q_c, 0), gh.predict(Xte) + q_c
tert = pd.qcut(mu, 3, labels=["basse", "moyenne", "haute"])
for nom, (l, h) in {"largeur constante": (lo, hi), "CQR": (lo2, hi2)}.items():
    ok = (yte_r >= l) & (yte_r <= h)
    print(f"{nom:18s} couverture {ok.mean():.3f} largeur moyenne {np.mean(h - l):6.1f} par tiers de
↳ prévision :", {t: round(float(ok[tert == t].mean()), 3) for t in tert.categories})

```

```

largeur constante  couverture 0.910  largeur moyenne 210.1  par tiers de prévision : {'basse': 0.994,
↳ 'moyenne': 0.975, 'haute': 0.76}
CQR                couverture 0.910  largeur moyenne 185.3  par tiers de prévision : {'basse': 0.922,
↳ 'moyenne': 0.916, 'haute': 0.892}

```

Les deux méthodes couvrent 91 % des clients, mais la régression quantile conformalisée est plus étroite en moyenne (185 € contre 210 €) et beaucoup plus homogène : la couverture par tiers de prévision vaut 0,92 ; 0,92 ; 0,89, contre 0,99 ; 0,98 ; 0,76 pour les intervalles de largeur constante.

Exercices

Exercice 5.1 ★ — Matrice de confusion à la main (section 5.1.2)

Sur 1 000 clients, 75 vont partir. Le modèle émet 60 alertes, dont 45 sont justes.

1. Complétez la matrice de confusion.
2. Calculez l'exactitude, la précision, le rappel, la spécificité, le F_1 , le F_2 et le coefficient de Matthews.
3. Comparez l'exactitude à celle du modèle « personne ne part ».

Exercice 5.2 ★ — Mesures F_β (section 5.1.2)

Un modèle a une précision de 0,8 et un rappel de 0,5. Calculez F_1 , F_2 et $F_{0,5}$. Manquer un départ coûte quatre fois plus cher qu'une fausse alerte : laquelle des trois mesures est la plus cohérente avec cette situation ? Pourquoi ne remplace-t-elle pas un calcul de coût ?

Exercice 5.3 ★★ — L'AUC par les paires (section 5.1.4)

Quatre clients qui partent ont les scores 0,9 ; 0,8 ; 0,55 et 0,4. Cinq clients fidèles ont les scores 0,7 ; 0,5 ; 0,4 ; 0,2 et 0,1.

1. Combien y a-t-il de paires (partant, fidèle) ?
2. Calculez l'AUC comme proportion de paires bien ordonnées (une égalité compte pour une demi-paire).
3. Donnez les points (FPR, TPR) de la courbe ROC et retrouvez l'AUC par trapèzes.

Exercice 5.4 ★★ — ROC contre précision-rappel (section 5.1.5)

Un détecteur a un rappel de 0,8 et un taux de fausses alertes de 0,1. Exprimez sa précision en fonction de la prévalence π , puis calculez-la pour $\pi = 10\%$, 1% et $0,1\%$. Que devient le point ROC ? Quelle conséquence pour le choix entre l'AUC et l'AP ?

Exercice 5.5 ★★ — Le seuil par les coûts et la calibration (section 5.1.7)

Une campagne coûte 8 € par contact, réussit avec la probabilité 0,25 et rapporte 120 € par succès.

1. Quel est le seuil théorique ?
2. Le modèle annonce des probabilités **deux fois trop grandes**. Que se passe-t-il si l'on applique le seuil de la question 1 à ces probabilités ? Quel seuil faut-il appliquer ? Vérifiez sur les probabilités du boosting (jeu de test).

Exercice 5.6 ★ — Log-loss et score de Brier (section 5.1.6)

Cinq prédictions $\hat{p} = (0,9; 0,8; 0,3; 0,6; 0,1)$ pour les résultats $y = (1; 1; 0; 1; 0)$. Calculez la log-loss et le score de Brier, puis le score de Brier du modèle constant, et le « score de compétence » du modèle.

Exercice 5.7 ★★ — L'ECE à la main (section 5.2.1)

Quatre classes de 100 clients : probabilité moyenne prédite 0,05 ; 0,20 ; 0,50 ; 0,80 ; taux observés 0,08 ; 0,15 ; 0,60 ; 0,70. Calculez l'ECE. Dans quelles classes le modèle est-il optimiste ou pessimiste ? Quelles différences sont plausiblement dues au hasard ?

Exercice 5.8 ★★★ — Poids de classes et décalage de l'ordonnée (section 5.2.6)

Montrez que, si l'on entraîne une régression logistique avec des poids qui donnent à chaque classe la moitié du poids total, l'ordonnée à l'origine est décalée de $-\ln \frac{\pi}{1-\pi}$ par rapport à celle d'un modèle sans poids, et que les pentes sont inchangées. Vérifiez par simulation.

Exercice 5.9 ★★ — Valeurs de Shapley à la main (section 5.3.5)

Trois variables A, B, C ont les valeurs de coalition $v(\emptyset) = 0$, $v(A) = 10$, $v(B) = 20$, $v(C) = 0$, $v(AB) = 40$, $v(AC) = 10$, $v(BC) = 20$, $v(ABC) = 45$.

1. Calculez les valeurs de Shapley de A, B et C en listant les six ordres d'arrivée.
2. Vérifiez l'efficacité.
3. La variable C n'apporte rien seule : pourquoi n'est-elle pas un « joueur nul » ?

Exercice 5.10 ★★★ — SHAP d'un modèle linéaire (section 5.3.6)

Pour $f(x) = \beta_0 + \sum_j \beta_j x_j$ et des variables indépendantes, montrez que la valeur de Shapley (version « on remplace les variables absentes par leur distribution ») de la variable j est $\varphi_j = \beta_j (x_j - E[X_j])$. Vérifiez sur $\beta = (2, -1, 0, 5)$, $x = (3, 0, 2)$.

Exercice 5.11 ★★ — Les critères d'équité (section 5.4.2)

Deux groupes ont les matrices de confusion suivantes (VP, FP, FN, VN) : groupe A : (40, 20, 20, 120) ; groupe B : (30, 10, 30, 230). Calculez pour chacun le taux de défaut, le taux d'alerte, le rappel, le taux de fausses alertes et la précision. Lesquels des critères (parité démographique, égalité des chances, parité prédictive, égalité des fausses alertes) sont satisfaits ?

Exercice 5.12 ★★★ — L'impossibilité (section 5.4.4)

Démontrez l'identité $FPR = \frac{p}{1-p} \cdot \frac{1-PPV}{PPV} \cdot (1-FNR)$. Application : deux groupes ont la même précision (0,6) et le même rappel (0,5) mais des prévalences de 30 % et de 10 %. Quels taux de fausses alertes en résulte-t-il ? Qu'en concluez-vous ?

Exercice 5.13 ★★ — Le quantile conforme (section 5.5.2)

Dix-neuf scores de calibration (triés) : 0,02 ; 0,05 ; 0,07 ; 0,09 ; 0,12 ; 0,15 ; 0,18 ; 0,21 ; 0,25 ; 0,29 ; 0,33 ; 0,38 ; 0,41 ; 0,46 ; 0,52 ; 0,58 ; 0,66 ; 0,74 ; 0,91.

1. Calculez $\hat{q}$ pour $\alpha = 0,10$ puis $\alpha = 0,20$.
2. Un client a une probabilité de départ de 0,30 (score $1 - \hat{p}_y$ pour chaque classe). Donnez son ensemble de prédiction dans les deux cas.
3. Que devient $\hat{q}$ avec seulement 8 scores et $\alpha = 0,10$? Interprétez.

Exercice 5.14 ★★★ — La loi de la couverture (section 5.5.3)

Si les scores sont continus, montrez que, **conditionnellement au jeu de calibration**, la couverture d'un nouveau score suit une loi Bêta(k , $n + 1 - k$). Calculez sa moyenne et son écart-type pour $n = 99$ et $\alpha = 0,10$, puis vérifiez par simulation avec des scores uniformes.

Corrigé 5.1

1. $VP = 45$, $FP = 60 - 45 = 15$, $FN = 75 - 45 = 30$, $VN = 1\,000 - 45 - 15 - 30 = 910$.
2. Exactitude = $955/1000 = 0,955$; précision = $45/60 = 0,75$; rappel = $45/75 = 0,60$; spécificité = $910/925 \approx 0,984$; $F_1 = \frac{2 \times 0,75 \times 0,6}{1,35} \approx 0,667$; $F_2 = \frac{5 \times 0,75 \times 0,6}{4 \times 0,75 + 0,6} = 0,625$; $MCC = \frac{45 \times 910 - 15 \times 30}{\sqrt{60 \cdot 75 \cdot 925 \cdot 940}} \approx 0,648$.
3. « Personne ne part » a une exactitude de $925/1000 = 0,925$: le modèle ne gagne que trois points d'exactitude, alors qu'il détecte 60 % des départs.

VP, FP, FN, VN = 45, 15, 30, 910

n = VP + FP + FN + VN; P_, R_ = VP / (VP + FP), VP / (VP + FN)

print("exactitude", (VP + VN) / n, "| précision", P_, "| rappel", R_, "| spécificité", round(VN / (VN + FP), 4), "| majoritaire", (FP + VN) / n)

print("F1", round(2 * P_ * R_ / (P_ + R_), 4), "| F2", round(5 * P_ * R_ / (4 * P_ + R_), 4), "| MCC", round((VP * VN - FP * FN) / math.sqrt((VP + FP) * (VP + FN) * (VN + FP) * (VN + FN)), 4))

exactitude 0.955 | précision 0.75 | rappel 0.6 | spécificité 0.9838 | majoritaire 0.925
F1 0.6667 | F2 0.625 | MCC 0.6475

Corrigé 5.2

$F_1 = \frac{2 \times 0,8 \times 0,5}{1,3} \approx 0,615$; $F_2 = \frac{5 \times 0,8 \times 0,5}{4 \times 0,8 + 0,5} = \frac{2}{3,7} \approx 0,541$; $F_{0,5} = \frac{1,25 \times 0,8 \times 0,5}{0,25 \times 0,8 + 0,5} = \frac{0,5}{0,7} \approx 0,714$. Dans le F_β , le rappel compte β^2 fois plus que la précision ; si manquer un départ coûte quatre fois plus cher qu'une fausse alerte, $\beta = 2$ (soit $\beta^2 = 4$) est cohérent, d'où $F_2 \approx 0,54$. Mais le F_β est un compromis *heuristique* : il ignore le nombre de vrais négatifs et la valeur réelle des gains. Pour décider, on calcule le coût attendu (section 5.1.7).

P_, R_ = 0.8, 0.5

for beta in [1, 2, 0.5]:

print(f"F{beta} =", round((1 + beta ** 2) * P_ * R_ / (beta ** 2 * P_ + R_), 4))

F1 = 0.6154
F2 = 0.5405
F0.5 = 0.7143

Corrigé 5.3

1. $4 \times 5 = 20$ paires.
2. Paires gagnées : 0,9 bat les 5 fidèles (5) ; 0,8 bat les 5 (5) ; 0,55 bat 0,5 ; 0,4 ; 0,2 ; 0,1 mais pas 0,7 (4) ; 0,4 bat 0,2 et 0,1 (2) et est à égalité avec le 0,4 fidèle (0,5). Total $5+5+4+2+0,5 = 16,5$, d'où $AUC = 16,5/20 = 0,825$.
3. En balayant les seuils de haut en bas : (0; 0) $\rightarrow$ (0; 0,25) $\rightarrow$ (0; 0,5) (après 0,9 et 0,8) $\rightarrow$ (0,2; 0,5) (0,7 fidèle) $\rightarrow$ (0,2; 0,75) (0,55) $\rightarrow$ (0,4; 0,75) (0,5 fidèle) $\rightarrow$ (0,6; 1) (0,4 : deux clients à égalité, un de chaque classe) $\rightarrow$ (0,8; 1) $\rightarrow$ (1; 1). Aire par trapèzes : $0,2 \times 0,5 + 0,2 \times 0,75 + 0,2 \times \frac{0,75+1}{2} + 0,2 \times 1 + 0,2 \times 1 = 0,1 + 0,15 + 0,175 + 0,2 + 0,2 = 0,825$.

pos = np.array([0.9, 0.8, 0.55, 0.4]); neg = np.array([0.7, 0.5, 0.4, 0.2, 0.1])

gagnees = (pos[:, None] > neg[None, :]).sum() + 0.5 * (pos[:, None] == neg[None, :]).sum()

print("paires :", pos.size * neg.size, "| gagnées (égalité = 1/2) :", gagnees, "| AUC =", gagnees / (pos.size * neg.size))

print("AUC scikit-learn :", M.roc_auc_score([1] * 4 + [0] * 5, np.r_[pos, neg]))

paires : 20 | gagnées (égalité = 1/2) : 16.5 | AUC = 0.825
AUC scikit-learn : 0.825

Corrigé 5.4

La précision est $\frac{VP}{VP + FP} = \frac{TPR \cdot \pi}{TPR \cdot \pi + FPR \cdot (1 - \pi)}$ (on normalise par la taille de l'échantillon : les positifs sont πn , les négatifs $(1 - \pi)n$). Avec $TPR = 0,8$ et $FPR = 0,1$: $\pi = 10\%$: $\frac{0,08}{0,08 + 0,09} = 0,471$; $\pi = 1\%$: $\frac{0,008}{0,008 + 0,099} = 0,075$; $\pi = 0,1\%$: $\frac{0,0008}{0,0008 + 0,0999} = 0,0079$. Le point ROC (0,1; 0,8) n'a pas bougé, mais moins d'une alerte sur cent est justifiée à 0,1 % de prévalence. L'AUC ne voit pas ce phénomène ; la précision moyenne, si.

```
tpr, fpr = 0.8, 0.1
for prev in [0.1, 0.01, 0.001]:
    print(f"prévalence {prev:5.3f} : précision = {tpr * prev / (tpr * prev + fpr * (1 - prev)):.4f}")

prévalence 0.100 : précision = 0.4706
prévalence 0.010 : précision = 0.0748
prévalence 0.001 : précision = 0.0079
```

Corrigé 5.5

1. $t^* = \frac{c}{sV} = \frac{8}{0,25 \times 120} = \frac{8}{30} \approx 0,267$.
2. Si le modèle annonce $\hat{p} = 2p$, le client p est contacté dès que $2p \geq 0,267$, c'est-à-dire $p \geq 0,133$: on contacte trop de clients (585 au lieu de 373 sur le jeu de test), parmi lesquels beaucoup ne sont pas rentables. Il faut appliquer le seuil **corrigé** $2t^* = 0,533$ aux probabilités gonflées, ce qui revient au même seuil sur les vraies probabilités. Vérification : le gain est de 3 496 € avec les vraies probabilités au seuil théorique, 3 180 € avec les probabilités doublées au même seuil (perte de 316 €), et de nouveau 3 496 € au seuil corrigé.

```
c_, s_, V_ = 8.0, 0.25, 120.0
t_th = c_ / (s_ * V_); print("seuil théorique :", round(t_th, 4))
for facteur in [1.0, 2.0]:
    # le modèle annonce-t-il la vraie probabilité (x1) ou le
    # double (x2) ?
    pk = np.clip(p * facteur, 0, 1)
    print(f"probabilités x{facteur:.0f} : gain au seuil théorique {gain_net(pk, t_th, c_, s_, V_):7.1f}
    ↪ € ({int((pk >= t_th).sum())} contacts) | au seuil corrigé {facteur * t_th:.3f} : {gain_net(pk,
    ↪ facteur * t_th, c_, s_, V_):7.1f} € ({int((pk >= facteur * t_th).sum())} contacts)")

seuil théorique : 0.2667
probabilités x1 : gain au seuil théorique 3496.0 € (373 contacts) | au seuil corrigé 0.267 : 3496.0
↪ € (373 contacts)
probabilités x2 : gain au seuil théorique 3180.0 € (585 contacts) | au seuil corrigé 0.533 : 3496.0
↪ € (373 contacts)
```

Corrigé 5.6

Log-loss : $-\ln 0,9 = 0,105$; $-\ln 0,8 = 0,223$; $-\ln 0,7 = 0,357$; $-\ln 0,6 = 0,511$; $-\ln 0,9 = 0,105$; moyenne $\approx 0,260$. Brier : $(0,01 + 0,04 + 0,09 + 0,16 + 0,01)/5 = 0,062$. Le modèle constant annonce la fréquence 0,6 : $(3 \times 0,16 + 2 \times 0,36)/5 = 0,24$. Score de compétence : $1 - 0,062/0,24 \approx 0,74$.

```
ph = np.array([0.9, 0.8, 0.3, 0.6, 0.1]); yv = np.array([1, 1, 0, 1, 0])
print("log-loss", round(float(-(yv * np.log(ph) + (1 - yv) * np.log(1 - ph)).mean()), 4), "| Brier",
    ↪ round(float(((ph - yv) ** 2).mean()), 4), "| Brier du modèle constant", round(float(((yv.mean() -
    ↪ yv) ** 2).mean()), 4))
```

log-loss 0.2603 | Brier 0.062 | Brier du modèle constant 0.24

Corrigé 5.7

$ECE = \frac{1}{4}(0,03 + 0,05 + 0,10 + 0,10) = 0,07$. Classes 1 et 3 : le modèle est **pessimiste** (il annonce moins que le taux observé : 0,05 pour 0,08 ; 0,50 pour 0,60) ; classes 2 et 4 : **optimiste** (0,20 pour 0,15 ; 0,80 pour 0,70). Avec 100 clients par classe, le taux observé fluctue de $\sqrt{p(1-p)/100}$, soit 0,022 ; 0,040 ; 0,050 ; 0,040 : les écarts de 0,03 et de 0,05 (classes 1 et 2) valent environ 1,2 à 1,4 écart-type et peuvent être dus au hasard ; les écarts de 0,10 (classes 3 et 4) valent 2 à 2,5 écarts-types : ils sont plus probablement réels.

```
classes = np.array([[100, 0.05, 0.08], [100, 0.20, 0.15], [100, 0.50, 0.60], [100, 0.80, 0.70]])
print("ECE =", round(float(np.sum(classes[:, 0] * np.abs(classes[:, 1] - classes[:, 2])) / classes[:,
↪ 0].sum()), 4))
```

ECE = 0.07

Corrigé 5.8

Notons π la vraie prévalence et $LR(x)$ le rapport de vraisemblance $P(x | Y = 1)/P(x | Y = 0)$. La vraie cote *a posteriori* est $\frac{\pi}{1-\pi}LR(x)$: si le modèle logistique est correct, $\text{logit } P(Y = 1 | x) = \ln \frac{\pi}{1-\pi} + \ln LR(x) = \beta_0 + \beta^T x$. Avec des poids qui donnent la moitié du poids total à chaque classe, la vraisemblance pondérée est celle d'un échantillon où les deux classes sont à égalité : la prévalence apparente est $\pi' = \frac{1}{2}$, la cote *a posteriori* apprise est $\frac{\pi'}{1-\pi'}LR(x) = LR(x)$. Le logit appris est donc $\ln LR(x) = \beta_0 + \beta^T x - \ln \frac{\pi}{1-\pi}$: seule l'ordonnée est décalée, de $-\ln \frac{\pi}{1-\pi}$. Dans la simulation (vraie ordonnée -2 , prévalence 0,164), le modèle ordinaire estime $-2,019$, le modèle pondéré $-0,389$: le décalage est de 1,631 et $-\ln \frac{0,1638}{0,8362} = 1,630$; les pentes sont les mêmes à 0,001 près.

```
rng = np.random.default_rng(0); n = 60000; xs = rng.normal(size=(n, 2)); b0 = -2.0
vrai = 1 / (1 + np.exp(-(b0 + 1.0 * xs[:, 0] - 0.5 * xs[:, 1])))
yy = rng.binomial(1, vrai)
m_n = LogisticRegression(C=1e6).fit(xs, yy); m_b = LogisticRegression(C=1e6,
↪ class_weight="balanced").fit(xs, yy)
pi_ = yy.mean()
print("prévalence", round(pi_, 4), "| ordonnées : normale", round(float(m_n.intercept_[0]), 3),
↪ "pondérée", round(float(m_b.intercept_[0]), 3),
    "| écart", round(float(m_b.intercept_[0] - m_n.intercept_[0]), 3), "| -ln(pi/(1-pi)) =",
    ↪ round(float(-np.log(pi_ / (1 - pi_))), 3))
print("pentes : normale", np.round(m_n.coef_[0], 3), "pondérée", np.round(m_b.coef_[0], 3))
```

prévalence 0.1638 | ordonnées : normale -2.019 pondérée -0.389 | écart 1.631 | $-\ln(\pi/(1-\pi)) = 1.63$
pentes : normale [1.025 -0.516] pondérée [1.024 -0.515]

Corrigé 5.9

Ordres (apports de A, B, C) : ABC : (10 ; 30 ; 5) ; ACB : (10 ; 35 ; 0) ; BAC : (20 ; 20 ; 5) ; BCA : (25 ; 20 ; 0) ; CAB : (10 ; 35 ; 0) ; CBA : (25 ; 20 ; 0). (Exemple, ordre CAB : $v(C) - v(\emptyset) = 0$, puis $v(AC) - v(C) = 10$, puis $v(ABC) - v(AC) = 35$.) Moyennes : $\varphi_A = 100/6 \approx 16,67$, $\varphi_B = 160/6 \approx 26,67$, $\varphi_C = 10/6 \approx 1,67$. La somme est 45 = $v(ABC) - v(\emptyset)$: efficacité vérifiée. C n'est pas un joueur nul parce qu'il apporte 5 lorsqu'il arrive après A et B (45 - 40), même s'il n'apporte rien avec A seul ou avec B seul : un **joueur nul** est un joueur qui n'apporte **jamais** rien, quelle que soit la coalition.

```

v = {(): 0, ("A",): 10, ("B",): 20, ("C",): 0, ("A", "B"): 40, ("A", "C"): 10, ("B", "C"): 20, ("A",
    ↪ "B", "C"): 45}
val = lambda S: v[tuple(sorted(S))]
phi = {j: 0.0 for j in "ABC"}
for ordre in itertools.permutations("ABC"):
    pris = []
    for j in ordre:
        phi[j] += (val(pris + [j]) - val(pris)) / 6; pris.append(j)
print({k: round(x, 4) for k, x in phi.items()}, "| somme", round(sum(phi.values()), 4))

{'A': 16.6667, 'B': 26.6667, 'C': 1.6667} | somme 45.0

```

Corrigé 5.10

Avec des variables indépendantes et la valeur de coalition $v(S) = E[f(X) \mid X_S = x_S] = \beta_0 + \sum_{j \in S} \beta_j x_j + \sum_{j \notin S} \beta_j E[X_j]$ (le modèle est linéaire, l'espérance se décompose), l'apport marginal de j à une coalition S qui ne le contient pas est $v(S \cup \{j\}) - v(S) = \beta_j(x_j - E[X_j])$, **indépendamment de S** . La moyenne pondérée de quantités constantes est cette constante : $\varphi_j = \beta_j(x_j - E[X_j])$. Numériquement, avec $\beta = (2, -1, 0, 5)$ et $x = (3, 0, 2)$, l'énumération exacte et la formule donnent les mêmes trois valeurs (4,0128 ; 0,9944 ; 0,5620 sur ce fond de 500 tirages), dont la somme est $f(x) - E[f(X)]$.

```

rng = np.random.default_rng(1); beta_ = np.array([2.0, -1.0, 0.5]); fond = rng.normal(1.0, 1.0,
    ↪ size=(500, 3)); x0 = np.array([3.0, 0.0, 2.0])
def v(S):
    Z = fond.copy(); Z[:, list(S)] = x0[list(S)]; return float((Z @ beta_).mean())
phi = [sum(math.factorial(len(S)) * math.factorial(2 - len(S)) / 6 * (v(S + (j,)) - v(S)) for r in
    ↪ range(3) for S in itertools.combinations([k for k in range(3) if k != j], r)) for j in range(3)]
print("Shapley exact :", np.round(phi, 4), "| beta_j (x_j - moyenne_j) :", np.round(beta_ * (x0 -
    ↪ fond.mean(axis=0)), 4))

```

Shapley exact : [4.0128 0.9944 0.562] | beta_j (x_j - moyenne_j) : [4.0128 0.9944 0.562]

Corrigé 5.11

	Groupe A	Groupe B
Taux de défaut	0,300	0,200
Taux d'alerte	0,300	0,133
Rappel	0,667	0,500
Fausses alertes	0,143	0,042
Précision	0,667	0,750

Les écarts sont de 0,167 pour le taux d'alerte (parité démographique), 0,167 pour le rappel (égalité des chances), 0,101 pour les fausses alertes et 0,083 pour la précision : **aucun des critères n'est satisfait**. On vérifie l'identité de l'exercice 5.12 : pour A, $\frac{0,3}{0,7} \cdot \frac{1/3}{2/3} \cdot 0,667 = 0,143$.

```

groupes = {"A": dict(TP=40, FP=20, FN=20, TN=120), "B": dict(TP=30, FP=10, FN=30, TN=230)}
for g, d in groupes.items():
    n = sum(d.values()); pos = d["TP"] + d["FN"]
    print(g, "| n", n, "| défaut", round(pos / n, 3), "| alerte", round((d["TP"] + d["FP"]) / n, 3),
        ↪ "| rappel", round(d["TP"] / pos, 3),
            "| fausses alertes", round(d["FP"] / (d["FP"] + d["TN"]), 3), "| précision", round(d["TP"] /
            ↪ (d["TP"] + d["FP"]), 3))

```

A | n 200 | défaut 0.3 | alerte 0.3 | rappel 0.667 | fausses alertes 0.143 | précision 0.667
 B | n 300 | défaut 0.2 | alerte 0.133 | rappel 0.5 | fausses alertes 0.042 | précision 0.75

Corrigé 5.12

Les effectifs d'un groupe de taille n sont $VP = (1 - \text{FNR}) p n$ et $FP = \text{FPR} (1 - p) n$. La précision est $\text{PPV} = \frac{VP}{VP + FP}$, soit $FP = VP \cdot \frac{1 - \text{PPV}}{\text{PPV}}$. En remplaçant : $\text{FPR} (1 - p) n = (1 - \text{FNR}) p n \cdot \frac{1 - \text{PPV}}{\text{PPV}}$, d'où l'identité. Application : le facteur $\frac{1 - \text{PPV}}{\text{PPV}} (1 - \text{FNR}) = \frac{0,4}{0,6} \times 0,5 = \frac{1}{3}$ est le même pour les deux groupes ; il reste $\text{FPR} = \frac{p}{1 - p} \cdot \frac{1}{3} : 0,143$ pour $p = 0,30$ et $0,037$ pour $p = 0,10$. **Deux groupes de prévalences différentes ne peuvent pas avoir à la fois la même précision, le même rappel et les mêmes fausses alertes, sauf pour un classifieur parfait.**

```
for nom, prev in [("A", 0.30), ("B", 0.10)]:
    fpr = prev / (1 - prev) * (1 - 0.6) / 0.6 * 0.5
    print(f"groupe {nom} : prévalence {prev} -> taux de fausses alertes imposé {fpr:.4f}")
```

```
groupe A : prévalence 0.3 -> taux de fausses alertes imposé 0.1429
groupe B : prévalence 0.1 -> taux de fausses alertes imposé 0.0370
```

Corrigé 5.13

1. $\alpha = 0,10 : k = \lceil 20 \times 0,9 \rceil = 18 : \hat{q}$ est le 18^e plus petit score, **0,74**. $\alpha = 0,20 : k = \lceil 20 \times 0,8 \rceil = 16 : \hat{q} = 0,58$.
2. Pour $\hat{p}_{\text{partant}} = 0,30$: le score de la classe « partant » est $1 - 0,30 = 0,70$ et celui de « fidèle » est $0,30$. Avec $\hat{q} = 0,74$, les deux sont $\leq \hat{q}$: l'ensemble est {fidèle, partant}. Avec $\hat{q} = 0,58$, seul « fidèle » ($0,30$) reste : l'ensemble est {fidèle}. Un niveau de confiance plus exigeant (90 % au lieu de 80 %) rend les ensembles plus gros.
3. Avec $n = 8$ et $\alpha = 0,10 : k = \lceil 9 \times 0,9 \rceil = 9 > 8$, donc $\hat{q} = +\infty$ et l'ensemble contient **toutes les classes** : avec trop peu de données de calibration, la garantie à 90 % ne peut être tenue qu'en ne disant rien. Il faut $n \geq \frac{1}{\alpha} - 1 = 9$ pour que $\hat{q}$ soit fini (ici $n = 9$ donne le plus grand des scores, $0,25$).

```
scores = np.array([0.02, 0.05, 0.07, 0.09, 0.12, 0.15, 0.18, 0.21, 0.25, 0.29, 0.33, 0.38, 0.41, 0.46,
                  0.52, 0.58, 0.66, 0.74, 0.91])
for n_, a in [(19, 0.10), (19, 0.20), (9, 0.10), (8, 0.10)]:
    k = math.ceil((n_ + 1) * (1 - a)); print(f"n = {n_:2d}, alpha = {a} : k = {k}", "-> q =",
    np.sort(scores[:n_])[k - 1] if k <= n_ else "infini (ensemble = toutes les classes)")
```

```
n = 19, alpha = 0.1 : k = 18 -> q = 0.74
n = 19, alpha = 0.2 : k = 16 -> q = 0.58
n = 9, alpha = 0.1 : k = 9 -> q = 0.25
n = 8, alpha = 0.1 : k = 9 -> q = infini (ensemble = toutes les classes)
```

Corrigé 5.14

Si les scores sont continus, la couverture d'un nouveau score, conditionnellement au jeu de calibration, est $F(\hat{q})$ où F est la fonction de répartition des scores et $\hat{q} = S_{(k)}$ est le k -ième plus petit des n scores de calibration. Or $F(S_{(k)})$ est le k -ième plus petit de n variables uniformes (le théorème de la transformation intégrale de probabilité), de loi Bêta($k, n + 1 - k$). Sa moyenne est $\frac{k}{n + 1}$ et sa variance $\frac{k(n + 1 - k)}{(n + 1)^2(n + 2)}$. Avec $n = 99$ et $\alpha = 0,10 : k = \lceil 100 \times 0,9 \rceil = 90$, donc moyenne $0,9$

et écart-type $\sqrt{\frac{90 \times 10}{100^2 \times 101}} \approx 0,0299$. La simulation (20 000 jeux de calibration uniformes) donne bien 0,9000 et 0,0299.

```
rng = np.random.default_rng(0); n, alpha_ = 99, 0.10; k = math.ceil((n + 1) * (1 - alpha_)); cov = []
for _ in range(20000):
    cal = np.sort(rng.random(n)); cov.append(cal[k - 1])           # pour des scores U(0,1), la
    ↪ couverture d'un jeu de calibration est le k-ième plus petit score
print("k =", k, "| couverture moyenne", round(np.mean(cov), 4), "écart-type", round(np.std(cov), 4),
    ↪ "| Bêta(k, n+1-k) :", round(k / (n + 1), 4), round(math.sqrt(k * (n + 1 - k) / ((n + 1) ** 2 * (n
    ↪ + 2))), 4))
```

```
k = 90 | couverture moyenne 0.9 écart-type 0.0299 | Bêta(k, n+1-k) : 0.9 0.0299
```

Ce que vous avez pratiqué. Le calcul à la main de toutes les métriques et de leurs liens (matrice de confusion, AUC, précision-rappel, coûts, log-loss, Brier) ; la calibration, de la mesure à la réparation ; l'interprétabilité (permutation, SHAP, LIME, Shapley exact) ; l'audit d'équité sur un jeu réel, ses limites mathématiques et statistiques ; la prédiction conforme avec sa preuve et sa vérification par simulation.

Chapitre 6 : ⊕ Détection d'anomalies et de fraude — exercices et applications

Ce chapitre du cahier accompagne le chapitre 6 du livre (chapitre complémentaire). Les **applications** reprennent pas à pas, avec le code complet, ce que le livre n'a fait que résumer : mesurer un détecteur sous déséquilibre extrême, les scores de distance et de densité, une forêt d'isolement écrite à la main, l'autoencodeur et l'ACP, et le tri des alertes. Les **exercices** sont corrigés à la fin du chapitre. Données : `donnees/transactions.csv` (60 000 commandes, 0,8 % de fraude, deux types). Le chapitre est **autonome** : il recharge ses données et refait la séparation apprentissage/test du livre ; mais ses applications s'appuient les unes sur les autres (une fonction définie dans une application sert aux suivantes), il faut donc les exécuter dans l'ordre.

Préparation

Une première cellule charge les bibliothèques et construit les dix variables du livre (avec l'heure codée par un cercle, car 23 h est voisine de 0 h) ; la suivante fait la séparation apprentissage/test (graine 0, stratifiée sur la fraude), standardise, et prépare un échantillon de référence de 10 000 commandes pour les méthodes de voisinage.

```
import sys, warnings
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import average_precision_score, roc_auc_score
sys.path.insert(0, "build"); import style; style.setup()      # style des figures du volume
warnings.filterwarnings("ignore")

t = pd.read_csv("donnees/transactions.csv")
X = pd.DataFrame({
    "log_montant": np.log(t["montant"]),
    "sin_heure": np.sin(2 * np.pi * t["heure"] / 24), "cos_heure": np.cos(2 * np.pi * t["heure"] / 24),
    "log_distance": np.log1p(t["distance_facturation_livraison_km"]), "nb_cmd_24h":
        ↪ t["nb_commandes_24h"],
    "log_age_compte": np.log1p(t["age_compte_jours"]), "ip_différent": t["ip_pays_différent"],
    "appareil_inconnu": 1 - t["appareil_connu"], "log_délai":
        ↪ np.log1p(t["délai_depuis_derniere_cmd_h"]), "nb_articles": t["nb_articles"]})
```

```

print(X.shape, "| fraudes :", int(t["fraude"].sum()), "soit", round(100 * t["fraude"].mean(), 2), "%")

(60000, 10) | fraudes : 486 soit 0.81 %

(X_app, X_test, y_app, y_test, type_app, type_test, montant_app, montant_test) = train_test_split(
    X, t["fraude"].values, t["type_fraude"].values, t["montant"].values, test_size=0.3,
    ↪ random_state=0, stratify=t["fraude"].values)
scaler = StandardScaler().fit(X_app)
Z_app, Z_test = scaler.transform(X_app), scaler.transform(X_test)
reference = np.random.default_rng(0).choice(len(Z_app), 10000, replace=False)
valid = np.setdiff1d(np.arange(len(Z_app)), reference) # 32 000 commandes étiquetées, pour
    ↪ régler les méthodes
Z_val, y_val = Z_app[valid], y_app[valid]
budget = int(np.ceil(0.01 * len(y_test))) # 180 alertes : 1 % du jeu de test
R = {}
def evaluer(nom, score):
    """AUC, AP, précision au budget, et part des fraudes de chaque type retrouvées parmi les `budget`
    ↪ alertes."""
    alertes = np.argsort(-score)[:budget]
    R[nom] = dict(AUC=roc_auc_score(y_test, score), AP=average_precision_score(y_test, score),
    ↪ precision=y_test[alertes].mean(),
        rappel1=np.isin(np.where(type_test == 1)[0], alertes).mean(),
    ↪ rappel2=np.isin(np.where(type_test == 2)[0], alertes).mean())
    return {k: round(float(v), 3) for k, v in R[nom].items()}
print("apprentissage :", len(y_app), "| validation :", len(y_val), "| test :", len(y_test), "dont",
    ↪ int(y_test.sum()), "fraudes | budget :", budget)

apprentissage : 42000 | validation : 32000 | test : 18000 dont 146 fraudes | budget : 180

```

Applications

Application 6.1 — Mesurer un détecteur sous déséquilibre extrême (section 6.1)

Contexte. La gérante dispose d'un modèle supervisé et veut savoir ce qu'il vaut réellement, et combien d'alertes il est raisonnable de traiter chaque jour. **Objectif.** Calculer à la main la matrice de confusion, la précision, le rappel et l'AP, puis chercher le nombre d'alertes qui minimise le coût selon le prix d'une vérification.

Étape 1 : un modèle supervisé de référence. Un gradient boosting avec pondération des classes (chapitre 2, section 2.4 ; chapitre 4, section 4.3), entraîné sur le jeu d'apprentissage :

```

from sklearn.ensemble import HistGradientBoostingClassifier

modele = HistGradientBoostingClassifier(class_weight="balanced", max_iter=150,
    ↪ random_state=0).fit(X_app, y_app)
score_gbm = modele.predict_proba(X_test)[: , 1]
print(evaluer("Gradient boosting (supervisé)", score_gbm))

{'AUC': 0.971, 'AP': 0.646, 'precision': 0.55, 'rappel1': 0.716, 'rappel2': 0.631}

```

Étape 2 : la matrice de confusion à plusieurs budgets. Pour un nombre n d'alertes (les n commandes les plus suspectes), on compte les vraies alertes (VP), les fausses (FP), les fraudes manquées (FN) et les commandes normales laissées passer (VN).

```

def mesures(score, n_alertes):
    alertes = np.argsort(-score)[:n_alertes]
    vp = int(y_test[alertes].sum()); fp = n_alertes - vp; fn = int(y_test.sum()) - vp; vn =
    ↪ len(y_test) - n_alertes - fn

```

```

return dict(alertes=n_alertes, VP=vp, FP=fp, FN=fn, VN=vn, exactitude=(vp + vn) / len(y_test),
            ↪ precision=vp / n_alertes, rappel=vp / y_test.sum())

print(pd.DataFrame([mesures(score_gbm, n) for n in (50, 100, 180, 300,
            ↪ 500)]).round(3).to_string(index=False))

```

alertes	VP	FP	FN	VN	exactitude	precision	rappel
50	47	3	99	17851	0.994	0.94	0.322
100	71	29	75	17825	0.994	0.71	0.486
180	99	81	47	17773	0.993	0.55	0.678
300	108	192	38	17662	0.987	0.36	0.740
500	120	380	26	17474	0.977	0.24	0.822

Lecture. Quand le budget passe de 50 à 500 alertes, la précision s'effondre (de 94 % à 24 %) et le rappel progresse (de 32 % à 82 %), tandis que l'exactitude ne bouge presque pas (de 99,4 % à 97,7 %) : elle ne distingue rien d'utile. Aucune mesure de la table ne dit à elle seule combien d'alertes traiter : il faut les coûts (étape 4).

Étape 3 : la précision moyenne, recalculée à la main. Si l'on parcourt les commandes de la plus suspecte à la moins suspecte, la précision moyenne est la moyenne des précisions mesurées aux rangs où l'on trouve une fraude :

```

ordre = np.argsort(-score_gbm)
est_fraude = y_test[ordre] == 1
precision_rang = np.cumsum(y_test[ordre]) / np.arange(1, len(ordre) + 1)    # précision après n
            ↪ alertes
ap_main = precision_rang[est_fraude].mean()                                # moyenne aux rangs des
            ↪ fraudes
print("AP à la main :", round(float(ap_main), 4), "| scikit-learn :",
            ↪ round(float(average_precision_score(y_test, score_gbm)), 4))

```

```
AP à la main : 0.6463 | scikit-learn : 0.6463
```

Étape 4 : le nombre d'alertes optimal dépend du coût de vérification. On suppose qu'une fraude manquée coûte le montant de la commande et qu'une vérification coûte revue euros.

```

def courbe_cout(score, revue):
    ordre = np.argsort(-score)
    manque = (y_test * montant_test).sum() - np.cumsum(y_test[ordre] * montant_test[ordre])
    return manque + revue * np.arange(1, len(ordre) + 1)

cout_sans = (y_test * montant_test).sum()
print("coût sans aucune alerte :", round(float(cout_sans)), "€")
for revue in (1, 4, 10, 25):
    c = courbe_cout(score_gbm, revue); n_opt = int(c.argmin() + 1)
    print(f"vérification à {revue:2d} € : optimum à {n_opt:4d} alertes, coût {c[n_opt - 1]:7.0f} €,
            ↪ gain {100 * (1 - c[n_opt - 1] / cout_sans):.0f} %")

```

```

coût sans aucune alerte : 14404 €
vérification à 1 € : optimum à 1569 alertes, coût      2853 €, gain 80 %
vérification à 4 € : optimum à 461 alertes, coût      4453 €, gain 69 %
vérification à 10 € : optimum à 175 alertes, coût     6260 €, gain 57 %
vérification à 25 € : optimum à 175 alertes, coût     8885 €, gain 38 %

```

Lecture. Plus la vérification est chère, moins il faut d'alertes et plus le gain par rapport à l'absence de détecteur diminue : l'optimum passe de 1 569 alertes (vérification à 1 €) à 461 (à 4 €), puis à 175 alertes pour une vérification à 10 € et à 25 € (le même nombre : au-delà de la 175e alerte, les fraudes deviennent trop rares dans la liste pour que la décision dépende de la valeur exacte du coût dans cette fourchette). La règle sous-jacente est simple : **une alerte supplémentaire vaut la peine tant que (probabilité de fraude de la commande) × (montant moyen d'une fraude) est**

supérieur au coût de vérification (exercice 6.4).

Application 6.2 — z-score, Mahalanobis et masquage (section 6.2.1 et 6.2.2)

Objectif. Refaire les calculs du livre sur les montants d'un compte, comparer trois façons de combiner les z-scores, vérifier la distance de Mahalanobis, et mesurer l'effet du **masquage** quand la contamination augmente.

Étape 1 : un z-score masqué par son propre outlier.

```
x = np.array([12, 15, 14, 13, 16, 15, 480.])
mad = np.median(np.abs(x - np.median(x)))
print("z classique de 480 :", round(float((480 - x.mean()) / x.std(ddof=1)), 2), "| z robuste :",
      ↪ round(float((480 - np.median(x)) / (1.4826 * mad)), 1))
print("constante 1,4826 : MAD d'une loi normale réduite (simulation) =",
      ↪ round(float(np.median(np.abs(np.random.default_rng(0).normal(size=1_000_000)))), 4), "; 1/1,4826
      ↪ =", round(1 / 1.4826, 4))
```

```
z classique de 480 : 2.27 | z robuste : 313.6
constante 1,4826 : MAD d'une loi normale réduite (simulation) = 0.6757 ; 1/1,4826 = 0.6745
```

Étape 2 : trois variantes de z-score sur les six variables continues. Sommer les valeurs absolues des z-scores ; pour la version robuste, deux traitements du MAD nul (variable ignorée, ou repli sur l'écart absolu moyen multiplié par 1,2533).

```
cont = ["log_montant", "log_distance", "nb_cmd_24h", "log_age_compte", "log_delai", "nb_articles"]
mu, sd = X_app[cont].mean(), X_app[cont].std()
med = X_app[cont].median(); ecarts = (X_app[cont] - med).abs(); mad = ecarts.median()
repli = pd.Series(np.where(mad > 0, 1.4826 * mad, 1.2533 * ecarts.mean()), index=cont)
print("variables à MAD nul :", [c for c in cont if mad[c] == 0])
print("classique      ", evaluer("z classique", ((X_test[cont] - mu) / sd).abs().sum(axis=1).values))
print("robuste (MAD)   ", evaluer("z robuste (MAD seul)", ((X_test[cont] - med) / (1.4826 *
      ↪ mad).replace(0, np.nan)).abs().fillna(0).sum(axis=1).values))
print("robuste (repli) ", evaluer("z robuste (repli)", ((X_test[cont] - med) /
      ↪ repli).abs().sum(axis=1).values))
```

```
variables à MAD nul : ['nb_cmd_24h']
classique      {'AUC': 0.939, 'AP': 0.433, 'precision': 0.378, 'rappel1': 0.457, 'rappel2': 0.477}
robuste (MAD)   {'AUC': 0.89, 'AP': 0.296, 'precision': 0.317, 'rappel1': 0.568, 'rappel2': 0.169}
robuste (repli) {'AUC': 0.93, 'AP': 0.331, 'precision': 0.317, 'rappel1': 0.222, 'rappel2': 0.6}
```

Lecture. Les trois variantes ne trouvent pas les mêmes fraudes : regardez les deux colonnes `rappel1` et `rappel2`. La variante qui ignore la variable à MAD nul (le nombre de commandes des dernières 24 heures) manque presque toutes les prises de contrôle (type 2), signées justement par des rafales de commandes.

Étape 3 : la distance de Mahalanobis, calculée deux fois. À la main avec l'inverse de la matrice de covariance, puis avec `scikit-learn`. Les deux calculs ne diffèrent que par la convention de la covariance (`np.cov` divise par $n - 1$, `EmpiricalCovariance` par n), c'est-à-dire d'un facteur $n/(n - 1)$ très proche de 1 :

```
from sklearn.covariance import EmpiricalCovariance

precision_app = np.linalg.inv(np.cov(Z_app.T))
d2_main = np.einsum("ij,jk,ik->i", Z_test, precision_app, Z_test)
d2_sk = EmpiricalCovariance().fit(Z_app).mahalanobis(Z_test)
print("écart maximal entre les deux calculs :", float(np.abs(d2_main - d2_sk).max()))
print(evaluer("Mahalanobis", d2_main))
```

```
écart maximal entre les deux calculs : 0.0038964685454629944
{'AUC': 0.946, 'AP': 0.382, 'precision': 0.311, 'rappel1': 0.198, 'rappel2': 0.615}
```

Étape 4 : le masquage en fonction de la contamination. On simule un groupe normal de 1 000 points corrélés ($\rho = 0,8$) auquel on ajoute des points contaminants regroupés loin du centre, dans une proportion croissante. On compte la part des contaminants détectés (au-dessus du quantile 97,5 % du χ^2 à 2 degrés de liberté) par la distance classique et par la distance robuste (MCD).

```
from scipy.stats import chi2
from sklearn.covariance import MinCovDet

rng = np.random.default_rng(5); lignes = []
normaux = rng.multivariate_normal([0, 0], [[1, 0.8], [0.8, 1]], 1000)
for taux in (0.02, 0.05, 0.10, 0.20, 0.30):
    n_c = int(1000 * taux / (1 - taux))
    contam = rng.multivariate_normal([4.5, -2.5], 0.25 * np.eye(2), n_c)
    donnees = np.vstack([normaux, contam]); seuil = chi2.ppf(0.975, 2)
    d_c = np.einsum("ij,jk,ik->i", donnees - donnees.mean(0), np.linalg.inv(np.cov(donnees.T)),
    ↪ donnees - donnees.mean(0))
    d_r = MinCovDet(random_state=0, support_fraction=0.7).fit(donnees).mahalanobis(donnees)
    lignes.append(dict(contamination=taux, contaminants=n_c,
    ↪ detectes_classique=round(float((d_c[1000:] > seuil).mean()), 2),
    ↪ detectes_robuste=round(float((d_r[1000:] > seuil).mean()), 2)))
print(pd.DataFrame(lignes).to_string(index=False))
```

contamination	contaminants	detectes_classique	detectes_robuste
0.02	20	1.00	1.0
0.05	52	1.00	1.0
0.10	111	0.75	1.0
0.20	250	0.00	1.0
0.30	428	0.00	1.0

Lecture. Jusqu'à 5 % de contamination, la distance classique détecte tous les contaminants ; à 10 %, elle n'en détecte plus que 75 % ; à 20 % et 30 %, **aucun** : le groupe contaminant étire tellement la covariance estimée qu'il se masque lui-même. L'estimateur robuste (MCD, qui travaille sur 70 % des points) les détecte tous, même à 30 %. En théorie, il ne peut pas tolérer plus de contamination que la part de points qu'il exclut (ici 30 %) : au-delà, son sous-ensemble « sain » contiendrait lui-même des contaminants.

Application 6.3 — Plus proches voisins et LOF (section 6.2.3 et 6.2.4)

Objectif. Choisir le nombre de voisins sur le jeu de validation, mesurer l'effet de la standardisation et de la taille de l'échantillon de référence, écrire le LOF « à la main » et le vérifier, puis régler le LOF.

Étape 1 : choisir k sur la validation, avec et sans standardisation. Pour chaque k , la précision moyenne sur le jeu de validation (qui sert à choisir) et sur le jeu de test (qui sert à juger) :

```
from sklearn.neighbors import NearestNeighbors

def grille_knn(Z_ref, Z_v, Z_t, ks=(1, 5, 10, 30, 100)):
    nn = NearestNeighbors(n_neighbors=max(ks)).fit(Z_ref)
    dv, dt = nn.kneighbors(Z_v)[0], nn.kneighbors(Z_t)[0]
    return pd.DataFrame({"k": ks, "AP validation": [average_precision_score(y_val, dv[:,
    ↪ :k]).mean(axis=1)) for k in ks],
    ↪ "AP test": [average_precision_score(y_test, dt[:, :k]).mean(axis=1)) for k in
    ↪ ks]}).round(3)

standard = grille_knn(Z_app[reference], Z_val, Z_test)
brut = grille_knn(X_app.values[reference], X_app.values[valid], X_test.values)
```

```
k_knn = int(standard.loc[standard["AP validation"].idxmax(), "k"])
print("variables standardisées\n", standard.to_string(index=False), "\nvariables brutes\n",
      ↪ brut.to_string(index=False), "\nk choisi :", k_knn)
```

```
variables standardisées
  k AP validation AP test
1   0.261      0.303
5   0.350      0.411
10  0.382      0.444
30  0.400      0.460
100 0.399      0.448
variables brutes
  k AP validation AP test
1   0.279      0.402
5   0.355      0.486
10  0.386      0.503
30  0.426      0.511
100 0.446      0.508
k choisi : 30
```

Lecture. La validation retient $k = 30$ (AP de 0,400 ; $k = 100$ donne 0,399) et le jeu de test classe les réglages dans le même ordre. Autre résultat : sur les **variables brutes**, les voisins font mieux pour tous les k (0,511 pour $k = 30$ contre 0,460 avec la standardisation). Une hypothèse explique cet écart : la standardisation multiplie les deux variables binaires (IP étrangère, appareil inconnu), d'écart-type 0,20 et 0,30, par 5 et 3,3, alors que les variables continues, d'écart-type voisin de 1, ne bougent presque pas. Vérifions-la en ne standardisant **que** les variables continues :

```
binaires = [list(X.columns).index(c) for c in ("ip_différent", "appareil_inconnu")]
Z_mixte_app, Z_mixte_test = Z_app.copy(), Z_test.copy()
Z_mixte_app[:, binaires] = X_app.values[:, binaires]; Z_mixte_test[:, binaires] = X_test.values[:,
↪ binaires] # binaires laissées en 0/1
d = NearestNeighbors(n_neighbors=k_knn).fit(Z_mixte_app[reference]).kneighbors(Z_mixte_test)[0]
↪ ].mean(axis=1)
print("AP test, continues standardisées et binaires en 0/1 :",
      ↪ round(float(average_precision_score(y_test, d)), 3))
```

```
AP test, continues standardisées et binaires en 0/1 : 0.513
```

Le résultat (voir la sortie) est du même ordre que celui des variables brutes : l'hypothèse est confirmée. Moralité : **standardiser mécaniquement toutes les colonnes n'est pas toujours neutre**, surtout avec des variables binaires rares.

Étape 2 : la taille de l'échantillon de référence. Comparer chaque commande à un échantillon plus grand coûte plus de temps ; qu'y gagne-t-on ?

```
rng_r = np.random.default_rng(1)
for taille in (500, 2000, 5000, 10000, 20000):
    idx = rng_r.choice(len(Z_app), taille, replace=False)
    d = NearestNeighbors(n_neighbors=k_knn).fit(Z_app[idx]).kneighbors(Z_test)[0].mean(axis=1)
    print(f"référence de {taille:6d} commandes : AP test = {average_precision_score(y_test, d):.3f}")
```

```
référence de   500 commandes : AP test = 0.370
référence de  2000 commandes : AP test = 0.458
référence de  5000 commandes : AP test = 0.465
référence de 10000 commandes : AP test = 0.468
référence de 20000 commandes : AP test = 0.476
```

Étape 3 : le LOF écrit à la main. Une fonction qui calcule le LOF pour des points de dimension quelconque, avec les définitions de la section 6.2.4 (k -distance, distance d'atteignabilité, densité locale), vérifiée contre scikit-learn sur 60 points aléatoires.

```

from sklearn.neighbors import LocalOutlierFactor

def lof_main(P, k):
    D = np.sqrt(((P[:, None, :] - P[None, :, :]) ** 2).sum(axis=2))
    ordre = np.argsort(D, axis=1)
    voisins = ordre[:, 1:k + 1]  # les k plus proches voisins (sans le
    ↪ point lui-même)
    kdists = D[np.arange(len(P)), ordre[:, k]]  # k-distance de chaque point
    atteint = np.maximum(kdists[voisins], np.take_along_axis(D, voisins, axis=1))  # distance
    ↪ d'atteignabilité à chaque voisin
    lrd = 1 / atteint.mean(axis=1)  # densité d'atteignabilité locale
    return lrd[voisins].mean(axis=1) / lrd

P = np.random.default_rng(2).normal(size=(60, 2))
ecart = np.abs(lof_main(P, 5) +
    ↪ LocalOutlierFactor(n_neighbors=5).fit(P).negative_outlier_factor_).max()
print("écart maximal avec scikit-learn :", float(ecart))

```

```

écart maximal avec scikit-learn : 3.012434746096915e-10

```

Étape 4 : le LOF sur les transactions, réglé sur la validation.

```

lignes = []
for k in (10, 30, 100, 300):
    lof = LocalOutlierFactor(n_neighbors=k, novelty=True).fit(Z_app[reference])
    lignes.append(dict(k=k, AP_validation=average_precision_score(y_val, -lof.score_samples(Z_val)),
    ↪ score_test=-lof.score_samples(Z_test)))
k_lof = max(lignes, key=lambda r: r["AP_validation"])[k]
for r in lignes:
    print(f"k = {r['k']:3d} : AP validation {r['AP_validation']:.3f} |", evaluer(f"LOF k={r['k']}"),
    ↪ r["score_test"])
print("k choisi sur la validation :", k_lof)

```

```

k = 10 : AP validation 0.114 | {'AUC': 0.886, 'AP': 0.113, 'precision': 0.183, 'rappel1': 0.309,
    ↪ 'rappel2': 0.123}
k = 30 : AP validation 0.243 | {'AUC': 0.945, 'AP': 0.306, 'precision': 0.322, 'rappel1': 0.432,
    ↪ 'rappel2': 0.354}
k = 100 : AP validation 0.408 | {'AUC': 0.956, 'AP': 0.471, 'precision': 0.444, 'rappel1': 0.58,
    ↪ 'rappel2': 0.508}
k = 300 : AP validation 0.433 | {'AUC': 0.944, 'AP': 0.492, 'precision': 0.411, 'rappel1': 0.519,
    ↪ 'rappel2': 0.492}
k choisi sur la validation : 300

```

Lecture. Le LOF est **extrêmement sensible** à k : sur la validation, la précision moyenne passe de 0,114 ($k = 10$) à 0,243 ($k = 30$), 0,408 ($k = 100$) puis 0,433 ($k = 300$) ; le jeu de test classe les réglages dans le même ordre (0,113 ; 0,306 ; 0,471 ; 0,492). À $k = 300$, le LOF retrouve 52 % des fraudes de type 1 et 49 % de celles de type 2, mieux que les voisins simples (AP de 0,460). Le meilleur k est en bout de grille : essayez $k = 500$ ou 1 000 (exercice laissé au lecteur) et observez si la courbe plafonne.

Application 6.4 — Une forêt d'isolement écrite à la main (section 6.3)

Objectif. Construire des arbres d'isolement avec quelques lignes de numpy, les assembler en forêt, calculer le score $s = 2^{-E[h]/c(n)}$, et comparer avec scikit-learn.

Étape 1 : un arbre d'isolement. À chaque nœud, on tire une variable au hasard, puis une coupure uniforme entre son minimum et son maximum, jusqu'à isoler les points ou atteindre la hauteur maximale. Un nœud est un quadruplet (variable, coupure, enfant gauche, enfant droit) ; une feuille est le nombre de points qu'elle contient.

```
def c_de_n(n):
    return 0.0 if n <= 1 else (1.0 if n == 2 else 2 * (np.log(n - 1) + 0.5772156649) - 2 * (n - 1) / n)

def arbre(P, hauteur_max, rng, h=0):
    if h >= hauteur_max or len(P) <= 1:
        return len(P) # feuille : nombre de points restants
    j = rng.integers(P.shape[1]); bas, haut = P[:, j].min(), P[:, j].max()
    if bas == haut:
        return len(P)
    coupure = rng.uniform(bas, haut); gauche = P[:, j] < coupure
    return (j, coupure, arbre(P[gauche], hauteur_max, rng, h + 1), arbre(P[~gauche], hauteur_max, rng,
        ↪ h + 1))
```

Étape 2 : la profondeur d'isolement de chaque point. Pour une feuille qui contient encore $m > 1$ points (hauteur maximale atteinte), on ajoute $c(m)$, la longueur moyenne attendue pour finir de les isoler.

```
def profondeurs(noeud, P, h=0):
    if not isinstance(noeud, tuple):
        return np.full(len(P), h + c_de_n(noeud))
    j, coupure, g, d = noeud
    masque = P[:, j] < coupure; sortie = np.empty(len(P))
    sortie[masque] = profondeurs(g, P[masque], h + 1); sortie[~masque] = profondeurs(d, P[~masque], h
    ↪ + 1)
    return sortie

# petit test : les 8 valeurs du livre (section 6.3.1)
v = np.array([2, 3, 3.5, 4, 4.5, 5, 5.5, 30]); rng = np.random.default_rng(0)
moy = np.mean([profondeurs(arbre(v, 8, rng), v) for _ in range(3000)], axis=0)
print("coupures moyennes :", dict(zip([2, 3, 3.5, 4, 4.5, 5, 5.5, 30], moy.round(2))))

coupures moyennes : {2: np.float64(3.0), 3: np.float64(4.21), 3.5: np.float64(4.68), 4:
    ↪ np.float64(4.77), 4.5: np.float64(4.66), 5: np.float64(4.35), 5.5: np.float64(3.54), 30:
    ↪ np.float64(1.13)}
```

Étape 3 : la forêt, le score et la comparaison avec scikit-learn. 100 arbres de 256 points, hauteur maximale $\lceil \log_2 256 \rceil = 8$.

```
from scipy.stats import spearmanr
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(0); psi = 256
foret = [arbre(Z_app[rng.choice(len(Z_app)), psi, replace=False], 8, rng) for _ in range(100)]
esperance_h = np.mean([profondeurs(a, Z_test) for a in foret], axis=0)
score_main = 2 ** (-esperance_h / c_de_n(psi))
score_sk = IsolationForest(n_estimators=100, max_samples=psi,
    ↪ random_state=0).fit(Z_app).score_samples(Z_test)
print("ma forêt :", evaluer("Forêt d'isolement (à la main)", score_main))
print("scikit-learn :", evaluer("Forêt d'isolement (sklearn)", score_sk))
print("corrélation de rang entre les deux scores :", round(float(spearmanr(score_main, score_sk)[0]),
    ↪ 3), "| score moyen d'une commande normale :", round(float(score_main[y_test == 0].mean()), 3))
```

```

ma forêt      : {'AUC': 0.935, 'AP': 0.325, 'precision': 0.306, 'rappel1': 0.111, 'rappel2': 0.708}
scikit-learn  : {'AUC': 0.924, 'AP': 0.293, 'precision': 0.278, 'rappel1': 0.099, 'rappel2': 0.646}
corrélation de rang entre les deux scores : 0.968 | score moyen d'une commande normale : 0.462

```

Lecture. Notre forêt de 100 arbres obtient une précision moyenne de 0,325 et celle de scikit-learn, avec 100 arbres aussi, de 0,293 : les rangs des scores sont très corrélés (0,968) mais pas identiques, car les deux forêts tirent des variables et des coupures différentes. L'écart de 0,03 est de l'ordre du **bruit dû à l'aléa** d'une forêt de 100 arbres : deux forêts construites de la même façon avec des graines différentes ne donnent pas exactement le même détecteur. Les deux retrouvent surtout le type 2 (71 % et 65 %) et peu le type 1 (11 % et 10 %). Le score moyen d'une commande normale est de 0,462, proche des 0,5 que prévoit la théorie pour un point moyen.

Étape 4 : de quel seuil parle-t-on ? En calculant le score sur le jeu d'apprentissage (sans étiquette), on obtient la distribution des scores « normaux » ; fixer une contamination c , c'est retenir le quantile $1 - c$ de cette distribution comme seuil.

```

esperance_app = np.mean([profondeurs(a, Z_app[:10000]) for a in foret], axis=0)
score_app = 2 ** (-esperance_app / c_de_n(psi))
for c in (0.005, 0.0081, 0.02, 0.05):
    seuil = np.quantile(score_app, 1 - c); drapeau = score_main >= seuil
    print(f"contamination {c:6.4f} : seuil {seuil:.3f}, {drapeau.sum():5d} alertes, précision
          ↳ {y_test[drapeau].mean():.3f}, rappel {y_test[drapeau].sum() / y_test.sum():.3f}")

```

```

contamination 0.0050 : seuil 0.609,    109 alertes, précision 0.413, rappel 0.308
contamination 0.0081 : seuil 0.598,    158 alertes, précision 0.335, rappel 0.363
contamination 0.0200 : seuil 0.572,    367 alertes, précision 0.188, rappel 0.473
contamination 0.0500 : seuil 0.546,    910 alertes, précision 0.103, rappel 0.644

```

Lecture. Le nombre d'alertes suit la contamination choisie (158 alertes pour 0,0081, 367 pour 0,02 et 910 pour 0,05, sur 18 000 commandes). Plus on alerte, plus le rappel augmente (de 31 % à 64 %) et plus la précision s'effondre (de 41 % à 10 %) : fixer la contamination, c'est choisir un point de cette courbe, donc un budget d'alertes.

Application 6.5 — Autoencodeur et ACP (section 6.4)

Objectif. Vérifier numériquement qu'un autoencodeur linéaire se comporte comme une ACP, examiner la convergence d'un réseau, tester l'hypothèse que l'instabilité vient d'un entraînement trop court, et reproduire la **procédure de sélection sur un jeu de validation** du livre.

Étape 1 : un autoencodeur linéaire est une ACP à une composante. Un MLPRegressor avec une couche cachée d'un seul neurone et une activation identité, entraîné à reconstruire ses entrées, calcule une projection linéaire sur une droite. L'erreur de reconstruction doit donc ressembler à celle de l'ACP à une composante :

```

from sklearn.decomposition import PCA
from sklearn.neural_network import MLPRegressor

pca1 = PCA(n_components=1).fit(Z_app)
err_pca = ((Z_test - pca1.inverse_transform(pca1.transform(Z_test))) ** 2).sum(axis=1)
ae_lin = MLPRegressor(hidden_layer_sizes=(1,), activation="identity", max_iter=500,
↳ random_state=0).fit(Z_app[reference], Z_app[reference])
err_ae = ((ae_lin.predict(Z_test) - Z_test) ** 2).sum(axis=1)
print("corrélation entre les deux erreurs de reconstruction :", round(float(np.corrcoef(err_pca,
↳ err_ae)[0, 1]), 3))
print("ACP (1 composante)      :", evaluer("ACP 1 composante", err_pca))
print("autoencodeur linéaire   :", evaluer("AE linéaire (1 neurone)", err_ae))

```

```

corrélation entre les deux erreurs de reconstruction : 1.0
ACP (1 composante)      : {'AUC': 0.946, 'AP': 0.391, 'precision': 0.311, 'rappel1': 0.198, 'rappel2':
↳ 0.615}
autoencodeur linéaire   : {'AUC': 0.945, 'AP': 0.386, 'precision': 0.322, 'rappel1': 0.198, 'rappel2':
↳ 0.646}

```

Lecture. Les deux erreurs de reconstruction sont corrélées à plus de 0,999 et donnent la même précision moyenne (0,39) : l'autoencodeur linéaire entraîné par descente de gradient retrouve l'ACP, comme le prévoit le théorème d'Eckart-Young (exercice 6.10).

Étape 2 : la convergence. Le livre entraîne les réseaux sur 100 itérations seulement ; a-t-on convergé ? La courbe de perte et le nombre d'itérations effectuées répondent :

```

ae = MLPRegressor(hidden_layer_sizes=(6, 3, 6), activation="tanh", max_iter=100,
↳ random_state=0).fit(Z_app[reference], Z_app[reference])
fig, ax = plt.subplots(figsize=(6.5, 3.4))
ax.plot(ae.loss_curve_, color=style.BLEU); ax.set_yscale("log")
ax.set_xlabel("itération (époque)"); ax.set_ylabel("perte d'entraînement (échelle log)")
plt.tight_layout(); plt.savefig("figures/ch06-cahier-perte.png", dpi=200, bbox_inches="tight");
↳ plt.close()
print("itérations effectuées :", ae.n_iter_, "sur 100 permises | perte finale :",
↳ round(float(ae.loss_curve_[-1]), 4), "| 10 itérations avant :", round(float(ae.loss_curve_[-11]),
↳ 4))

```

```

itérations effectuées : 100 sur 100 permises | perte finale : 0.2699 | 10 itérations avant : 0.271

```

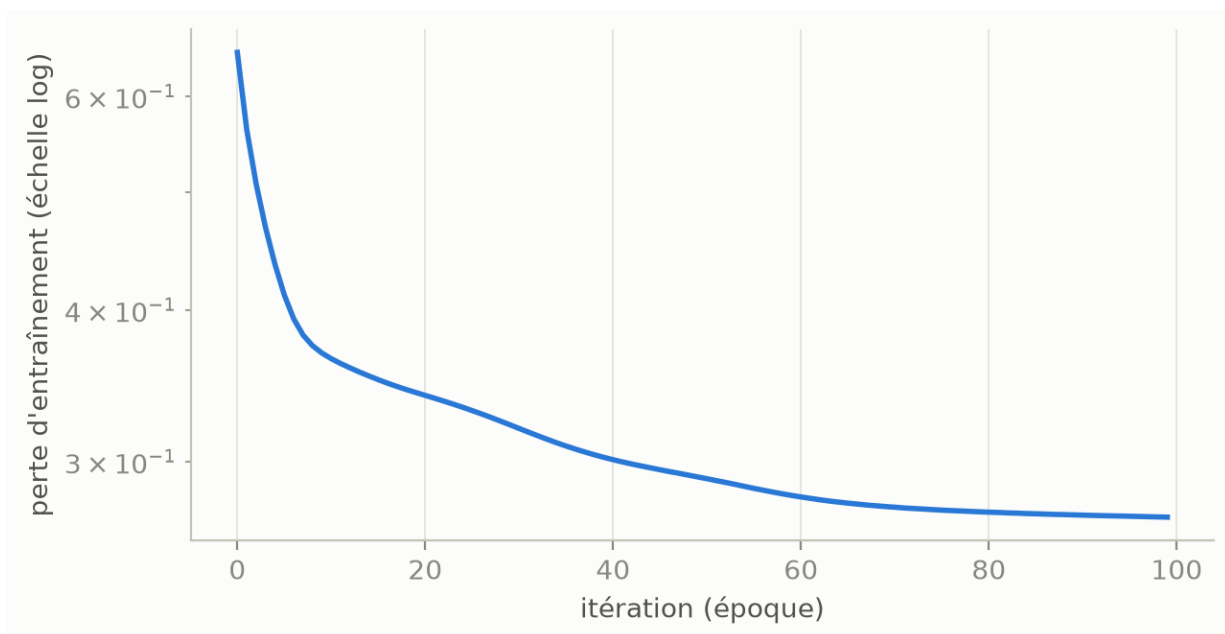


Figure 1: Perte d'entraînement de l'autoencodeur (6, 3, 6) au fil des itérations, en échelle logarithmique.

Étape 3 : l'instabilité vient-elle d'un entraînement trop court ? Pour deux architectures instables dans le livre, on compare 100 et 400 itérations, sur trois graines. *Attention : cet examen utilise le jeu de test pour étudier un phénomène, pas pour choisir un réglage.*

```

def ap_ae(h, iters, graine):
    r = MLPRegressor(hidden_layer_sizes=h, activation="tanh", max_iter=iters,
↳ random_state=graine).fit(Z_app[reference], Z_app[reference])
    return average_precision_score(y_test, ((r.predict(Z_test) - Z_test) ** 2).sum(axis=1))

```

```
for h in ((5,), (16, 8, 16)):
    for iters in (100, 400):
        aps = [ap_ae(h, iters, g) for g in range(3)]
        print(f"{str(h):12s} {iters:3d} itérations : AP par graine {np.round(aps, 3)}, moyenne
              ↳ {np.mean(aps):.3f}")
```

```
(5,)          100 itérations : AP par graine [0.068 0.176 0.109], moyenne 0.117
(5,)          400 itérations : AP par graine [0.067 0.176 0.109], moyenne 0.117
(16, 8, 16)   100 itérations : AP par graine [0.135 0.127 0.087], moyenne 0.116
(16, 8, 16)   400 itérations : AP par graine [0.135 0.127 0.091], moyenne 0.118
```

Lecture. Au bout des 100 itérations permises, la perte de l'autoencodeur (6, 3, 6) ne diminue presque plus (0,2699 contre 0,2710 dix itérations plus tôt). Et laisser 400 itérations aux deux architectures les moins bonnes ne change **pas** leur précision moyenne (0,117 dans les deux cas pour (5,) ; 0,116 puis 0,118 pour (16, 8, 16)) : l'instabilité observée dans le livre ne vient donc pas d'un entraînement trop court. Chaque réseau converge vers une solution qui dépend de son initialisation.

Choix sur la validation (étape 4). Le comité de trois réseaux (6, 3, 6) l'emporte nettement sur les deux autres architectures, à la fois sur la validation (0,452 contre 0,241 et 0,183) et sur le test (0,484 contre 0,256 et 0,201) : la validation désigne la bonne architecture.

Étape 4 : choisir l'architecture sur le jeu de validation. La procédure du livre en miniature : trois architectures, trois graines, comité de trois réseaux ; on choisit sur la validation, on juge sur le test.

```
def comite(h, graines=(0, 1, 2)):
    sv, st = [], []
    for g in graines:
        r = MLPRegressor(hidden_layer_sizes=h, activation="tanh", max_iter=100,
        ↳ random_state=g).fit(Z_app[reference], Z_app[reference])
        ev = ((r.predict(Z_val) - Z_val) ** 2).sum(axis=1); et = ((r.predict(Z_test) - Z_test) **
        ↳ 2).sum(axis=1)
        sv.append((ev - ev.mean()) / ev.std()); st.append((et - et.mean()) / et.std())      #
        ↳ échelle fixée sur la validation
    return np.mean(sv, axis=0), np.mean(st, axis=0)

resultat = {h: comite(h) for h in ((3,), (6, 3, 6), (10, 5, 10))}
for h, (sv, st) in resultat.items():
    print(f"{str(h):10s} AP validation {average_precision_score(y_val, sv):.3f} | AP test
          ↳ {average_precision_score(y_test, st):.3f}")
choix = max(resultat, key=lambda h: average_precision_score(y_val, resultat[h][0]))
print("architecture choisie sur la validation :", choix)
```

```
(3,)          AP validation 0.241 | AP test 0.256
(6, 3, 6)     AP validation 0.452 | AP test 0.484
(10, 5, 10)   AP validation 0.183 | AP test 0.201
architecture choisie sur la validation : (6, 3, 6)
```

Application 6.6 — Trier les alertes : combiner les détecteurs (section 6.4.5)

Objectif. Comparer ce que trouvent trois détecteurs de familles différentes, tester des combinaisons, utiliser les scores non supervisés comme **variables d'un modèle supervisé**, et traduire le tout en euros.

Étape 1 : trois détecteurs, ajustés sans étiquette sur l'échantillon de référence. Les distances aux voisins, la forêt d'isolement et un comité de trois autoencodeurs (6, 3, 6). Comme aucun n'a vu les commandes du jeu de validation, on peut y calculer leurs scores sans biais.

```

from sklearn.ensemble import IsolationForest

voisins = NearestNeighbors(n_neighbors=k_knn).fit(Z_app[reference])      # k_knn : choisi à
↳ l'application 6.3
iso = IsolationForest(n_estimators=200, random_state=0).fit(Z_app[reference])
s_val, s_test = {}, {}
for nom, Z, sortie in (("validation", Z_val, s_val), ("test", Z_test, s_test)):
    sortie["voisins"] = voisins.kneighbors(Z)[0].mean(axis=1)
    sortie["isolement"] = -iso.score_samples(Z)
sv, st = comite((6, 3, 6), graines=(0, 1, 2, 3))
s_val["autoencodeur"], s_test["autoencodeur"] = sv, st
for nom in s_test:
    print(f"{nom:13s}", evaluer(nom, s_test[nom]))

voisins      {'AUC': 0.959, 'AP': 0.46, 'precision': 0.367, 'rappel1': 0.259, 'rappel2': 0.692}
isolement    {'AUC': 0.935, 'AP': 0.332, 'precision': 0.289, 'rappel1': 0.111, 'rappel2': 0.662}
autoencodeur  {'AUC': 0.962, 'AP': 0.526, 'precision': 0.478, 'rappel1': 0.519, 'rappel2': 0.677}

```

Étape 2 : se recouvrent-ils ? Si deux détecteurs retrouvent les mêmes commandes, les combiner n'apporte rien. On mesure le recouvrement de leurs 180 alertes (indice de Jaccard : taille de l'intersection divisée par la taille de l'union) :

```

alertes = {nom: set(np.argsort(-s)[:budget]) for nom, s in s_test.items()}
for a, b in (("voisins", "isolement"), ("voisins", "autoencodeur"), ("isolement", "autoencodeur")):
    inter = len(alertes[a] & alertes[b])
    print(f"{a:10s} et {b:13s} : {inter:3d} alertes communes, Jaccard = {inter / len(alertes[a] |
↳ alertes[b]):.2f}")

```

```

voisins      et isolement      : 133 alertes communes, Jaccard = 0.59
voisins      et autoencodeur    : 119 alertes communes, Jaccard = 0.49
isolement    et autoencodeur    : 85 alertes communes, Jaccard = 0.31

```

Lecture. Les voisins et la forêt d'isolement se recouvrent beaucoup (133 alertes communes sur 180, Jaccard de 0,59) : ils voient en grande partie les mêmes fraudes, les combiner n'ajoutera pas grand-chose. La forêt d'isolement et l'autoencodeur se recouvrent beaucoup moins (85 alertes communes, Jaccard de 0,31) : ils sont plus complémentaires.

Étape 3 : des combinaisons. Moyenne des rangs de deux ou trois détecteurs, ou maximum des rangs (une alerte dès qu'un détecteur la juge très suspecte).

```

from scipy.stats import rankdata
rang = {nom: rankdata(s) / len(s) for nom, s in s_test.items()}
combos = {"rang moyen (3)": sum(rang.values()) / 3, "rang moyen (voisins + autoencodeur)":
↳ (rang["voisins"] + rang["autoencodeur"]) / 2,
    "rang maximal (3)": np.maximum.reduce(list(rang.values()))}
for nom, s in combos.items():
    print(f"{nom:13s}", evaluer(nom, s))

rang moyen (3)                                {'AUC': 0.957, 'AP': 0.435, 'precision': 0.356, 'rappel1':
↳ 0.259, 'rappel2': 0.662}
rang moyen (voisins + autoencodeur)           {'AUC': 0.962, 'AP': 0.507, 'precision': 0.439, 'rappel1':
↳ 0.444, 'rappel2': 0.662}
rang maximal (3)                              {'AUC': 0.96, 'AP': 0.497, 'precision': 0.439, 'rappel1': 0.407,
↳ 'rappel2': 0.708}

```

Lecture. Aucune combinaison ne dépasse l'autoencodeur seul (précision moyenne de 0,526). La meilleure, la moyenne des rangs des voisins et de l'autoencodeur (0,507), s'en rapproche ; la moyenne des trois (0,435) est tirée vers le bas par la forêt d'isolement, nettement moins bonne. Combiner n'aide que si les membres sont de qualité comparable.

Étape 4 : les scores non supervisés comme variables d'un modèle supervisé. On entraîne un

gradient boosting **sur le jeu de validation** (32 000 commandes étiquetées, qui n'ont servi ni à ajuster les détecteurs ni à les juger), avec les dix variables seules, ou avec les dix variables plus les trois scores. Quelques centaines de fraudes seulement : le réglage du modèle compte, et nous le choisissons par **validation croisée à 5 plis sur le jeu de validation**, sans regarder le test.

```
from sklearn.model_selection import StratifiedKFold, cross_val_predict

X_val_df, X_test_df = X_app.iloc[valid].reset_index(drop=True), X_test.reset_index(drop=True)
plus = lambda X_, s: X_.assign(**{f"score_{k}": v for k, v in s.items()})
jeux = {"variables seules": (X_val_df, X_test_df), "variables + 3 scores": (plus(X_val_df, s_val),
    ↪ plus(X_test_df, s_test))}
reglages = {"profondeur libre": {}, "profondeur 3": dict(max_depth=3, learning_rate=0.05)}
cv = StratifiedKFold(5, shuffle=True, random_state=0); lignes = []
for nj, (A, _) in jeux.items():
    for nr, kw in reglages.items():
        m = HistGradientBoostingClassifier(class_weight="balanced", max_iter=150, random_state=0,
    ↪ **kw)
        p_cv = cross_val_predict(m, A, y_val, cv=cv, method="predict_proba")[:, 1]
        lignes.append(dict(jeu=nj, reglage=nr,
    ↪ AP_validation_croisee=round(float(average_precision_score(y_val, p_cv)), 3)))
res_cv = pd.DataFrame(lignes); print(res_cv.to_string(index=False))
```

	jeu	reglage	AP_validation_croisee
	variables seules	profondeur libre	0.537
	variables seules	profondeur 3	0.595
	variables + 3 scores	profondeur libre	0.426
	variables + 3 scores	profondeur 3	0.498

```
meilleur = res_cv.loc[res_cv["AP_validation_croisee"].idxmax()]
A, B = jeux[meilleur["jeu"]]
sup_choisi = HistGradientBoostingClassifier(class_weight="balanced", max_iter=150, random_state=0,
    ↪ **reglages[meilleur["reglage"]]).fit(A, y_val)
score_sup = sup_choisi.predict_proba(B)[:, 1]
print("choisi par validation croisée :", dict(meilleur[["jeu", "reglage"]]))
print("sur le jeu de test :", evaluer("GBM choisi par validation croisée", score_sup))
```

```
choisi par validation croisée : {'jeu': 'variables seules', 'reglage': 'profondeur 3'}
sur le jeu de test : {'AUC': 0.957, 'AP': 0.655, 'precision': 0.511, 'rappel1': 0.654, 'rappel2': 0.6}
```

Lecture. La validation croisée est claire : **les trois scores n'aident pas** le modèle supervisé (précision moyenne de 0,426 avec les scores contre 0,537 sans, à profondeur libre ; 0,498 contre 0,595 avec des arbres de profondeur 3). On retient donc les variables seules avec des arbres de profondeur 3, qui obtiennent 0,655 sur le jeu de test, soit autant que le modèle du livre entraîné sur les 42 000 commandes (0,646) alors qu'ici il n'en a vu que 32 000. Pourquoi les scores n'apportent-ils rien ? Une explication plausible, que nous n'avons pas testée, est qu'ils sont des fonctions des mêmes dix variables, que le gradient boosting exploite déjà directement, et que trois variables de plus augmentent le risque de sur-ajustement avec seulement 246 fraudes pour apprendre. Retenez la **méthode** (régler par validation croisée avant de regarder le test) plus que le résultat.

Étape 5 : en euros. Pour chaque méthode, le coût total au budget de 180 alertes (fraudes manquées plus 4 € par vérification), en pourcentage du coût sans détecteur :

```
sans = (y_test * montant_test).sum()
ligne = lambda nom, s: f"{nom:34s} {courbe_cout(s, 4.0)[budget - 1]:7.0f} € ({100 * (1 -
    ↪ courbe_cout(s, 4.0)[budget - 1] / sans):3.0f} % d'économie)"
for nom, s_ in [("gradient boosting (apprentissage)", score_gbm), ("GBM choisi par validation
    ↪ croisée", score_sup),
    ↪ ("voisins", s_test["voisins"]), ("forêt d'isolement", s_test["isolement"]),
    ↪ ("autoencodeur (comité de 4)", s_test["autoencodeur"]),
    ↪ ("rang moyen (3)", combos["rang moyen (3)"])]]:
    print(ligne(nom, s_))
```

gradient boosting (apprentissage)	5230 €	(64 % d'économie)
GBM choisi par validation croisée	6018 €	(58 % d'économie)
voisins	8932 €	(38 % d'économie)
forêt d'isolement	10205 €	(29 % d'économie)
autoencodeur (comité de 4)	6572 €	(54 % d'économie)
rang moyen (3)	8697 €	(40 % d'économie)

Exercices

Exercice 6.1 ★ — Deux détecteurs, une même exactitude (section 6.1.3)

Sur 5 000 commandes, dont 40 frauduleuses, le détecteur A ne déclenche jamais d'alerte ; le détecteur B déclenche 60 alertes, dont 30 sont de vraies fraudes. Calculez l'exactitude, la précision, le rappel et la mesure F1 de chacun. Que remarque-t-on ?

Exercice 6.2 ★ — Précision moyenne à la main (section 6.1.5)

Un détecteur classe huit commandes de la plus suspecte à la moins suspecte ; les étiquettes dans cet ordre sont : fraude, normale, fraude, normale, normale, fraude, normale, normale (soit 3 fraudes sur 8). (a) Donnez la précision et le rappel après 1, 3 et 6 alertes. (b) Calculez la précision moyenne (AP) comme la moyenne des précisions aux rangs des fraudes.

Exercice 6.3 ★★ — Taux de base et formule de Bayes (section 6.1.3)

Une boutique reçoit 0,5 % de commandes frauduleuses. Un détecteur retrouve 90 % des fraudes et déclenche une fausse alerte sur 2 % des commandes normales. (a) Quelle est la probabilité qu'une alerte soit une vraie fraude ? (b) Quel taux de fausses alertes faudrait-il pour que la moitié des alertes soient justes (à rappel inchangé) ?

Exercice 6.4 ★★ — Combien d'alertes traiter ? (section 6.1.6)

Une fraude manquée coûte en moyenne 70 € ; une vérification coûte 5 €. Sur un jeu de 100 fraudes, un détecteur en retrouve 40 dans ses 100 premières alertes, 60 dans les 200 premières, 70 dans les 300, 75 dans les 400 et 78 dans les 500. (a) Calculez le coût total pour chaque nombre d'alertes. (b) À partir de quelle précision marginale une alerte supplémentaire cesse-t-elle de valoir la peine ? Vérifiez-le sur les tranches de 100 alertes.

Exercice 6.5 ★ — z-score classique contre z-score robuste (section 6.2.1)

Les montants (en €) d'un compte sont 20, 22, 21, 19, 23, 20, 21, 22 et 95. Calculez le z-score de 95 avec la moyenne et l'écart-type, puis avec la médiane et le MAD. Lequel signale la commande ?

Exercice 6.6 ★★ — Mahalanobis à la main (section 6.2.2)

Deux variables standardisées ont pour corrélation $\rho = 0,6$. Calculez la distance de Mahalanobis au carré des points $A = (1, 1)$, $B = (1, -1)$ et $C = (2, 0)$, et comparez avec leur distance euclidienne au carré. Lesquels dépassent le seuil du χ^2 à 2 degrés de liberté au niveau 95 % (5,99) ?

Exercice 6.7 ★★ — LOF sur cinq points (section 6.2.4)

Cinq points sur une droite sont aux abscisses 0; 1; 2,2; 3,5; 9. Avec $k = 2$, calculez pour chaque point sa 2-distance, ses voisins, sa densité d'atteignabilité locale, puis son LOF. Quel point est le plus anormal ?

Exercice 6.8 ★ — La constante $c(n)$ et le score d'isolement (section 6.3.2)

Pour $n = 16$ points, calculez $c(16)$, puis le score d'anomalie d'un point isolé en moyenne en 3 coupures, d'un autre isolé en 6 coupures, et d'un point dont la longueur moyenne vaut $c(16)$. Lesquels sont jugés anormaux (score supérieur à 0,5) ?

Exercice 6.9 ★★ — Contamination et seuil (section 6.3.4)

Sur 1 000 commandes, 8 sont frauduleuses. On fixe la contamination d'une forêt d'isolement à 2 %, ce qui déclenche 20 alertes ; parmi elles, 60 % des fraudes sont retrouvées. (a) Quelle est la précision ? (b) On fixe maintenant la contamination à 5 % : 50 alertes, 85 % des fraudes retrouvées. Calculez la précision. (c) Qu'est-ce qui, dans la vie réelle, devrait guider le choix de la contamination ?

Exercice 6.10 ★★★ — Un autoencodeur linéaire est une ACP (section 6.4.2)

Soit X un tableau centré de n lignes et p colonnes, de décomposition en valeurs singulières $X = U\Sigma V^\top$. (a) Montrez que l'approximation de rang q de X au sens des moindres carrés est $XV_q V_q^\top$, où V_q contient les q premières colonnes de V , et que l'erreur totale vaut $\sum_{j>q} \sigma_j^2$. (b) Montrez que l'erreur de reconstruction d'une ligne x_i vaut $\|x_i\|^2 - \|V_q^\top x_i\|^2$. (c) Vérifiez numériquement ces deux résultats sur un tableau simulé.

Exercice 6.11 ★★ — Pourquoi limiter le goulot ? (section 6.4.3)

On dispose de 1 000 points normaux de deux variables corrélées ($\rho = 0,9$) et de 10 anomalies placées à contre-courant de la corrélation. On entraîne deux autoencodeurs linéaires (activation identité) : un goulot d'un neurone, un goulot de deux neurones. (a) Que prévoyez-vous, **en théorie**, pour l'erreur de reconstruction moyenne des points normaux et des anomalies, dans chaque cas ? (b) Vérifiez par simulation, avec deux optimiseurs : le solveur par défaut (adam, par descente de gradient) et le solveur lbfgs (qui résout le problème à haute précision). Que constatez-vous ?

Exercice 6.12 ★★★ — Un budget, deux détecteurs (section 6.4.5)

Avec les scores des voisins et de la forêt d'isolement de l'application 6.6, comparez trois façons d'utiliser les 180 alertes du budget : (a) les 180 premières du détecteur des voisins ; (b) les 90 premières de chaque détecteur (sans doublon, en complétant au besoin) ; (c) les 180 premières de la moyenne de leurs rangs. Laquelle retrouve le plus de fraudes ? Expliquez ce que cela montre sur la complémentarité.

Corrigés

Corrigé 6.1

Détecteur A : $VP = 0$, $FP = 0$, $FN = 40$, $VN = 4960$. Exactitude $4960/5000 = 99,2\%$; précision indéfinie (aucune alerte) ; rappel 0. Détecteur B : $VP = 30$, $FP = 30$, $FN = 10$, $VN = 4930$. Exactitude $(30 + 4930)/5000 = 99,2\%$; précision $30/60 = 50\%$; rappel $30/40 = 75\%$; $F_1 = 2 \times 0,5 \times 0,75 / (0,5 + 0,75) = 0,6$. **Les deux détecteurs ont exactement la même exactitude (99,2 %)**, alors que A est inutile et que B retrouve les trois quarts des fraudes. L'exactitude ne mesure ici que la proportion de commandes normales.

Corrigé 6.2

- (a) Rangs : 1 (fraude), 2 (normale), 3 (fraude), 4, 5 (normales), 6 (fraude), 7, 8 (normales).
Après 1 alerte : 1 vraie fraude, précision $1/1 = 1$, rappel $1/3$. Après 3 : 2 fraudes, précision $2/3 \approx 0,667$, rappel $2/3$. Après 6 : 3 fraudes, précision $3/6 = 0,5$, rappel $3/3 = 1$.
- (b) Les précisions aux rangs des fraudes (1, 3 et 6) sont 1 , $2/3$ et $1/2$: $AP = (1 + 0,667 + 0,5)/3 \approx 0,722$.

```
print("AP à la main :", round(float(np.mean([1 / 1, 2 / 3, 3 / 6])), 4))
```

```
AP à la main : 0.7222
```

Corrigé 6.3

- (a) Par la formule de Bayes, avec $P(F) = 0,005$, $P(A | F) = 0,9$, $P(A | \bar{F}) = 0,02$:

$$P(F | A) = \frac{0,9 \times 0,005}{0,9 \times 0,005 + 0,02 \times 0,995} = \frac{0,0045}{0,0244} \approx 0,184.$$

Moins de deux alertes sur dix sont de vraies fraudes.

- (b) On veut $P(F | A) = 0,5$, c'est-à-dire autant de vraies que de fausses alertes : $0,0045 = 0,995x$, d'où $x \approx 0,0045$: un taux de fausses alertes d'environ **0,45 %**, soit quatre fois moins que le taux de départ.

```
p, rec, fpr = 0.005, 0.9, 0.02
print("précision :", round(rec * p / (rec * p + fpr * (1 - p)), 3), "| taux de fausses alertes pour 50 % :", round(rec * p / (1 - p), 4))
```

```
précision : 0.184 | taux de fausses alertes pour 50 % : 0.0045
```

Corrigé 6.4

- (a) Coût = $(100 - \text{trouvées}) \times 70 + 5n$:

Alertes	Fraudes trouvées	Manquées	Coût de manque	Coût de vérification	Total
100	40	60	4 200	500	4 700
200	60	40	2 800	1 000	3 800
300	70	30	2 100	1 500	3 600
400	75	25	1 750	2 000	3 750
500	78	22	1 540	2 500	4 040

Le minimum est à **300 alertes** (3 600 €).

- (b) Une alerte supplémentaire rapporte, en moyenne, $p \times 70$ € (où p est la probabilité qu'elle soit une vraie fraude) et coûte 5 € : elle vaut la peine si $p > 5/70 \approx 7,1$ %. Les tranches : de 200 à 300, 10 fraudes de plus pour 100 alertes, $p = 10\% > 7,1\%$: on continue ; de 300 à 400, 5 fraudes de plus, $p = 5\% < 7,1\%$: on s'arrête. L'optimum est bien à 300.

Corrigé 6.5

Moyenne : $263/9 \approx 29,2$; écart-type (avec $n - 1$) : $\approx 24,7$; $z = (95 - 29,2)/24,7 \approx 2,66$: **sous le seuil de 3**, la commande n'est pas signalée (masquage). Médiane : 21 ; écarts absolus à 21 : 1, 1, 0, 2, 2, 1, 0, 1, 74, rangés 0, 0, 1, 1, 1, 1, 2, 2, 74, de médiane **1** ; $MAD = 1$; $z^{\text{rob}} = (95 - 21)/(1,4826 \times 1) \approx 49,9$: la commande est signalée sans ambiguïté.

```
x = np.array([20, 22, 21, 19, 23, 20, 21, 22, 95.])
print("z classique :", round(float((95 - x.mean()) / x.std(ddof=1)), 2), "| z robuste :",
      round(float((95 - np.median(x)) / (1.4826 * np.median(np.abs(x - np.median(x))))), 1))
```

z classique : 2.66 | z robuste : 49.9

Corrigé 6.6

Avec $\rho = 0,6$: $\Sigma^{-1} = \frac{1}{1 - 0,36} \begin{pmatrix} 1 & -0,6 \\ -0,6 & 1 \end{pmatrix} = \frac{1}{0,64} \begin{pmatrix} 1 & -0,6 \\ -0,6 & 1 \end{pmatrix}$. Pour $x = (a, b)$: $d_M^2 = \frac{a^2 - 1,2ab + b^2}{0,64}$.

- $A = (1, 1) : \frac{1 - 1,2 + 1}{0,64} = \frac{0,8}{0,64} = 1,25$ (distance euclidienne² : 2) ;
- $B = (1, -1) : \frac{1 + 1,2 + 1}{0,64} = \frac{3,2}{0,64} = 5,0$ (euclidienne² : 2) ;
- $C = (2, 0) : \frac{4}{0,64} = 6,25$ (euclidienne² : 4).

Seul C dépasse 5,99. A et B sont à la même distance euclidienne, mais B va à contre-courant de la corrélation positive et sa distance de Mahalanobis est quatre fois supérieure à celle de A ; B reste pourtant sous le seuil, et C, simplement plus éloigné du centre, le franchit.

```
S = np.array([[1, 0.6], [0.6, 1]])
for p in [(1, 1), (1, -1), (2, 0)]:
    p = np.array(p); print(p, "euclidienne^2 =", float(p @ p), "| Mahalanobis^2 =", round(float(p @
    np.linalg.inv(S) @ p), 3))
```

```
[1 1] euclidienne^2 = 2.0 | Mahalanobis^2 = 1.25
[ 1 -1] euclidienne^2 = 2.0 | Mahalanobis^2 = 5.0
[2 0] euclidienne^2 = 4.0 | Mahalanobis^2 = 6.25
```

Corrigé 6.7

Distances entre points voisins : 1 ; 1,2 ; 1,3 ; 5,5.

Point	2 voisins	2-distance	densité lrd	LOF
0	1 ; 2,2	2,2	0,588	0,945
1	0 ; 2,2	1,2	0,571	0,988
2,2	1 ; 3,5	1,3	0,541	1,015

Point	2 voisins	2-distance	densité lrd	LOF
3,5	2,2 ; 1	2,5	0,526	1,056
9	3,5 ; 2,2	6,8	0,163	3,281

(Par exemple, pour le point 9 : distances d'atteignabilité à 3,5 et à 2,2 : $\max\{2,5; 5,5\} = 5,5$ et $\max\{1,3; 6,8\} = 6,8$, moyenne 6,15, d'où $\text{lrd} = 1/6,15 \approx 0,163$; les voisins ont des densités 0,526 et 0,541, et $\text{LOF} = \frac{(0,526 + 0,541)/2}{0,163} \approx 3,28$.) **Le point 9** est le plus anormal ; les quatre autres ont un LOF voisin de 1.

```
from sklearn.neighbors import LocalOutlierFactor
pts = np.array([0, 1, 2.2, 3.5, 9.0]).reshape(-1, 1)
print("LOF (scikit-learn) :",
      (-LocalOutlierFactor(n_neighbors=2).fit(pts).negative_outlier_factor_).round(3))
```

```
LOF (scikit-learn) : [0.945 0.988 1.015 1.056 3.281]
```

Corrigé 6.8

$c(16) = 2(\ln 15 + 0,5772) - \frac{2 \times 15}{16} = 2 \times 3,285 - 1,875 \approx 4,70$. Scores : $E[h] = 3 : 2^{-3/4,70} \approx 0,64$; $E[h] = 6 : 2^{-6/4,70} \approx 0,41$; $E[h] = c(16) : 2^{-1} = 0,5$. Seul le premier point (isolé en moyenne en 3 coupures) est jugé anormal (score $> 0,5$) ; le deuxième, enfoui plus profondément que la moyenne, est jugé très normal.

```
c16 = 2 * (np.log(15) + 0.5772156649) - 2 * 15 / 16
print(round(float(c16), 3), [round(float(2 ** (-e / c16)), 3) for e in (3, 6, c16)])
```

```
4.696 [0.642, 0.412, 0.5]
```

Corrigé 6.9

- 8 fraudes $\times 60\% = 4,8$ retrouvées sur 20 alertes : précision $4,8/20 = 24\%$.
- 8 fraudes $\times 85\% = 6,8$ retrouvées sur 50 alertes : précision $6,8/50 = 13,6\%$; le rappel monte de 60% à 85% , mais le prix est une précision divisée par près de deux.
- La contamination n'est pas une propriété des données (on ne connaît pas la vraie proportion de fraudes) : c'est un **choix opérationnel**. Il doit être guidé par le **budget d'alertes** que l'on peut traiter et par le **coût** d'une fraude manquée comparé au coût d'une vérification (section 6.1.6), idéalement évalué sur un petit jeu étiqueté.

Corrigé 6.10

- Par le théorème d'Eckart-Young (volume I, section 1.1.4), la meilleure approximation de rang $\leq q$ de X au sens de la norme de Frobenius est $X_q = U_q \Sigma_q V_q^T$, avec une erreur $\|X - X_q\|_F^2 = \sum_{j>q} \sigma_j^2$. Or $XV_q = U_q \Sigma_q$ (car $V^T V = I$), donc $X_q = XV_q V_q^T$. Un autoencodeur linéaire reconstruit X par XAB avec $A \in \mathbb{R}^{p \times q}$, $B \in \mathbb{R}^{q \times p}$: AB est de rang $\leq q$, donc XAB aussi ; la meilleure reconstruction possible est X_q , atteinte pour $A = V_q$ et $B = V_q^T$. L'autoencodeur linéaire optimal projette donc sur le sous-espace des q premières composantes principales : c'est l'ACP.
- Pour une ligne x_i , la reconstruction est $\hat{x}_i = V_q V_q^T x_i$, projection orthogonale de x_i sur le sous-espace engendré par V_q . Par le théorème de Pythagore, $\|x_i\|^2 = \|\hat{x}_i\|^2 + \|x_i - \hat{x}_i\|^2$ et $\|\hat{x}_i\|^2 = \|V_q^T x_i\|^2$ (car les colonnes de V_q sont orthonormées), d'où $\|x_i - \hat{x}_i\|^2 = \|x_i\|^2 - \|V_q^T x_i\|^2$.

(c) Vérification :

```

rng = np.random.default_rng(0)
A = rng.normal(size=(5, 5)); M = rng.normal(size=(300, 5)) @ A # un tableau à 5
↳ colonnes corrélées
M = M - M.mean(axis=0); q = 2
U, sig, Vt = np.linalg.svd(M, full_matrices=False); Vq = Vt[:q].T
erreur_totale = ((M - M @ Vq @ Vq.T) ** 2).sum()
print("erreur totale :", round(float(erreur_totale), 4), "| somme des sigma_j^2 (j > q) :",
↳ round(float((sig[q:] ** 2).sum()), 4))
par_ligne = (M ** 2).sum(axis=1) - ((M @ Vq) ** 2).sum(axis=1)
print("écart maximal sur l'erreur par ligne :", float(np.abs(par_ligne - ((M - M @ Vq @ Vq.T) **
↳ 2).sum(axis=1)).max()))

```

```

erreur totale : 1162.9741 | somme des sigma_j^2 (j > q) : 1162.9741
écart maximal sur l'erreur par ligne : 1.931788062847724e-14

```

Corrigé 6.11

- (a) Avec **un neurone** de goulot, le réseau ne peut représenter qu'une droite, celle de la corrélation : les points normaux, proches de cette droite, ont une petite erreur ; les anomalies, à contre-courant, en sont éloignées et ont une grande erreur. Avec **deux neurones** pour deux variables, le goulot n'est plus un goulot : le réseau peut représenter l'identité et reconstruire **tous** les points, anomalies comprises. En théorie, les deux erreurs sont alors **presque nulles** et les anomalies sont invisibles.

- (b) Simulation :

```

from sklearn.neural_network import MLPRegressor
rng = np.random.default_rng(0)
normaux = rng.multivariate_normal([0, 0], [[1, 0.9], [0.9, 1]], 1000)
anomalies = np.column_stack([rng.uniform(1.5, 2.5, 10), -rng.uniform(1.5, 2.5, 10)]) # à
↳ contre-courant : x et y de signes opposés
for solveur in ("adam", "lbfgs"):
    for q in (1, 2):
        ae = MLPRegressor(hidden_layer_sizes=(q,), activation="identity", max_iter=2000,
↳ random_state=0, solver=solveur).fit(normaux, normaux)
        err = lambda D: ((ae.predict(D) - D) ** 2).sum(axis=1).mean()
        print(f"{solveur:5s} goulot de {q} : erreur des normaux {err(normaux):.2e}, erreur des
↳ anomalies {err(anomalies):.2e}")

```

```

adam goulot de 1 : erreur des normaux 1.01e-01, erreur des anomalies 7.58e+00
adam goulot de 2 : erreur des normaux 1.59e-02, erreur des anomalies 1.03e+00
lbfgs goulot de 1 : erreur des normaux 9.61e-02, erreur des anomalies 7.30e+00
lbfgs goulot de 2 : erreur des normaux 1.06e-09, erreur des anomalies 2.04e-08

```

Lecture. Avec un neurone, l'erreur des anomalies est environ 75 fois celle des points normaux, quel que soit l'optimiseur (7,6 contre 0,10 avec adam ; 7,3 contre 0,096 avec lbfgs) : le réseau est contraint de projeter sur une droite. Avec deux neurones et le solveur lbfgs, le réseau apprend l'identité : **les deux erreurs sont presque nulles** (de l'ordre de 10^{-9} et 10^{-8}), et l'anomalie est parfaitement reconstruite, donc invisible. Avec adam, en revanche, l'optimisation s'arrête avant d'avoir appris l'identité : l'erreur des anomalies reste élevée (1,03 contre 0,016 pour les normaux) et le détecteur *fonctionne encore*. Une explication plausible, que nous n'avons pas testée, est que la direction de faible variance (0,1 sur un total de 2) fournit un gradient faible et est apprise lentement. Ce qui est établi, c'est que **la capacité effective d'un réseau dépend de l'optimisation autant que de l'architecture** : un goulot trop large peut sembler sans danger tant que l'entraînement est insuffisant, et cesser de l'être avec un meilleur optimiseur. C'est une raison de plus de régler sur un jeu de validation.

Corrigé 6.12

On réutilise les scores `s_test` de l'application 6.6 :

```
voisins_s, iso_s = s_test["voisins"], s_test["isolement"]
def trouvees(indices): return int(y_test[list(indices)].sum())
a = np.argsort(-voisins_s)[:budget]
moitie = budget // 2
ens = list(np.argsort(-voisins_s)[:moitie])
for i in np.argsort(-iso_s): # on complète avec la forêt, sans doublon
    if len(ens) >= budget: break
    if i not in ens: ens.append(i)
c = np.argsort(-(rankdata(voisins_s) + rankdata(iso_s))[:budget])
print("(a) 180 premières des voisins          :", trouvees(a), "fraudes sur", int(y_test.sum()))
print("(b) 90 + 90 sans doublon                :", trouvees(ens))
print("(c) 180 premières du rang moyen          :", trouvees(c))
```

```
(a) 180 premières des voisins          : 66 fraudes sur 146
(b) 90 + 90 sans doublon                : 59
(c) 180 premières du rang moyen          : 57
```

Lecture. Les 180 premières alertes des voisins retrouvent **66** fraudes sur 146, la combinaison 90 + 90 en retrouve 59 et la moyenne des rangs 57 : **aucune des combinaisons ne fait mieux que le meilleur détecteur seul**. La raison est double. D'une part, la forêt d'isolement est moins bonne que les voisins sur ces données (AP de 0,33 contre 0,46) : lui consacrer la moitié du budget retire des alertes de meilleure qualité. D'autre part, les deux détecteurs se recouvrent fortement (133 alertes communes sur 180, indice de Jaccard de 0,59, application 6.6) : la forêt d'isolement apporte peu de commandes que les voisins n'avaient pas déjà dans leurs 180 premières, et la combinaison (b) remplace les rangs 91 à 180 des voisins par des alertes de la forêt, de moindre qualité. Combiner demande des détecteurs **comparables en qualité** et **complémentaires** (peu de recouvrement).

Chapitre 7 : ⊕ Systèmes de recommandation — exercices et applications

Ce chapitre du cahier accompagne le chapitre 7 du livre. Les **applications** reprennent, pas à pas et avec le code, ce que le livre a seulement résumé : la matrice d'interactions, les voisins, l'ALS écrit à la main, la factorisation sur les notes, les métriques, le démarrage à froid, le découpage temporel, la boucle de rétroaction et les compromis de diversité. Les **exercices** se travaillent d'abord à la main, puis se vérifient par le calcul. Données : `interactions.csv` et `produits_ml.csv` (boutique simulée). Prérequis : le chapitre 7 du livre et la section 1.1 sur la séparation des données.

Préparation

Une seule cellule charge les bibliothèques et les fichiers ; les applications qui suivent s'en servent. Le **protocole d'évaluation** (découpage des achats en entraînement, validation et test, calcul des métriques) et les simulateurs sont fournis par le script `build/outils_ch07.py`, comme dans le livre.

Fonction	Rôle
<code>charger()</code>	lit les fichiers et retourne les interactions, les produits et la matrice binaire clients × produits
<code>decouper(R, seed)</code>	retire ~25 % des achats de chaque client (test), puis ~20 % du reste (validation)
<code>evaluer(S, R_connu, cible, users, k)</code>	calcule précision, rappel, AP, NDCG et succès @k pour chaque client
<code>resume(df), ic_bootstrap(v)</code>	moyennes, intervalle de confiance par bootstrap
<code>scores_*</code>	méthodes du livre (popularité, contenu, voisins, SVD, ALS), pour comparaison
<code>monde_en_derive(), boucle_retroaction()</code>	simulateurs des sections 7.4.3 et 7.4.6

```
import sys
sys.path.insert(0, "build")
import numpy as np
import pandas as pd
import scipy.sparse as sp
import matplotlib
```

```

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from outils_ch07 import *

inter, prod, R = charger()                                # R : matrice binaire clients x produits (creuse)
d = decouper(R, seed=7)                                   # mêmes jeux qu'au livre
Rt, Rv, users = d["R_train"], d["val"], d["users"]
print(R.shape, int(R.nnz), "| clients évalués :", len(users), "| achats entraînement/validation/test
↪ :", Rt.nnz, int(Rv.nnz), int(d["test"].nnz))

def ndcg_moyen(S):                                         # raccourci : NDCG@10 moyen sur la validation
    return evaluer(S, Rt, Rv, users)["ndcg"].mean()

```

```
(3000, 150) 27687 | clients évalués : 2365 | achats entraînement/validation/test : 17177 3987 6523
```

Applications

Application 7.1 — La matrice d'interactions, la popularité et le contenu à la main

Sections du livre : 7.1.3 à 7.1.6. **Objectif :** construire la matrice à partir du fichier d'achats, mesurer la longue traîne, écrire soi-même la popularité et le filtrage par contenu, et voir ce qui arrive quand on oublie de mettre le prix à la même échelle que les catégories.

Étape 1 — De la table « un achat par ligne » à la matrice.

```

M = inter.groupby(["id_client", "id_produit"]).size().unstack(fill_value=0)
M = M.reindex(index=range(1, 3001), columns=range(1, 151), fill_value=0)    # clients sans achat
↪ inclus
print("identique à la matrice creuse :", np.array_equal((M.values > 0).astype(float), R.toarray()))
print("cases remplies :", int(R.nnz), "sur", R.shape[0] * R.shape[1], "| densité :", round(R.nnz /
↪ (R.shape[0] * R.shape[1]), 4))
par_produit = np.sort(np.asarray(R.sum(axis=0)).ravel())[::-1]
print("part des 10 produits les plus achetés :", round(par_produit[:10].sum() / R.nnz, 3), "| des 50
↪ premiers :", round(par_produit[:50].sum() / R.nnz, 3))

```

```

identique à la matrice creuse : True
cases remplies : 27687 sur 450000 | densité : 0.0615
part des 10 produits les plus achetés : 0.211 | des 50 premiers : 0.629

```

Lecture. La table « un achat par ligne » et la matrice creuse contiennent exactement la même information. Les 10 produits les plus achetés concentrent 21,1 % des achats et les 50 premiers 62,9 % : c'est la longue traîne du livre, vue par un autre bout.

Étape 2 — La popularité écrite à la main. Un simple comptage des achats d'entraînement.

```

achats = np.asarray(Rt.sum(axis=0)).ravel()
top = np.argsort(-achats)[:10]
print("10 produits les plus achetés (id) :", top + 1, "| acheteurs :", achats[top].astype(int))
print("catégories :", "".join(prod.categorie.values[top]))
r_pop = evaluer(scores_popularite(Rt), Rt, Rv, users)
print("rappel@10 :", round(r_pop.rappel.mean(), 4), "| NDCG@10 :", round(r_pop.ndcg.mean(), 4))
print("liste du client", users[0] + 1, ":", np.array(r_pop.recommandes.iloc[0]) + 1)

```

```

10 produits les plus achetés (id) : [ 29  48 121  69 116  50  15 137  11  44] | acheteurs : [652 509
↪ 396 390 339 312 290 287 277 271]
catégories : CBDCABADAC
rappel@10 : 0.2409 | NDCG@10 : 0.1431
liste du client 1 : [121  69 116  15 137  11  44  10  23  60]

```

Lecture. Les dix produits les plus achetés comptent de 271 à 652 acheteurs et couvrent les quatre catégories. La liste du premier client évalué ne contient pas les produits 29, 48 et 50 : il les a déjà achetés, et l'on ne recommande que ce qu'il ne connaît pas encore. Le `rappel@10` (24,1 %) et le `NDCG@10` (0,143) sont ceux du livre.

Étape 3 — Le contenu, avec et sans mise à l'échelle du prix.

```
def contenu(R_, prix_mis_a_l_echelle=True, avec_prix=True):
    F = pd.get_dummies(prod.categorie, dtype=float).to_numpy()
    if avec_prix:
        p = np.log(prod.prix.to_numpy()) if prix_mis_a_l_echelle else prod.prix.to_numpy() #
        # sinon : prix brut en euros
        p = (p - p.mean()) / p.std() if prix_mis_a_l_echelle else p
        F = np.hstack([F, p[:, None]])
        n = np.asarray(R_.sum(axis=1)).ravel(); n[n == 0] = 1
        P = (R_ @ F) / n[:, None] # profil : moyenne des produits
        # achetés
        P = P / np.maximum(np.linalg.norm(P, axis=1, keepdims=True), 1e-12)
        return P @ (F / np.linalg.norm(F, axis=1, keepdims=True)).T # cosinus profil / produit

print("même scores que le livre :", np.allclose(contenu(Rt), scores_contenu(Rt, prod)))
print("prix moyen du catalogue :", round(prod.prix.mean(), 2), "€ | corrélation prix-popularité :",
      round(np.corrcoef(prod.prix, achats)[0, 1], 3))
for nom, S in (("catégorie seule", contenu(Rt, avec_prix=False)), ("catégorie + prix standardisé",
      contenu(Rt)), ("catégorie + prix brut (en euros)", contenu(Rt, prix_mis_a_l_echelle=False))):
    r = evaluer(S, Rt, Rv, users)
    print(f"{nom:34s} NDCG@10 = {r.ndcg.mean():.4f} | prix moyen des produits recommandés :
          {prod.prix.values[np.vstack(r.recommandes.values)].mean():.1f} €")
```

```
même scores que le livre : True
prix moyen du catalogue : 30.44 € | corrélation prix-popularité : 0.085
catégorie seule                NDCG@10 = 0.0538 | prix moyen des produits recommandés : 29.3 €
catégorie + prix standardisé    NDCG@10 = 0.0588 | prix moyen des produits recommandés : 31.4 €
catégorie + prix brut (en euros) NDCG@10 = 0.0658 | prix moyen des produits recommandés : 52.8 €
```

Lecture. Les trois variantes ont des `NDCG` proches (0,054 ; 0,059 ; 0,066), très loin de la popularité (0,143). Le résultat surprenant est que la variante « mal mise à l'échelle », avec le prix brut en euros, obtient le **meilleur** score. L'explication se lit dans la dernière colonne : dominé par le prix, le cosinus recommande surtout des produits **chers** (52,8 € en moyenne, contre 30,4 € pour le catalogue), et comme les produits chers sont un tout petit peu plus achetés que les autres (corrélation prix-popularité de 0,085), le score en profite. Il ne s'agit donc pas d'une meilleure compréhension des goûts mais d'un **effet de popularité indirect**. Deux leçons : on ne juge jamais un modèle de contenu à son seul score, on regarde aussi **ce qu'il recommande** ; et ces écarts de l'ordre d'un à deux centièmes, donnés sans intervalle de confiance, se lisent avec prudence.

Pour aller plus loin. Remplacez la moyenne des vecteurs produits par une moyenne pondérée par `nb_achats` (colonne de `interactions.csv`) : le profil change-t-il beaucoup ?

Application 7.2 — Les voisins de produits et de clients écrits à la main

Sections du livre : 7.2.2 à 7.2.5. **Objectif :** retrouver la similarité cosinus à partir des co-achats, écrire le score de voisinage avec rétrécissement, et explorer une variante de la similarité.

Étape 1 — Co-achats et cosinus.

```
co = (Rt.T @ Rt).toarray() # co[i, j] : nombre de clients ayant acheté i et j
n = np.diag(co).copy()
S = co / np.sqrt(np.outer(n, n)); np.fill_diagonal(S, 0)
from sklearn.metrics.pairwise import cosine_similarity
S2 = cosine_similarity(Rt.T); np.fill_diagonal(S2, 0)
i, j = np.unravel_index(S.argmax(), S.shape)
```

```
print("identique à scikit-learn :", np.allclose(S, S2), "| paire la plus proche :", i + 1, j + 1,
      ↪ "cosinus", round(S.max(), 3), "co-achats", int(co[i, j]))
iu = np.triu_indices(150, 1)
print("co-achats par paire : médiane", int(np.median(co[iu])), "| maximum", int(co[iu].max()), "|
      ↪ paires à moins de 5 co-achats :", round((co[iu] < 5).mean(), 3))
```

```
identique à scikit-learn : True | paire la plus proche : 29 48 sinus 0.231 co-achats 133
co-achats par paire : médiane 3 | maximum 133 | paires à moins de 5 co-achats : 0.646
```

Lecture. Notre cosinus est identique à celui de scikit-learn. La paire de produits la plus proche est (29, 48), les deux produits les plus achetés (133 co-achats), pour un cosinus de seulement 0,231. Les co-achats sont rares : médiane de 3 par paire, et 64,6 % des paires en ont moins de 5. C'est ce qui rend les similarités individuellement bruitées, et justifie le rétrécissement.

Étape 2 — Le score de voisinage avec rétrécissement.

```
def voisins_produits(R_, k=149, lam=0.0):
    co = (R_.T @ R_).toarray(); n = np.diag(co)
    S = co / np.sqrt(np.outer(n, n))
    if lam > 0:
        S = S * co / (co + lam) # rétrécissement n_ij / (n_ij + lambda)
    np.fill_diagonal(S, 0)
    if k < 149:
        seuil = -np.sort(-S, axis=1)[:k - 1][:, None]; S = np.where(S >= seuil, S, 0)
    return np.asarray(R_ @ S)

print("même scores que le livre :", np.allclose(voisins_produits(Rt, 20, 10),
      ↪ scores_voisins_articles(Rt, 20, 10)))
grille = pd.DataFrame({f"k = {k}": {f"λ = {lam}": ndcg_moyen(voisins_produits(Rt, k, lam)) for lam in
      ↪ (0, 5, 20, 100)} for k in (5, 10, 50, 149)})
print(grille.round(4).to_string())
```

```
même scores que le livre : True
      k = 5  k = 10  k = 50  k = 149
λ = 0    0.1406  0.1526  0.1564  0.1584
λ = 5    0.1478  0.1534  0.1560  0.1576
λ = 20   0.1478  0.1534  0.1567  0.1570
λ = 100  0.1477  0.1508  0.1547  0.1547
```

Lecture. Avec 5 voisins, un rétrécissement de $\lambda = 5$ fait passer le NDCG de 0,141 à 0,148 ; avec tous les voisins, il n'apporte rien (0,158 sans rétrécissement, 0,157 avec $\lambda = 5$), et un rétrécissement fort ($\lambda = 100$) fait même perdre un peu (0,155). Un λ de 5 à 20 est donc une assurance peu coûteuse : elle aide quand le voisinage est étroit et ne coûte presque rien quand il est large.

Étape 3 — Une variante : le cosinus asymétrique. On remplace $\sqrt{n_i n_j}$ par $n_i^\alpha n_j^{1-\alpha}$; $\alpha = 0,5$ redonne le cosinus.

```
def voisins_alpha(R_, alpha):
    co = (R_.T @ R_).toarray(); n = np.diag(co)
    S = co / np.outer(n ** alpha, n ** (1 - alpha)); np.fill_diagonal(S, 0)
    return np.asarray(R_ @ S)

print({a: round(float(ndcg_moyen(voisins_alpha(Rt, a))), 4) for a in (0.0, 0.25, 0.5, 0.75, 1.0)})
print("voisins de clients, k = 30, 100, 300, 1000 :",
      ↪ [round(float(ndcg_moyen(scores_voisins_clients(Rt, k))), 4) for k in (30, 100, 300, 1000)])

{0.0: 0.0333, 0.25: 0.115, 0.5: 0.1584, 0.75: 0.1604, 1.0: 0.1579}
voisins de clients, k = 30, 100, 300, 1000 : [0.1337, 0.1587, 0.1646, 0.1567]
```

Lecture. Le cosinus asymétrique avec $\alpha = 0,75$ obtient 0,160 contre 0,158 pour le cosinus ($\alpha = 0,5$) : un écart que la validation ne permet pas de distinguer du bruit. En revanche, les valeurs

extrêmes sont catastrophiques ($\alpha = 0 : 0,033$), car on ne normalise plus par la popularité des produits. Pour les voisins de clients, on retrouve les valeurs du livre : 0,134 ($k = 30$), 0,159 (100), 0,165 (300) et 0,157 (1 000).

Application 7.3 — L'ALS implicite écrit à la main

Sections du livre : 7.3.3 à 7.3.7. **Objectif :** programmer les moindres carrés alternés avec confiance, vérifier que l'objectif ne fait que baisser, puis explorer l'effet de la confiance α , du nombre d'itérations et regarder les facteurs appris.

Étape 1 — L'étape de mise à jour, puis la boucle.

```
def pas_als(A, F, lam=100.0, alpha=8.0):
    """Facteurs des lignes de A (creuse, binaire), les facteurs F de l'autre côté étant fixés."""
    k = F.shape[1]; FtF = F.T @ F + lam * np.eye(k)
    X = np.zeros((A.shape[0], k))
    for i in range(A.shape[0]):
        idx = A.indices[A.indptr[i]:A.indptr[i + 1]] # produits achetés par la ligne i
        if len(idx):
            Fi = F[idx]
            X[i] = np.linalg.solve(FtF + alpha * Fi.T @ Fi, (1 + alpha) * Fi.sum(axis=0))
    return X

def objectif(U, V, A, lam=100.0, alpha=8.0):
    X = A.toarray(); C = 1 + alpha * X
    return (C * (X - U @ V.T) ** 2).sum() + lam * ((U ** 2).sum() + (V ** 2).sum())

rng = np.random.default_rng(0); k = 6
U = 0.1 * rng.standard_normal((Rt.shape[0], k)); V = 0.1 * rng.standard_normal((Rt.shape[1], k))
Rtt = Rt.T.tocsr(); suivi = []
for it in range(10):
    U = pas_als(Rtt, V); V = pas_als(Rtt, U)
    suivi.append(objectif(U, V, Rt))
print("objectif après chaque itération :", np.round(suivi, 0))
print("il ne fait que baisser :", bool(np.all(np.diff(suivi) <= 1e-6)))
print("mêmes scores que le livre :", np.allclose(U @ V.T, scores_als(Rt, k=6, lam=100.0, alpha=8.0,
    ↪ n_iter=10, seed=0)))
```

```
objectif après chaque itération : [153830. 130128. 124731. 124472. 124308. 124212. 124156. 124122.
    ↪ 124102.
    124092.]
il ne fait que baisser : True
mêmes scores que le livre : True
```

Lecture. L'objectif passe de 153 830 à 124 092 en dix itérations et ne fait **que baisser**, comme le garantit la méthode ; les trois premières itérations font l'essentiel du travail (124 731 à la troisième). Nos scores sont identiques à ceux du livre.

Étape 2 — La confiance α et le nombre d'itérations.

```
print("confiance  $\alpha$  :", {a: round(float(ndcg_moyen(scores_als(Rt, k=6, lam=100.0, alpha=a,
    ↪ n_iter=10))), 4) for a in (1, 4, 8, 16, 32)})
print("itérations :", {t: round(float(ndcg_moyen(scores_als(Rt, k=6, lam=100.0, alpha=8.0,
    ↪ n_iter=t))), 4) for t in (1, 2, 3, 5, 10, 20)})
```

```
confiance  $\alpha$  : {1: 0.1397, 4: 0.1402, 8: 0.1681, 16: 0.1636, 32: 0.1529}
itérations : {1: 0.0991, 2: 0.1431, 3: 0.1494, 5: 0.1575, 10: 0.1681, 20: 0.1685}
```

Lecture. La confiance α interagit avec la régularisation. Avec $\lambda = 100$ fixé, $\alpha = 8$ est le meilleur (0,168) ; avec $\alpha = 1$ ou 4 la régularisation écrase les facteurs et le NDCG retombe au niveau de la popularité (0,140) ; avec $\alpha = 32$, le modèle accorde trop de poids aux achats et surapprend

(0,153). Côté itérations, une seule ne suffit pas (0,099), deux donnent 0,143, cinq 0,158, dix 0,168, et vingt n'apportent presque plus rien (0,1685) : la convergence est atteinte vers dix.

Étape 3 — Les voisins d'un produit dans l'espace des facteurs.

```
Vf = V / np.linalg.norm(V, axis=1, keepdims=True)
sim_lat = Vf @ Vf.T; np.fill_diagonal(sim_lat, -1)
voisins = np.argsort(-sim_lat[0])[5:]
print("produit 1, catégorie", prod.categorie[0], "| ses 5 voisins :", voisins + 1, "catégories",
      ↪ "".join(prod.categorie.values[voisins]))
meme = np.mean([(prod.categorie.values[np.argsort(-sim_lat[i])[5:]] == prod.categorie[i]).mean() for
      ↪ i in range(150)])
print("part des 5 plus proches voisins de la même catégorie, en moyenne :", round(meme, 3), "| attendu
      ↪ au hasard :", round(((prod.categorie.value_counts() / 150) ** 2).sum(), 3))
```

```
produit 1, catégorie A | ses 5 voisins : [ 5 103 31 43 21] catégories ABBBB
part des 5 plus proches voisins de la même catégorie, en moyenne : 0.86 | attendu au hasard : 0.258
```

Lecture. Dans l'espace complet des six facteurs, en moyenne 86 % des cinq plus proches voisins d'un produit sont de sa catégorie, contre 26 % attendus au hasard : les facteurs ont retrouvé l'organisation du catalogue sans qu'on la leur donne. Le produit 1 est une exception instructive (un seul de ses cinq voisins est de la catégorie A) : les facteurs reflètent les **achats**, pas les étiquettes, et un produit peut très bien être acheté par le même public que ceux d'une autre catégorie.

Pour aller plus loin. Le livre a traité les achats comme un signal binaire. Remplacez la confiance $1 + \alpha R_{ui}$ par $1 + \alpha \ln(1 + n_{ui})$, où n_{ui} est la colonne nb_achats du fichier interactions.csv (un tiers des achats concernent plusieurs exemplaires) : la quantité achetée améliore-t-elle le NDCG ? Il faut pour cela construire une matrice de confiance creuse et adapter pas_als.

Application 7.4 — La factorisation sur les notes explicites, par SGD

Sections du livre : 7.3.2 et 7.3.3. **Objectif :** prédire les **notes** (un quart des achats en a une) avec un modèle à biais et facteurs, appris par descente de gradient stochastique, et le comparer à des références. Ici l'objectif n'est plus un classement mais une erreur de prédiction (RMSE).

Étape 1 — Jeu d'apprentissage, jeu de test, références.

```
notes = inter.dropna(subset=["note"])[["id_client", "id_produit", "note"]].reset_index(drop=True)
masque = np.random.default_rng(4).random(len(notes)) < 0.8
tr, te = notes[masque], notes[~masque]
mu = tr.note.mean()
rmse = lambda pred: float(np.sqrt(np.mean((te.note.values - pred) ** 2)))
bi = tr.assign(e=tr.note - mu).groupby("id_produit").e.agg(lambda s: s.sum() / (len(s) + 10))
    ↪ # biais produit rétréci
bu = tr.assign(e=tr.note - mu - tr.id_produit.map(bi)).groupby("id_client").e.agg(lambda s: s.sum() /
    ↪ (len(s) + 10))
pred_biais = mu + te.id_produit.map(bi).fillna(0).values + te.id_client.map(bu).fillna(0).values
print(len(notes), "notes dont", len(te), "en test | moyenne", round(mu, 3))
print("RMSE moyenne globale :", round(rmse(mu), 3), "| modèle à biais :", round(rmse(pred_biais), 3))
```

```
6883 notes dont 1384 en test | moyenne 3.996
RMSE moyenne globale : 1.023 | modèle à biais : 0.97
```

Lecture. On dispose de 6 883 notes, dont 1 384 gardées pour le test. Prédire la moyenne globale (3,996) donne une RMSE de 1,023 ; le simple modèle à biais (produit et client, rétrécis) la ramène à 0,970, soit 5 % de moins.

Étape 2 — SGD avec biais et facteurs.

```
def sgd_mf(k=8, lam=0.05, gamma=0.01, epochs=30, seed=0):
    rng = np.random.default_rng(seed)
    P = 0.1 * rng.standard_normal((3001, k)); Q = 0.1 * rng.standard_normal((151, k)); bu_ =
    ↪ np.zeros(3001); bi_ = np.zeros(151)
    u, i, r = tr.id_client.values, tr.id_produit.values, tr.note.values
    historique = []
    for ep in range(epochs):
        for t in rng.permutation(len(r)):
            a, b = u[t], i[t]
            e = r[t] - (mu + bu_[a] + bi_[b] + P[a] @ Q[b])
            bu_[a] += gamma * (e - lam * bu_[a]); bi_[b] += gamma * (e - lam * bi_[b])
            P[a], Q[b] = P[a] + gamma * (e * Q[b] - lam * P[a]), Q[b] + gamma * (e * P[a] - lam * Q[b])
            pred = np.clip(mu + bu_[te.id_client.values] + bi_[te.id_produit.values] +
            ↪ (P[te.id_client.values] * Q[te.id_produit.values]).sum(axis=1), 1, 5)
            historique.append(rmse(pred))
    return historique

h = sgd_mf()
print("RMSE de test après 1, 5, 10, 20, 30 époques :", [round(h[e - 1], 3) for e in (1, 5, 10, 20,
    ↪ 30)])
```

```
RMSE de test après 1, 5, 10, 20, 30 époques : [0.998, 0.977, 0.971, 0.972, 0.982]
```

Lecture. La RMSE de test descend de 0,998 (une époque) à 0,971 (dix époques), puis **remonte** (0,972 à vingt, 0,982 à trente) : le modèle commence à apprendre le bruit des notes d'entraînement. C'est le surapprentissage, visible sur une courbe, et la raison pour laquelle on arrête l'apprentissage sur la validation (arrêt précoce).

Étape 3 — Régularisation.

```
for lam in (0.0, 0.05, 0.2, 0.5):
    h = sgd_mf(lam=lam, epochs=25)
    print(f"λ = {lam:4f}: meilleure RMSE de test {min(h):.3f} (époque {int(np.argmin(h)) + 1}) | à
    ↪ l'époque 25 : {h[-1]:.3f}")
```

```
λ = 0.0: meilleure RMSE de test 0.971 (époque 12) | à l'époque 25 : 0.984
λ = 0.05: meilleure RMSE de test 0.970 (époque 13) | à l'époque 25 : 0.976
λ = 0.2: meilleure RMSE de test 0.968 (époque 16) | à l'époque 25 : 0.971
λ = 0.5: meilleure RMSE de test 0.970 (époque 16) | à l'époque 25 : 0.971
```

Lecture. Sans régularisation, la meilleure RMSE est de 0,971 (époque 12) mais elle se dégrade ensuite (0,984 à l'époque 25) ; avec $\lambda = 0,2$, elle atteint 0,968 et **reste** à 0,971. La régularisation stabilise l'apprentissage. Mais le résultat principal est **négatif** : la meilleure factorisation (0,968) est à égalité avec le modèle à biais seul (0,970). Avec environ 5 500 notes d'entraînement pour 3 000 clients, les facteurs n'apportent rien de mesurable sur les biais. Ce n'est pas un échec de la méthode : sur les **achats**, où l'on dispose de 27 687 observations contre 6 883 notes (plus de quatre fois plus), les facteurs aident nettement.

Application 7.5 — Les métriques et leurs intervalles de confiance, écrits à la main

Sections du livre : 7.4.1 et 7.4.2. **Objectif** : écrire les métriques de classement pour une liste, vérifier le résultat sur l'exemple du livre, puis comparer deux méthodes sur le **jeu de test** avec un intervalle de confiance apparié.

Étape 1 — Les métriques pour une liste.

```
def metriques_liste(recommandes, pertinents, k=10):
    hits = np.isin(recommandes[:k], list(pertinents)).astype(float)
    rangs = np.arange(1, k + 1)
    n_rel = len(pertinents)
```

```

ap = (hits * np.cumsum(hits) / rangs).sum() / min(n_rel, k)
dcg = (hits / np.log2(rangs + 1)).sum()
idcg = (1 / np.log2(np.arange(1, min(n_rel, k) + 1) + 1)).sum()
return hits.sum() / k, hits.sum() / n_rel, ap, dcg / idcg

print("exemple du livre (précision, rappel, AP, NDCG) :", np.round(metriques_liste(np.array([7, 3, 9,
↪ 1, 4]), {3, 1, 12}, k=5), 4))
r = evaluer(scores_popularite(Rt), Rt, Rv, users)
cibles = [set(Rv.indices[Rv.indptr[u]:Rv.indptr[u + 1]]) for u in users[:200]]
mm = np.mean([metriques_liste(np.array(l), c) for l, c in zip(r.recommandes.iloc[:200], cibles)],
↪ axis=0)
print("moyenne de nos fonctions sur 200 clients :", mm.round(4))
print("moyenne de evaluer() sur les mêmes      :", r.iloc[:200][["precision", "rappel", "ap",
↪ "ndcg"]].mean().values.round(4))

```

```

exemple du livre (précision, rappel, AP, NDCG) : [0.4    0.6667 0.3333 0.4982]
moyenne de nos fonctions sur 200 clients : [0.0395 0.2492 0.0934 0.1434]
moyenne de evaluer() sur les mêmes      : [0.0395 0.2492 0.0934 0.1434]

```

Lecture. Nos fonctions redonnent l'exemple du livre (précision 0,4 ; rappel 0,667 ; AP 0,333 ; NDCG 0,498) et la même moyenne que `evaluer()` sur 200 clients (précision 0,0395, rappel 0,2492, AP 0,0934, NDCG 0,1434).

Étape 2 — Comparaison sur le jeu de test avec un intervalle de confiance apparié. Les modèles sont réentraînés sur l'entraînement et la validation, avec les hyperparamètres choisis en validation.

```

Rtv, T = d["R_trainval"], d["test"]
res_als = evaluer(scores_als(Rtv, k=6, lam=100.0, alpha=8.0, n_iter=10), Rtv, T, users)
res_cli = evaluer(scores_voisins_clients(Rtv, 300), Rtv, T, users)
res_pop = evaluer(scores_popularite(Rtv), Rtv, T, users)
def ic_apparie(a, b, col="ndcg"):
    delta = a[col].values - b[col].values
    return round(delta.mean(), 4), np.round(ic_bootstrap(delta), 4)
print("ALS - voisins de clients :", [round(float(v), 4) for v in (ic_apparie(res_als, res_cli)[0],
↪ *ic_apparie(res_als, res_cli)[1])], "(écart, borne basse, borne haute)")
print("ALS - popularité          :", [round(float(v), 4) for v in (ic_apparie(res_als, res_pop)[0],
↪ *ic_apparie(res_als, res_pop)[1])])
print("rappel@10 : ALS", round(res_als.rappel.mean(), 3), "| voisins de clients",
↪ round(res_cli.rappel.mean(), 3), "| popularité", round(res_pop.rappel.mean(), 3))

```

```

ALS - voisins de clients : [0.0068, 0.0022, 0.011] (écart, borne basse, borne haute)
ALS - popularité          : [0.042, 0.0357, 0.0481]
rappel@10 : ALS 0.295 | voisins de clients 0.288 | popularité 0.244

```

Lecture. Sur le jeu de test, l'ALS dépasse la popularité de 0,042 de NDCG (intervalle [0,036 ; 0,048]) et les voisins de clients de 0,0068 (intervalle [0,002 ; 0,011], qui exclut zéro de justesse), comme dans le livre.

Étape 3 — Le choix de k pour l'évaluation.

```

S_als, S_pop = scores_als(Rtv, k=6, lam=100.0, alpha=8.0, n_iter=10), scores_popularite(Rtv)
for k_eval in (1, 3, 5, 10, 20):
    a = resume(evaluer(S_als, Rtv, T, users, k=k_eval)); p = resume(evaluer(S_pop, Rtv, T, users,
↪ k=k_eval))
    print(f"@{k_eval}<3d> rappel ALS {a.rappel:.3f} vs popularité {p.rappel:.3f} | NDCG ALS
↪ {a.ndcg:.3f} vs popularité {p.ndcg:.3f}")

```

```

@1  rappel ALS 0.060 vs popularité 0.047 | NDCG ALS 0.176 vs popularité 0.142
@3  rappel ALS 0.132 vs popularité 0.109 | NDCG ALS 0.155 vs popularité 0.124
@5  rappel ALS 0.186 vs popularité 0.153 | NDCG ALS 0.171 vs popularité 0.137
@10 rappel ALS 0.295 vs popularité 0.244 | NDCG ALS 0.216 vs popularité 0.174

```

@20 rappel ALS 0.437 vs popularité 0.394 | NDCG ALS 0.265 vs popularité 0.225

Lecture. L'avantage **relatif** de l'ALS sur la popularité fond quand la liste s'allonge : le rapport des rappels vaut 1,28 pour $k = 1$ (0,060 contre 0,047), 1,21 pour $k = 10$ et 1,11 pour $k = 20$ (0,437 contre 0,394). Avec une longue liste, tout le monde finit par retrouver les produits populaires. Le k de l'évaluation doit donc refléter l'usage réel (une bannière, une vitrine de cinq, un défilement).

Application 7.6 — Le démarrage à froid

Section du livre : 7.4.5. Objectif : refaire l'expérience du nouveau client avec un « fold-in », la compléter par un hybride qui mélange popularité et personnalisation tant que l'on connaît peu d'achats, puis explorer les variantes du contenu pour un nouveau produit.

Étape 1 — Mise en place : clients « froids » et modèles entraînés sans eux.

```
rng = np.random.default_rng(21)
Rc = R.tocsr(); nb = np.asarray(Rc.sum(axis=1)).ravel()
eligibles = np.where(nb >= 11)[0]
froids = np.sort(rng.choice(eligibles, len(eligibles) // 4, replace=False))
R_tr = Rc.tolil(); R_tr[froids, :] = 0; R_tr = R_tr.tocsr(); R_tr.eliminate_zeros()
S_it = cosinus_colonnes(R_tr) # similarités entre produits
U_f, V_f = als_implicit(R_tr, k=6, lam=100.0, alpha=8.0, n_iter=10, seed=0)
pop_tr = scores_popularite(R_tr)[0]
print(len(eligibles), "clients à 11 achats ou plus ;", len(froids), "simulés comme nouveaux")
```

1000 clients à 11 achats ou plus ; 250 simulés comme nouveaux

Étape 2 — La courbe, avec un hybride. On mélange les scores normalisés : $(1 - w)$ popularité + w personnalisé, avec $w = m/(m + c)$ qui croît avec le nombre m d'achats connus ($c = 3$).

```
def normalise(S):
    return S / np.maximum(S.max(axis=1, keepdims=True), 1e-12)

def courbe(m, rng_loc):
    L_r, C_r, L_c, C_c = [], [], [], []
    S_p, S_i, S_a, S_h = (np.zeros((3000, 150)) for _ in range(4))
    for u in froids:
        items = rng_loc.permutation(Rc.indices[Rc.indptr[u]:Rc.indptr[u + 1]])
        vus, cibles = items[:m], items[m:]
        L_r += [u] * len(vus); C_r += list(vus); L_c += [u] * len(cibles); C_c += list(cibles)
        S_p[u] = pop_tr
        S_i[u] = S_it[vus].sum(axis=0) if m else pop_tr
        S_a[u] = vecteur_client(V_f, vus, lam=100.0, alpha=8.0) @ V_f.T if m else pop_tr
        w = m / (m + 3)
        S_h[u] = (1 - w) * normalise(pop_tr[None, :])[0] + w * normalise(S_a[u][None, :])[0]
    A = sp.csr_matrix((np.ones(len(C_r)), (L_r, C_r)), shape=(3000, 150)); B =
    ↪ sp.csr_matrix((np.ones(len(C_c)), (L_c, C_c)), shape=(3000, 150))
    return [evaluer(S, A, B, froids)["rappel"].mean() for S in (S_p, S_i, S_a, S_h)]

res = pd.DataFrame({m: courbe(m, np.random.default_rng(100 + m)) for m in (0, 1, 2, 3, 5, 8)},
    ↪ index=["popularité", "voisins de produits", "ALS (fold-in)", "hybride"]).T
print(res.round(3).to_string())
```

	popularité	voisins de produits	ALS (fold-in)	hybride
0	0.198	0.198	0.198	0.198
1	0.203	0.204	0.212	0.221
2	0.203	0.226	0.232	0.227
3	0.213	0.259	0.259	0.248
5	0.214	0.259	0.268	0.254
8	0.223	0.290	0.285	0.269

Lecture. Sans achat connu, les quatre méthodes sont à égalité (19,8 %). Avec **un** achat, l'hybride (22,1 %) devance l'ALS (21,2 %) et la popularité (20,3 %) ; dès **deux** achats, l'ALS repasse devant (23,2 % contre 22,7 %) et à huit achats elle est nettement meilleure que l'hybride (28,5 % contre 26,9 %). Le poids $w = m/(m+3)$ est trop prudent : il ne sert qu'au tout début. Ces chiffres diffèrent un peu de ceux du livre (par exemple 23,2 % au lieu de 24,0 % pour l'ALS à deux achats) parce que les achats révélés sont tirés avec d'autres graines : avec 250 clients, des écarts d'un point sont dans le bruit.

Étape 3 — Un nouveau produit : quel contenu ? Les 26 produits récents sont retirés ; on les classe pour chaque client qui en a acheté, à partir de ses autres achats.

```
nouv = np.where(prod.nouveaute.values == 1)[0]; anciens = np.where(prod.nouveaute.values == 0)[0]
cat = pd.get_dummies(prod.categorie, dtype=float).to_numpy()
lp = np.log(prod.prix.values); prix_z = ((lp - lp.mean()) / lp.std())[:, None]
Rn = Rc[:, nouv].toarray() > 0; Ra = Rc[:, anciens]
cibles_u = np.where(Rn.sum(axis=1) >= 1)[0]; Y = Rn[cibles_u]

def scores_nouveaux(F):
    n = np.asarray(Ra.sum(axis=1)).ravel(); n[n == 0] = 1
    P = (Ra @ F[anciens]) / n[:, None]
    P = P / np.maximum(np.linalg.norm(P, axis=1, keepdims=True), 1e-12)
    Fn = F[nouv] / np.maximum(np.linalg.norm(F[nouv], axis=1, keepdims=True), 1e-12)
    return (P @ Fn.T)[cibles_u]
```

On mesure ensuite, pour chaque client, le **rappel@5** parmi les 26 produits récents et l'**AUC** (probabilité qu'un produit réellement acheté soit classé devant un produit non acheté), puis on compare quatre jeux de caractéristiques.

```
def rappel_auc(S, Y, k=5):
    rec, auc = [], []
    for s, y in zip(S, Y):
        rec.append(y[np.argsort(-s, kind="stable")[:k]].sum() / y.sum())
        pos, neg = s[y], s[~y]
        auc.append((pos[:, None] > neg[None, :]).mean() + 0.5 * (pos[:, None] == neg[None, :]).mean())
    if len(neg) < np.nan:
        return round(np.mean(rec), 3), round(np.nanmean(auc), 3)

variantes = {"catégorie seule": cat, "prix seul": prix_z, "catégorie + prix": np.hstack([cat,
    prix_z]), "catégorie × 2 + prix": np.hstack([2 * cat, prix_z])}
for nom, F in variantes.items():
    rec_, auc_ = rappel_auc(scores_nouveaux(F), Y)
    print(f"{nom:22s} rappel@5 = {float(rec_):.3f} | AUC = {float(auc_):.3f}")
print("hasard : rappel@5 =", round(5 / 26, 3), "| AUC = 0,5")
```

```
catégorie seule      rappel@5 = 0.296 | AUC = 0.617
prix seul            rappel@5 = 0.150 | AUC = 0.493
catégorie + prix     rappel@5 = 0.290 | AUC = 0.592
catégorie × 2 + prix rappel@5 = 0.307 | AUC = 0.613
hasard : rappel@5 = 0.192 | AUC = 0,5
```

Lecture. La catégorie seule retrouve 29,6 % des achats (AUC de 0,617) contre 19,2 % pour le hasard. Le **prix seul** ne vaut rien : 15,0 % (AUC de 0,493), en dessous du hasard, ce qui est cohérent avec des données où le prix n'est lié à aucun goût. Donner plus de poids à la catégorie (30,7 %) améliore légèrement, d'un point que nous ne saurions distinguer du bruit.

Application 7.7 — Découpage aléatoire contre découpage temporel

Section du livre : 7.4.3. Objectif : refaire la simulation du livre en faisant varier le nombre de produits dont la popularité change entre les deux périodes, et mesurer l'inflation des résultats par le découpage aléatoire.

```
def inflation(n_bouge, seed=11):
    R1, R2 = monde_en_derive(n_bouge=n_bouge, seed=seed)
    Rall = ((R1 + R2) > 0).astype(float).tocsr()
    ds = decouper(Rall, seed=3, part_test=0.25, part_val=0.0001)
    T2 = (R2 - R2.multiply(R1)).tocsr(); T2.eliminate_zeros()
    us2 = np.where(np.asarray(T2.sum(axis=1)).ravel() >= 1)[0]
    out = {}
    for nom, f in (("popularité", scores_popularite), ("ALS", lambda X: scores_als(X, k=4, lam=30.0,
        ↪ alpha=8.0, n_iter=8))):
        alea = resume(evaluer(f(ds["R_trainval"])), ds["R_trainval"], ds["test"], ds["users"])).rappel
        temps = resume(evaluer(f(R1.tocsr()), R1.tocsr(), T2, us2)).rappel
        out[nom] = (alea, temps, alea / temps)
    return out

lignes = []
for nb_ in (0, 10, 30, 60):
    o = inflation(nb_)
    lignes.append({"produits qui changent": nb_, **{f"{nom} {c}": v for nom, t in o.items() for c, v
        ↪ in zip(("aléatoire", "temporel", "rapport"), t)}})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
```

produits qui changent		popularité aléatoire	popularité temporel	popularité rapport	ALS aléatoire	
↪	ALS temporel	ALS rapport				
		0	0.288	0.263	1.097	0.355
↪	0.301	1.182				
		10	0.306	0.216	1.417	0.371
↪	0.281	1.318				
		30	0.328	0.221	1.483	0.398
↪	0.260	1.531				
		60	0.353	0.248	1.425	0.412
↪	0.251	1.644				

Lecture. Même sans aucun changement de popularité, le découpage aléatoire donne des résultats supérieurs de 10 % (popularité, 0,288 contre 0,263) à 18 % (ALS, 0,355 contre 0,301) : cet écart ne vient pas de la dérive mais de la définition même des deux protocoles (voir le corrigé 7.12). Quand la popularité change, l'écart grimpe à 1,3 à 1,6 fois : 1,42, 1,48 et 1,43 pour la popularité avec 10, 30 et 60 produits qui changent, 1,32, 1,53 et 1,64 pour l'ALS, plus sensible. Le rapport de la popularité n'est pas monotone : c'est une simulation unique, à lire comme un ordre de grandeur.

Pour aller plus loin. Le rapport « aléatoire / temporel » reste supérieur à 1 même sans changement de popularité : pourquoi ? (Indice : dans le découpage temporel, on ne teste que sur des produits **nouveaux pour le client** ; dans le découpage aléatoire, un client peut avoir, dans l'entraînement, des achats de la seconde période.)

Application 7.8 — La boucle de rétroaction

Section du livre : 7.4.6. Objectif : explorer le simulateur : quelle quantité d'exploration, quelle taille de vitrine, quelle règle de classement ?

Étape 1 — Exploration et règle de classement.

```
def final(eps, taux, vitrine=5, graines=range(4)):
    moy = np.mean([boucle_retroaction(eps, taux, vitrine=vitrine, seed=s).iloc[-1][["part_top5",
        ↪ "produits_achetes", "correlation_vrai", "bons_en_vitrine"]].astype(float).values for s in
        ↪ graines], axis=0)
```

```

    return moy

lignes = []
for taux in (False, True):
    for eps in (0.0, 0.05, 0.1, 0.2, 0.5):
        m = final(eps, taux)
        lignes.append({"classement": "taux d'achat" if taux else "ventes", "exploration": eps, "part
↳ top 5": m[0], "produits achetés": m[1], "corrélation": m[2], "bons en vitrine": m[3]})
print(pd.DataFrame(lignes).round(2).to_string(index=False))

```

classement	exploration	part top 5	produits achetés	corrélation	bons en vitrine
ventes	0.00	0.96	48.00	0.78	0.25
ventes	0.05	0.91	55.75	0.80	0.25
ventes	0.10	0.87	59.25	0.80	0.25
ventes	0.20	0.78	60.00	0.81	0.25
ventes	0.50	0.53	60.00	0.84	0.50
taux d'achat	0.00	0.79	46.25	0.99	5.00
taux d'achat	0.05	0.83	55.75	0.97	4.75
taux d'achat	0.10	0.85	59.25	0.94	5.00
taux d'achat	0.20	0.82	60.00	0.95	4.50
taux d'achat	0.50	0.66	60.00	0.98	5.00

Lecture. Avec le classement par **ventes**, la vitrine ne contient presque jamais les bons produits (0,25 des 5 meilleurs, 0,5 avec 50 % d'exploration) et la corrélation avec l'attrait réel reste entre 0,78 et 0,84 ; l'exploration, elle, répartit les achats (48 produits achetés sans exploration, 60 avec 20 %) et fait chuter la concentration (de 96 % à 53 % des achats sur cinq produits). Avec le classement par **taux d'achat**, la vitrine contient de 4,5 à 5 des 5 meilleurs produits et la corrélation est de 0,94 à 0,99 quelle que soit l'exploration. L'exploration répartit les ventes ; c'est la **règle de classement** qui rétablit la lecture des goûts.

Étape 2 — La taille de la vitrine.

```

for v in (3, 5, 10):
    for nom, (e, t) in {"ventes seules": (0.0, False), "taux d'achat + 20 % de hasard": (0.2,
↳ True)}.items():
        m = final(e, t, vitrine=v)
        print(f"vitrine de {v:2d} produits, {nom:30s}: corrélation {float(m[2]):.2f} | bons produits
↳ parmi les 5 premiers du classement {float(m[3]):.2f}")

```

```

vitrine de 3 produits, ventes seules : corrélation 0.88 | bons produits parmi les 5
↳ premiers du classement 2.25
vitrine de 3 produits, taux d'achat + 20 % de hasard : corrélation 0.96 | bons produits parmi les 5
↳ premiers du classement 4.00
vitrine de 5 produits, ventes seules : corrélation 0.78 | bons produits parmi les 5
↳ premiers du classement 0.25
vitrine de 5 produits, taux d'achat + 20 % de hasard : corrélation 0.95 | bons produits parmi les 5
↳ premiers du classement 4.50
vitrine de 10 produits, ventes seules : corrélation 0.65 | bons produits parmi les 5
↳ premiers du classement 0.75
vitrine de 10 produits, taux d'achat + 20 % de hasard : corrélation 0.96 | bons produits parmi les 5
↳ premiers du classement 5.00

```

Lecture. Plus la vitrine est grande, plus la lecture des ventes est biaisée : la corrélation avec l'attrait réel passe de 0,88 (3 produits) à 0,78 (5) puis 0,65 (10), parce que davantage de produits profitent de l'exposition. Le classement par taux d'achat, lui, reste à 0,95 ou 0,96 et trouve les cinq meilleurs produits avec une vitrine de dix.

Application 7.9 — Mélanger les méthodes, et diversifier

Sections du livre : 7.4.4. **Objectif** : mélanger la popularité et l'ALS, puis réordonner les listes pour augmenter la diversité des catégories, et mesurer ce que l'on gagne et ce que l'on perd.

Étape 1 — Un mélange des scores.

```
def couverture(df):
    return len(np.unique(np.vstack(df["recommandes"].values))) / 150

S_pop, S_als = normalise(scores_popularite(Rt)), normalise(scores_als(Rt, k=6, lam=100.0, alpha=8.0,
    ↪ n_iter=10))
lignes = []
for w in (0.0, 0.25, 0.5, 0.75, 1.0):
    r = evaluer((1 - w) * S_pop + w * S_als, Rt, Rv, users)
    lignes.append({"poids de l'ALS": w, "NDCG@10": r.ndcg.mean(), "rappel@10": r.rappel.mean(),
    ↪ "couverture": couverture(r)})
print(pd.DataFrame(lignes).round(4).to_string(index=False))
```

poids de l'ALS	NDCG@10	rappel@10	couverture
0.00	0.1431	0.2409	0.1400
0.25	0.1508	0.2551	0.1467
0.50	0.1571	0.2658	0.2000
0.75	0.1645	0.2732	0.2533
1.00	0.1681	0.2761	0.2800

Lecture. Le NDCG augmente régulièrement avec le poids de l'ALS (de 0,143 pour la popularité à 0,168 pour l'ALS seul), et la couverture aussi (de 14 % à 28 %). Sur ce jeu, **aucun mélange ne fait mieux que l'ALS seul** : l'hybride n'apporte rien. Il serait utile si le modèle personnalisé était instable, par exemple avec très peu de données.

Étape 2 — Réordonner pour la diversité. On construit chaque liste de façon gloutonne parmi les 30 meilleurs candidats : à chaque pas, on retire μ au score d'un candidat pour chaque produit déjà choisi de la même catégorie.

```
cat_ = prod.categorie.values
def diversifier(S, mu):
    S2 = np.full_like(S, -1.0)
    for u in users:
        cand = list(np.argsort(-np.where(np.asarray(Rt[u].todense()).ravel() > 0, -np.inf,
    ↪ S[u]))[:30])
        choisis = []
        for rang in range(10):
            pen = [S[u, c] - mu * sum(cat_[c] == cat_[x] for x in choisis) for c in cand]
            c = cand.pop(int(np.argmax(pen))); choisis.append(c); S2[u, c] = 100 - rang
    return S2

S_base = 0.5 * S_pop + 0.5 * S_als
lignes = []
for mu in (0.0, 0.05, 0.1, 0.2):
    r = evaluer(diversifier(S_base, mu), Rt, Rv, users)
    L = np.vstack(r.recommandes.values)
    div = np.mean([np.mean([cat_[a] != cat_[b] for ia, a in enumerate(row) for b in row[ia + 1:]]) for
    ↪ row in L])
    lignes.append({"pénalité  $\mu$ ": mu, "NDCG@10": r.ndcg.mean(), "rappel@10": r.rappel.mean(),
    ↪ "diversité": div, "couverture": couverture(r)})
print(pd.DataFrame(lignes).round(4).to_string(index=False))
```

pénalité μ	NDCG@10	rappel@10	diversité	couverture
0.00	0.1571	0.2658	0.7879	0.2000
0.05	0.1556	0.2641	0.8166	0.1933
0.10	0.1551	0.2627	0.8208	0.2000
0.20	0.1543	0.2625	0.8221	0.2000

Lecture. Pénaliser les doublons de catégorie fait passer la diversité de 0,788 à 0,822 (+0,034) pour une perte de 0,003 de NDCG (0,157 à 0,154) et de 0,003 de rappel : un coût faible pour un gain modeste. La couverture ne bouge pas (0,20), car on ne réordonne que les 30 meilleurs candidats : pour couvrir plus de produits, il faut élargir l'ensemble de candidats.

Exercices

Exercice 7.1 ★ — Cosinus et Jaccard à la main (section 7.1.4)

On reprend la matrice de 5 clients et 6 produits du livre ($U1 : P1, P2, P4$; $U2 : P1, P3, P4$; $U3 : P2, P3, P4, P5$; $U4 : P1, P2, P6$; $U5 : P1, P3, P4, P5$). (a) Calculez le cosinus et l'indice de Jaccard des produits P1 et P2. (b) Parmi les produits P1 à P5, quelle paire a le cosinus le plus élevé ? (c) Pourquoi le cosinus de P5 et P6 vaut-il 0, et que vaudrait celui de la corrélation de Pearson ?

Exercice 7.2 ★ — Popularité et contenu pour un petit client (sections 7.1.5 et 7.1.6)

- (a) Dans la même matrice, quels produits la popularité recommande-t-elle au client U4, dans quel ordre ? (b) Quatre produits ont pour caractéristiques (cat. A, cat. B, prix en dizaines d'euros) : $P1 = (1, 0, 2)$, $P2 = (1, 0, 3)$, $P3 = (0, 1, 2, 5)$, $P4 = (1, 0, 2, 5)$. Un client a acheté P1 et P2. Mettez le prix à l'échelle (centré, réduit sur ces quatre produits) et recalculez les cosinus avec P3 et P4. Que constatez-vous par rapport au livre ?

Exercice 7.3 ★★ — Pourquoi stocker en format creux (section 7.1.3)

La matrice de la boutique a 3 000 lignes, 150 colonnes et 27 687 cases remplies. (a) Quelle mémoire occupe-t-elle en tableau dense de flottants de 8 octets, et en format creux CSR (8 octets par valeur, 4 par indice de colonne, 4 par pointeur de ligne) ? (b) Même question pour un site de 1 million de clients et 10 000 produits, à 10 achats par client en moyenne.

Exercice 7.4 ★ — Voisins de produits pour U4 (section 7.2.3)

Toujours sur la même matrice : calculez à la main le score de voisinage $\hat{s}_{uj} = \sum_{i \in I_u} \cos(i, j)$ du client U4 (qui a acheté P1, P2 et P6) pour les candidats P3, P4 et P5. Quel classement obtenez-vous ? Est-il identique à celui de la popularité ?

Exercice 7.5 ★★ — Le rétrécissement renverse un classement (section 7.2.4)

Trois paires de produits ont pour (cosinus ; co-achats) : $a = (0,9 ; 3)$, $b = (0,5 ; 30)$, $c = (0,7 ; 12)$. (a) Classez-les avant rétrécissement. (b) Appliquez $\text{sim}' = \text{sim} \times n_{ij} / (n_{ij} + \lambda)$ avec $\lambda = 10$ et reclassez. (c) Pour quelle valeur de λ les paires b et c échangent-elles leur rang ?

Exercice 7.6 ★★ — Prédire une note avec les voisins (section 7.2.2)

Avec le tableau de notes du livre (clients u_1 à u_4 , produits A à D), prédisez la note de u_1 pour D en ne gardant que les voisins de **similarité positive**. Comparez avec la prédiction obtenue avec les trois voisins (4,43).

Exercice 7.7 ★★ — Une étape d'ALS à la main (section 7.3.4)

Dans l'exemple 4×4 de la section 7.3.4, on a trouvé après l'étape des produits $q \approx (1,652 ; 0,715 ; 1,724 ; 1,249)$. Avec $\lambda = 1$, recalculez à la main le facteur du client 2 (qui a noté $P1 = 4$ et $P4 = 1$), puis le facteur du client 1 ($P1 = 5$, $P2 = 3$, $P4 = 1$).

Exercice 7.8 ★★★ — L'astuce de calcul de l'ALS implicite (section 7.3.5)

Soit C_u la matrice diagonale de confiance du client u , de coefficients $c_{ui} = 1 + \alpha R_{ui}$. (a) Montrez que $Q^T C_u Q = Q^T Q + \alpha Q_u^T Q_u$, où Q_u regroupe les lignes de Q des produits achetés par u . (b) Comparez le nombre d'opérations pour le calcul direct et pour l'astuce, en fonction du nombre de produits m , du nombre d_u d'achats du client et du nombre de facteurs k .

Exercice 7.9 ★★ — Un pas de gradient stochastique (section 7.3.3)

On a $p_u = (0,5 ; 0,2)$, $q_i = (0,4 ; 0,6)$, une note $r_{ui} = 4$, un pas $\gamma = 0,1$ et $\lambda = 0,02$. Calculez l'erreur, puis les nouvelles valeurs de p_u et q_i (en utilisant les anciennes valeurs pour les deux mises à jour), et la nouvelle prédiction. L'erreur a-t-elle diminué ?

Exercice 7.10 ★ — Les cinq métriques pour une liste de dix (section 7.4.1)

Une liste de $k = 10$ produits a la pertinence $(1,0,0,1,0,0,0,1,0,0)$; le client avait en tout 4 produits pertinents dans le jeu de test. Calculez précision@10, rappel@10, succès@10, AP@10 et NDCG@10.

Exercice 7.11 ★★ — Quand les métriques ne s'accordent pas (section 7.4.1)

Un client a 3 produits pertinents. La liste A de 5 produits a la pertinence $(1,0,0,0,0)$; la liste B a $(0,0,1,1,1)$. Calculez pour chacune précision@5, rappel@5, AP@5 et NDCG@5, ainsi que « le premier produit est-il pertinent ? ». Quelle liste est la meilleure ? Dans quel usage choisiriez-vous l'autre ?

Exercice 7.12 ★★★ — Sans dérive, le découpage aléatoire est-il innocent ? (sections 7.4.3 et 7.4.6)

- (a) Avec `monde_en_derive(n_bouge=0)`, comparez le rappel@10 de la popularité en découpage aléatoire et en découpage temporel. Le rapport vaut-il 1 ? (b) Expliquez en deux phrases le mécanisme qui reste. (c) Dans la boucle de rétroaction, pourquoi « ventes + 20 % d'exploration » ne change-t-il pas le nombre de bons produits en vitrine ?

Corrigés

Corrigé 7.1

- (a) P1 est acheté par U1, U2, U4, U5 ($n_1 = 4$); P2 par U1, U3, U4 ($n_2 = 3$); les deux par U1 et U4 ($n_{12} = 2$). Cosinus : $2/\sqrt{4 \times 3} = 2/\sqrt{12} \approx 0,577$; Jaccard : $2/(4 + 3 - 2) = 0,4$. (b) On calcule tous les cosinus par $n_{ij}/\sqrt{n_i n_j}$; le plus élevé est celui de **P3 et P4** ($n_3 = 3, n_4 = 4, n_{34} = 3 : 3/\sqrt{12} \approx 0,866$). (c) P5 et P6 n'ont aucun acheteur commun, donc le produit scalaire est nul. La corrélation de Pearson, qui centre les vecteurs sur leur moyenne, vaut au contraire $-0,41$: elle lit l'absence d'achat commun comme une **opposition** (les deux produits ne sont jamais achetés ensemble, alors que chacun l'est par d'autres clients). Le cosinus dit « aucun lien », la corrélation dit « lien négatif » ; pour recommander, « aucun lien » est la lecture utile.

```
M = np.array([[1, 1, 0, 1, 0, 0], [1, 0, 1, 1, 0, 0], [0, 1, 1, 1, 1, 0], [1, 1, 0, 0, 0, 1], [1, 0, 1, 1, 1, 0]], float)
n = M.sum(axis=0); cos = (M.T @ M) / np.sqrt(np.outer(n, n)); np.fill_diagonal(cos, 0)
print("cos(P1,P2) =", cos[0, 1].round(3), "| Jaccard =", (M[:, 0] @ M[:, 1]) / (n[0] + n[1] - M[:, 0] @ M[:, 1]))
i, j = np.unravel_index(cos[:5, :5].argmax(), (5, 5)); print("paire la plus proche parmi P1..P5 :", i + 1, j + 1, cos[i, j].round(3))
print("corrélation de Pearson P5-P6 :", np.corrcoef(M[:, 4], M[:, 5])[0, 1].round(3))
```

```
cos(P1,P2) = 0.577 | Jaccard = 0.4
paire la plus proche parmi P1..P5 : 3 4 0.866
corrélation de Pearson P5-P6 : -0.408
```

Corrigé 7.2

- (a) U4 a acheté P1, P2, P6 ; il reste P3 (3 acheteurs), P4 (4) et P5 (2). La popularité recommande donc **P4, puis P3, puis P5**. (b) Prix en dizaines d'euros : (2; 3; 2,5; 2,5), moyenne 2,5, variance $(0,25+0,25+0+0)/4 = 0,125$ et écart-type $\sqrt{0,125} \approx 0,354$; prix centrés réduits : $(-1,414; +1,414; 0; 0)$. Les vecteurs deviennent $P1 = (1, 0, -1,414)$, $P2 = (1, 0, +1,414)$, $P3 = (0, 1, 0)$, $P4 = (1, 0, 0)$. Profil du client : $(1, 0, 0)$. Cosinus avec P4 : 1 ; avec P3 : 0. **La mise à l'échelle a rétabli la hiérarchie** : sans elle, P3 obtenait 0,862 (le prix dominait) ; avec elle, un produit d'une autre catégorie n'a plus aucune ressemblance avec le profil.

```
prix = np.array([2, 3, 2.5, 2.5]); z = (prix - prix.mean()) / prix.std()
F = np.array([[1, 0, z[0]], [1, 0, z[1]], [0, 1, z[2]], [1, 0, z[3]]]); profil = F[:2].mean(axis=0)
c = lambda a, b: a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print("prix centrés réduits :", z.round(3), "| profil :", profil.round(3), "| cos P3 =", round(c(profil, F[2]), 3), "| cos P4 =", round(c(profil, F[3]), 3))
```

```
prix centrés réduits : [-1.414  1.414  0.      0.    ] | profil : [1.  0.  0.] | cos P3 = 0.0 | cos P4 = 1.0
```

Corrigé 7.3

- (a) Dense : $3\,000 \times 150 \times 8 = 3,6$ Mo. CSR : $27\,687 \times (8 + 4) + 3\,001 \times 4 \approx 344$ ko, soit environ **10 fois moins**. (b) Dense : $10^6 \times 10^4 \times 8 = 80$ Go (impossible à garder en mémoire). Creux : 10^7 achats $\times 12$ octets ≈ 124 Mo (avec les pointeurs de ligne), soit un rapport de **plus de 600**. Le gain croît avec la **dilution** de la matrice : à 0,1 % de cases remplies, on ne stocke que ce qui existe.

```
dense = 3000 * 150 * 8; creux = 27687 * 12 + 3001 * 4
print("boutique : dense", dense / 1e6, "Mo | creux", round(creux / 1e3), "ko | rapport", round(dense / creux, 1))
dense2 = 1e6 * 1e4 * 8; creux2 = 1e6 * 10 * 12 + (1e6 + 1) * 4
print("grand site : dense", dense2 / 1e9, "Go | creux", round(creux2 / 1e6), "Mo | rapport", round(dense2 / creux2))
```

boutique : dense 3.6 Mo | creux 344 ko | rapport 10.5
 grand site : dense 80.0 Go | creux 124 Mo | rapport 645

Corrigé 7.4

U4 a acheté P1, P2, P6. Avec les cosinus du livre : pour **P3**, $\cos(P3, P1) = 2/\sqrt{12} = 0,577$, $\cos(P3, P2) = 1/\sqrt{9} = 0,333$, $\cos(P3, P6) = 0$: score 0,911. Pour **P4** : $\cos(P4, P1) = 3/\sqrt{16} = 0,75$, $\cos(P4, P2) = 2/\sqrt{12} = 0,577$, $\cos(P4, P6) = 0$: score 1,327. Pour **P5** : $1/\sqrt{8} = 0,354$, $1/\sqrt{6} = 0,408$, 0 : score 0,762. Le classement est **P4, P3, P5**, identique à celui de la popularité sur ce petit tableau (la personnalisation ne se voit que sur des historiques plus variés).

```
S = (M.T @ M) / np.sqrt(np.outer(M.sum(axis=0), M.sum(axis=0))); np.fill_diagonal(S, 0)
sc = M[3] @ S
print("scores de U4 pour P3, P4, P5 :", sc[[2, 3, 4]].round(3))
```

scores de U4 pour P3, P4, P5 : [0.911 1.327 0.762]

Corrigé 7.5

- (a) Avant rétrécissement : $a(0,9) > c(0,7) > b(0,5)$. (b) Avec $\lambda = 10$: $a' = 0,9 \times \frac{3}{13} \approx 0,208$; $b' = 0,5 \times \frac{30}{40} = 0,375$; $c' = 0,7 \times \frac{12}{22} \approx 0,382$. Nouveau classement : $c > b > a$: la paire a , fondée sur 3 co-achats, passe **de la première à la dernière** place. (c) b et c échangent leur rang quand $0,5 \frac{30}{30+\lambda} = 0,7 \frac{12}{12+\lambda}$, soit $15(12+\lambda) = 8,4(30+\lambda)$, donc $6,6\lambda = 72$ et $\lambda \approx 10,9$: pour $\lambda < 10,9$ c'est c qui est devant, au-delà c'est b (le plus fiable).

```
paires = {"a": (0.9, 3), "b": (0.5, 30), "c": (0.7, 12)}
for lam in (0, 10, 10.909, 20):
    print(f"λ = {lam:6.1f}:", {k: round(s * n / (n + lam), 3) for k, (s, n) in paires.items()})
```

```
λ =      0: {'a': 0.9, 'b': 0.5, 'c': 0.7}
λ =     10: {'a': 0.208, 'b': 0.375, 'c': 0.382}
λ = 10.909: {'a': 0.194, 'b': 0.367, 'c': 0.367}
λ =     20: {'a': 0.117, 'b': 0.3, 'c': 0.262}
```

Corrigé 7.6

Les voisins de similarité positive sont u_2 (0,653, écart +0,25) et u_4 (0,816, écart +0,5). Prédiction : $4 + \frac{0,653 \times 0,25 + 0,816 \times 0,5}{0,653 + 0,816} = 4 + \frac{0,571}{1,469} \approx 4,39$, contre 4,43 avec le voisin de similarité négative. La différence est faible ici, mais le raisonnement est plus sûr : on n'utilise pas l'hypothèse fragile « il n'aime pas ce que j'aime, donc ce qu'il déteste me plaira ».

```
notes = np.array([[5, 3, 4, np.nan], [4, 2, 5, 4], [2, 5, 1, 2], [5, 4, 4, 5]]); moy =
↳ np.nanmean(notes, axis=1); cent = notes - moy[:, None]
sims = np.array([cent[0, :3] @ cent[v, :3] / (np.linalg.norm(cent[0, :3]) * np.linalg.norm(cent[v,
↳ :3])) for v in (1, 2, 3)])
ec = cent[1:, :3]; pos = sims > 0
print("similarités :", sims.round(3), "| prédiction (3 voisins) :", round(moy[0] + (sims * ec).sum() /
↳ np.abs(sims).sum(), 3), "| (voisins positifs) :", round(moy[0] + (sims[pos] * ec[pos]).sum() /
↳ sims[pos].sum(), 3))
```

similarités : [0.653 -0.717 0.816] | prédiction (3 voisins) : 4.425 | (voisins positifs) : 4.389

Corrigé 7.7

Client 2 : $p_2 = \frac{4 \times 1,652 + 1 \times 1,249}{1,652^2 + 1,249^2 + 1} = \frac{7,857}{5,289} \approx 1,486$. Client 1 : $p_1 = \frac{5 \times 1,652 + 3 \times 0,715 + 1 \times 1,249}{1,652^2 + 0,715^2 + 1,249^2 + 1} = \frac{11,654}{5,800} \approx 2,009$. On retrouve exactement les valeurs de la seconde passe du livre.

```
q = np.array([1.652, 0.715, 1.724, 1.249])
print("client 2 :", round((4 * q[0] + 1 * q[3]) / (q[0]**2 + q[3]**2 + 1), 3), "| client 1 :",
      round((5 * q[0] + 3 * q[1] + 1 * q[3]) / (q[0]**2 + q[1]**2 + q[3]**2 + 1), 3))
```

```
client 2 : 1.486 | client 1 : 2.009
```

Corrigé 7.8

- (a) $C_u = I + \alpha \text{diag}(R_u)$, où $\text{diag}(R_u)$ est la matrice diagonale des achats du client. Donc $Q^T C_u Q = Q^T Q + \alpha Q^T \text{diag}(R_u) Q$. Or $\text{diag}(R_u)$ ne conserve que les lignes de Q correspondant aux produits achetés : $Q^T \text{diag}(R_u) Q = Q_u^T Q_u$. D'où le résultat. (b) Le calcul direct de $Q^T C_u Q$ coûte $O(mk^2)$ pour chaque client. L'astuce calcule $Q^T Q$ **une fois pour tous** les clients ($O(mk^2)$ au total) et ajoute $\alpha Q_u^T Q_u$, qui coûte $O(d_u k^2)$; il reste à résoudre un système $k \times k$ ($O(k^3)$). Le coût par client passe de $O(mk^2)$ à $O(d_u k^2 + k^3)$: avec $m = 150$, $d_u \approx 9$ et $k = 6$, c'est un facteur d'environ 15 sur ce terme, et bien plus avec un grand catalogue.

```
rng = np.random.default_rng(0); Q = rng.standard_normal((150, 6)); achat = rng.random(150) < 0.06;
alpha = 8.0
direct = Q.T @ np.diag(1 + alpha * achat) @ Q
astuce = Q.T @ Q + alpha * Q[achat].T @ Q[achat]
print("identiques :", np.allclose(direct, astuce), "| produits achetés :", int(achat.sum()))
```

```
identiques : True | produits achetés : 8
```

Corrigé 7.9

Prédiction : $\hat{r} = 0,5 \times 0,4 + 0,2 \times 0,6 = 0,32$; erreur $e = 4 - 0,32 = 3,68$. Mises à jour (avec les anciennes valeurs) : $p_1 = 0,5 + 0,1 (3,68 \times 0,4 - 0,02 \times 0,5) = 0,6462$; $p_2 = 0,2 + 0,1 (3,68 \times 0,6 - 0,02 \times 0,2) = 0,4204$; $q_1 = 0,4 + 0,1 (3,68 \times 0,5 - 0,02 \times 0,4) = 0,5832$; $q_2 = 0,6 + 0,1 (3,68 \times 0,2 - 0,02 \times 0,6) = 0,6724$. Nouvelle prédiction : $0,6462 \times 0,5832 + 0,4204 \times 0,6724 \approx 0,660$, soit une erreur de 3,34 : **elle a diminué** (de 3,68 à 3,34), comme le veut la descente de gradient.

```
p = np.array([0.5, 0.2]); q = np.array([0.4, 0.6]); r, g, lam = 4.0, 0.1, 0.02
e = r - p @ q; p2 = p + g * (e * q - lam * p); q2 = q + g * (e * p - lam * q)
print("erreur", round(e, 3), "| p", p2.round(4), "| q", q2.round(4), "| nouvelle prédiction",
      round(p2 @ q2, 4), "| nouvelle erreur", round(r - p2 @ q2, 3))
```

```
erreur 3.68 | p [0.6462 0.4204] | q [0.5832 0.6724] | nouvelle prédiction 0.6595 | nouvelle erreur 3.34
```

Corrigé 7.10

Positions des succès : rangs 1, 4 et 8. Précision@10 = $3/10 = 0,3$; rappel@10 = $3/4 = 0,75$; succès@10 = 1. AP@10 = $\frac{1/1 + 2/4 + 3/8}{\min(4, 10)} = \frac{1,875}{4} \approx 0,469$. DCG = $1 + \frac{1}{\log_2 5} + \frac{1}{\log_2 9} = 1 + 0,431 + 0,315 = 1,746$; IDCG (4 produits pertinents aux rangs 1 à 4) = $1 + 0,631 + 0,5 + 0,431 = 2,562$; NDCG $\approx 0,682$.

```
print(np.round(metrics_liste(np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]), {0, 3, 7, 100}, k=10), 4))
```

[0.3 0.75 0.4688 0.6817]

Corrigé 7.11

	Précision@5	Rappel@5	AP@5	NDCG@5	1 ^{er} produit pertinent ?
Liste A (1,0,0,0,0)	0,2	0,333	0,333	0,469	oui
Liste B (0,0,1,1,1)	0,6	1	0,478	0,618	non

La liste B est meilleure pour les quatre métriques « de liste » : elle retrouve tout ce que le client voulait. La liste A n'est meilleure que sur « le premier produit est-il pertinent ? ». Si l'on ne montre qu'un **seul** produit (une bannière), A est préférable ; si l'on affiche une vitrine de cinq, B vaut mieux. Le choix de la métrique doit donc suivre l'usage.

```
rg = np.arange(1, 6); idcg = (1 / np.log2(np.arange(1, 4) + 1)).sum()
for nom, rel in (("A", [1, 0, 0, 0, 0]), ("B", [0, 0, 1, 1, 1])):
    hits = np.array(rel, float)
    ap = (hits * np.cumsum(hits) / rg).sum() / 3; ndcg = (hits / np.log2(rg + 1)).sum() / idcg
    print(nom, "précision", hits.sum() / 5, "| rappel", round(hits.sum() / 3, 3), "| AP", round(ap,
↵ 3), "| NDCG", round(ndcg, 3))
```

A précision 0.2 | rappel 0.333 | AP 0.333 | NDCG 0.469

B précision 0.6 | rappel 1.0 | AP 0.478 | NDCG 0.618

Corrigé 7.12

- (a) **Non** : sans changement de popularité, le rapport vaut environ **1,10** pour la popularité (0,288 contre 0,263) et **1,18** pour l'ALS (0,355 contre 0,301). (b) Les deux protocoles ne posent pas la même question. Dans le découpage temporel, on ne teste que sur des produits **que le client n'avait pas achetés** en première période : c'est une tâche plus dure. Dans le découpage aléatoire, le client garde dans son entraînement des achats de la même période que ceux qu'on cache, et un produit acheté aux deux périodes peut être « retrouvé ». La dérive s'ajoute à cet écart de définition et le fait passer de 1,1 à 1,5 environ. (c) Ajouter de l'exploration (20 % de produits tirés au hasard) fait découvrir plus de produits (60 produits achetés au lieu de 48), mais le nombre de bons produits en vitrine reste à 0,25 sur 5 : le classement par **ventes** continue de confondre « acheté » et « montré » : les produits qui, par hasard, ont été exposés davantage gardent un avantage. Seul un classement qui divise par les **expositions** (le taux d'achat) corrige la lecture.

```
o = inflation(0)
for nom, t in o.items():
    print(f"{nom:11s} aléatoire {float(t[0]):.3f} | temporel {float(t[1]):.3f} | rapport
↵ {float(t[2]):.2f}")
```

popularité aléatoire 0.288 | temporel 0.263 | rapport 1.10

ALS aléatoire 0.355 | temporel 0.301 | rapport 1.18

Chapitre 8 : ⊕ Apprentissage semi-supervisé et actif — exercices et applications

Ce chapitre du cahier accompagne le chapitre 8 du livre (apprentissage semi-supervisé et actif). Vous y trouverez **huit applications guidées**, où l'on écrit à la main ce que le livre a seulement montré (courbes d'apprentissage, auto-apprentissage, propagation d'étiquettes, co-apprentissage, boucle active, comité, biais d'échantillonnage, coût), puis **douze exercices** avec corrigés. Les données sont les deux jeux du livre : les **chiffres manuscrits** (réels, `load_digits`) et les **clients de la boutique** (simulés, `donnees/clients_ml.csv`).

Préparation

Le chargement des deux jeux (réservoir d'entraînement et jeu de test, variables centrées-réduites pour les clients) est dans `build/outils_ch08.py`. Le reste, c'est-à-dire tout l'intéressant, est écrit ici, à la main.

```
import sys
sys.path.insert(0, "build")
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
import outils_ch08 as o                                     # charger_digits, charger_churn, tirer_etiquettes...

Xp, Xt, yp, yt = o.charger_digits()                        # chiffres : réservoir (1 347 images) et test (450)
Xc, Xct, yc, yct = o.charger_churn()                      # clients : réservoir (9 000) et test (3 000)
print("chiffres", Xp.shape, Xt.shape, "| clients", Xc.shape, Xct.shape, "| taux de résiliation",
      ↪ round(yc.mean(), 3))
```

```
chiffres (1347, 64) (450, 64) | clients (9000, 49) (3000, 49) | taux de résiliation 0.14
```

Applications

Application 8.1 — La courbe d'apprentissage selon le budget d'étiquettes

Objectif. Construire la courbe de référence du livre (section 8.1.5) et mesurer à quel point un tirage unique est trompeur.

Étape 1 : un seul tirage de 10 étiquettes. On choisit 10 images au hasard, on entraîne, on mesure sur le jeu de test.

```
rng = np.random.default_rng(0)
idx = rng.choice(len(Xp), 10, replace=False)
modele = LogisticRegression(max_iter=2000).fit(Xp[idx], yp[idx])
print("classes présentes parmi les 10 étiquettes :", len(set(yp[idx])), "sur 10 | précision sur le
↳ test :", round(modele.score(Xt, yt), 3))
```

```
classes présentes parmi les 10 étiquettes : 8 sur 10 | précision sur le test : 0.398
```

Étape 2 : répéter le tirage. Pour chaque budget, 10 tirages indépendants (graines 100 à 109). On garde la moyenne, l'écart-type, le minimum et le maximum.

```
budgets = [10, 20, 50, 100, 200, 400]
lignes = []
for nl in budgets:
    scores = []
    for s in range(10):
        i = np.random.default_rng(100 + s).choice(len(Xp), nl, replace=False)
        scores.append(LogisticRegression(max_iter=2000).fit(Xp[i], yp[i]).score(Xt, yt))
    lignes.append((nl, np.mean(scores), np.std(scores), np.min(scores), np.max(scores)))
tab = pd.DataFrame(lignes, columns=["étiquettes", "moyenne", "écart-type", "min", "max"]).round(3)
print(tab.to_string(index=False))
```

étiquettes	moyenne	écart-type	min	max
10	0.429	0.082	0.331	0.571
20	0.577	0.056	0.489	0.651
50	0.782	0.042	0.729	0.844
100	0.874	0.010	0.856	0.891
200	0.927	0.008	0.913	0.940
400	0.947	0.009	0.933	0.960

Étape 3 : combien de tirages faut-il ? L'erreur-type de la moyenne de n tirages est $\sigma/\sqrt{n}$. À 10 étiquettes, avec l'écart-type observé, combien de tirages pour que la moyenne soit connue à $\pm 0,01$ près (une erreur-type) ?

```
sigma10 = tab.loc[tab["étiquettes"] == 10, "écart-type"].iloc[0]
print("écart-type à 10 étiquettes :", sigma10, "| erreur-type de la moyenne de 10 tirages :",
↳ round(sigma10 / np.sqrt(10), 3))
print("tirages nécessaires pour une erreur-type de 0,01 :", int(np.ceil((sigma10 / 0.01) ** 2)))
```

```
écart-type à 10 étiquettes : 0.082 | erreur-type de la moyenne de 10 tirages : 0.026
tirages nécessaires pour une erreur-type de 0,01 : 68
```

Pour aller plus loin. Refaites la courbe avec un autre modèle, par exemple les 3 plus proches voisins, et comparez : la méthode de référence change-t-elle l'allure de la courbe ?

```
from sklearn.neighbors import KNeighborsClassifier
lignes = []
for nl in (10, 20, 50, 100, 200):
    sc_lr, sc_knn = [], []
    for s in range(10):
        i = np.random.default_rng(100 + s).choice(len(Xp), nl, replace=False)
```

```

sc_lr.append(LogisticRegression(max_iter=2000).fit(Xp[i], yp[i]).score(Xt, yt))
sc_knn.append(KNeighborsClassifier(3).fit(Xp[i], yp[i]).score(Xt, yt))
lignes.append((nl, np.mean(sc_lr), np.mean(sc_knn)))
print(pd.DataFrame(lignes, columns=["étiquettes", "logistique", "3
↪ voisins"]).round(3).to_string(index=False))

```

étiquettes	logistique	3 voisins
10	0.429	0.312
20	0.577	0.502
50	0.782	0.764
100	0.874	0.866
200	0.927	0.930

Application 8.2 — Auto-apprentissage : écrire la boucle, regarder les pseudo-étiquettes

Objectif. Écrire l'algorithme de la section 8.2.1 sans `SelfTrainingClassifier`, puis regarder, tour par tour, **combien** de pseudo-étiquettes sont ajoutées et **quelle part est juste**.

Étape 1 : la boucle. À chaque tour : on entraîne sur les points étiquetés, on prédit les non étiquetés, on ajoute ceux dont la confiance dépasse le seuil. On garde une trace de la précision des pseudo-étiquettes, que nous pouvons calculer car nous connaissons les vraies étiquettes (`yp`) : dans une vraie étude, ce serait impossible.

```

def auto_apprentissage(X, y_part, y_vrai, seuil=0.9, tours=10):
    y = y_part.copy()
    journal = []
    for t in range(1, tours + 1):
        lab = y != -1
        modele = LogisticRegression(max_iter=2000).fit(X[lab], y[lab])
        libres = np.where(~lab)[0]
        P = modele.predict_proba(X[libres])
        sur = P.max(axis=1) >= seuil
        if not sur.any():
            break
        ajoutes = libres[sur]
        y[ajoutes] = modele.classes_[P[sur].argmax(axis=1)]
        journal.append((t, len(ajoutes), (y[ajoutes] == y_vrai[ajoutes]).mean()))
    return LogisticRegression(max_iter=2000).fit(X[y != -1], y[y != -1]), journal

```

Étape 2 : un tirage de 50 étiquettes, seuil 0,7. (Au seuil 0,9, ce tirage ne contient aucun point assez sûr : la boucle n'ajouterait rien. Le seuil de 0,7 permet de voir le mécanisme à l'œuvre.)

```

rng = np.random.default_rng(100)
idx = rng.choice(len(Xp), 50, replace=False)
y_part = np.full(len(Xp), -1)
y_part[idx] = yp[idx]
modele_auto, journal = auto_apprentissage(Xp, y_part, yp, seuil=0.7)
seul = LogisticRegression(max_iter=2000).fit(Xp[idx], yp[idx])
print(pd.DataFrame(journal, columns=["tour", "pseudo-étiquettes ajoutées", "part
↪ juste"]).round(3).to_string(index=False))
print("précision test : auto-apprentissage", round(modele_auto.score(Xt, yt), 3), "| supervisé seul",
↪ round(seul.score(Xt, yt), 3))

```

tour	pseudo-étiquettes ajoutées	part juste
1	216	1.000
2	402	0.978
3	234	0.889
4	120	0.550
5	80	0.338
6	43	0.326

7	20	0.650
8	10	0.800
9	9	1.000
10	8	0.875

précision test : auto-apprentissage 0.802 | supervisé seul 0.831

Étape 3 : vérifier contre scikit-learn, puis balayer le seuil sur 10 tirages.

```
from sklearn.semi_supervised import SelfTrainingClassifier
lignes = []
for seuil in (0.7, 0.8, 0.9, 0.95):
    ma, sk, ba = [], [], []
    for s in range(10):
        i = np.random.default_rng(100 + s).choice(len(Xp), 50, replace=False)
        yp_ = np.full(len(Xp), -1); yp_[i] = yp[i]
        ma.append(auto_apprentissage(Xp, yp_, yp, seuil)[0].score(Xt, yt))
        sk.append(SelfTrainingClassifier(LogisticRegression(max_iter=2000), threshold=seuil).fit(Xp,
        ↪ yp_).score(Xt, yt))
        ba.append(LogisticRegression(max_iter=2000).fit(Xp[i], yp[i]).score(Xt, yt))
    lignes.append((seuil, np.mean(ma), np.mean(sk), np.mean(ba)))
print(pd.DataFrame(lignes, columns=["seuil", "ma boucle", "scikit-learn", "supervisé
↪ seul"]).round(3).to_string(index=False))
```

seuil	ma boucle	scikit-learn	supervisé seul
0.70	0.698	0.698	0.782
0.80	0.593	0.593	0.782
0.90	0.706	0.706	0.782
0.95	0.781	0.781	0.782

Lecture. Le journal montre l'histoire en deux temps. Aux **deux premiers tours**, la boucle ajoute 216 puis 402 pseudo-étiquettes, **justes à 100 % puis 98 %** : ce sont les cas faciles, très loin de la frontière. Mais dès le **quatrième tour**, les points ajoutés ne sont plus justes qu'à 55 %, puis 34 % et 33 % aux tours 5 et 6 : la boucle continue d'ajouter des points qu'elle se trompe à classer, qui deviennent des données d'entraînement. Le modèle final (0,802) est **moins bon** que le supervisé seul (0,831). C'est le biais de confirmation de la section 8.2.2. Dans le balayage des seuils, **ma boucle et celle de scikit-learn donnent exactement les mêmes précisions** (0,698 ; 0,593 ; 0,706 ; 0,781) et aucune ne dépasse le supervisé seul (0,782 en moyenne) : le seuil 0,95 le rejoint seulement parce qu'il n'ajoute presque rien.

Application 8.3 — La propagation d'étiquettes, du graphe à la solution fermée

Objectif. Calculer $F^* = (1 - \alpha)(I - \alpha S)^{-1}Y$ (section 8.2.4) sur un graphe de plus proches voisins, vérifier qu'on retrouve LabelSpreading, et **voir** pourquoi un trop petit nombre de voisins est catastrophique.

Étape 1 : la solution fermée sur un graphe de voisinage.

```
import scipy.sparse as sp
from scipy.sparse.linalg import spsolve
from sklearn.neighbors import kneighbors_graph

def propagation(X, y_part, k=7, alpha=0.2, nb_classes=10):
    W = kneighbors_graph(X, k, mode="connectivity", include_self=False)
    W = ((W + W.T) > 0).astype(float) # graphe symétrique
    d = np.asarray(W.sum(axis=1)).ravel()
    S = sp.diags(1 / np.sqrt(d)) @ W @ sp.diags(1 / np.sqrt(d))
    Y = np.zeros((len(X), nb_classes))
    lab = y_part != -1
    Y[lab, y_part[lab]] = 1
    return spsolve((sp.eye(len(X)) - alpha * S).tocsc(), (1 - alpha) * Y), W
```

Étape 2 : comparer à scikit-learn. Même tirage de 50 étiquettes qu'à l'application précédente.

```
from sklearn.semi_supervised import LabelSpreading
F, W = propagation(Xp, y_part, k=7, alpha=0.2)
ma_classe = F.argmax(axis=1)
sk = LabelSpreading(kernel="knn", n_neighbors=7, alpha=0.2, max_iter=200).fit(Xp, y_part)
libres = y_part == -1
print("précision sur les points non étiquetés : ma solution", round((ma_classe[libres] ==
↳ yp[libres]).mean(), 3),
      "| scikit-learn", round((sk.transduction_[libres] == yp[libres]).mean(), 3))
print("accord entre les deux sur les classes :", round((ma_classe == sk.transduction_).mean(), 3))
```

```
précision sur les points non étiquetés : ma solution 0.928 | scikit-learn 0.925
accord entre les deux sur les classes : 0.932
```

Étape 3 : le graphe est-il connexe ? Un graphe en plusieurs morceaux a des îlots que les étiquettes n'atteignent jamais. On compte les composantes connexes, et celles qui ne contiennent **aucune** étiquette.

```
from scipy.sparse.csgraph import connected_components
lignes = []
for k in (3, 5, 7, 15):
    F_k, W_k = propagation(Xp, y_part, k=k)
    nb_comp, comp = connected_components(W_k, directed=False)
    sans_etiquette = sum(1 for c in range(nb_comp) if not (y_part[comp == c] != -1).any())
    acc = (F_k.argmax(axis=1)[libres] == yp[libres]).mean()
    lignes.append((k, nb_comp, sans_etiquette, int((~np.isin(comp, np.unique(comp[y_part !=
↳ -1]))).sum()), acc))
print(pd.DataFrame(lignes, columns=["k", "composantes", "dont sans étiquette", "points non atteints",
↳ "précision (non étiquetés)"]).round(3).to_string(index=False))
```

k	composantes	dont sans étiquette	points non atteints	précision (non étiquetés)
3	2	0	0	0.915
5	2	0	0	0.922
7	1	0	0	0.928
15	1	0	0	0.924

Lecture. Notre solution fermée et LabelSpreading s'accordent sur 93 % des classes et donnent des précisions voisines sur les points non étiquetés (0,928 contre 0,925). L'écart vient du **graphe** : le nôtre est symétrisé, celui de scikit-learn ne l'est pas. Cela change tout pour les petits k : notre graphe est **connexe dès** $k = 7$ (une seule composante), et même à $k = 3$ ou $k = 5$ il n'a que deux composantes, **toutes deux contenant des étiquettes** (aucune sans étiquette, aucun point non atteint). Du coup, avec 50 étiquettes, notre implémentation atteint **0,915 dès** $k = 3$. L'effondrement de LabelSpreading à $k = 3$ (0,29 avec 20 étiquettes, section 8.2.5 du livre) tient donc au graphe **orienté** de scikit-learn, où 86 % des points ne reçoivent aucun score, et non à un manque de voisins en soi. **Leçon : symétrisez le graphe**, ou prenez $k \geq 7$.

Application 8.4 — Co-apprentissage : un essai instructif

Objectif. Tester le co-apprentissage de la section 8.2.3 sur deux « vues » d'un chiffre, la moitié haute et la moitié basse de l'image, et constater que les **hypothèses** de la méthode ne sont pas réunies.

Étape 1 : les deux vues, et leur valeur individuelle. Une image 8×8 est stockée sous forme de 64 pixels ; les 32 premiers sont les 4 lignes du haut, les 32 suivantes celles du bas. Avec 100 étiquettes, que vaut chaque moitié seule ? Et les deux ensemble ?

```
vue1, vue2 = slice(0, 32), slice(32, 64)
rng = np.random.default_rng(100)
idx = rng.choice(len(Xp), 100, replace=False)
```

```

a1 = LogisticRegression(max_iter=2000).fit(Xp[idx][:, vue1], yp[idx]).score(Xt[:, vue1], yt)
a2 = LogisticRegression(max_iter=2000).fit(Xp[idx][:, vue2], yp[idx]).score(Xt[:, vue2], yt)
a12 = LogisticRegression(max_iter=2000).fit(Xp[idx], yp[idx]).score(Xt, yt)
print("moitié haute seule :", round(a1, 3), "| moitié basse seule :", round(a2, 3), "| image entière
↪ :", round(a12, 3))

```

```
moitié haute seule : 0.72 | moitié basse seule : 0.722 | image entière : 0.864
```

Étape 2 : la boucle de co-apprentissage. À chaque tour, chaque modèle étiquette les 10 points non étiquetés dont il est le plus sûr, et ces étiquettes sont ajoutées au jeu **commun**. La prédiction finale multiplie les probabilités des deux modèles.

```

def co_apprentissage(X, idx, y, tours=10, ajout=10):
    etiq = {int(i): int(y[i]) for i in idx}
    for _ in range(tours):
        for v in (vue1, vue2):
            L = np.array(sorted(etiq))
            m = LogisticRegression(max_iter=2000).fit(X[L][:, v], [etiq[i] for i in L])
            U = np.array([i for i in range(len(X)) if i not in etiq])
            P = m.predict_proba(X[U][:, v])
            meilleurs = np.argsort(-P.max(axis=1))[:ajout]
            for j in meilleurs:
                etiq[int(U[j])] = int(m.classes_[P[j].argmax()])
    return etiq

etiq = co_apprentissage(Xp, idx[:30], yp) # on part de 30 étiquettes seulement
ajoutes = [i for i in etiq if i not in set(idx[:30])]
print("pseudo-étiquettes ajoutées :", len(ajoutes), "| part juste :", round(np.mean([etiq[i] == yp[i]
↪ for i in ajoutes]), 3))

```

```
pseudo-étiquettes ajoutées : 200 | part juste : 0.305
```

Étape 3 : l'effet final.

```

L = np.array(sorted(etiq))
yl = np.array([etiq[i] for i in L])
m1 = LogisticRegression(max_iter=2000).fit(Xp[L][:, vue1], yl)
m2 = LogisticRegression(max_iter=2000).fit(Xp[L][:, vue2], yl)
pred = m1.classes_[m1.predict_proba(Xt[:, vue1]) * m2.predict_proba(Xt[:, vue2])).argmax(axis=1)]
base = LogisticRegression(max_iter=2000).fit(Xp[idx[:30]], yp[idx[:30]]).score(Xt, yt)
print("précision test : co-apprentissage", round((pred == yt).mean(), 3), "| supervisé seul avec les
↪ mêmes 30 étiquettes", round(base, 3))

```

```
précision test : co-apprentissage 0.376 | supervisé seul avec les mêmes 30 étiquettes 0.671
```

Lecture. Chaque moitié d'image, avec 100 étiquettes, donne environ **0,72** de précision, contre **0,86** pour l'image entière : chaque vue est **informative mais insuffisante**. Avec seulement 30 étiquettes, c'est pire : les pseudo-étiquettes ajoutées (200) ne sont justes que dans **30,5 %** des cas, et le co-apprentissage finit à **0,376**, contre **0,671** pour le supervisé seul. Trois raisons : (1) aucune vue ne **suffit** à elle seule ; (2) les deux vues ne sont pas **indépendantes sachant la classe** : elles décrivent le même tracé, et quand l'une se trompe, l'autre se trompe souvent de la même façon, au lieu de la corriger ; (3) 200 pseudo-étiquettes à 30 % de justesse **noient** les 30 vraies étiquettes. Les hypothèses de la section 8.2.3 sont des conditions, pas des détails.

Application 8.5 — Quand les graphes échouent : et si l'on choisissait mieux les variables ?

Objectif. Reprendre le diagnostic de la section 8.2.6 (homophilie) sur les clients, et tester une idée : construire le graphe non sur les 49 variables, mais sur six variables de comportement.

Étape 1 : l'homophilie selon les variables retenues.

```
from sklearn.neighbors import NearestNeighbors
c = pd.read_csv("donnees/clients_ml.csv")
colonnes = ["recence_jours", "nb_commandes_12m", "satisfaction_moy", "nb_tickets_support_12m",
            ↪ "nb_retours_12m", "part_achats_promo"]
X6 = c[colonnes].copy()
X6["satisfaction_moy"] = X6["satisfaction_moy"].fillna(X6["satisfaction_moy"].median())
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
A6, B6, a6, b6 = train_test_split(X6.to_numpy(), c["churn_90j"].to_numpy(), test_size=0.25,
            ↪ random_state=0, stratify=c["churn_90j"])
A6 = StandardScaler().fit(A6).transform(A6)

def homophilie(X, y, k=7):
    voisins = NearestNeighbors(n_neighbors=k + 1).fit(X).kneighbors(X, return_distance=False)[: , 1:]
    return (y[voisins] == y[:, None]).mean(), y[voisins[y == 1]].mean()

for nom, (X_, y_) in {"49 variables": (Xc, yc), "6 variables de comportement": (A6, a6)}.items():
    h, h1 = homophilie(X_, y_)
    print(f"{nom:28s} voisins de même étiquette : {h:.3f} | parmi les résiliations, voisins qui
            ↪ résilient : {h1:.3f}")
```

```
49 variables          voisins de même étiquette : 0.805 | parmi les résiliations, voisins qui
↪ résilient : 0.269
6 variables de comportement voisins de même étiquette : 0.822 | parmi les résiliations, voisins qui
↪ résilient : 0.358
```

Étape 2 : la propagation sur ces deux graphes. Même protocole qu'au livre (étiquettes tirées en respectant la proportion de résiliations ; AUC sur les clients non étiquetés atteints), 6 tirages.

```
from sklearn.semi_supervised import LabelSpreading
from sklearn.metrics import roc_auc_score

def auc_propagation(X, y, nl, graines=6):
    res = []
    for s in range(graines):
        e = o.tirer_etiquettes(y, nl, np.random.default_rng(10 + s), stratifie=True)
        y_p = np.full(len(X), -1); y_p[e] = y[e]
        D = LabelSpreading(kernel="knn", n_neighbors=10, alpha=0.2, max_iter=100).fit(X,
            ↪ y_p).label_distributions_
        z = D.sum(axis=1)
        ok = (y_p == -1) & (z > 0)
        sup = LogisticRegression(max_iter=2000).fit(X[ok], y[ok]).predict_proba(X[ok])[:, 1]
        res.append((roc_auc_score(y[ok], D[ok, 1] / z[ok]), roc_auc_score(y[ok], sup)))
    return np.mean(res, axis=0)

for nl in (100, 300):
    for nom, (X_, y_) in {"49 variables": (Xc, yc), "6 variables": (A6, a6)}.items():
        p, s_ = auc_propagation(X_, y_, nl)
        print(f"{nl:3d} étiquettes, graphe sur {nom:12s} : AUC propagation {p:.3f} | AUC supervisé
            ↪ (mêmes points) {s_:.3f}")
```

```
100 étiquettes, graphe sur 49 variables : AUC propagation 0.527 | AUC supervisé (mêmes points) 0.727
100 étiquettes, graphe sur 6 variables : AUC propagation 0.646 | AUC supervisé (mêmes points) 0.782
300 étiquettes, graphe sur 49 variables : AUC propagation 0.577 | AUC supervisé (mêmes points) 0.787
300 étiquettes, graphe sur 6 variables : AUC propagation 0.681 | AUC supervisé (mêmes points) 0.810
```

Lecture. Avec six variables de comportement, le graphe est un peu plus « propre » : la part de voisins de même étiquette passe de 0,805 à 0,822, et surtout, parmi les clients qui résilient, la part de voisins qui résilient aussi passe de 0,269 à **0,358** (pour un taux de base de 0,14). La propagation en profite : son AUC passe de **0,527 à 0,646** (100 étiquettes) et de **0,577 à 0,681** (300). **Choisir les variables qui comptent aide beaucoup**, mais cela ne suffit pas : le modèle supervisé, évalué sur les mêmes clients, atteint **0,782 et 0,810**. Ici, la classe dépend de **seuils** et d'**interactions** (une récence supérieure à un seuil *et* une satisfaction basse, par exemple), qu'un graphe de proximité ne sait pas exprimer, alors qu'un modèle supervisé le peut.

Application 8.6 — La boucle d'apprentissage actif, écrite à la main

Objectif. Écrire la boucle de la section 8.3.1 avec la stratégie de la marge, et la comparer au tirage au hasard sur les chiffres.

Étape 1 : la boucle. Elle part de n_0 étiquettes tirées au hasard, puis demande lot étiquettes à chaque tour, jusqu'au budget.

```
def boucle(X, y, Xtest, ytest, strategie, n0=10, lot=10, budget=100, graine=0):
    rng = np.random.default_rng(graine)
    etiq = list(rng.choice(len(X), n0, replace=False))
    courbe = []
    while True:
        m = LogisticRegression(max_iter=1000).fit(X[etiq], y[etiq])
        courbe.append((len(etiq), m.score(Xtest, ytest)))
        if len(etiq) >= budget:
            return np.array(courbe)
        libres = np.setdiff1d(np.arange(len(X)), etiq)
        if strategie == "aleatoire":
            choix = rng.choice(libres, lot, replace=False)
        else:
            P = np.zeros((len(libres), 10))
            P[:, m.classes_] = m.predict_proba(X[libres])          # les classes jamais vues ont la
↪ probabilité 0
            tri = np.sort(P, axis=1)
            choix = libres[np.argsort(tri[:, -1] - tri[:, -2])[:lot]] # petite marge = grande
↪ hésitation
            etiq += [int(i) for i in choix]
```

Étape 2 : dix tirages par stratégie.

```
res = {st: np.array([boucle(Xp, yp, Xt, yt, st, graine=s)[:, 1] for s in range(10)]) for st in
↪ ("aleatoire", "marge")}
budgets = list(range(10, 101, 10))
tab = pd.DataFrame({"étiquettes": budgets, "aléatoire": res["aleatoire"].mean(axis=0), "marge":
↪ res["marge"].mean(axis=0)}).round(3)
print(tab.iloc[[1, 2, 4, 5, 9]].to_string(index=False))
d = res["marge"][:, 5] - res["aleatoire"][:, 5]
print("à 60 étiquettes : écart moyen", round(d.mean(), 3), "| tirages où la marge gagne :", int((d >
↪ 0).sum()), "sur 10")
```

étiquettes	aléatoire	marge
20	0.572	0.584
30	0.646	0.727
50	0.786	0.833
60	0.834	0.874
100	0.888	0.927

à 60 étiquettes : écart moyen 0.04 | tirages où la marge gagne : 8 sur 10

Pour aller plus loin : la taille du lot. Demander les étiquettes **une à une** est le plus efficace, mais le moins pratique (un réentraînement par étiquette). Que perd-on avec des lots plus gros ? (5 tirages, budget 100.)

```
lignes = []
for lot in (1, 5, 10, 25):
    r = [boucle(Xp, yp, Xt, yt, "marge", lot=lot, budget=100, graine=s)[-1, 1] for s in range(5)]
    lignes.append((lot, np.mean(r)))
print(pd.DataFrame(lignes, columns=["taille du lot", "précision à 100
↳ étiquettes"]).round(3).to_string(index=False))
```

taille du lot	précision à 100 étiquettes
1	0.928
5	0.928
10	0.926
25	0.931

Lecture. Notre boucle retrouve le résultat du livre : la marge dépasse le hasard à 30 étiquettes (0,727 contre 0,646), à 60 (0,874 contre 0,834) et à 100 (0,927 contre 0,888). Mais l'**écart à 60 étiquettes est de 4 points** (la marge gagne dans 8 tirages sur 10), contre 7 points et 10 tirages sur 10 dans le livre : ici le jeu initial est tiré avec le même générateur que les tirages aléatoires, donc les 10 tirages ne sont pas les mêmes. Le **sens** du résultat est stable ; son **amplitude** dépend du détail du protocole, d'où l'intérêt de répéter. Pour la taille du lot, **rien de mesurable** : 0,928 (lots de 1 ou de 5), 0,926 (10), 0,931 (25), des différences très inférieures à la variabilité entre tirages (5 tirages seulement). À ce budget, on peut demander les étiquettes par gros lots sans rien perdre.

Application 8.7 — Comité et densité

Objectif. Écrire à la main les deux critères des sections 8.3.3 et 8.3.4, puis les faire tourner dans la même boucle (o.boucle_active est la boucle de l'application précédente, avec ces critères branchés dedans).

Étape 1 : les deux scores.

```
def score_comite(Xl, yl, Xlibres, nb_membres=5, graine=0):
    rng = np.random.default_rng(graine)
    votes = []
    for _ in range(nb_membres):
        b = rng.choice(len(Xl), len(Xl), replace=True)
        votes.append(LogisticRegression(max_iter=1000).fit(Xl[b], yl[b]).predict(Xlibres))
    votes = np.array(votes)
    freq = np.stack([(votes == k).mean(axis=0) for k in range(10)], axis=1)
    return -(freq * np.log(freq + 1e-12)).sum(axis=1)

def densite(X, nb_ref=400, graine=0):
    ref = X[np.random.default_rng(graine).choice(len(X), nb_ref, replace=False)]
    A = X / np.linalg.norm(X, axis=1, keepdims=True); B = ref / np.linalg.norm(ref, axis=1,
↳ keepdims=True)
    return (A @ B.T).mean(axis=1)

m = LogisticRegression(max_iter=1000).fit(Xp[:30], yp[:30])
print("désaccord maximal du comité sur 5 modèles :", round(score_comite(Xp[:30], yp[:30], Xp[30:],
↳ 5).max(), 3), "(ln 5 = 1,609)")
print("densité : minimum", round(densite(Xp).min(), 3), "| médiane", round(np.median(densite(Xp)),
↳ 3), "| maximum", round(densite(Xp).max(), 3))
```

```
désaccord maximal du comité sur 5 modèles : 1.609 (ln 5 = 1,609)
densité : minimum 0.548 | médiane 0.689 | maximum 0.788
```

Étape 2 : l'effet du nombre de membres et de l'exposant de densité. (5 tirages, 10 étiquettes de départ, lots de 10, budget 60.)

```
def precision_a_60(strategie, **kw):
    r = []
```

```

for s in range(5):
    idx0 = o.tirer_etiquettes(yp, 10, np.random.default_rng(500 + s))
    r.append(o.boucle_active(Xp, yp, Xt, yt, strategie, idx0, 10, 60, graine=s, k=10, **kw)[-1, 1])
return np.mean(r)

lignes = [("aléatoire", precision_a_60("aleatoire")), ("marge", precision_a_60("marge"))]
for M in (3, 5, 15):
    lignes.append(("comité de {M} modèles", precision_a_60("comite", nb_comite=M)))
for beta in (0.0, 1.0, 2.0):
    lignes.append(("entropie × densité^{beta:g}", precision_a_60("densite", beta=beta)))
print(pd.DataFrame(lignes, columns=["stratégie", "précision à 60
↳ étiquettes"]).round(3).to_string(index=False))

```

stratégie	précision à 60 étiquettes
aléatoire	0.778
marge	0.880
comité de 3 modèles	0.828
comité de 5 modèles	0.823
comité de 15 modèles	0.860
entropie × densité ⁰	0.743
entropie × densité ¹	0.731
entropie × densité ²	0.712

Lecture. À 60 étiquettes (5 tirages) : tirage au hasard **0,778**, marge **0,880**. Le comité est entre les deux : **0,828** avec 3 modèles, **0,823** avec 5, **0,860** avec 15. Plus de membres aident plutôt (15 contre 3 ou 5), avec un coût proportionnel et un bruit d'échantillonnage non négligeable sur 5 tirages. Côté densité, l'exposant $\beta = 0$ revient à l'entropie seule (**0,743**, sous le hasard) ; $\beta = 1$ donne 0,731 et $\beta = 2$ donne 0,712 : sur les chiffres, **plus on pondère par la densité, plus c'est mauvais**, ce qui confirme le constat du livre (8.3.5).

Application 8.8 — Biais d'échantillonnage, démarrage à froid et coût

Objectif. Sur les clients de la boutique, reproduire le biais de la section 8.3.6, le corriger par un échantillon aléatoire, puis chiffrer le tout en euros.

Étape 1 : la stratégie d'incertitude sur les clients. 5 tirages, 40 étiquettes de départ, lots de 20, budget 300 ; le modèle final de chaque tirage est conservé.

```

resultats = {}
for st in ("aleatoire", "incertitude"):
    lignes = []
    for s in range(5):
        idx0 = o.tirer_etiquettes(yc, 40, np.random.default_rng(700 + s), stratifie=True)
        courbe, etiq = o.boucle_active(Xc, yc, Xct, yct, st, idx0, 20, 300, graine=s, k=2,
↳ metrique="auc", renvoyer_etiq=True)
        m = LogisticRegression(max_iter=1000).fit(Xc[etiq], yc[etiq])
        lignes.append((courbe[-1, 1], yc[etiq].mean(), m.predict_proba(Xct)[: , 1].mean(), m, etiq))
    resultats[st] = lignes
    print(f"{st:12s} AUC {np.mean([l[0] for l in lignes]):.3f} | part de résiliations parmi les
↳ étiquetés {np.mean([l[1] for l in lignes]):.3f} | probabilité moyenne prédite {np.mean([l[2]
↳ for l in lignes]):.3f}")
print("taux réel dans le test :", round(yct.mean(), 3))

```

aleatoire	AUC 0.813 part de résiliations parmi les étiquetés 0.148 probabilité moyenne prédite
↳	0.142
incertitude	AUC 0.829 part de résiliations parmi les étiquetés 0.371 probabilité moyenne prédite
↳	0.077
taux réel dans le test : 0.14	

Étape 2 : recalibrer avec 200 étiquettes aléatoires de plus. On ajuste une régression logistique à une variable sur le score du modèle (mise à l'échelle de Platt).

```

from sklearn.metrics import brier_score_loss
lignes = []
for st, res in resultats.items():
    av, ap, ba, bp = [], [], [], []
    for s, (_, _, _, m, etiq) in enumerate(res):
        libres = np.setdiff1d(np.arange(len(Xc)), etiq)
        cal = np.random.default_rng(900 + s).choice(libres, 200, replace=False)
        platt = LogisticRegression(C=1e6).fit(m.decision_function(Xc[cal]).reshape(-1, 1), yc[cal])
        p0 = m.predict_proba(Xct)[: , 1]
        p1 = platt.predict_proba(m.decision_function(Xct).reshape(-1, 1))[:, 1]
        av.append(p0.mean()); ap.append(p1.mean()); ba.append(brier_score_loss(yct, p0));
        bp.append(brier_score_loss(yct, p1))
    lignes.append((st, np.mean(av), np.mean(ap), np.mean(ba), np.mean(bp)))
print(pd.DataFrame(lignes, columns=["stratégie", "proba moyenne avant", "après", "Brier avant",
    "après"]).round(4).to_string(index=False))

```

stratégie	proba moyenne avant	après	Brier avant	après
aleatoire	0.1419	0.1394	0.1078	0.0994
incertitude	0.0771	0.1293	0.1079	0.0957

Étape 3 : le démarrage à froid, en nombre de classes couvertes. Combien de chiffres différents contiennent 10 images tirées au hasard, comparé aux 10 médoïdes d'un k-means ?

```

rng = np.random.default_rng(0)
alea = np.mean([len(set(yp[rng.choice(len(Xp), 10, replace=False)])) for _ in range(1000)])
med = np.mean([len(set(yp[o.init_medoides(Xp, 10, graine=s)])) for s in range(10)])
print("classes couvertes : tirage aléatoire", round(alea, 2), "| médoïdes", round(med, 2))

```

```
classes couvertes : tirage aléatoire 6.54 | médoïdes 9.2
```

Étape 4 : le coût. Avec 4 € par étiquette, 200 étiquettes de calibrage et une économie de s étiquettes grâce à la stratégie active, à partir de quelle économie s l'opération est-elle rentable si l'on a besoin de probabilités calibrées ? Et si l'on ne veut que classer ?

```

cout_etiquette, etiquettes_calibrage = 4.0, 200
print("économie minimale pour amortir le calibrage :", etiquettes_calibrage, "étiquettes (",
    cout_etiquette * etiquettes_calibrage, "€ )")
print("si l'on ne veut que classer : toute économie est bonne à prendre (aucun calibrage nécessaire)")

```

```
économie minimale pour amortir le calibrage : 200 étiquettes ( 800.0 € )
si l'on ne veut que classer : toute économie est bonne à prendre (aucun calibrage nécessaire)
```

Lecture. Biais. Avec 300 étiquettes (5 tirages), l'AUC est de 0,829 pour l'incertitude et de 0,813 pour le hasard. Mais la part de résiliations parmi les clients étiquetés est de **0,371** (contre 0,148 pour le hasard) et la probabilité moyenne prédite est de **0,077** (contre 0,142 pour le hasard) alors que le taux réel est de 0,14. **Recalibrage.** Avec seulement 200 étiquettes aléatoires de plus, la probabilité moyenne du modèle actif remonte à **0,129** (pas tout à fait 0,14 : 200 étiquettes sont peu ; le livre en utilise 300 et atteint 0,141), et son score de Brier passe de 0,1079 à **0,0957**, meilleur que celui du modèle aléatoire recalibré (0,0994). **Démarrage à froid.** 10 images aléatoires couvrent 6,5 chiffres sur 10, 10 médoïdes en couvrent 9,2. **Coût.** À 4 € l'étiquette, le calibrage de 200 étiquettes coûte 800 € : le choix actif doit donc économiser *plus de 200 étiquettes* pour être rentable si l'on a besoin de probabilités fiables ; s'il ne s'agit que de classer, le calibrage est inutile et toute économie est bonne à prendre.

Exercices

Exercice 8.1 ★ — Une étape d'EM dur à la main (section 8.1.2)

Une seule variable x , deux classes A et B. Deux points étiquetés : A en -1 , B en $+0,4$. Six points sans étiquette : $-2,2, -1,9, -1,5, +1,2, +1,6, +2,1$. (a) Où est la frontière « milieu des deux points étiquetés » ? (b) Étiquetez provisoirement les six points, recalculez les centres, puis la frontière. (c) Une deuxième étape change-t-elle quelque chose ?

Exercice 8.2 ★★ — Combien de tirages ? (section 8.1.5)

À 10 étiquettes, la précision d'un modèle supervisé a un écart-type de 0,082 d'un tirage à l'autre. (a) Quelle est l'erreur-type de la moyenne de 10 tirages ? (b) Combien de tirages faut-il pour connaître la moyenne à $\pm 0,01$ (une erreur-type) ? (c) Pourquoi une comparaison **appariée** (mêmes étiquettes pour les deux méthodes) demande-t-elle en général moins de tirages ?

Exercice 8.3 ★★ — Quelle hypothèse est en jeu ? (section 8.1.3)

Pour chaque situation, dites laquelle des quatre hypothèses (lissage, groupes, basse densité, variété) est en jeu, et si elle est plausible. (a) Classer des photos de produits en défaut ou non, avec 30 photos étiquetées et 20 000 sans étiquette, les photos de produits défectueux formant un groupe visuellement distinct. (b) Prédire la résiliation des clients à partir de 49 variables de natures très différentes. (c) Reconnaître des gestes de la main à partir de capteurs : les gestes forment des trajectoires continues dans un espace de grande dimension. (d) Séparer deux populations dont les distributions se chevauchent presque entièrement.

Exercice 8.4 ★ — Pseudo-étiquettes et erreurs attendues (section 8.2.1)

Un modèle produit, pour cinq points sans étiquette, les confiances suivantes : 0,97, 0,93, 0,91, 0,88, 0,55. (a) Avec un seuil de 0,9, quels points sont pseudo-étiquetés ? (b) Si ces confiances étaient de vraies probabilités, combien d'erreurs attend-on parmi les points ajoutés ? (c) Sur les chiffres, au seuil 0,8, la part de pseudo-étiquettes justes était de 0,73 (livre, 8.2.1). Que dit-elle de la calibration du modèle ?

Exercice 8.5 ★★ — Propagation sur quatre nœuds (section 8.2.4)

Chaîne de quatre nœuds $1 - 2 - 3 - 4$ (arêtes 1-2, 2-3, 3-4, poids 1). Le nœud 1 est étiqueté A, le nœud 4 est étiqueté B. Avec $\alpha = 0,5$: (a) calculez les degrés et la matrice S ; (b) calculez le premier tour de la diffusion pour la colonne A ; (c) calculez la solution fermée F^* (à la machine) et les classes ; (d) expliquez le résultat.

Exercice 8.6 ★★★ — Pourquoi la propagation converge (section 8.2.4)

Montrez que les valeurs propres de $S = D^{-1/2}WD^{-1/2}$ sont de module au plus 1, puis que la récurrence $F^{(t+1)} = \alpha SF^{(t)} + (1 - \alpha)Y$ converge vers $(1 - \alpha)(I - \alpha S)^{-1}Y$ pour tout $\alpha \in]0, 1[$.

Exercice 8.7 ★ — Trois critères, un classement (section 8.3.2)

Trois candidats, trois classes : $a = (0,7; 0,2; 0,1)$, $b = (0,4; 0,4; 0,2)$, $c = (0,34; 0,33; 0,33)$. Calculez pour chacun la confiance minimale, l'écart entre les deux meilleures classes et l'entropie, puis classez-les du plus utile au moins utile selon chaque critère.

Exercice 8.8 ★★ — Équivalence à deux classes (section 8.3.2)

Pour **deux** classes, avec $p = P(1 | x)$, montrez que la confiance minimale, la marge et l'entropie classent les points dans le même ordre.

Exercice 8.9 ★★ — Entropie du vote (section 8.3.3)

Un comité de 7 modèles vote entre 3 classes. Calculez l'entropie du vote pour les répartitions $(7, 0, 0)$, $(4, 2, 1)$ et $(3, 3, 1)$ et classez-les.

Exercice 8.10 ★★ — Un test des signes (section 8.3.5)

Sur 10 tirages, la marge fait mieux que le hasard à 60 étiquettes dans les 10 tirages (livre, 8.3.5). Sous l'hypothèse « aucune différence », chaque tirage donne l'avantage à l'une ou l'autre stratégie avec probabilité $1/2$. Quelle est la probabilité d'un résultat aussi extrême (10 sur 10 dans un sens ou dans l'autre) ? Que conclure ? Quelles sont les limites de ce raisonnement ?

Exercice 8.11 ★★★ — Corriger le biais d'échantillonnage par les proportions ? (section 8.3.6)

Un modèle est entraîné sur un jeu où la part de positifs est π_s alors que la vraie part est π . (a) Si le jeu a été obtenu en **sur-échantillonnant** les positifs, indépendamment des variables x , montrez que les cotes (*odds*) du modèle sont à corriger d'un facteur $\frac{\pi/(1-\pi)}{\pi_s/(1-\pi_s)}$. (b) Vérifiez-le par simulation sur les clients. (c) Appliquez la même correction au modèle entraîné par **apprentissage actif** (stratégie d'incertitude, 400 étiquettes, 38 % de positifs). Que constatez-vous ? Expliquez.

Exercice 8.12 ★★ — Quand l'apprentissage actif est-il rentable ? (section 8.3.8)

Mettre en place une boucle active coûte, une fois, 1 000 € d'ingénierie. Chaque projet économise ensuite 60 étiquettes à 4 € l'unité. (a) Combien de projets pour amortir l'investissement ? (b) Même question si chaque étiquette ne coûte que 0,40 € et que l'économie est de 50 étiquettes. (c) Quel autre coût faut-il compter si l'on a besoin de probabilités calibrées ?

Corrigés**Corrigé 8.1**

(a) La frontière est au milieu de -1 et $0,4$: $\frac{-1 + 0,4}{2} = -0,3$.

(b) Les trois points de gauche $(-2,2, -1,9, -1,5)$ sont à gauche de $-0,3$: provisoirement A.

Les trois points de droite sont B. Nouveaux centres : $\bar{x}_A = \frac{-1 - 2,2 - 1,9 - 1,5}{4} = -1,65$ et $\bar{x}_B = \frac{0,4 + 1,2 + 1,6 + 2,1}{4} = 1,325$. Nouvelle frontière : $\frac{-1,65 + 1,325}{2} = -0,1625$.

- (c) Aucun point ne change de côté de $-0,1625$: l'étape est **stable**, l'algorithme a convergé. La frontière est passée de $-0,3$ à $-0,1625$, soit plus près du creux (autour de 0 si les classes sont symétriques).

```
a, b = -1.0, 0.4
U = np.array([-2.2, -1.9, -1.5, 1.2, 1.6, 2.1])
f = (a + b) / 2
for tour in range(1, 4):
    A = [a] + [u for u in U if u < f]
    B = [b] + [u for u in U if u >= f]
    f_nouveau = (np.mean(A) + np.mean(B)) / 2
    print("tour", tour, ":", "centres", round(np.mean(A), 4), round(np.mean(B), 4), "| frontière",
          round(f_nouveau, 4))
    f = f_nouveau
```

```
tour 1 : centres -1.65 1.325 | frontière -0.1625
tour 2 : centres -1.65 1.325 | frontière -0.1625
tour 3 : centres -1.65 1.325 | frontière -0.1625
```

Corrigé 8.2

- (a) $0,082/\sqrt{10} \approx 0,026$. (b) On veut $0,082/\sqrt{n} \leq 0,01$, soit $n \geq (0,082/0,01)^2 = 67,24$: **68 tirages**. (c) Quand les deux méthodes reçoivent les **mêmes étiquettes**, la variabilité due au tirage affecte les deux de la même façon et **s'annule en grande partie dans la différence** : l'écart-type de la *différence* est bien plus faible que celui de chaque précision, donc moins de tirages suffisent pour détecter un écart.

```
print("erreur-type pour 10 tirages :", round(0.082 / np.sqrt(10), 3), "| tirages pour 0,01 :",
      int(np.ceil((0.082 / 0.01) ** 2)))
```

```
erreur-type pour 10 tirages : 0.026 | tirages pour 0,01 : 68
```

Corrigé 8.3

- (a) **Groupes** (et basse densité) : les défauts forment un groupe distinct, la frontière passe dans le creux : plausible, le semi-supervisé a de bonnes chances d'aider. (b) **Lissage** : « deux clients proches ont la même réaction » est **douteux** quand les variables sont de natures très différentes et que la classe dépend de seuils et d'interactions (livre, 8.2.6). Prudence. (c) **Variété** : des trajectoires continues dans un grand espace, le graphe de voisinage est adapté : plausible. (d) **Aucune** n'est vérifiée : il n'y a pas de creux, la forme de $p(x)$ ne renseigne pas sur la frontière : le semi-supervisé n'apportera rien (voire nuira).

Corrigé 8.4

- (a) Aux confiances $\geq 0,9$: les trois premiers points (0,97, 0,93, 0,91). (b) Si ces confiances étaient exactes, les erreurs attendues seraient $(1-0,97) + (1-0,93) + (1-0,91) = 0,03 + 0,07 + 0,09 = 0,19$ sur 3 points, soit **6,3 %**. (c) Au seuil 0,8, le modèle annonce **au moins 80 %** de confiance sur chaque pseudo-étiquette, donc il devrait avoir raison au moins 80 % du temps. Il n'a raison que dans 73 % des cas : il est **trop sûr de lui** (mal calibré, section 5.2), ce qui explique le biais de confirmation.

Corrigé 8.5

(a) Degrés : $d = (1, 2, 2, 1)$. $S_{12} = \frac{1}{\sqrt{1 \cdot 2}} \approx 0,7071$, $S_{23} = \frac{1}{\sqrt{2 \cdot 2}} = 0,5$, $S_{34} = \frac{1}{\sqrt{2 \cdot 1}} \approx 0,7071$.

(b) Colonne A, $F^{(0)} = (1, 0, 0, 0)$: nœud 1 : $0,5 \cdot 1 = 0,5$ (le terme de voisinage est nul au départ) ; nœud 2 : $0,5 \cdot 0,7071 \cdot 1 \approx 0,354$; nœuds 3 et 4 : 0.

```
W = np.array([[0, 1, 0, 0], [1, 0, 1, 0], [0, 1, 0, 1], [0, 0, 1, 0]], float)
d = W.sum(axis=1)
S = W / np.sqrt(np.outer(d, d))
Y = np.zeros((4, 2)); Y[0, 0] = 1; Y[3, 1] = 1
al = 0.5
F1 = al * S @ Y + (1 - al) * Y
Fs = (1 - al) * np.linalg.inv(np.eye(4) - al * S) @ Y
print("S =\n", S.round(4)); print("tour 1, colonne A :", F1[:, 0].round(3))
print("F* (A, B) :\n", Fs.round(3)); print("classes :", np.array(list("AB"))[Fs.argmax(axis=1)])
```

```
S =
[[0.    0.7071 0.    0.   ]
 [0.7071 0.    0.5   0.   ]
 [0.    0.5   0.    0.7071]
 [0.    0.    0.7071 0.   ]]
tour 1, colonne A : [0.5  0.354 0.    0.   ]
F* (A, B) :
[[0.578 0.022]
 [0.22  0.063]
 [0.063 0.22 ]
 [0.022 0.578]]
classes : ['A' 'A' 'B' 'B']
```

(c) et (d) : voir la sortie ci-dessus. La chaîne étant **symétrique**, les nœuds 2 et 3 sont les symétriques l'un de l'autre : le nœud 2, plus proche de 1, reçoit A ; le nœud 3, plus proche de 4, reçoit B ; la frontière est exactement au milieu de l'arête 2-3.

Corrigé 8.6

Valeurs propres de S . Posons $P = D^{-1}W$: ses entrées sont positives et chaque ligne somme à 1 (matrice **stochastique**). Si $Pv = \lambda v$ et i est l'indice de la composante de v de plus grand module, $|\lambda||v_i| = |\sum_j P_{ij}v_j| \leq \sum_j P_{ij}|v_j| \leq |v_i|$, donc $|\lambda| \leq 1$. Or $S = D^{1/2}PD^{-1/2}$ est **semblable** à P : mêmes valeurs propres. Donc toutes celles de S sont dans $[-1, 1]$.

Convergence. Le rayon spectral de αS est au plus $\alpha < 1$, donc $(\alpha S)^t \rightarrow 0$. En dépliant, $F^{(t)} = (\alpha S)^t Y + (1 - \alpha) \sum_{s=0}^{t-1} (\alpha S)^s Y$. Le premier terme tend vers 0, et la série de Neumann $\sum_s (\alpha S)^s$ converge vers $(I - \alpha S)^{-1}$ (car $I - \alpha S$ est inversible, ses valeurs propres étant $1 - \alpha\lambda \geq 1 - \alpha > 0$). D'où $F^{(t)} \rightarrow (1 - \alpha)(I - \alpha S)^{-1}Y$. Vérification numérique sur le graphe à six nœuds du livre :

```
W6 = np.zeros((6, 6))
for i, j in [(0, 1), (0, 2), (1, 2), (2, 3), (3, 4), (3, 5), (4, 5)]:
    W6[i, j] = W6[j, i] = 1
d6 = W6.sum(axis=1); S6 = W6 / np.sqrt(np.outer(d6, d6))
Y6 = np.zeros((6, 2)); Y6[0, 0] = 1; Y6[5, 1] = 1
F = Y6.copy()
for t in range(60):
    F = 0.5 * S6 @ F + 0.5 * Y6
Fstar = 0.5 * np.linalg.inv(np.eye(6) - 0.5 * S6) @ Y6
print("valeurs propres de S dans [-1, 1] :", np.linalg.eigvalsh(S6).round(3))
print("écart maximal entre l'itération (60 tours) et la solution fermée :", float(np.abs(F -
    ↪ Fstar).max()))
```

valeurs propres de S dans $[-1, 1]$: $[-0.629 \ -0.5 \ -0.5 \ -0.167 \ 0.795 \ 1. \]$
 écart maximal entre l'itération (60 tours) et la solution fermée : $1.1102230246251565e-16$

Corrigé 8.7

```
cand = {"a": [0.7, 0.2, 0.1], "b": [0.4, 0.4, 0.2], "c": [0.34, 0.33, 0.33]}
lignes = []
for nom, p in cand.items():
    p = np.array(p)
    tri = np.sort(p)[::-1]
    lignes.append((nom, 1 - tri[0], tri[0] - tri[1], -(p * np.log(p)).sum()))
tab = pd.DataFrame(lignes, columns=["candidat", "1 - max", "écart", "entropie"]).set_index("candidat")
print(tab.round(4).to_string())
print("classement par confiance minimale :", list(tab["1 - max"].sort_values(ascending=False).index))
print("classement par marge                :", list(tab["écart"].sort_values().index))
print("classement par entropie             :", list(tab["entropie"].sort_values(ascending=False).index))
```

	1 - max	écart	entropie
candidat			
a	0.30	0.50	0.8018
b	0.60	0.00	1.0549
c	0.66	0.01	1.0985

classement par confiance minimale : ['c', 'b', 'a']
 classement par marge : ['b', 'c', 'a']
 classement par entropie : ['c', 'b', 'a']

Le candidat b a **exactement** deux classes à égalité (0,4 et 0,4) : la marge est nulle, elle le désigne comme le plus incertain. Le candidat c est presque uniforme sur les trois classes : la confiance minimale et l'entropie le placent en tête. Seule la marge les départage différemment.

Corrigé 8.8

Pour deux classes, $P(1 | x) = p$ et $P(0 | x) = 1 - p$. La confiance minimale vaut $1 - \max(p, 1 - p) = \frac{1}{2} - |p - \frac{1}{2}|$; la marge (écart entre les deux meilleures) vaut $|2p - 1| = 2|p - \frac{1}{2}|$; l'entropie $H(p) = -p \ln p - (1 - p) \ln(1 - p)$ est une fonction **symétrique** autour de $\frac{1}{2}$ et **décroissante en** $|p - \frac{1}{2}|$ (sa dérivée $\ln \frac{1-p}{p}$ est positive pour $p < \frac{1}{2}$). Les trois sont donc des fonctions **monotones** de la distance $|p - \frac{1}{2}|$: ils classent les points dans le même ordre (le plus incertain étant celui dont p est le plus proche de $\frac{1}{2}$).

Corrigé 8.9

```
for votes in [(7, 0, 0), (4, 2, 1), (3, 3, 1)]:
    f = np.array(votes) / 7
    f = f[f > 0]
    print(votes, "entropie du vote :", round(float(-(f * np.log(f)).sum()), 4))
```

(7, 0, 0) entropie du vote : -0.0
 (4, 2, 1) entropie du vote : 0.9557
 (3, 3, 1) entropie du vote : 1.0042

Le classement, du plus utile au moins utile : (3,3,1) (deux classes presque à égalité, une troisième marginale), puis (4,2,1), puis (7,0,0) (unanimité : score nul, rien à apprendre du désaccord).

Corrigé 8.10

La probabilité que les 10 tirages donnent l'avantage à la **même** stratégie, dans un sens ou dans l'autre, est $2 \times (1/2)^{10} = 2/1024 \approx 0,002$: très peu probable si les deux stratégies étaient équivalentes. On peut donc écarter l'hypothèse « aucune différence » à 60 étiquettes sur ces données.

Limites : le test des signes ignore la **taille** des écarts ; les 10 tirages partagent le même jeu de test et le même réservoir, donc ne sont pas indépendants au sens strict ; le résultat vaut pour ces données, ce modèle et ce budget, pas en général.

```
from scipy.stats import binom
print("P(10 sur 10 dans un sens ou l'autre) =", round(2 * binom.pmf(10, 10, 0.5), 5))
```

```
P(10 sur 10 dans un sens ou l'autre) = 0.00195
```

Corrigé 8.11

- (a) Si la sélection d'un point dépend **seulement** de son étiquette y (pas de x), alors $P_s(x | y) = P(x | y)$ et seule la proportion de classes change. Par Bayes, $\frac{P_s(1 | x)}{P_s(0 | x)} = \frac{P(x | 1) \pi_s}{P(x | 0) (1 - \pi_s)}$, tandis que $\frac{P(1 | x)}{P(0 | x)} = \frac{P(x | 1) \pi}{P(x | 0) (1 - \pi)}$. D'où cotes vraies = cotes du modèle $\times \frac{\pi / (1 - \pi)}{\pi_s / (1 - \pi_s)}$.
- (b) et (c) : on le vérifie sur les clients, d'abord pour un sur-échantillonnage des positifs indépendant de x (cas (a)), puis pour le jeu étiqueté par apprentissage actif.

```
def corriger(p, pi_s, pi=0.14):
    cotes = p / (1 - p) * (pi / (1 - pi)) / (pi_s / (1 - pi_s))
    return cotes / (1 + cotes)

res_a, res_b = [], []
for s in range(10):
    rng = np.random.default_rng(1000 + s)
    pos, neg = np.where(yc == 1)[0], np.where(yc == 0)[0]
    e = np.r_[rng.choice(pos, 150, replace=False), rng.choice(neg, 250, replace=False)] # 37,5 %
    ← de positifs, tirés sans regarder x
    p = LogisticRegression(max_iter=1000).fit(Xc[e], yc[e]).predict_proba(Xct)[: , 1]
    res_a.append((p.mean(), corriger(p, yc[e].mean()).mean()))
    idx0 = o.tirer_etiquettes(yc, 40, np.random.default_rng(700 + s), stratifie=True)
    _, ea = o.boucle_active(Xc, yc, Xct, yct, "incertitude", idx0, 20, 400, graine=s, k=2,
    ← metrique="auc", renvoyer_etiq=True)
    pa = LogisticRegression(max_iter=1000).fit(Xc[ea], yc[ea]).predict_proba(Xct)[: , 1]
    res_b.append((pa.mean(), corriger(pa, yc[ea].mean()).mean(), yc[ea].mean()))
print("sur-échantillonnage aléatoire : proba moyenne avant", round(np.mean([r[0] for r in res_a]),
    ← 3), "| après correction", round(np.mean([r[1] for r in res_a]), 3))
print("apprentissage actif : proba moyenne avant", round(np.mean([r[0] for r in res_b]), 3),
    ← "| après correction", round(np.mean([r[1] for r in res_b]), 3), "| part de positifs",
    ← round(np.mean([r[2] for r in res_b]), 3))
print("taux réel :", round(yct.mean(), 3))
```

```
sur-échantillonnage aléatoire : proba moyenne avant 0.279 | après correction 0.152
apprentissage actif : proba moyenne avant 0.078 | après correction 0.046 | part de positifs
    ← 0.384
taux réel : 0.14
```

Résultats. Pour un **sur-échantillonnage aléatoire** des positifs (37,5 % de positifs), le modèle brut **surestime** le risque (probabilité moyenne 0,279) et la correction de la question (a) le ramène à **0,152**, proche du taux réel de 0,14 : la correction marche, comme prévu. Pour le jeu étiqueté par **apprentissage actif** (38,4 % de positifs), c'est l'inverse : le modèle brut **sous-estime** le risque (0,078) et la correction l'abaisse encore, à **0,046**, plus loin de la vérité. Le biais de l'apprentissage actif n'est donc **pas** un simple changement de proportion de classes. Les points ont été choisis en

fonction de leurs **variables** x (ils sont près de la frontière du modèle), si bien que l'hypothèse de la question (a), « la sélection ne dépend que de y », est fausse ; la correction par les proportions ne s'applique pas. Nous n'avons pas isolé par une expérience le mécanisme exact de la sous-estimation. Le remède éprouvé est celui du livre : **recalibrer sur un petit échantillon aléatoire**.

Corrigé 8.12

- (a) Une économie de $60 \times 4 = 240$ € par projet : $1\,000/240 \approx 4,2$, soit **5 projets**. (b) Économie de $50 \times 0,40 = 20$ € par projet : $1\,000/20 = 50$ projets. (c) Si l'on a besoin de **probabilités calibrées**, il faut compter les étiquettes **aléatoires** supplémentaires du recalibrage (livre, 8.3.6 : 300 étiquettes, soit 1 200 € à 4 € l'unité), qui peuvent dépasser l'économie réalisée sur le choix des points ; il faut aussi compter le coût de **maintenance** du système (réentraînement, files d'attente d'annotation).

```
import math
print("projets pour amortir (cas a) :", math.ceil(1000 / (60 * 4)), " | (cas b) :", math.ceil(1000 /
↵ (50 * 0.40)))
```

```
projets pour amortir (cas a) : 5 | (cas b) : 50
```

Chapitre 9 : ⊕ Bases de l'apprentissage par renforcement — exercices et applications

Ce chapitre du cahier accompagne le chapitre 9 du livre (chapitre complémentaire). Il contient six **applications** guidées, qui reconstruisent pas à pas les environnements du livre (le problème de stock, les quatre bannières, le couloir d'entrepôt), et douze **exercices** corrigés. Tout est simulé, avec des graines fixes, et le chapitre est **autonome** : il refait ses imports et redéfinit ses fonctions. Les applications se lisent dans l'ordre : certaines réutilisent les fonctions définies plus haut.

Applications

Préparation (à exécuter d'abord)

Nous commençons par le **problème de stock** du livre (section 9.1.9) : cinq niveaux de stock (0 à 4), une commande avant l'ouverture, une demande aléatoire de 0 à 3 clients. Les paramètres économiques sont regroupés dans un dictionnaire, pour pouvoir les faire varier.

```
import numpy as np
import pandas as pd
from scipy import stats

dem = np.array([0.2, 0.4, 0.3, 0.1])      # probabilités de demande : 0, 1, 2 ou 3 clients
S = 5                                     # stocks possibles : 0 à 4

def parametres(frais=4.0, prix=3.0, achat=1.0, garde=0.5, penurie=1.0):
    return dict(frais=frais, prix=prix, achat=achat, garde=garde, penurie=penurie)

def recompense(s, a, d, p):
    """Récompense de la journée et stock du lendemain : stock s, commande a, demande d."""
    stock = s + a; vendu = min(stock, d); reste = stock - vendu
    gain = (p["prix"] * vendu - p["achat"] * a - p["frais"] * (a > 0)
            - p["garde"] * reste - p["penurie"] * max(d - stock, 0))
    return gain, reste
```

Deux fonctions : l'**itération de la valeur** (la solution exacte) et le **gain moyen** d'une politique, mesuré par simulation.

```
def iteration_valeur(p, gamma=0.95, tol=1e-9):
    """Retourne V*, Q* et la politique optimale (quantité commandée pour chaque stock)."""
```

```

V = np.zeros(S)
while True:
    Q = np.full((S, S), -np.inf)
    for s in range(S):
        for a in range(S - s):
            # on ne peut pas dépasser 4 en stock
            Q[s, a] = sum(dem[d] * (lambda r, nx: r + gamma * V[nx])(*recompense(s, a, d, p)) for d
↪ in range(4))
        Vn = Q.max(1)
        if np.abs(Vn - V).max() < tol: return Vn, Q, Q.argmax(1)
        V = Vn

def gain_moyen(politique, p, jours=100000, seed=1):
    rng = np.random.default_rng(seed); s = 2; total = 0.0
    for d in rng.choice(4, size=jours, p=dem):
        r, s = recompense(s, politique[s], d, p); total += r
    return total / jours

```

Application 9.1 — Résoudre le problème de stock et mesurer l'effet des frais fixes (section 9.1.9)

Contexte. La gérante hésite sur sa politique de commande. Le livre a montré que, avec 4 € de frais de livraison, il vaut mieux **attendre la rupture, puis remplir**. Mais que se passe-t-il si le transporteur change ses tarifs, ou si la gérante accorde moins de valeur à l'avenir ?

Étape 1 — la solution de référence. Résolvons le problème avec les paramètres du livre et vérifions les valeurs.

```

p0 = parametres()
V, Q, pi = iteration_valeur(p0)
print("valeurs optimales V* :", V.round(2))
print("politique optimale (quantité à commander pour un stock de 0 à 4) :", pi.tolist())
print("gain moyen par jour :", round(gain_moyen(pi.tolist(), p0), 3))

```

```

valeurs optimales V* : [ 2.05  4.14  6.73  8.62 10.05]
politique optimale (quantité à commander pour un stock de 0 à 4) : [4, 0, 0, 0, 0]
gain moyen par jour : 0.269

```

Étape 2 — faire varier les frais fixes. Pour chaque tarif de livraison, nous recalculons la politique optimale et son gain.

```

lignes = []
for frais in [0.0, 1.0, 2.0, 4.0, 6.0]:
    p = parametres(frais=frais)
    _, _, pol = iteration_valeur(p)
    lignes.append({"frais fixes (€)": frais, "politique": pol.tolist(), "gain moyen par jour (€)":
↪ round(gain_moyen(pol.tolist(), p), 3)})
print(pd.DataFrame(lignes).to_string(index=False))

```

frais fixes (€)	politique	gain moyen par jour (€)
0.0	[2, 1, 0, 0, 0]	1.900
1.0	[3, 2, 0, 0, 0]	1.270
2.0	[4, 0, 0, 0, 0]	0.851
4.0	[4, 0, 0, 0, 0]	0.269
6.0	[4, 0, 0, 0, 0]	-0.313

Lecture. Sans frais de livraison, l'agent commande **peu et souvent** : la politique [2, 1, 0, 0, 0] revient à « compléter jusqu'à 2 articles ». À 1 € de frais, il remonte à 3 articles dès que le stock tombe à 1 ou moins ([3, 2, 0, 0, 0]). À partir de 2 €, la politique devient « attendre la rupture, puis remplir jusqu'à 4 ». Plus les frais fixes sont élevés, plus il devient rentable de **grouper** les commandes, quitte à accepter des ruptures. On reconnaît la structure classique d'une politique

(s, S).

Étape 3 — faire varier l'actualisation. Le facteur γ mesure la valeur que la gérante accorde à l'avenir.

```
for gamma in [0.5, 0.8, 0.9, 0.95, 0.99]:
    _, _, pol = iteration_valeur(p0, gamma=gamma)
    print(f"gamma = {gamma:4.2f} -> politique {pol.tolist()}")
```

```
gamma = 0.50 -> politique [0, 0, 0, 0, 0]
gamma = 0.80 -> politique [4, 0, 0, 0, 0]
gamma = 0.90 -> politique [4, 0, 0, 0, 0]
gamma = 0.95 -> politique [4, 0, 0, 0, 0]
gamma = 0.99 -> politique [4, 0, 0, 0, 0]
```

Lecture. Avec $\gamma = 0,5$, l'agent est **myope** : il **ne commande jamais** ([0, 0, 0, 0, 0]), car les 4 € de frais sont payés tout de suite et leur bénéfice vient plus tard. Dès $\gamma = 0,8$, il retrouve la politique « attendre, puis remplir ». Un facteur d'actualisation trop faible n'est donc pas un détail technique : il change la décision.

Pour aller plus loin. Modifiez le coût de stockage (garde) et la pénalité de rupture (penurie) : à partir de quelle pénalité l'agent cesse-t-il d'accepter des ruptures ?

Application 9.2 — Les quatre bannières : comparer les stratégies de bandit (section 9.2)

Contexte. La gérante a quatre bannières, de taux de conversion inconnus (4,0 ; 5,2 ; 5,8 ; 7,0 %). On veut comparer l'A/B test, ϵ -glouton, UCB et Thompson sur 10 000 visiteurs, avec 200 répétitions pour estimer le **regret** moyen.

Étape 1 — le simulateur. Pour aller vite, la fonction traite les 200 répétitions **en parallèle** (chaque ligne d'un tableau est une répétition indépendante).

```
P = np.array([0.040, 0.052, 0.058, 0.070])          # vrais taux : inconnus des algorithmes

def jouer(algo, P=P, T=10000, R=200, seed=900, c=2.0, commit=2000, eps=0.1):
    rng = np.random.default_rng(seed); K = len(P)
    n = np.zeros((R, K)); s = np.zeros((R, K)); regret = np.zeros((R, T)); rr = np.arange(R)
    for t in range(T):
        moy = s / np.maximum(n, 1)                    # taux observés
        if algo == "ab":                               # tourner à égalité, puis s'engager
            a = np.full(R, t % K) if t < commit else np.argmax(moy, axis=1)
        elif algo == "eps":                            # ε-glouton
            a = np.where(rng.random(R) < eps, rng.integers(0, K, R), np.argmax(moy + (n == 0) * 1e9,
↪ axis=1))
        elif algo == "ucb":                            # indice = taux observé + bonus
            a = np.full(R, t) if t < K else np.argmax(moy + np.sqrt(c * np.log(t + 1) / np.maximum(n,
↪ 1)), axis=1)
        else:                                           # Thompson : un tirage dans chaque loi Bêta
            a = np.argmax(rng.beta(1 + s, 1 + n - s), axis=1)
            x = rng.random(R) < P[a]; n[rr, a] += 1; s[rr, a] += x
            regret[:, t] = P.max() - P[a]
    return np.cumsum(regret, axis=1), n
```

Étape 2 — lancer les cinq stratégies (environ 40 secondes) et résumer leur regret final.

```
strategies = {"A/B (2 000 visiteurs)": dict(algo="ab"), "ε-glouton": dict(algo="eps"),
              "UCB1 (c = 2)": dict(algo="ucb", c=2.0), "UCB réduit (c = 0,1)": dict(algo="ucb",
↪ c=0.1), "Thompson": dict(algo="ts")}

lignes = []
for nom, kw in strategies.items():
    cr, n = jouer(**kw); f = cr[:, -1]
```

```
lignes.append({"stratégie": nom, "regret moyen": f.mean().round(1), "écart-type":
↳ f.std().round(1),
               "90e centile": np.quantile(f, 0.9).round(1), "mauvais bras le plus joué (%)":
               ↳ round(100 * float((np.argmax(n, axis=1) != 3).mean()), 1)})
print(pd.DataFrame(lignes).to_string(index=False))
```

	stratégie	regret moyen	écart-type	90e centile	mauvais bras le plus joué (%)
A/B (2 000 visiteurs)		47.9	36.6	125.0	14.5
ε-glouton		64.6	50.9	138.6	28.0
UCB1 (c = 2)		123.4	5.3	130.3	1.0
UCB réduit (c = 0,1)		56.6	14.6	74.9	1.0
Thompson		46.2	23.2	74.1	7.0

Lecture. Vous retrouvez les chiffres du livre : Thompson (46,2) et l'A/B (47,9) ont un regret moyen proche, mais l'A/B se trompe de bannière dans 14,5 % des répétitions et son 90e centile est bien plus haut ; UCB1 est le plus cher (123,4) parce que son bonus est trop grand pour des taux de l'ordre de 5 % ; réduit à $c = 0,1$, il redevient compétitif.

Étape 3 — l'effet de l'écart entre les bannières. Quand les bannières sont plus proches, le problème devient plus difficile. Refaisons l'expérience avec des taux de 5,0 ; 5,5 ; 6,0 et 6,5 % pour trois stratégies (environ 20 secondes).

```
P_proches = np.array([0.050, 0.055, 0.060, 0.065])
lignes = []
for nom, kw in {"A/B (2 000 visiteurs)": dict(algo="ab"), "UCB réduit (c = 0,1)": dict(algo="ucb",
↳ c=0.1), "Thompson": dict(algo="ts")}.items():
    cr, n = jouer(P=P_proches, **kw); f = cr[:, -1]
    lignes.append({"stratégie": nom, "regret moyen": f.mean().round(1), "90e centile": np.quantile(f,
↳ 0.9).round(1),
                  "mauvais bras le plus joué (%)": round(100 * float((np.argmax(n, axis=1) !=
↳ 3).mean()), 1)})
print(pd.DataFrame(lignes).to_string(index=False))
```

	stratégie	regret moyen	90e centile	mauvais bras le plus joué (%)
A/B (2 000 visiteurs)		34.6	61.7	35.0
UCB réduit (c = 0,1)		46.5	63.3	21.0
Thompson		40.9	61.8	30.5

Lecture. Avec des bannières très proches, aucune stratégie ne peut distinguer sûrement la meilleure en 10 000 visiteurs : le pourcentage de répétitions où la bannière la plus jouée est la mauvaise passe de 1 à 7 % (au plus 14,5 %, pour l'A/B) à 21 à 35 %. Le regret reste pourtant modeste, parce que se tromper entre 6,0 % et 6,5 % coûte peu. Remarquez surtout que **l'A/B a ici le regret moyen le plus faible (34,6)**, devant Thompson (40,9) et UCB réduit (46,5) : quand les écarts sont minuscules, les stratégies adaptatives n'ont presque plus rien à gagner. Les stratégies ne se classent pas une fois pour toutes ; leur avantage dépend de l'écart entre les options.

Pour aller plus loin. Tracez le regret cumulé moyen en fonction du nombre de visiteurs pour chaque stratégie ($cr.mean(0)$), et vérifiez que Thompson ralentit progressivement.

Application 9.3 — Un bandit contextuel selon le canal d'arrivée (section 9.2.8)

Contexte. Le meilleur choix dépend du canal d'arrivée du visiteur. On compare Thompson aveugle au canal et Thompson avec une croyance par canal, sur 10 000 visiteurs et 200 répétitions (graine 77).

Étape 1 — les taux par canal. Chaque ligne est un canal (Boutique, Site, Réseaux), chaque colonne une bannière.

```
Pc = np.array([[0.060, 0.040, 0.030, 0.045],      # Boutique : la bannière A est la meilleure
               [0.040, 0.045, 0.070, 0.050],      # Site : la bannière C
               [0.035, 0.040, 0.045, 0.075]])      # Réseaux : la bannière D
pc = np.array([0.3, 0.4, 0.3])                    # part des visiteurs par canal
print("meilleure bannière unique :", ["A", "B", "C", "D"][int(np.argmax(pc @ Pc))], "| taux moyen :",
      ↪ round(float((pc @ Pc).max()), 4))
print("meilleure bannière par canal : taux moyen", round(float(pc @ Pc.max(1)), 4))
```

```
meilleure bannière unique : D | taux moyen : 0.056
meilleure bannière par canal : taux moyen 0.0685
```

Étape 2 — le simulateur contextuel. Pour chaque visiteur, on tire son canal, puis la bannière selon la croyance (globale ou propre au canal).

```
def jouer_ctx(mode, T=10000, R=200, seed=77):
    rng = np.random.default_rng(seed); n = np.zeros((R, 3, 4)); s = np.zeros((R, 3, 4)); regret =
    ↪ np.zeros(R); rr = np.arange(R)
    for t in range(T):
        ctx = rng.choice(3, R, p=pc)
        N = n.sum(1) if mode == "aveugle" else n[rr, ctx]      # croyance globale ou par canal
        Sc = s.sum(1) if mode == "aveugle" else s[rr, ctx]
        a = np.argmax(rng.beta(1 + Sc, 1 + N - Sc), axis=1)
        x = rng.random(R) < Pc[ctx, a]
        n[rr, ctx, a] += 1; s[rr, ctx, a] += x
        regret += Pc[ctx].max(1) - Pc[ctx, a]                  # regret par rapport au meilleur choix
    ↪ du canal
    return regret

for mode in ("aveugle", "contextuel"):
    r = jouer_ctx(mode); print(f"{mode:11s} regret moyen {r.mean():6.1f} écart-type {r.std():5.1f}")
```

```
aveugle      regret moyen  164.5 écart-type  16.0
contextuel   regret moyen   80.1 écart-type  20.5
```

Lecture. Le Thompson aveugle accumule environ 164 conversions de regret, le contextuel environ 80 : connaître le canal **divise le regret par deux**.

Pour aller plus loin. Que se passe-t-il si les trois canaux ont les mêmes taux ? Le bandit contextuel perd-il quelque chose par rapport à l'aveugle ? (Remplacez Pc par trois lignes identiques.)

Application 9.4 — Le Q-learning sur le problème de stock (section 9.3)

Contexte. L'agent ne connaît pas la loi de la demande ; il apprend sa politique en jouant 300 000 jours simulés. On compare un pas d'apprentissage **constant** à un pas **décroissant** et on confronte la table apprise à la solution exacte Q^* de l'application 9.1.

Étape 1 — précalculer les récompenses et les transitions (pour aller vite), puis écrire l'apprentissage.

```
REC = [[[recompense(s, a, d, p0)[0] for d in range(4)] if a < S - s else None for a in range(S)] for s
      ↪ in range(S)]
NXT = [[[recompense(s, a, d, p0)[1] for d in range(4)] if a < S - s else None for a in range(S)] for s
      ↪ in range(S)]
```

```
LEG = [list(range(S - s)) for s in range(S)] # commandes possibles pour chaque stock
gamma = 0.95
```

```
def q_learning(pas=300000, seed=5, mode="décroissant", alpha=0.1, k=50, suivi=5000):
    rng = np.random.default_rng(seed); Qt = [[0.0] * S for _ in range(S)]; N = [[0] * S for _ in
    ↪ range(S)]; s = 2
    dem_t = rng.choice(4, size=pas, p=dem).tolist(); u = rng.random(pas).tolist(); tr =
    ↪ rng.random(pas).tolist()
    temps, erreurs = [], []
    for t in range(pas):
        eps = max(0.05, 1 - t / (0.5 * pas)); L = LEG[s] # exploration décroissante
        a = L[int(tr[t] * len(L)) % len(L)] if u[t] < eps else max(L, key=Qt[s].__getitem__)
        d = dem_t[t]; r = REC[s][a][d]; nx = NXT[s][a][d]
        N[s][a] += 1; al = alpha if mode == "constant" else 1.0 / (1.0 + N[s][a] / k)
        Qt[s][a] += al * (r + gamma * max(Qt[nx][x] for x in LEG[nx]) - Qt[s][a]); s = nx
        if (t + 1) % suivi == 0:
            temps.append(t + 1); erreurs.append(max(abs(Qt[x][y] - Q[x, y]) for x in range(S) for y in
    ↪ LEG[x]))
    return np.array(Qt), np.array(temps), np.array(erreurs)
```

Étape 2 — apprendre avec les deux réglages et comparer à la solution exacte (Q et pi viennent de l'application 9.1).

```
for mode in ("constant", "décroissant"):
    Qm, temps, err = q_learning(mode=mode); pol = Qm.argmax(1).tolist()
    print(f"{mode:12s} politique {pol} (optimale : {pi.tolist()}) | gain moyen {gain_moyen(pol,
    ↪ p0):.3f}")
    print(" erreur max |Q - Q*| après 10 000, 50 000, 150 000, 300 000 pas :",
    ↪ [round(float(err[temps == k][0]), 2) for k in (10000, 50000, 150000, 300000)])
```

```
constant      politique [4, 0, 0, 0, 0] (optimale : [4, 0, 0, 0, 0]) | gain moyen 0.269
               erreur max |Q - Q*| après 10 000, 50 000, 150 000, 300 000 pas : [0.55, 0.71, 1.22, 0.79]
décroissant   politique [4, 0, 0, 0, 0] (optimale : [4, 0, 0, 0, 0]) | gain moyen 0.269
               erreur max |Q - Q*| après 10 000, 50 000, 150 000, 300 000 pas : [1.02, 0.24, 0.16, 0.25]
```

Étape 3 — la robustesse : 10 graines différentes.

```
for mode in ("constant", "décroissant"):
    bon, erreurs = 0, []
    for graine in range(10):
        Qg, _, e = q_learning(seed=100 + graine, mode=mode, suivi=300000)
        bon += Qg.argmax(1).tolist() == pi.tolist(); erreurs.append(e[-1])
    print(f"{mode:12s} politique optimale retrouvée {bon}/10 | erreur maximale moyenne
    ↪ {np.mean(erreurs):.2f} €")
```

```
constant      politique optimale retrouvée 8/10 | erreur maximale moyenne 1.22 €
décroissant   politique optimale retrouvée 10/10 | erreur maximale moyenne 0.20 €
```

Lecture. Avec un pas décroissant, l'agent retrouve la politique optimale à chaque fois (10 fois sur 10) avec une erreur de valeur d'environ 0,20 €, alors qu'avec un pas constant l'erreur reste autour de 1,2 € et la politique optimale n'est trouvée que 8 fois sur 10. **Quand l'écart de valeur entre deux actions est du même ordre que le bruit résiduel**, un pas constant peut les confondre ; un pas décroissant finit par éteindre ce bruit.

Pour aller plus loin. Divisez le nombre de pas par dix (30 000) : combien de graines retrouvent encore la politique optimale ? À partir de quel nombre de jours simulés la politique est-elle fiable ?

Application 9.5 — SARSA contre Q-learning sur le couloir d'entrepôt (section 9.3.4)

Contexte. L'agent traverse un couloir de 4×10 cases, du départ S au quai G ; la rangée du bas est une zone dangereuse (-100 et retour au départ), chaque pas coûte -1 . L'agent **ne connaît pas la carte** et l'apprend en explorant avec $\varepsilon = 0,1$.

Étape 1 — la grille et ses règles.

```
H, W = 4, 10; depart, arrivee = (3, 0), (3, 9); danger = {(3, c) for c in range(1, 9)}
mouv = [(-1, 0), (0, 1), (1, 0), (0, -1)] # haut, droite, bas, gauche

def pas_grille(s, a):
    r, c = min(max(s[0] + mouv[a][0], 0), H - 1), min(max(s[1] + mouv[a][1], 0), W - 1)
    if (r, c) in danger: return depart, -100.0, False # zone dangereuse : retour au départ
    return (r, c), -1.0, (r, c) == arrivee
```

Étape 2 — l'apprentissage par différence temporelle, avec une seule fonction pour les deux algorithmes : la seule différence est la **cible**.

```
def td_couloir(algo, episodes=500, seed=3, alpha=0.5, eps=0.1):
    rng = np.random.default_rng(seed); Qg = np.zeros((H, W, 4)); retours = []
    choisir = lambda s: int(rng.integers(4)) if rng.random() < eps else int(np.argmax(Qg[s]))
    for _ in range(episodes):
        s = depart; a = choisir(s); G = 0.0
        for _ in range(500):
            s2, r, fin = pas_grille(s, a); G += r; a2 = choisir(s2)
            cible = r + (0.0 if fin else (Qg[s2][a2] if algo == "sarsa" else Qg[s2].max())) # SARSA :
            ↪ a2 ; Q-learning : le max
            Qg[s][a] += alpha * (cible - Qg[s][a]); s, a = s2, a2
            if fin: break
        retours.append(G)
    return Qg, np.array(retours)

def chemin_glouton(Qg):
    s = depart; chemin = [s]
    for _ in range(40):
        s, _, fin = pas_grille(s, int(np.argmax(Qg[s]))); chemin.append(s)
        if fin: break
    return chemin
```

Étape 3 — comparer les deux méthodes (moyenne de 10 graines pour le retour pendant l'apprentissage, une graine pour les chemins).

```
for algo in ("sarsa", "q"):
    retours = np.mean([td_couloir(algo, seed=k)[1] for k in range(10)], axis=0)
    ch = chemin_glouton(td_couloir(algo, seed=3)[0])
    print(f"{algo:6s} retour moyen (100 derniers épisodes) : {retours[-100:].mean():6.1f} | chemin
    ↪ glouton : {len(ch) - 1} pas, rangée la plus haute : {min(p[0] for p in ch)}")
```

```
sarsa  retour moyen (100 derniers épisodes) : -26.6 | chemin glouton : 15 pas, rangée la plus haute :
↪ 0
q      retour moyen (100 derniers épisodes) : -39.5 | chemin glouton : 11 pas, rangée la plus haute : 2
```

Lecture. Vous retrouvez le résultat du livre : SARSA obtient un meilleur retour **pendant** l'apprentissage (environ -27 contre -40) parce qu'il évite le bord de la zone dangereuse, tandis que le Q-learning apprend le chemin le plus court (11 pas) qui la longe.

Étape 4 — l'effet de l'exploration. Que devient la différence quand ε varie ? Nous comparons le retour pendant l'apprentissage (moyenne de 5 graines).

```
lignes = []
for eps in (0.01, 0.1, 0.3):
    ligne = {"ε": eps}
```

```

for algo in ("sarsa", "q"):
    ligne[algo] = round(float(np.mean([td_couloir(algo, seed=k, eps=eps)[1][-100:].mean() for k in
↪ range(5)])), 1)
    lignes.append(ligne)
print(pd.DataFrame(lignes).to_string(index=False))

```

ϵ	sarsa	q
0.01	-14.5	-12.5
0.10	-23.2	-38.1
0.30	-55.1	-124.7

Lecture. Plus ϵ est grand, plus le Q-learning **souffre** de longer le précipice, et plus l'écart avec SARSA se creuse (−124,7 contre −55,1 pour $\epsilon = 0,3$). À l'inverse, pour $\epsilon = 0,01$, le Q-learning est même **un peu meilleur** (−12,5 contre −14,5) : ses dérapages sont si rares que le chemin court ne coûte presque rien. La différence entre les deux méthodes est liée à l'exploration, pas à une supériorité intrinsèque de l'une. (Cette expérience utilise 5 graines, d'où les valeurs légèrement différentes de celles de l'étape 3 pour $\epsilon = 0,1$.)

Application 9.6 — Une récompense mal posée (section 9.4.6)

Contexte. « L'agent maximise ce que l'on mesure, pas ce que l'on veut. » Reprenons le problème de stock en changeant la **récompense**, puis évaluons les politiques obtenues avec la **vraie** mesure (le profit).

Étape 1 — récompense = nombre d'articles vendus (on oublie tous les coûts), puis récompense avec tous les coûts sauf les frais de livraison fixes.

```

p_ventes = parametres(frais=0.0, achat=0.0, garde=0.0, penurie=0.0)    # récompense = 3 € par article
↪ vendu
p_sans_frais = parametres(frais=0.0)                                   # on a oublié seulement les
↪ frais fixes
_, _, pi_ventes = iteration_valeur(p_ventes)
_, _, pi_sans_frais = iteration_valeur(p_sans_frais)
print("politique si la récompense = ventes seules :", pi_ventes.tolist())
print("politique si l'on oublie les frais fixes   :", pi_sans_frais.tolist())

```

```

politique si la récompense = ventes seules : [3, 2, 1, 0, 0]
politique si l'on oublie les frais fixes   : [2, 1, 0, 0, 0]

```

Étape 2 — évaluer chaque politique avec le vrai profit (p_0).

```

for nom, pol in [("politique optimale (vraie récompense)", pi), ("récompense = ventes seules",
↪ pi_ventes), ("frais fixes oubliés", pi_sans_frais)]:
    print(f"{nom:40s} gain réel moyen par jour : {gain_moyen(pol.tolist(), p0):6.3f} €")

```

```

politique optimale (vraie récompense)    gain réel moyen par jour : 0.269 €
récompense = ventes seules                gain réel moyen par jour : -1.451 €
frais fixes oubliés                      gain réel moyen par jour : -1.302 €

```

Lecture. L'agent dont la récompense ne compte que les ventes apprend à **réapprovisionner chaque jour** jusqu'à 3 articles (de quoi satisfaire toute la demande possible), parce que commander ne lui coûte rien : il perd en réalité environ 1,45 € par jour. Oublier **seulement** les frais fixes suffit à transformer un gain de 0,27 € en une perte de 1,30 € par jour. L'algorithme n'a rien « mal » fait : il a parfaitement optimisé la mauvaise récompense.

Pour aller plus loin. Ajoutez un à un les coûts oubliés (achat, stockage, rupture, frais fixes) et suivez l'évolution du gain réel de la politique apprise : quel coût manquant fait le plus de dégâts ?

Exercices

Exercice 9.1 ★ — Retour actualisé (section 9.1.2)

- (a) Une suite de récompenses vaut 2, 0, 5, 1 puis plus rien. Calculez le retour G_0 avec $\gamma = 0,9$.
 (b) Une récompense constante de 3 € par jour pendant un temps infini, avec $\gamma = 0,95$: quel est le retour ? Quel est l'horizon effectif ?

Exercice 9.2 ★★ — Évaluer deux politiques (section 9.1.5)

Dans le cycle de vie du client du livre (3 états, $\gamma = 0,9$), calculez la valeur de chaque état pour (a) la politique « toujours attendre » ; (b) la politique « offre sauf pour un client fidèle ». Quelle politique est meilleure, et dans quels états ?

Exercice 9.3 ★★ — Itération de la valeur à la main (section 9.1.7)

Deux états, A et B, $\gamma = 0,5$. En A : l'action x donne 2 € et reste en A ; l'action y donne 0 € et passe en B. En B : l'action x donne 1 € et passe en A ; l'action y donne 3 € et reste en B. Faites trois itérations de la valeur à partir de $V_0 = (0, 0)$, puis déterminez V^* en résolvant les équations d'optimalité.

Exercice 9.4 ★★★ — Combien d'itérations ? (section 9.1.8)

Dans le cycle de vie du client, l'erreur de l'itération de la valeur après k itérations vaut exactement $40 \times 0,9^k$. Combien d'itérations faut-il pour que l'erreur passe sous 0,01 ? Quelle borne la contraction de Bellman donne-t-elle ? Vérifiez par le calcul.

Exercice 9.5 ★ — Calculer un regret (section 9.2.2)

Les quatre bannières ont pour taux 4,0 ; 5,2 ; 5,8 et 7,0 %. Une stratégie a affiché la bannière A 300 fois, B 200 fois, C 100 fois et D 400 fois. Quel est son regret ?

Exercice 9.6 ★★ — L'indice UCB (section 9.2.5)

Au visiteur $t = 500$, trois bannières ont été affichées 100, 150 et 250 fois, pour 5, 9 et 22 conversions. Calculez les indices UCB1 (bonus $\sqrt{2 \ln t/n}$) et dites quelle bannière est choisie. Même question avec un bonus réduit $\sqrt{0,1 \ln t/n}$. Commentez.

Exercice 9.7 ★★ — Thompson à la main (section 9.2.6)

La bannière A a obtenu 12 conversions sur 200 affichages, la bannière B 9 sur 100. Avec un *a priori* uniforme, donnez les lois *a posteriori*, les moyennes *a posteriori*, et estimez la probabilité que B soit meilleure que A. Avec quelle fréquence Thompson affichera-t-il B ?

Exercice 9.8 ★★★ — Test A/B contre bandit (sections 9.2.3 et 9.2.9)

Deux bannières, C (5,8 %) et D (7,0 %). (a) Combien de visiteurs faut-il, au total, pour un test A/B classique (risque 5 %, puissance 80 %) ? (b) Si l'on partage ces visiteurs à égalité, combien de conversions perd-on, en moyenne, par rapport à n'afficher que D ? (c) Comparez au regret moyen de Thompson sur le même nombre de visiteurs. (d) Que perd-on, en échange, du côté de l'inférence ?

Exercice 9.9 ★ — L'erreur de différence temporelle (section 9.3.2)

$V(s) = 5$, on observe la récompense $r = 2$ et l'état suivant s' avec $V(s') = 6$. Avec $\gamma = 0,9$ et $\alpha = 0,1$, calculez l'erreur de différence temporelle δ et la nouvelle valeur de $V(s)$. L'état était-il meilleur ou moins bon que prévu ?

Exercice 9.10 ★★ — Q-learning ou SARSA ? (sections 9.3.3 et 9.3.4)

On est dans l'état s , on joue l'action a et on observe $r = -1$ et l'état s' . On a $Q(s, a) = 4$, et dans s' : $Q(s', \text{gauche}) = 3$, $Q(s', \text{droite}) = 7$. L'agent explore et joue ensuite $a' = \text{gauche}$. Avec $\gamma = 0,9$ et $\alpha = 0,5$, calculez la mise à jour de $Q(s, a)$ (a) par Q-learning ; (b) par SARSA. Pourquoi différentes ?

Exercice 9.11 ★★ — Quels pas d'apprentissage garantissent la convergence ? (section 9.3.6)

Pour chacun des pas $\alpha_n = 1/n$, $1/\sqrt{n}$, $1/n^{0,7}$ et $0,1$, dites si les conditions de Robbins et Monro ($\sum \alpha_n = \infty$, $\sum \alpha_n^2 < \infty$) sont satisfaites. Illustrez par les sommes partielles jusqu'à $n = 100\,000$.

Exercice 9.12 ★★★ — La malédiction de la dimension (sections 9.3.8 et 9.4.1)

- (a) Dix articles ont chacun un stock de 0 à 20. Combien d'états ? (b) Si l'agent visite un nouvel état par jour, combien d'années faudrait-il pour tous les voir une fois ? (c) Un modèle linéaire $Q_\theta(s, a) = \theta_0 + \sum_j \theta_j \text{stock}_j + \sum_j \theta_{10+j} \text{commande}_j$ a combien de paramètres ? Que gagne-t-on, et que risque-t-on ?

Corrigés**Corrigé 9.1**

- (a) $G_0 = 2 + 0,9 \times 0 + 0,9^2 \times 5 + 0,9^3 \times 1 = 2 + 0 + 4,05 + 0,729 = 6,779$. (b) $G = 3 / (1 - 0,95) = 60$ €, avec un horizon effectif de $1 / (1 - 0,95) = 20$ jours.

```
print("(a)", 2 + 0.9 * 0 + 0.9 ** 2 * 5 + 0.9 ** 3 * 1, "| (b)", 3 / (1 - 0.95), "| horizon", 1 / (1 - 0.95))
```

```
(a) 6.779000000000001 | (b) 59.99999999999995 | horizon 19.99999999999982
```

Corrigé 9.2

On résout le système linéaire $(I - \gamma P_\pi)V = r_\pi$ (équation de Bellman d'espérance), où P_π est la matrice de transition sous la politique.

```
suiv = np.array([[0, 1], [1, 2], [2, 2]]) # état suivant [état, action] (0
↳ attendre, 1 offre)
rec = np.array([[0.0, -1.0], [1.0, 0.0], [4.0, 3.0]]) # récompense [état, action]
for nom, pol in [("toujours attendre", [0, 0, 0]), ("offre sauf fidèle", [1, 1, 0])]:
    P = np.zeros((3, 3)); r = np.zeros(3)
    for s in range(3): P[s, suiv[s, pol[s]]] = 1; r[s] = rec[s, pol[s]]
    print(f"{nom:20s} V =", np.linalg.solve(np.eye(3) - 0.9 * P, r).round(3))
```

```
toujours attendre   V = [ 0. 10. 40.]
offre sauf fidèle   V = [31.4 36. 40.]
```

- (a) « Toujours attendre » : $V = (0; 10; 40)$ (un régulier qui attend vaut $1/(1 - 0,9) = 10$, un fidèle 40, un occasionnel 0). (b) « Offre sauf fidèle » : $V = (31,4; 36; 40)$. La seconde est meilleure dans les **deux** premiers états (un gain de 31,4 € pour l'occasionnel, 26 € pour le régulier) et égale pour le fidèle.

Corrigé 9.3

V_1 : en A, x donne $2 + 0,5 \times 0 = 2$, y donne 0 : max 2 ; en B, x donne 1, y donne 3 : max 3. $V_1 = (2, 3)$.
 V_2 : en A, x : $2 + 0,5 \times 2 = 3$; y : $0 + 0,5 \times 3 = 1,5$: max 3. En B, x : $1 + 0,5 \times 2 = 2$; y : $3 + 0,5 \times 3 = 4,5$: max 4,5. $V_2 = (3; 4,5)$.
 V_3 : en A, x : $2 + 0,5 \times 3 = 3,5$; y : $0 + 0,5 \times 4,5 = 2,25$: 3,5. En B, x : $1 + 0,5 \times 3 = 2,5$; y : $3 + 0,5 \times 4,5 = 5,25$: 5,25. $V_3 = (3,5; 5,25)$.
 Optimalité : on devine x en A et y en B, donc $V^*(A) = 2 + 0,5V^*(A) \Rightarrow V^*(A) = 4$ et $V^*(B) = 3 + 0,5V^*(B) \Rightarrow V^*(B) = 6$.
 Vérification : en A, y vaudrait $0 + 0,5 \times 6 = 3 < 4$; en B, x vaudrait $1 + 0,5 \times 4 = 3 < 6$. La politique « x en A, y en B » est donc bien optimale, et $V^* = (4, 6)$.

```
V = np.zeros(2)
for k in range(1, 4):
    V = np.array([max(2 + 0.5 * V[0], 0 + 0.5 * V[1]), max(1 + 0.5 * V[0], 3 + 0.5 * V[1])]);
    print(f"V_{k} =", V)
```

```
V_1 = [2. 3.]
V_2 = [3. 4.5]
V_3 = [3.5 5.25]
```

Corrigé 9.4

Il faut $40 \times 0,9^k < 0,01$, soit $k > \ln(0,01/40) / \ln(0,9) \approx 78,7$: **79 itérations**. La contraction de Bellman donne exactement cette borne $\|V_k - V^*\| \leq \gamma^k \|V_0 - V^*\|$, ici atteinte puisque l'erreur est dominée par l'état fidèle.

```
print("k minimal :", int(np.ceil(np.log(0.01 / 40) / np.log(0.9))))
print("erreur à k = 78 :", round(40 * 0.9 ** 78, 5), "| à k = 79 :", round(40 * 0.9 ** 79, 5))
```

```
k minimal : 79
erreur à k = 78 : 0.01079 | à k = 79 : 0.00971
```

Corrigé 9.5

Les écarts à la meilleure bannière sont $\Delta_A = 0,030$, $\Delta_B = 0,018$, $\Delta_C = 0,012$, $\Delta_D = 0$. Le regret vaut $300 \times 0,030 + 200 \times 0,018 + 100 \times 0,012 + 400 \times 0 = 9 + 3,6 + 1,2 = 13,8$ conversions perdues.

Corrigé 9.6

```
t = 500; n = np.array([100, 150, 250]); conv = np.array([5, 9, 22])
for c in (2.0, 0.1):
    indice = conv / n + np.sqrt(c * np.log(t) / n)
    print(f"c = {c:3.1f} : indices {indice.round(3)} -> bannière choisie :")
    print(f"    {'ABC'[int(np.argmax(indice))]}")
```

```
c = 2.0 : indices [0.403 0.348 0.311] -> bannière choisie : A
c = 0.1 : indices [0.129 0.124 0.138] -> bannière choisie : C
```

Avec le bonus classique, les indices valent environ 0,403 ; 0,348 ; 0,311 : on affiche **A**, la moins affichée, malgré un taux observé de 5 % seulement (son bonus est le plus grand). Avec $c = 0,1$, ils valent environ 0,129 ; 0,124 ; 0,138 : on affiche **C**, la meilleure observée (8,8 %). Le réglage du bonus détermine le comportement : grand c , on explore la bannière la moins connue ; petit c , on exploite la meilleure.

Corrigé 9.7

Lois a posteriori : A suit Bêta(13, 189) (moyenne $13/202 \approx 0,064$), B suit Bêta(10, 92) (moyenne $10/102 \approx 0,098$).

```
rng = np.random.default_rng(3)
a = rng.beta(13, 189, 200000); b = rng.beta(10, 92, 200000)
print("moyennes a posteriori :", round(13 / 202, 4), round(10 / 102, 4), "| P(B > A) =",
      round(float((b > a).mean()), 3))
```

```
moyennes a posteriori : 0.0644 0.098 | P(B > A) = 0.842
```

On trouve $P(\text{B meilleure}) \approx 0,84$: Thompson affichera B environ **84 % du temps** et A 16 %. B est probablement meilleure, mais l'incertitude (100 affichages seulement) justifie de continuer à regarder A.

Corrigé 9.8

- (a) Avec $p_1 = 0,058$, $p_2 = 0,070$, $\bar{p} = 0,064$: $n = 2(1,96 + 0,84)^2 \bar{p}(1 - \bar{p}) / (p_2 - p_1)^2 \approx 6530$ par bannière, soit environ **13 060** visiteurs en tout (13 061 avec les arrondis du code). (b) La moitié des visiteurs voit C, qui perd 0,012 conversion par visiteur : $6530 \times 0,012 \approx 78$ conversions perdues. (c) Simulons Thompson sur 13 060 visiteurs (200 répétitions) :

```
n_bras = 2 * (stats.norm.ppf(0.975) + stats.norm.ppf(0.8)) ** 2 * 0.064 * 0.936 / 0.012 ** 2
print("visiteurs par bannière :", round(n_bras), "| total :", round(2 * n_bras), "| regret du test A/B",
      round(n_bras * 0.012, 1))
cr, _ = jouer("ts", P=np.array([0.058, 0.070]), T=int(2 * n_bras), R=200, seed=12)
print("regret moyen de Thompson sur le même nombre de visiteurs :", round(float(cr[:, -1].mean()), 1))
```

```
visiteurs par bannière : 6530 | total : 13061 | regret du test A/B : 78.4
regret moyen de Thompson sur le même nombre de visiteurs : 22.3
```

Thompson perd nettement moins de conversions : environ 22 en moyenne, contre 78 pour le test A/B. (d) En échange, il fournit une **moins bonne inférence** : la bannière C est affichée beaucoup moins, son taux est estimé avec moins de précision, et le biais d'estimation de l'allocation adaptative (section 9.2.9) complique la comparaison formelle. Si la décision doit être **justifiée** auprès de tiers, le test A/B reste l'outil adapté.

Corrigé 9.9

$\delta = r + \gamma V(s') - V(s) = 2 + 0,9 \times 6 - 5 = 2,4$. Nouvelle valeur : $V(s) = 5 + 0,1 \times 2,4 = 5,24$. $\delta > 0$: l'état était **meilleur que prévu**, on relève sa valeur (d'un dixième de la surprise).

Corrigé 9.10

- (a) **Q-learning** : cible = $r + \gamma \max_{a'} Q(s', a') = -1 + 0,9 \times 7 = 5,3$; $Q(s, a) \leftarrow 4 + 0,5 \times (5,3 - 4) = 4,65$. (b) **SARSA** : cible = $r + \gamma Q(s', a') = -1 + 0,9 \times 3 = 1,7$; $Q(s, a) \leftarrow 4 + 0,5 \times (1,7 - 4) = 2,85$. Elles diffèrent parce que le Q-learning suppose que l'agent jouera **le meilleur coup** (droite) à l'étape suivante, alors que SARSA tient compte de l'action **réellement jouée** (gauche, un coup d'exploration moins bon). C'est exactement la source de la différence de prudence observée dans le couloir.

Corrigé 9.11

- $\alpha_n = 1/n$: $\sum 1/n = \infty$ (série harmonique) et $\sum 1/n^2 = \pi^2/6 < \infty$: **convient**.
- $\alpha_n = 1/\sqrt{n}$: $\sum \alpha_n = \infty$, mais $\sum \alpha_n^2 = \sum 1/n = \infty$: **ne convient pas** (le bruit ne s'éteint pas assez vite).
- $\alpha_n = 1/n^{0,7}$: $\sum \alpha_n = \infty$ (exposant < 1) et $\sum \alpha_n^2 = \sum n^{-1,4} < \infty$ (exposant > 1) : **convient**.
- $\alpha_n = 0,1$: $\sum \alpha_n = \infty$ mais $\sum \alpha_n^2 = \infty$: **ne convient pas**.

```
pas = {"1/n": lambda k: 1 / k, "1/sqrt(n)": lambda k: k ** -0.5, "1/n^0,7": lambda k: k ** -0.7,
      ↪ "constant 0,1": lambda k: 0.1}
for nom, f in pas.items():
    print(f"{nom:13s} somme des pas : {sum(f(k) for k in range(1, 100001)):9.2f} | somme des carrés :
      ↪ {sum(f(k) ** 2 for k in range(1, 100001)):9.4f}")
```

1/n	somme des pas :	12.09		somme des carrés :	1.6449
1/sqrt(n)	somme des pas :	631.00		somme des carrés :	12.0901
1/n^0,7	somme des pas :	102.63		somme des carrés :	3.0805
constant 0,1	somme des pas :	10000.00		somme des carrés :	1000.0000

La somme des carrés se stabilise (1,64 ; 3,08) pour les pas qui conviennent et continue de croître (12,09 ; 1 000) pour les autres.

Corrigé 9.12

- (a) $21^{10} = 16\,679\,880\,978\,201$ états, soit près de 17 000 milliards. (b) À un état par jour, il faudrait environ $1,67 \times 10^{13} / 365 \approx 4,6 \times 10^{10}$ années (46 milliards d'années) pour les voir tous **une seule fois**. (c) Le modèle linéaire a $1 + 10 + 10 = 21$ paramètres. On **gagne** la possibilité de généraliser : l'expérience acquise dans quelques états informe sur les autres. On **risque** un modèle trop simple (la vraie valeur n'est pas linéaire : voir l'effet de seuil du stock) et l'**instabilité** de la triade mortelle (section 9.4.4), puisque l'approximation de fonction est combinée à l'amorçage et à l'apprentissage hors politique.

```
print("états :", 21 ** 10, "| années pour tout voir une fois :", f"{21 ** 10 / 365:.2e}", "|
      ↪ paramètres :", 1 + 10 + 10)
```

```
états : 16679880978201 | années pour tout voir une fois : 4.57e+10 | paramètres : 21
```

Projet du volume et auto-évaluation — exercices et applications

Ce chapitre du cahier clôt le volume III. Il contient **le projet du volume** (un pipeline d'apprentissage automatique complet sur un jeu de données **réel**, du modèle de référence à l'interprétation), puis l'**auto-évaluation**. Il ne demande aucune notion nouvelle : chaque étape renvoie à la section du livre qui l'explique. Le plus profitable : lire le cahier des charges, essayer seul, puis comparer.

Projet du volume

P.1 Le cahier des charges

Un établissement de crédit possède l'historique de **30 000 clients** titulaires d'une carte de crédit : limite de crédit, âge, situation, six mois d'historique de retards, de factures et de paiements. Pour chacun, on sait si le client a **fait défaut le mois suivant** (22 % des cas). Il veut un **modèle de risque** pour décider qui contacter en priorité (relance, révision de la limite).

Les données sont réelles. Il s'agit du jeu « Default of Credit Card Clients » de l'UCI, publié sous licence CC0 (Yeh et Lien, 2009) : 30 000 clients d'une banque d'Asie en 2005. Le fichier `donnees/credit_default.csv` est fourni, aucune connexion n'est nécessaire. Les colonnes sont décrites en tête de `build/telecharger_credit.py`. Contrairement aux données simulées de la boutique, **on ne connaît pas la vérité** : on ne peut que mesurer, comparer et rester prudent.

La méthode suit onze étapes, chacune appuyée sur un chapitre du livre :

Étape	Question	Chapitre du livre
P.2 Cadrer	Quelle décision ? Quelle métrique ? Quel coût d'erreur ?	1.1, 5.1
P.3 Auditer	Les données sont-elles saines ? Y a-t-il une fuite ?	1.1, 4.1
P.4 Séparer et références	Quel est le score à battre ?	1.1, 1.4
P.5 Variables	Peut-on aider le modèle ?	4.1, 4.2
P.6 Comparer	Quel modèle, avec quelle incertitude ?	1.2, 2.1 à 2.4

Étape	Question	Chapitre du livre
P.7 Régler	Que gagne-t-on à ajuster les hyperparamètres ?	1.5
P.8 Décider	Quel seuil, au vu des coûts ?	4.3.5, 5.1.7
P.9 Calibrer	Les probabilités sont-elles fiables ?	5.2
P.10 Évaluer une fois	Combien vaut le modèle final, avec sa marge d'erreur ?	1.1, 1.4, 5.1
P.11 Comprendre et vérifier l'équité	Pourquoi ces prédictions ? Sont-elles équitables ?	5.3, 5.4
P.12 Rapporter	Que dire, avec quelles limites ?	1.4

P.2 Étape 1 : cadrer

Avant de coder, on écrit **ce que le modèle servira à décider**. Ici : classer les clients par risque de défaut pour **choisir qui relancer**, avec un budget de contacts limité. Deux conséquences :

- on a besoin d'un **bon classement** (AUC, précision moyenne) et de **probabilités fiables** (calibration), pas seulement d'une étiquette ;
- les deux erreurs ne coûtent pas la même chose. Nous posons une **hypothèse de coût**, à discuter avec le métier : laisser passer un défaut coûte **5 unités**, une relance inutile en coûte **1** (livre, section 5.1.7).

La métrique principale sera la **précision moyenne** (*average precision*, section 5.1) car la classe positive est minoritaire, avec l'AUC et le **coût attendu** en complément. Les décisions de modélisation se prendront sur des données d'**entraînement** et de **validation** ; le **jeu de test** ne sera utilisé **qu'une fois**, à l'étape P.10.

P.3 Étape 2 : auditer les données

```
import warnings
import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")
d = pd.read_csv("donnees/credit_default.csv")
print(d.shape, "| taux de défaut :", round(d["default"].mean(), 4), "| valeurs manquantes :",
      ↪ int(d.isna().sum().sum()))
for col in ["sex", "education", "marriage"]:
    print(col, d[col].value_counts().sort_index().to_dict())
print("pay_1 :", d["pay_1"].value_counts().sort_index().to_dict())
```

```
(30000, 24) | taux de défaut : 0.2212 | valeurs manquantes : 0
sex {1: 11888, 2: 18112}
education {0: 14, 1: 10585, 2: 14030, 3: 4917, 4: 123, 5: 280, 6: 51}
marriage {0: 54, 1: 13659, 2: 15964, 3: 323}
pay_1 : {-2: 2759, -1: 5686, 0: 14737, 1: 3688, 2: 2667, 3: 322, 4: 76, 5: 26, 6: 11, 7: 9, 8: 19}
```

Trois constats d'audit (livre, section 4.1) :

1. **Aucune valeur manquante**, mais des **codes non documentés** : education prend les valeurs 0, 5 et 6 et marriage la valeur 0, que la documentation ne prévoit pas. On les regroupera avec la modalité « autre ».
2. pay_1 à pay_6 mêlent des **états** (−2, −1, 0 : pas de retard) et des **mois de retard** (1 à 8) : c'est une variable ordinale, pas une grandeur continue.

3. **Pas de fuite évidente** : toutes les colonnes décrivent l'historique de septembre à avril, la cible porte sur le mois suivant. En revanche, **le jeu n'a pas de date par client** : on ne peut pas faire de séparation temporelle, et on le reconnaîtra dans les limites.

P.4 Étape 3 : séparer et fixer des références

On sépare 60 % / 20 % / 20 % (entraînement, validation, test) en conservant la proportion de défauts dans chaque jeu (stratification), avec une graine fixe.

```
from sklearn.model_selection import train_test_split

X, y = d.drop(columns="default", d["default"])
X_tv, X_te, y_tv, y_te = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
X_tr, X_va, y_tr, y_va = train_test_split(X_tv, y_tv, test_size=0.25, random_state=42, stratify=y_tv)
print(len(X_tr), len(X_va), len(X_te), "| taux de défaut :", [round(float(v.mean()), 3) for v in
↪ (y_tr, y_va, y_te)])
```

```
18000 6000 6000 | taux de défaut : [0.221, 0.221, 0.221]
```

Les **références** (livre, section 1.4) fixent le score à battre : prédire toujours la proportion de défauts, appliquer une **règle simple** (« défaut si au moins un mois de retard au dernier relevé »), et une **régression logistique** standardisée.

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, brier_score_loss, roc_auc_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

def scores(nom, p):
    return {"modèle": nom, "AUC": roc_auc_score(y_va, p), "précision moy.":
↪ average_precision_score(y_va, p), "Brier": brier_score_loss(y_va, np.clip(p, 0, 1))}

dummy = DummyClassifier(strategy="prior").fit(X_tr, y_tr)
logit = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(X_tr, y_tr)
refs = pd.DataFrame([scores("toujours la proportion", dummy.predict_proba(X_va[:, 1])),
↪ scores("règle : retard ≥ 1 mois", (X_va["pay_1"] >= 1).astype(float)),
↪ scores("régression logistique", logit.predict_proba(X_va[:, 1]))])
print(refs.round(4).to_string(index=False))
```

	modèle	AUC	précision moy.	Brier
	toujours la proportion	0.5000	0.2212	0.1723
	règle : retard ≥ 1 mois	0.6793	0.3602	0.2222
	régression logistique	0.7242	0.4978	0.1455

La règle à une ligne obtient déjà une AUC de 0,68 ; la régression logistique atteint 0,72. Tout modèle plus complexe devra **justifier son surcroît de complexité** par un gain mesuré.

P.5 Étape 4 : des variables qui parlent au métier

Le livre (section 4.2) rappelle que de bonnes variables valent souvent mieux qu'un modèle plus complexe. Ici, le **raisonnement métier** suggère trois familles :

- le **taux d'utilisation** de la limite : facture divisée par limite de crédit ;
- le **taux de remboursement** : paiement divisé par la facture du mois ;
- des **résumés de l'historique de retards** : nombre de mois en retard, retard maximal, retard moyen.

On les écrit dans une fonction **sans état** (elle n'apprend rien sur les données : pas de risque de fuite), puis on la place dans un Pipeline.

```
def variables(X):
    X = X.copy()
    for k in range(1, 7):
        X[f"util_{k}"] = X[f"bill_amt{k}"] / X["limit_bal"]
        X[f"ratio_paiement_{k}"] = X[f"pay_amt{k}"] / (X[f"bill_amt{k}"].abs() + 1)
    retards = [f"pay_{k}" for k in range(1, 7)]
    X["nb_mois_retard"] = (X[retards] > 0).sum(axis=1)
    X["retard_max"], X["retard_moyen"] = X[retards].max(axis=1), X[retards].mean(axis=1)
    X["education"] = X["education"].replace({0: 4, 5: 4, 6: 4})
    X["marriage"] = X["marriage"].replace({0: 3})
    return X

F_tr, F_va = variables(X_tr), variables(X_va)
logit_f = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000)).fit(F_tr, y_tr)
print(pd.DataFrame([scores("logistique, variables brutes", logit.predict_proba(X_va)[:, 1]),
                        scores("logistique, variables métier", logit_f.predict_proba(F_va)[:,
                        1])]).round(4).to_string(index=False))
```

	modèle	AUC	précision moy.	Brier
logistique, variables brutes	0.7242		0.4978	0.1455
logistique, variables métier	0.7525		0.5046	0.1412

Le modèle **linéaire** gagne environ 0,03 d'AUC (0,724 → 0,753) ; la précision moyenne progresse peu (0,498 → 0,505). Les variables métier résument l'historique sous une forme plus directement exploitable par un modèle linéaire, mais le gros du signal était déjà dans pay_1.

P.6 Étape 5 : comparer des modèles, avec leur incertitude

On compare trois familles (livre, chapitre 2) par **validation croisée à 5 plis répétée** sur le jeu d'entraînement : la même découpe pour tous les modèles, de sorte que l'on puisse comparer les scores **pli par pli** (comparaison appariée, section 1.4).

```
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score

cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=2, random_state=0)
modeles = {
    "logistique": make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000)),
    "forêt aléatoire": RandomForestClassifier(100, min_samples_leaf=5, random_state=0, n_jobs=2),
    "boosting": HistGradientBoostingClassifier(random_state=0),
}
cv_scores = {nom: cross_val_score(m, F_tr, y_tr, cv=cv, scoring="average_precision") for nom, m in
    modeles.items()}
tab = pd.DataFrame([s.mean(), s.std(ddof=1)] for n, s in cv_scores.items()), index=["moyenne",
    "écart-type des plis"]
print(tab.round(4).to_string())
```

	moyenne	écart-type des plis
logistique	0.5125	0.0107
forêt aléatoire	0.5538	0.0119
boosting	0.5585	0.0128

Les scores des plis sont **corrélés** (les jeux d'entraînement se recouvrent) : un test de Student naïf sur leur différence serait trop optimiste. On utilise la **correction de Nadeau-Bengio** (livre, section 1.4.2), qui gonfle la variance du facteur $\frac{1}{k} + \frac{n_{test}}{n_{train}}$:

```
from scipy import stats

def difference_corrige(a, b, n_test_sur_train=0.25):
    diff = np.asarray(a) - np.asarray(b)
    k = len(diff)
```

```

t = diff.mean() / np.sqrt((1 / k + n_test_sur_train) * diff.var(ddof=1))
return diff.mean(), 2 * stats.t.sf(abs(t), k - 1)

for a, b in [("boosting", "logistique"), ("forêt aléatoire", "logistique"), ("boosting", "forêt
↳ aléatoire")]:
    ecart, p = difference_corrigee(cv_scores[a], cv_scores[b])
    print(f"{a:16s} - {b:16s} : écart de précision moyenne = {ecart:+.4f}    p corrigée = {p:.4f}")

```

boosting	- logistique	: écart de précision moyenne = +0.0460	p corrigée = 0.0000
forêt aléatoire	- logistique	: écart de précision moyenne = +0.0413	p corrigée = 0.0000
boosting	- forêt aléatoire	: écart de précision moyenne = +0.0048	p corrigée = 0.0639

Lecture : le boosting et la forêt battent la régression logistique d'environ 0,04 à 0,05 de précision moyenne, un écart **nettement supérieur au bruit** des plis (p corrigée proche de 0). En revanche, l'avantage du boosting sur la forêt (0,005) est **trop petit pour être établi** : la p-valeur corrigée (0,064) ne passe pas le seuil de 5 %. On retiendra le boosting pour sa rapidité d'entraînement et parce qu'il se prête mieux au réglage, pas parce qu'il serait prouvé meilleur.

P.7 Étape 6 : régler le boosting avec un budget limité

Le **réglage** est une optimisation : on la fait **sur l'entraînement uniquement**, avec la validation croisée, jamais sur le test (section 1.5). Un budget de **20 essais** d'Optuna explore la profondeur, le pas d'apprentissage et la régularisation d'un boosting LightGBM.

```

import lightgbm as lgb
import optuna
from sklearn.model_selection import StratifiedKFold

optuna.logging.set_verbosity(optuna.logging.WARNING)
cv3 = StratifiedKFold(n_splits=3, shuffle=True, random_state=1)

def objectif(trial):
    params = dict(n_estimators=300, learning_rate=trial.suggest_float("learning_rate", 0.01, 0.2,
↳ log=True),
                  num_leaves=trial.suggest_int("num_leaves", 4, 64, log=True),
↳ min_child_samples=trial.suggest_int("min_child_samples", 10, 200, log=True),
                  reg_lambda=trial.suggest_float("reg_lambda", 1e-3, 30, log=True), subsample=0.8,
↳ subsample_freq=1,
                  colsample_bytree=trial.suggest_float("colsample_bytree", 0.4, 1.0), random_state=0,
↳ n_jobs=2, verbose=-1)
    return cross_val_score(lgb.LGBMClassifier(**params), F_tr, y_tr, cv=cv3,
↳ scoring="average_precision").mean()

```

```

etude = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=0))
etude.optimize(objectif, n_trials=20)
print("meilleure précision moyenne en validation croisée :", round(etude.best_value, 4))
print({k: round(v, 4) if isinstance(v, float) else v for k, v in etude.best_params.items()})

```

```

meilleure précision moyenne en validation croisée : 0.5648
{'learning_rate': 0.0349, 'num_leaves': 10, 'min_child_samples': 32, 'reg_lambda': 0.0018,
↳ 'colsample_bytree': 0.9249}

```

```

base = cross_val_score(lgb.LGBMClassifier(n_estimators=300, random_state=0, n_jobs=2, verbose=-1),
↳ F_tr, y_tr, cv=cv3, scoring="average_precision").mean()
print(f"LightGBM par défaut : {base:.4f} | réglé : {etude.best_value:.4f} | gain :
↳ {etude.best_value - base:+.4f}")

```

```
LightGBM par défaut : 0.5383 | réglé : 0.5648 | gain : +0.0265
```

Le réglage apporte **+0,027 de précision moyenne** (0,538 → 0,565, soit environ 5 % de mieux) :

un gain réel mais modeste, obtenu avec 20 essais seulement. Notons que le meilleur score de la validation croisée est **légèrement optimiste** (on a retenu le maximum de 20 essais) : c'est l'une des raisons pour lesquelles on garde un jeu de test intact.

P.8 Étape 7 : choisir un seuil, en fonction des coûts

On ajuste le modèle réglé sur l'entraînement et on regarde le classement sur la **validation**. Pour décider **qui relancer**, il faut un **seuil**. Avec nos coûts (5 pour un défaut manqué, 1 pour une relance inutile), relancer un client est rentable dès que sa probabilité de défaut dépasse $\frac{1}{1+5} \approx 0,167$ (livre, section 5.1.7). On vérifie sur la validation en balayant les seuils.

```
modele = lgb.LGBMClassifier(n_estimators=300, **{**etude.best_params}, subsample=0.8,
    ↳ subsample_freq=1, random_state=0, n_jobs=2, verbose=-1).fit(F_tr, y_tr)
p_va = modele.predict_proba(F_va)[ :, 1]
cout = lambda seuil, y, p: 5 * np.sum((p < seuil) & (y == 1)) + 1 * np.sum((p >= seuil) & (y == 0))
seuils = np.linspace(0.05, 0.6, 56)
couts = np.array([cout(s, y_va.to_numpy(), p_va) for s in seuils])
s_opt = seuils[couts.argmin()]
print(f"seuil théorique 1/6 = {1/6:.3f} | seuil qui minimise le coût sur la validation = {s_opt:.2f}")
print(f"coût à ce seuil : {couts.min():.0f} | coût de « ne relancer personne » : {5 * y_va.sum()} |
    ↳ coût de « relancer tout le monde » : {(y_va == 0).sum()}")
print(f"à ce seuil : {np.mean(p_va >= s_opt):.1%} des clients relancés, {np.mean(p_va[y_va == 1] >=
    ↳ s_opt):.1%} des défauts attrapés")
seuils_voisins = [round(float(s_opt) - 0.05, 2), round(float(s_opt), 2), round(float(s_opt) + 0.05,
    ↳ 2)]
print("coûts aux seuils", seuils_voisins, ":", [int(cout(s, y_va.to_numpy(), p_va)) for s in
    ↳ seuils_voisins])
print(f"AUC en validation du modèle réglé : {roc_auc_score(y_va, p_va):.4f}")
```

```
seuil théorique 1/6 = 0.167 | seuil qui minimise le coût sur la validation = 0.17
coût à ce seuil : 3205 | coût de « ne relancer personne » : 6635 | coût de « relancer tout le monde »
    ↳ : 4673
à ce seuil : 41.5% des clients relancés, 74.4% des défauts attrapés
coûts aux seuils [0.12, 0.17, 0.22] : [3342, 3205, 3392]
AUC en validation du modèle réglé : 0.7873
```

Le seuil qui minimise le coût sur la validation (0,17) coïncide avec le seuil théorique (1/6), ce qui est rassurant : c'est le signe que les probabilités sont correctes. Relancer selon le modèle divise le coût par plus de deux par rapport à ne relancer personne (3 205 contre 6 635) et le réduit d'environ un tiers par rapport à relancer tout le monde (4 673). Les coûts aux seuils voisins (0,12 et 0,22 : 3 342 et 3 392, contre 3 205) montrent que la courbe est assez plate autour de l'optimum (+4 à +6 %) : mieux vaut un seuil choisi **d'après les coûts** qu'un 0,5 pris par habitude, mais sa valeur exacte compte peu.

P.9 Étape 8 : les probabilités sont-elles fiables ?

Le seuil théorique $\frac{1}{6}$ n'est valable que si les **probabilités sont calibrées** (livre, section 5.2). Vérifions par un **diagnostic de fiabilité** : on groupe les clients par probabilité prédite et on compare à la fréquence observée.

```
from sklearn.calibration import calibration_curve

frac, moy = calibration_curve(y_va, p_va, n_bins=8, strategy="quantile")
print(pd.DataFrame({"probabilité prédite moyenne": moy, "fréquence observée":
    ↳ frac}).round(3).to_string(index=False))
```

probabilité prédite moyenne	fréquence observée
0.041	0.052
0.068	0.075

0.098	0.091
0.131	0.131
0.164	0.167
0.224	0.235
0.355	0.349
0.660	0.671

Les probabilités prédites et les fréquences observées sont proches dans les huit groupes (par exemple 0,164 prédit contre 0,167 observé près du seuil de 0,17) : le modèle est **bien calibré**. C'est en partie parce qu'on l'a entraîné **sans pondérer les classes ni rééchantillonner** : ces techniques déforment les probabilités (livre, section 5.2). Si le modèle n'avait pas été calibré, on aurait appliqué une **régression isotonique** ou la méthode de Platt, à l'intérieur de la validation croisée.

P.10 Étape 9 : évaluer une seule fois sur le test

Le modèle est figé : variables, hyperparamètres, seuil. On l'**ajuste sur l'ensemble entraînement + validation** et on l'évalue **une seule fois** sur le test. Pour exprimer l'incertitude, on calcule un **intervalle de confiance par bootstrap** du jeu de test (livre, section 1.4).

```
F_tv, F_te = variables(X_tv), variables(X_te)
final = lgb.LGBMClassifier(n_estimators=300, **etude.best_params, subsample=0.8, subsample_freq=1,
    ↪ random_state=0, n_jobs=2, verbose=-1).fit(F_tv, y_tv)
p_te = final.predict_proba(F_te)[: , 1]
rng = np.random.default_rng(7)
boot = []
for _ in range(500):
    i = rng.integers(0, len(y_te), len(y_te))
    boot.append((roc_auc_score(y_te.iloc[i], p_te[i]), average_precision_score(y_te.iloc[i],
    ↪ p_te[i])))
boot = np.array(boot)
for j, nom in enumerate(["AUC", "précision moyenne"]):
    print(f"{nom:18s}: {[roc_auc_score, average_precision_score][j](y_te, p_te):.4f}   IC95 bootstrap
    ↪ [{np.percentile(boot[:, j], 2.5):.4f} ; {np.percentile(boot[:, j], 97.5):.4f}]" )
print(f"coût au seuil {s_opt:.2f} : {cout(s_opt, y_te.to_numpy(), p_te):.0f}   (référence « ne
    ↪ relancer personne » : {5 * y_te.sum():.0f})")
```

```
AUC                : 0.7822   IC95 bootstrap [0.7680 ; 0.7972]
précision moyenne : 0.5604   IC95 bootstrap [0.5343 ; 0.5887]
coût au seuil 0.17 : 3353   (référence « ne relancer personne » : 6635)
```

L'AUC sur le test (0,782) est proche de celle mesurée en validation (ligne « AUC en validation » de l'étape P.8) : pas de signe de sur-ajustement au jeu de validation. Surtout, **l'intervalle de confiance** rappelle l'incertitude : il s'étend sur environ $\pm 0,015$, de sorte que des différences d'AUC de quelques millièmes entre deux modèles seraient du bruit. Enfin le coût au seuil retenu (3 353) reste environ moitié de celui de « ne relancer personne » (6 635).

P.11 Étape 10 : comprendre, puis vérifier l'équité

Pourquoi ces prédictions ? Deux outils (livre, section 5.3) : l'**importance par permutation** (global, calculée sur le test) et les **valeurs de SHAP** (global et local).

```
from sklearn.inspection import permutation_importance

pi = permutation_importance(final, F_te, y_te, scoring="average_precision", n_repeats=5,
    ↪ random_state=0, n_jobs=2)
imp = pd.Series(pi.importances_mean, index=F_te.columns).sort_values(ascending=False)
print(imp.head(8).round(4).to_string())
```

```
pay_1      0.0973
retard_max 0.0310
bill_amt1  0.0145
nb_mois_retard 0.0124
retard_moyen 0.0079
pay_amt1   0.0047
limit_bal  0.0040
pay_amt2   0.0037
```

```
import shap

echantillon = F_te.sample(500, random_state=0)
valeurs = shap.TreeExplainer(final).shap_values(echantillon)
valeurs = valeurs[1] if isinstance(valeurs, list) else valeurs
moy_abs = pd.Series(np.abs(valeurs).mean(axis=0), index=F_te.columns).sort_values(ascending=False)
print(moy_abs.head(8).round(3).to_string())
```

```
pay_1      0.289
retard_max 0.289
nb_mois_retard 0.157
bill_amt1  0.124
util_2     0.106
pay_amt2   0.097
limit_bal  0.095
pay_amt1   0.085
```

Les deux méthodes classent en tête l'**historique de retards** : pay_1 (le statut de septembre), retard_max et nb_mois_retard, suivis par des montants de factures et de paiements. C'est ce que le métier attendrait ; la valeur de pay_1 est loin devant les autres dans l'importance par permutation (0,097 contre 0,031 pour la suivante). L'importance **n'est pas une causalité** : « le retard de septembre fait monter le risque » décrit ce que le modèle utilise, pas ce qui arriverait si l'on intervenait sur le retard.

Équité. Le jeu contient des attributs **sensibles** (sexe, âge, situation). Même si le modèle ne les utilise pas comme principale information, on doit **vérifier** s'il traite les groupes de façon comparable (livre, section 5.4). Nous ne les avons pas exclus du modèle ; nous mesurons trois indicateurs par groupe, au seuil retenu : le **taux de clients relancés**, le **taux de défauts attrapés** (rappel) et le **taux de fausses relances**.

```
def par_groupe(var, groupes):
    lignes = []
    for g, idx in groupes.items():
        y_g, p_g = y_te[idx].to_numpy(), p_te[idx].to_numpy()
        sel = p_g >= s_opt
        lignes.append({var: g, "clients": len(y_g), "taux de défaut": y_g.mean(), "relancés":
↪ sel.mean(),
                        "défauts attrapés": sel[y_g == 1].mean(), "fausses relances": sel[y_g ==
↪ 0].mean()})
    return pd.DataFrame(lignes)

sexe = X_te["sex"].map({1: "homme", 2: "femme"})
print(par_groupe("sexe", {g: sexe == g for g in ["homme", "femme"]}).round(3).to_string(index=False))
tranche = pd.cut(X_te["age"], [0, 29, 39, 49, 120], labels=["< 30", "30-39", "40-49", "50 et +"])
print(par_groupe("âge", {g: tranche == g for g in
↪ tranche.cat.categories}).round(3).to_string(index=False))
```

sexe	clients	taux de défaut	relancés	défauts attrapés	fausses relances
homme	2402	0.234	0.463	0.754	0.374
femme	3598	0.213	0.402	0.718	0.316
âge	clients	taux de défaut	relancés	défauts attrapés	fausses relances
< 30	1912	0.223	0.439	0.754	0.348

30-39	2198	0.206	0.392	0.717	0.308
40-49	1341	0.233	0.434	0.721	0.347
50 et +	549	0.246	0.497	0.748	0.415

Lecture chiffrée. Les hommes sont relancés plus souvent que les femmes (46 % contre 40 %), ce qui va avec un taux de défaut plus élevé (23 % contre 21 %) ; leur rappel est un peu plus haut (75 % contre 72 %), mais ils subissent aussi plus de fausses relances (37 % contre 32 %). Par âge, c'est le groupe des 50 ans et plus qui est le plus relancé (50 %) et celui qui a le plus de fausses relances (42 %), mais il ne compte que 549 clients : ses taux sont peu précis.

Lecture prudente. Des écarts de taux **ne prouvent pas** une discrimination, et leur absence ne prouve pas l'équité : les groupes ont des taux de défaut de base différents, et les critères d'équité (parité, égalité des chances, calibration) **ne peuvent pas tous être satisfaits en même temps** quand ces taux diffèrent (livre, section 5.4). Un tel tableau sert à **ouvrir la discussion** avec les juristes et le métier : quels écarts sont acceptables ? quel critère privilégier ? l'usage de l'âge ou du sexe est-il licite dans le contexte ? Ce ne sont pas des questions que le modèle peut trancher seul.

P.12 Étape 11 : le rapport

Comme dans les volumes précédents, le rapport est **généré à partir des résultats déjà calculés**, sans nombre recopié à la main.

```
def fr(x, d=1):
    return f"{x:,.{d}f}".replace(",", " ").replace(".", ",")

ic_auc = np.percentile(boot[:, 0], [2.5, 97.5])
rapport = f""RAPPORT : MODÈLE DE RISQUE DE DÉFAUT
{"=" * 60}
Données : {fr(len(d), 0)} clients, taux de défaut {fr(100 * d['default'].mean())} %. Séparation
↳ 60/20/20, graine fixe.

1. Références (validation)
- règle « retard ≥ 1 mois » : AUC {fr(roc_auc_score(y_va, (X_va['pay_1'] >= 1).astype(float)), 3)}
- régression logistique : AUC {fr(roc_auc_score(y_va, logit.predict_proba(X_va[:, 1]), 3)}

2. Modèle retenu : boosting (LightGBM) sur variables métier, hyperparamètres réglés (20 essais)
- test, une seule évaluation : AUC {fr(roc_auc_score(y_te, p_te), 3)} (IC95 : {fr(ic_auc[0], 3)} à
↳ {fr(ic_auc[1], 3)})
- seuil de relance {fr(s_opt, 2)} (coût défaut manqué = 5, relance inutile = 1) : {fr(100 *
↳ np.mean(p_te >= s_opt))} % des clients relancés,
{fr(100 * np.mean(p_te[y_te == 1] >= s_opt))} % des défauts attrapés

3. Limites
- un seul établissement, une seule période, pas de date par client : pas de test hors temps ;
- coûts des erreurs supposés, à valider avec le métier ;
- importance et valeurs de SHAP décrivent le modèle, pas des causes ;
- indicateurs d'équité calculés pour le sexe et l'âge : à discuter, pas à conclure.
"""
print(rapport)
```

RAPPORT : MODÈLE DE RISQUE DE DÉFAUT

=====

Données : 30 000 clients, taux de défaut 22,1 %. Séparation 60/20/20, graine fixe.

1. Références (validation)

- règle « retard ≥ 1 mois » : AUC 0,679
- régression logistique : AUC 0,724

2. Modèle retenu : boosting (LightGBM) sur variables métier, hyperparamètres réglés (20 essais)
 - test, une seule évaluation : AUC 0,782 (IC95 : 0,768 à 0,797)
 - seuil de relance 0,17 (coût défaut manqué = 5, relance inutile = 1) : 42,6 % des clients relancés, 73,3 % des défauts attrapés
3. Limites
 - un seul établissement, une seule période, pas de date par client : pas de test hors temps ;
 - coûts des erreurs supposés, à valider avec le métier ;
 - importance et valeurs de SHAP décrivent le modèle, pas des causes ;
 - indicateurs d'équité calculés pour le sexe et l'âge : à discuter, pas à conclure.

À retenir. Un projet d'apprentissage automatique solide, c'est : **cadrer** la décision et les coûts, **auditer** les données, fixer des **références**, séparer correctement et **ne toucher au test qu'une fois, comparer avec leur incertitude, choisir le seuil** en fonction des coûts, **vérifier la calibration, expliquer et vérifier l'équité**, puis **rapporter** les limites. Le modèle lui-même n'est qu'une des onze étapes.

P.13 Les limites de l'étude

- **Pas de séparation temporelle.** Le jeu n'a pas de date par client : on ne peut pas vérifier que le modèle tient dans le temps (dérive des données, changement de politique de crédit). En production, cette vérification est indispensable.
- **Un seul établissement et une seule période.** Les performances ne se transportent pas automatiquement à un autre pays ou à une autre année.
- **Les coûts sont une hypothèse.** Le seuil optimal dépend entièrement du rapport 5:1 posé au cadrage ; une analyse de **sensibilité** à ce rapport serait la suite naturelle.
- **Prédire n'est pas expliquer, ni décider.** Un modèle de risque ne dit pas **quelle action** réduirait le défaut ; pour cela, il faut des expériences (volume II, chapitre 7 et chapitre 8).
- **Les décisions affectent des personnes.** Relancer, réduire une limite ou refuser un crédit a des conséquences : documentation, contestation, surveillance de l'équité et supervision humaine font partie du projet.

Auto-évaluation

Mode d'emploi. Répondez à voix haute ou par écrit **avant** de lire le corrigé, en une ou deux phrases. Les sections à relire sont indiquées dans le corrigé. Trente-cinq bonnes réponses sur les quarante signalent un volume bien assimilé ; les questions 33 à 40 portent sur les chapitres facultatifs (6 à 9) et ne comptent que si vous les avez lus.

La démarche (chapitre 1)

1. Pourquoi l'erreur mesurée sur les données d'entraînement est-elle un mauvais indicateur de la performance future ?
2. Qu'appelle-t-on « fuite d'information » ? Donnez deux formes typiques.
3. Avec une validation croisée à 5 plis sur 1 000 observations, combien d'observations servent à l'entraînement et à l'évaluation à chaque pli, et combien de modèles ajuste-t-on ?
4. Pourquoi l'écart-type des scores des plis n'est-il pas une barre d'erreur valable pour la performance du modèle ?
5. Comment se décompose l'erreur quadratique attendue d'un modèle ? Que se passe-t-il quand on augmente la complexité ?

6. Un modèle obtient une AUC de 0,99 à l'entraînement et de 0,80 en validation. Que diagnostiquez-vous, et que tentez-vous ?
7. Pourquoi se compare-t-on toujours à une référence simple, et pourquoi n'ouvre-t-on le jeu de test qu'une fois ?

Les modèles supervisés (chapitre 2)

8. Pourquoi minimise-t-on une perte « logistique » plutôt que le taux d'erreur de classification ?
9. Quelle est l'impureté de Gini d'un nœud contenant 30 clients qui ont résilié et 70 qui n'ont pas résilié ?
10. Quelle est la variance de la moyenne de $B = 100$ prédictors de variance $\sigma^2 = 1$ et de corrélation $\rho = 0,3$? Que montre ce résultat sur les forêts aléatoires ?
11. Que « apprend » chaque nouvel arbre dans un gradient boosting ?
12. Pourquoi l'importance « par impureté » d'une forêt est-elle trompeuse ? Que lui préfère-t-on ?
13. Quel est le rôle du pas d'apprentissage dans un boosting, et à quoi sert l'arrêt précoce ?
14. Quels modèles exigent des variables à la même échelle, et lesquels s'en passent ?

Le non supervisé (chapitre 3)

15. Comment valider un regroupement quand on n'a aucune étiquette ?
16. Pour un point, la distance moyenne aux autres points de son groupe est 2 et la distance moyenne au groupe voisin le plus proche est 5. Quelle est sa silhouette ?
17. Pourquoi standardise-t-on les variables avant les k-means ?
18. L'ACP garde 70 % de la variance avec ses trois premières composantes. Quelle est l'erreur de reconstruction relative ?
19. Que ne faut-il surtout pas lire sur une carte t-SNE ou UMAP ?

Les variables et le déséquilibre (chapitre 4)

20. Pourquoi un encodage « par la cible » calculé naïvement est-il trompeur, et comment le corriger ?
21. Les arbres ont-ils besoin de mise à l'échelle ? Et la régression logistique pénalisée ?
22. Qu'est-ce qu'une valeur manquante « MNAR » ? Que faire ?
23. Une relance inutile coûte 1 € et un défaut manqué 5 €. À partir de quelle probabilité de défaut relance-t-on ?
24. Quel effet ont les poids de classes et le rééchantillonnage sur les probabilités prédites et sur le classement des clients ?
25. Pourquoi ne doit-on jamais appliquer SMOTE avant la validation croisée ?

Évaluation, calibration, interprétabilité (chapitre 5)

26. Une matrice de confusion donne $TP = 40$, $FP = 60$, $FN = 10$, $TN = 890$. Calculez exactitude, précision, rappel et F_1 .
27. Quelle est l'interprétation probabiliste de l'AUC ?
28. Pourquoi la précision moyenne est-elle plus informative que l'AUC quand la classe positive est rare ?
29. Qu'est-ce qu'un modèle calibré ? Comment le vérifie-t-on et comment le répare-t-on ?
30. Calculez le score de Brier des prédictions 0,9 ; 0,2 ; 0,7 pour les étiquettes 1 ; 0 ; 0.
31. Que garantit la propriété d'efficacité des valeurs de Shapley ? SHAP établit-il des causes ?
32. Citez trois critères d'équité. Peut-on tous les satisfaire ?

Les chapitres facultatifs (6 à 9)

33. Un détecteur de fraude affiche 99,19 % d'exactitude sur des données où 0,81 % des commandes sont frauduleuses. Que valent ces 99,19 % ?
34. Dans une forêt d'isolement, que vaut le score d'anomalie quand la longueur de chemin moyenne d'un point est égale à $c(n)$?
35. Trois clients achètent le produit A, quatre le produit B, et deux clients achètent les deux. Quelle est leur similarité cosinus ?
36. Un classement place des produits de pertinences (1, 0, 1). Calculez son NDCG@3.
37. Pourquoi la popularité est-elle si difficile à battre en recommandation ?
38. Quand l'apprentissage semi-supervisé peut-il nuire ?
39. Quel piège guette un jeu étiqueté constitué par apprentissage actif ?
40. Un agent met à jour $Q(s, a) = 2$ avec $\alpha = 0,5$, une récompense de 1, $\gamma = 0,9$ et $\max_{a'} Q(s', a') = 4$. Quelle est la nouvelle valeur ? Pourquoi un bandit n'est-il pas un test A/B ?

Corrigés des questions

Pour les questions numériques, **calculons** plutôt que de nous fier à la mémoire.

```
import numpy as np

print("Q3 plis de 1000/5 :", 1000 * 4 // 5, "pour l'entraînement,", 1000 // 5, "pour l'évaluation, 5  
↳ modèles")
print("Q9 Gini =", round(1 - (0.3**2 + 0.7**2), 3))
rho, sigma2, B = 0.3, 1.0, 100
print("Q10 variance de la moyenne =", round(rho * sigma2 + (1 - rho) * sigma2 / B, 4))
print("Q16 silhouette =", round((5 - 2) / max(2, 5), 3))
print("Q18 erreur de reconstruction relative =", round(1 - 0.70, 2))
print("Q23 seuil = 1 / (1 + 5) =", round(1 / 6, 4))
TP, FP, FN, TN = 40, 60, 10, 890
prec, rap = TP / (TP + FP), TP / (TP + FN)
print("Q26 exactitude", (TP + TN) / (TP + FP + FN + TN), "| précision", prec, "| rappel", rap, "| F1",  
↳ round(2 * prec * rap / (prec + rap), 4))
print("Q30 Brier =", round(np.mean((np.array([0.9, 0.2, 0.7]) - np.array([1, 0, 0])) ** 2), 4))
print("Q34 score = 2^(-1) =", 2 ** -1)
print("Q35 cosinus =", round(2 / np.sqrt(3 * 4), 4))
dcg = lambda rel: sum(r / np.log2(i + 2) for i, r in enumerate(rel))
print("Q36 NDCG@3 =", round(dcg([1, 0, 1]) / dcg([1, 1, 0]), 4))
print("Q40 nouvelle valeur de Q =", 2 + 0.5 * (1 + 0.9 * 4 - 2))
```

```
Q3 plis de 1000/5 : 800 pour l'entraînement, 200 pour l'évaluation, 5 modèles
Q9 Gini = 0.42
Q10 variance de la moyenne = 0.307
Q16 silhouette = 0.6
Q18 erreur de reconstruction relative = 0.3
Q23 seuil = 1 / (1 + 5) = 0.1667
Q26 exactitude 0.93 | précision 0.4 | rappel 0.8 | F1 0.5333
Q30 Brier = 0.18
Q34 score = 2^(-1) = 0.5
Q35 cosinus = 0.5774
Q36 NDCG@3 = 0.9197
Q40 nouvelle valeur de Q = 3.3
```

1. Le modèle a été ajusté pour bien expliquer **ces** données : il en épouse aussi le bruit. L'erreur d'entraînement est donc en moyenne **trop optimiste** ; il faut mesurer sur des données que l'ajustement n'a pas vues. (1.1)

2. Une information qui **ne serait pas disponible au moment de la prédiction** se glisse dans l'apprentissage. Formes typiques : une variable calculée **après** la date de prédiction (par exemple les commandes des trois mois suivants pour prédire la résiliation) ; un prétraitement (imputation, mise à l'échelle, sélection de variables, encodage par la cible) **ajusté sur toutes les données avant** le découpage. Le résultat est un score flatteur qui s'effondre en production. (1.1, 1.2)
3. 800 observations pour l'entraînement, 200 pour l'évaluation à chaque pli, et **5 modèles** ajustés (code ci-dessus). (1.2.2)
4. Les jeux d'entraînement de deux plis **se recouvrent largement** : les scores sont corrélés, et l'écart-type entre plis **sous-estime** l'incertitude réelle sur la performance. Pour comparer deux modèles, on utilise des différences appariées avec une correction (test t corrigé), ou un bootstrap. (1.2.4, 1.4.2)
5. $E[(y - \hat{f})^2] = \text{bruit} + \text{biais}^2 + \text{variance}$. Quand la complexité augmente, le biais diminue et la variance augmente : l'erreur de validation décrit une courbe en U, dont le minimum est le bon compromis. (1.3)
6. Un grand écart entre entraînement et validation est le signe d'un **surapprentissage** (variance trop élevée). Remèdes : simplifier le modèle (profondeur, régularisation), **plus de données**, moins de variables, arrêt précoce, moyenne de modèles (forêts). On le confirme par une courbe d'apprentissage. (1.3)
7. La référence (prédire la classe majoritaire, une règle métier, un modèle simple) donne **le score à battre** : un modèle sophistiqué qui ne la dépasse pas ne sert à rien. Le test ne sert qu'**une fois**, à la fin, parce que chaque consultation qui influence une décision en fait un jeu de validation, et son score devient optimiste. (1.4.1, 1.4.4)
8. Le taux d'erreur est **non convexe et non différentiable** : on ne sait pas l'optimiser efficacement. La perte logistique est un **substitut convexe** et dérivable, qui majore l'erreur et fournit des **probabilités**. (2.1)
9. $1 - (0,3^2 + 0,7^2) = 0,42$ (code ci-dessus). Un nœud pur vaut 0 ; le maximum pour deux classes est 0,5. (2.2.2)
10. $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 = 0,3 + 0,007 = 0,307$. Moyenner beaucoup d'arbres supprime presque toute la variance « individuelle », mais **pas** la part commune $\rho\sigma^2$: d'où l'idée de **décorréliser** les arbres (sous-ensemble aléatoire de variables à chaque coupure). (2.3.2, 2.3.3)
11. Il ajuste le **pseudo-résidu**, c'est-à-dire l'opposé du gradient de la perte par rapport aux prédictions actuelles : pour la perte quadratique, ce sont les résidus ordinaires. Le boosting est une descente de gradient **dans l'espace des fonctions**. (2.4)
12. L'importance par impureté **favorise les variables continues ou à nombreuses modalités** (elles offrent plus de découpages possibles) et se calcule sur les données d'entraînement. On préfère l'importance par **permutation** sur des données non vues, ou SHAP. (2.3.6, 5.3.2)
13. Un petit pas rend chaque arbre moins influent : le modèle apprend plus **lentement** mais généralise mieux, au prix de plus d'arbres. L'**arrêt précoce** interrompt l'ajout d'arbres quand l'erreur de validation cesse de s'améliorer, ce qui règle le nombre d'arbres automatiquement. (2.4)
14. Exigent la mise à l'échelle : les modèles à **distances** (k-NN, SVM, k-means), les modèles à **pénalité** (Ridge, Lasso) et la descente de gradient. Les **arbres** et leurs ensembles s'en passent, car ils ne comparent que l'ordre des valeurs. (2.1, 4.1)
15. Par **plusieurs épreuves qui se rejoignent** : critères internes (silhouette, Calinski-Harabasz, Davies-Bouldin), comparaison à une **référence sans structure, stabilité** par

sous-échantillonnage, et, si l'on dispose d'une vérité, des indices externes (indice de Rand ajusté, information mutuelle normalisée). Jamais par un seul chiffre. (3.1)

16. $s = \frac{b-a}{\max(a,b)} = \frac{5-2}{5} = 0,6$: le point est bien rangé dans son groupe. Une silhouette proche de 0 signale un point à la frontière, une valeur négative un point probablement mal classé. (3.1.2)

17. Les k-means utilisent des **distances** : une variable en euros écrase une variable comprise entre 0 et 1. Dans notre étude, l'indice de Rand ajusté passe de 0,49 avec les variables standardisées à 0,08 sans standardisation. (3.1.6)

18. $1 - 0,70 = 0,30$: l'erreur de reconstruction relative est la variance **non** conservée (théorème d'Eckart-Young). (3.2)

19. Les **distances entre groupes**, la **taille** des groupes et leurs formes fines. Ces projections préservent le voisinage local, pas la géométrie globale. (3.4)

20. Remplacer une modalité par la moyenne de la cible calculée **sur ses propres lignes** laisse fuir la cible : sur un code qui n'est que du bruit, on obtient une AUC de 0,678 à l'entraînement et 0,510 au test. Remède : calculer l'encodage **hors pli** (out-of-fold) avec un lissage, à l'intérieur du pipeline. (4.1)

21. Les arbres, **non**. La régression logistique pénalisée, **oui** : la pénalité dépend de l'échelle des coefficients, donc de celle des variables. (4.1, 2.1)

22. Une valeur est manquante **de façon non aléatoire** : la probabilité de l'absence dépend de la valeur elle-même (par exemple, les clients mécontents répondent moins à l'enquête de satisfaction). On ajoute un **indicateur d'absence**, on n'impute qu'à l'intérieur du pipeline, et on reconnaît que l'imputation ne récupère pas l'information perdue. (4.1)

23. $\frac{c_{FP}}{c_{FP}+c_{FN}} = \frac{1}{1+5} \approx 0,167$: on relance dès que la probabilité de défaut dépasse environ 17 %, loin du seuil de 0,5 pris par habitude. (4.3.5)

24. Ils **décalent les probabilités** (elles deviennent trop élevées pour la classe rare) et déplacent le bon seuil, mais **n'améliorent pas, en général, le classement** des clients. Si l'on a besoin de probabilités, on recalibre ; si l'on a besoin d'un classement, ils sont rarement utiles. (4.3.2, 5.2)

25. Les points synthétiques sont créés à partir de points qui se retrouveront **dans les plis d'évaluation** : on évalue sur des points qui ressemblent à des copies de l'entraînement, et le score devient absurde (jusqu'à 0,999 d'AUC). Le rééchantillonnage ne doit se faire que sur les plis d'entraînement, dans un pipeline. (4.5)

26. Exactitude = $\frac{40+890}{1000} = 0,93$; précision = $\frac{40}{100} = 0,4$; rappel = $\frac{40}{50} = 0,8$; $F_1 = \frac{2 \times 0,4 \times 0,8}{1,2} \approx 0,533$ (code ci-dessus). Une exactitude de 93 % cache une précision de 40 % : 5 % de positifs seulement. (5.1.2)

27. L'AUC est la **probabilité qu'un client positif tiré au hasard reçoive un score plus élevé qu'un client négatif** tiré au hasard (0,5 : aucun pouvoir de classement). Elle est liée à la statistique de Mann-Whitney. (5.1.4)

28. L'AUC est insensible à la prévalence et peut rester élevée alors que presque toutes les alertes sont fausses ; la **précision moyenne** mesure ce qu'on obtient dans la région utile (les premiers rangs) et son niveau de référence est la prévalence. Sur nos données de crédit, un classement au hasard donne une précision moyenne de 0,22 (la prévalence), et notre modèle atteint 0,56. (5.1.5)

29. Un modèle est **calibré** quand, parmi les clients à qui il attribue 30 %, environ 30 % ont l'événement. On le vérifie par un **diagramme de fiabilité** (et l'erreur de calibration), on le répare par la méthode de **Platt** (logistique sur le score) ou la **régression isotonique**, sur un jeu séparé de celui de l'ajustement. (5.2)

30. $\frac{(0,9-1)^2+(0,2-0)^2+(0,7-0)^2}{3} = \frac{0,01+0,04+0,49}{3} = 0,18$ (code ci-dessus). Plus il est bas, mieux c'est ; 0 serait parfait. (5.1.6)

31. L'**efficacité** : la somme des contributions des variables est **exactement** la différence entre la prédiction du client et la prédiction moyenne. Mais SHAP **décrit le modèle**, pas le monde : une contribution élevée ne dit pas qu'agir sur la variable changerait le résultat. (5.3.5, 5.3.7)

32. Par exemple la **parité démographique** (même taux de décisions positives dans chaque groupe), l'**égalité des chances** (même rappel), la **parité prédictive** (même précision) ou la **calibration par groupe**. Quand les taux de base diffèrent entre groupes, on **ne peut pas** tous les satisfaire en même temps (résultat d'impossibilité) : il faut choisir et justifier. (5.4)

33. Elles ne valent **rien** : un détecteur qui ne signale jamais rien obtient 99,19 %. On juge un détecteur par la **précision moyenne**, le **rappel à budget d'alertes fixé** ou la **précision@k**, jamais par l'exactitude. (6.1)

34. $s = 2^{-E[h]/c(n)} = 2^{-1} = 0,5$: un point moyennement isolable, ni franchement anormal (s proche de 1) ni franchement normal (s inférieur à 0,5). (6.3.2)

35. $\cos = \frac{n_{AB}}{\sqrt{n_A n_B}} = \frac{2}{\sqrt{3 \times 4}} \approx 0,577$. (7.1)

36. $DCG = \frac{1}{\log_2 2} + 0 + \frac{1}{\log_2 4} = 1,5$; le classement idéal (1, 1, 0) donne $1 + \frac{1}{\log_2 3} \approx 1,631$; $NDCG@3 \approx 0,920$ (code ci-dessus). (7.4.1)

37. Dans un catalogue, quelques produits concentrent l'essentiel des achats : recommander **les plus achetés** contente déjà beaucoup de monde, et la personnalisation n'apporte que quelques points, mesurables seulement avec un protocole rigoureux. (7.1, 7.4)

38. Quand les **hypothèses** qui font de l'étiquette inconnue une information utile sont **violées** : classes qui se chevauchent, graphe où les voisins ne partagent pas la même étiquette (par exemple les clients de la boutique : 80,5 % de voisins de même étiquette contre 75,9 % au hasard), ou étiquettes peu représentatives. (8.1, 8.2)

39. Le jeu étiqueté n'est **pas représentatif** : on a étiqueté surtout des cas ambigus ou rares (38 % de résiliations contre 14 % réellement), et les probabilités du modèle sont **faussées** (0,078 prédit pour 0,14 réel). Il faut un jeu de test tiré au hasard et, au besoin, un recalibrage. (8.3)

40. $Q \leftarrow 2 + 0,5(1 + 0,9 \times 4 - 2) = 3,3$ (code ci-dessus). Un **bandit** sert à **gagner en apprenant** : il alloue de plus en plus d'essais à ce qui marche (et paie un « regret »), alors qu'un test A/B sert à **conclure** avec une puissance connue et répartit les essais à égalité. (9.2, 9.3.3)

Votre grille d'auto-évaluation

Pour chaque ligne : **je sais l'expliquer** / **je sais le faire** / **à revoir**.

Compétence	Où la retravailler
Formuler un problème, séparer les données, éviter la fuite	1.1
Valider par validation croisée et en estimer l'incertitude	1.2
Diagnostiquer biais, variance, surapprentissage	1.3
Se comparer à une référence, comparer deux modèles	1.4
Régler des hyperparamètres sans se tromper	1.5

Compétence	Où la retravailler
Calculer un arbre, expliquer forêts et boosting	2.2, 2.3, 2.4
Valider un regroupement sans étiquettes	3.1
Choisir une réduction de dimension	3.2
Encoder, mettre à l'échelle, traiter les manquants	4.1
Créer et sélectionner des variables	4.2
Traiter des classes déséquilibrées, choisir un seuil par les coûts	4.3
Choisir et lire des métriques	5.1
Vérifier et réparer la calibration	5.2
Expliquer un modèle (permutation, PDP, LIME, SHAP)	5.3
Auditer l'équité, quantifier l'incertitude	5.4, 5.5
Détecter des anomalies, recommander, apprendre avec peu d'étiquettes, décider en séquence	6, 7, 8, 9
Mener un projet d'apprentissage automatique de bout en bout	Projet du volume